

Hybrid AC/DC Module Technical Notebook

Current C++ Architecture, Information Models, Mathematical Models, and Solver Workflows

HACDCPF C++ Module

May 7, 2026

Contents

1	Scope and Documentation Contract	10
1.1	Covered Module Families	10
1.2	Documentation Principle	11
1.3	Canonical Modeling Pipeline	11
1.4	Important Boundary Between “Implemented” and “Declared”	11
2	Nomenclature	12
2.1	Notation Policy	12
2.2	Index and Qualifier Conventions	12
2.3	Canonical Naming Grammar	13
2.4	Superscript/Subscript Rule	13
2.5	Integration Status Codes	13
3	Current C++ Architecture	13
3.1	Layered View	13
3.2	Runtime Data Flow	15
3.3	Public API Surface	15
3.4	Solver Families and Backends	16
3.5	Current Architectural Strengths	17
3.6	Current Architectural Friction Points	17
4	Engine Solver System	17
4.1	Layering Inside <code>hacdcpf::engine</code>	18
4.2	Canonical Problem Classes and Standard Forms	18
4.3	Validation and Dispatch Workflow	19
4.4	Solver Inventory and Current Implementation Status	20
4.5	Key Algorithm Workflows	22
4.5.1	Power-Flow-Specific Newton Kernel	22
4.5.2	Generic Nonlinear Equation Solver	23
4.5.3	Generic NLP Solver Used Today	23
4.5.4	Convex QP and LCQP Solver	24
4.5.5	High-Performance LP Interior-Point Method	24
4.5.6	Dual Simplex and Standard-Form Reoptimization	25
4.5.7	First-Order PDLP Solver	25
4.5.8	Mixed-Integer Branch-and-Cut	26
4.5.9	External Backend Wrapper Pattern	26

4.6	Assessment Against the Self-Contained Computation-Engine Target	27
4.6.1	Robustness Assessment	27
4.6.2	Performance Assessment	27
4.6.3	Readiness Summary	28
4.7	Recommended Reading Order for Maintainers	28
5	Native NLP Interior-Point Method: Theory, Algorithm, and Current Diagnostics	29
5.1	Standard NLP Form and Barrier KKT System	29
5.2	Current Driver Architecture in <code>ipm_solver.cpp</code>	30
5.3	Curvature Sources and Hessian Policy	30
5.4	Algorithmic Workflow of the Merit Driver	30
5.5	Algorithmic Workflow of the Filter Driver	31
5.6	What the Unit Tests Currently Prove	31
5.7	Structured Sparse Benchmark Results	32
5.8	Large Parity-OPF Comparison Results	32
5.9	Current Diagnostic Interpretation: Is the Code Still Wrong?	33
5.10	Most Likely Remaining Gap	34
5.11	Reviewer-Style Risk Checklist	34
6	Generic NativeIPM vs. Parity-Specific IPM: Algorithm Gap and Refactoring Roadmap	35
6.1	What Is Actually Being Compared	35
6.2	Algorithmic Differences	36
6.3	Why the Dedicated Parity Solver Still Wins	36
6.4	Current Performance Interpretation	37
6.5	Implementation Backlog: PR1 / PR2 / PR3	37
6.6	Practical Bottom Line	40
7	Symbols, Indexing, and Sign Convention	40
7.1	Electrical Bases	40
7.2	Sets and Indexing	40
7.3	Core AC and DC State Variables	40
7.4	Unified Sign Convention: Bus Injection Positive	41
7.5	Power-Balance Template	42
8	Information Models and Field Dictionaries	42
8.1	Modeling Layers Used by the Code	42
8.2	System Containers and Solver-Level Metadata	42
8.3	Component Inventory	45
8.4	AC Network Elements	46
8.5	Load Models and Demand-Side Components	48
8.6	Generation, Storage, and Aggregation Components	49
8.7	DC and Power-Electronic Components	50
8.8	Optional Three-Phase Models	51
9	I/O Contracts, Sheet Dictionaries, and JSON Keys	52
9.1	Serialization Families	52
9.2	Global Parsing and Workbook Rules	53
9.3	Hybrid System JSON Contract	54
9.4	Excel Input Workbook Contract	59
9.5	Compact Power-Flow Workbook Additions	67

9.6	Dedicated Result Workbooks	68
9.7	Current I/O Coverage Gaps That Matter to Engineering Work	72
9.8	Interpretation	73
10	Detailed I/O Field Dictionaries	73
10.1	System Metadata	73
10.2	AC Network and Demand Models	74
10.3	Generation and Storage Models	81
10.4	DC and Hybrid Coordination Models	84
10.5	Cross-Reference	90
11	Output and Report Field Dictionaries	90
11.1	Export Families	90
11.2	Result Semantic Conventions	91
11.3	Solver-Specific Result Notes	93
11.4	Consumer Contracts	95
11.5	Dedicated Power-Flow Result Sheets	98
11.6	Compact Power-Flow Result Columns	103
11.7	Dedicated OPF Sheets	103
11.8	Carbon Result JSON and Workbook Dictionaries	105
11.9	Short-Circuit and Network-Reconfiguration API Results	111
11.10	Current Output-Side Gaps	112
12	Power-Flow Models	113
12.1	Scope and Architecture Overview	113
12.2	Mathematical Problem Form	114
12.3	VSC Converter Model	115
12.4	DCDC Converter Model	116
12.5	Jacobian Structure	116
12.6	Hybrid Newton Algorithm	117
12.7	Fast-Decoupled Power Flow (FDPF)	118
12.8	DC-Only Power Flow	119
12.9	Adaptive Islanded Solver	119
12.10	HELM Solver	120
12.11	Error Handling and Diagnostics	120
12.12	Maintainability Roadmap: Code-Duplication Inventory	120
12.13	Numerical Performance (Release Mode)	121
13	Optimal Power Flow Models	122
13.1	Problem Form and Runtime Scope	122
13.2	Code Map	123
13.3	Pseudocode Map	123
13.4	Current Status	123
13.5	Top-Level Solver Routing	123
13.6	Fast Dispatch + PF Fallback (AC-Only Path)	124
13.7	Native Primal-Dual OPF Formulation	124
13.7.1	Decision Variables	124
13.7.2	Objective	124
13.7.3	Equality Constraints	124
13.7.4	Bounds and Nonlinear Limits	125
13.7.5	KKT System and Globalization	125
13.8	Parity Full-Space Formulation	125

13.8.1	Decision Variables	125
13.8.2	Objective	125
13.8.3	Equality Constraints	126
13.8.4	Inequality Constraints	127
13.8.5	Primal-Dual IPM System	127
13.9	Component Coverage by Solver Path	127
13.10	Fallback Behavior	128
13.11	DC Optimal Power Flow (<code>solve_dc_opf</code>)	128
13.11.1	Decision Variables	128
13.11.2	Network Model	128
13.11.3	Objective Function	129
13.11.4	Constraints	129
13.11.5	Solver Backend Selection	129
13.11.6	Locational Marginal Prices	130
13.12	Reactive Power Optimization (<code>solve_reactive_power_opt</code>)	130
13.12.1	Problem Structure	130
13.12.2	Objective Functions	130
13.12.3	Algorithm: Four-Phase Outer Approximation	130
13.12.4	Inner NLP Solver	131
13.12.5	Result Structure	131
14	Short-Circuit Models	131
14.1	Problem Form and Runtime Scope	131
14.2	Code Map	131
14.3	Pseudocode Map	131
14.4	Current Status	131
14.5	Implemented Solver Paths	131
14.6	Fault-Network Construction	132
14.7	Z-Bus Evaluation	132
14.8	Current Implemented Fault Equations	132
14.9	Reported Quantities	133
14.10	Detailed IEC Helper Functions	133
14.11	Practical Interpretation	133
15	Component-to-Module Integration Matrix	134
15.1	Status Codes	134
15.2	Module Preprocessing Differences	134
15.3	Detailed Integration Matrix	134
15.4	Notable Row-Level Notes	135
15.5	Interpretation	136
16	Optimal Network Reconfiguration	136
16.1	Problem Form and Runtime Scope	136
16.2	Code Map	136
16.3	Pseudocode Map	136
16.4	Current Status	136
16.5	Scope of the Current ONR Module	137
16.6	Decision Variables	137
16.7	Net-Injection Model	137
16.8	LinDistFlow MILP	137
16.9	Branch-and-Cut Configuration	138
16.10	Postprocessing and Verification	139

16.11	Current Modeling Boundary	139
17	Distribution Power Flow	139
17.1	Problem Form and Runtime Scope	139
17.2	Code Map	139
17.3	Pseudocode Map	139
17.4	Current Status	139
17.5	Scope of the Current DPF Module	139
17.6	Radial Network Requirement	140
17.7	Net-Injection Model	140
17.8	Backward-Forward Sweep Equations	140
17.9	Outer Loop for Reactive Limits	141
17.10	Reported Branch Power and Losses	141
17.11	Relationship to the Main Newton PF	141
17.12	Three-Phase Power Flow (abc Domain)	141
17.12.1	Phase Domain and API	141
17.12.2	3×3 Series Impedance from Sequence Parameters	142
17.12.3	Per-Phase Net Injection	142
17.12.4	Three-Phase Newton–Raphson Solver	142
17.12.5	Three-Phase Backward-Forward Sweep	144
17.12.6	Per-Phase Branch Power and Losses	145
17.12.7	Voltage Unbalance Factor (VUF)	145
17.12.8	Balanced-System Reduction	145
18	Distribution Reliability Assessment with Failure-Mode Enumeration	145
18.1	Problem Form and Runtime Scope	145
18.2	Code Map	146
18.3	Pseudocode Map	146
18.4	Current Status	146
18.5	Sets and Indices	146
18.6	Failure and Duration Parameterization	147
18.7	Contingency Enumeration and Weights	147
18.8	Reconfiguration-Aware Restoration Optimization	147
18.9	Microgrid and Grid-Forming BESS Formulation	148
18.9.1	Microgrid islanding states	148
18.9.2	Grid-forming BESS support constraints	149
18.10	Reliability Metrics for Distribution Assessment	149
18.10.1	Energy and adequacy indices	149
18.10.2	Customer-based distribution indices	149
18.10.3	Priority-weighted resilience metric	150
18.11	Modeling 30+ AC/DC Component Types in One Framework	150
18.12	Algorithmic Procedure (Deterministic FMEA)	150
18.13	Implementation Notes for the C++ Reliability Module	150
18.14	Component Reliability Catalog for Hybrid AC/DC FMEA	150
19	Distribution Resilience Assessment with Mobile Energy Storage Systems	152
19.1	Problem Form and Runtime Scope	152
19.2	Code Map	152
19.3	Pseudocode Map	152
19.4	Current Status	152
19.5	Sets and Indices	153
19.6	State Variables	153

19.7	Fault and Repair Process	154
19.8	Load Restoration and Prioritization	154
19.9	Island Supply Model	154
19.10	MESS Energy Dynamics	154
19.11	Transportation-Network Coupling	155
19.12	Radial Reconfiguration	155
19.13	Objective Function	155
19.14	Resilience Metrics	156
19.15	Implementation Notes	156
19.16	Validation Snapshot	157
20	Carbon Flow Analysis	157
20.1	Problem Form and Runtime Scope	157
20.2	Code Map	157
20.3	Pseudocode Map	157
20.4	Current Status	157
20.5	Purpose and Required Inputs	158
20.6	Implemented Scope and Current Boundary	158
20.7	Canonical Hybrid Graph	159
20.8	Source and Load Normalization	159
20.9	Directed Carrier Construction	160
20.10	Proportional Tracing on the Hybrid Graph	162
20.11	Matrix-Based Hybrid Bus Carbon Intensity	163
20.12	Emissions Summaries	163
20.13	Result Artifacts	164
20.14	Sankey Visualization of Carbon/SO ₂ Source-Sink Relations	164
21	Algorithm Flowcharts and Solver Workflows	165
21.1	Canonical Preprocessing Workflow	165
21.2	Hybrid Newton Power Flow	166
21.3	AC OPF Path Selection	167
21.4	Hybrid Carbon-Analysis Workflow	168
21.5	Distribution PF Workflow	169
21.6	Short-Circuit Workflow	170
21.7	ONR Workflow	170
21.8	Solver-Family Summary	171
21.9	Code-Aligned Pseudocode (All Primary Algorithms)	172
22	Native Branch-and-Cut Solver	180
22.1	Scope and Code Map	180
22.2	Active and Prepared Tree Data Structures	181
22.3	Problem Class and Outer Mathematical Model	181
22.4	MILP Presolve and Bound Strengthening	182
22.5	Standard Form and LP Relaxation Kernel	183
22.6	Warm Start by Dual Simplex	184
22.7	Cut Families and Admission Rules	185
22.8	Cut Pool and Solution Pool	186
22.9	Branching, Node Selection, and Primal Heuristics	186
22.10	Root-Node Strengthening Pipeline	187
22.11	Child Processing and Sequential Tree Search	190
22.12	Heterogeneous Parallel Tree Search	191
22.13	MINLP Path	192

22.14	Options and Statistics	193
22.15	Applications, Tests, and Code-Aligned Caveats	194
23	Validation Assets and Current Evidence	195
23.1	Validation Philosophy	195
23.2	Latest Release-Mode Core Snapshot	196
23.3	Cross-Validation Map	196
23.4	What Is Well Covered Today	197
23.5	What Still Needs Better Validation	198
23.6	Suggested Interpretation for Users	198
24	Current Project Implementation Audit	198
24.1	Audit Scope and Reading Order	198
24.2	Canonical Mathematical Form Shared by the Project	199
24.3	Mathematical Model Ledger	199
24.4	Algorithm and Implementation Ledger	201
24.5	Numerical Evidence and Performance Snapshot	203
24.6	Code-Quality Investigation by Module	204
24.7	Immediate Documentation Contract	206
25	Hybrid AC/DC Distribution DER Planning Under Carbon and Peak $N-1$ Reliability Constraints	206
25.1	Study Object and Assumptions	206
25.2	Sets, Parameters, and Planning Variables	207
25.2.1	Network and device sets	207
25.2.2	Planning variables	208
25.3	Three-Layer Planning Model	208
25.4	Normal-Operation Recourse Model	209
25.4.1	Operating variables	209
25.4.2	Operating objective	209
25.4.3	Carbon accounting	211
25.5	Peak-Time $N-1$ Reliability Model	211
25.5.1	Contingency variables	211
25.5.2	Peak-contingency constraints	211
25.5.3	FMEA-weighted reliability metric	213
25.6	Extreme-State $N-k$ Resilience Model	213
25.6.1	Extreme-state variables	213
25.6.2	Partition topology and source activation	214
25.6.3	Extreme-state electrical and load-service constraints	214
25.7	Discussion of Modeling Choices	216
25.8	Illustrative Test System for Case 10b	216
25.8.1	Network structure	216
25.8.2	Candidate DER placement	217
25.8.3	Representative operating states	217
25.8.4	Extreme-state library and $N-k$ families	217
25.8.5	Contingency set for peak-time FMEA	218
25.9	Baseline Parameter Set	218
25.10	Experimental Design	219
25.10.1	Evaluation metrics	219
25.10.2	Case 1: Base AC/DC planning without carbon or reliability pressure	219
25.10.3	Case 2: Carbon-constrained planning	220
25.10.4	Case 3: Peak-time $N-1$ reliability-constrained planning	220

25.10.5 Case 4: Joint carbon- and reliability-constrained planning	220
25.10.6 Case 5: Extreme-state $N-k$ resilience planning	220
25.10.7 Case 6: Architecture and device-value sensitivities	221
25.11 Recommended Reporting Tables and Figures	221
25.12 Expected Experimental Findings	221
25.13 Result-Display Skeleton	222
25.13.1 Case-summary and parameter-table templates	222
25.13.2 Figure placeholders	222
26 Improvement Plan	222
26.1 Completion Audit of the Previous Plan	222
26.2 Native Branch-and-Cut Proof-Tail Direction	224
26.3 Current Code-Quality Priorities	226
26.4 Module-Level Investigation and Improvement Matrix	228
26.5 Recommended Execution Order	231
27 Time-Series Analysis	231
27.1 Problem Form and Runtime Scope	231
27.2 Code Map	231
27.3 Pseudocode Map	232
27.4 Current Status	232
27.5 Code-Execution Path	232
27.6 Input Profiles and Runtime Scaling	233
27.7 Annual Production-Simulation Model (Implemented — L0/L2/L3)	233
27.7.1 Implementation Status	233
27.7.2 Position in the Existing Modeling Framework	233
27.7.3 Chronological Horizon and Sets	234
27.7.4 Decision Variables	234
27.7.5 Objective Function	235
27.7.6 Core Constraints	235
27.7.7 Hierarchical Temporal Decomposition	236
27.7.8 Coupling to OPF/PF Replay	240
27.7.9 Annual Output Metrics	240
27.8 Unit Commitment MILP (Implemented)	240
27.8.1 Sets and Variables	240
27.8.2 Objective Function	241
27.8.3 Constraints	241
27.8.4 Model-Class and Solution Output	242
27.8.5 Implementation Scope Notes (Important)	242
27.9 Distribution-System Counterpart: Time-Series Topology Reconfiguration	242
27.9.1 Sets, Parameters, and Decision Variables	243
27.9.2 Objective Function (Hybrid TS-TR)	243
27.9.3 Core Constraints	244
27.9.4 Component-Wise Sub-Equations (One-to-One with C++ Models)	245
27.9.5 Position in Time-Series Runtime Pipeline	248
27.9.6 Code-Aligned Pseudocode (Proposed)	248
27.9.7 C++ Interface Draft (time_series_pf-style, Proposed)	249
27.10 Time-Domain Power Flow (Sequential Hybrid PF)	251
27.10.1 Step State and Residual Equations	251
27.10.2 Per-Step Injection Update Rules	251
27.10.3 Sequential Outputs and Aggregation	251
27.11 Post-Processing Metrics	252

27.12	Current C++ API (Implemented)	252
28	Market Clearing Mathematical Models	253
28.1	Problem Form and Runtime Scope	253
28.2	Code Map	253
28.3	Pseudocode Map	253
28.4	Current Status	253
28.5	Nomenclature	254
28.5.1	Sets and Indices	254
28.5.2	Decision Variables	254
28.5.3	Parameters	255
28.6	SCUC – Security-Constrained Unit Commitment (§2.6.3)	256
28.6.1	Objective Function	256
28.6.2	Constraint 1: Power Balance (§2.6.3.1)	257
28.6.3	Constraint 2: Positive Reserve (§2.6.3.2)	257
28.6.4	Constraint 3: Negative Reserve (§2.6.3.3)	257
28.6.5	Constraint 4: Primary Frequency Regulation Reserve (§2.6.3.4)	257
28.6.6	Constraint 5: Must-Run Units (§2.6.3.5)	257
28.6.7	Constraint 6: Generator Output Expression (§2.6.3.6)	258
28.6.8	Constraint 7: Generator Output Limits (§2.6.3.8)	258
28.6.9	Constraint 8: Generator Group Output Limits (§2.6.3.9)	258
28.6.10	Constraint 9: Generator Group Energy Limits (§2.6.3.10)	258
28.6.11	Constraint 10: Ramp Rate (§2.6.3.11)	258
28.6.12	Constraint 11: Minimum Up/Down Time (§2.6.3.12)	258
28.6.13	Constraint 12: Maximum Startup/Shutdown Count (§2.6.3.13)	259
28.6.14	Constraint 13: Line Flow Limits (§2.6.3.14)	259
28.6.15	Constraint 14: Section Flow Limits (§2.6.3.15)	259
28.6.16	Constraint 15: Storage (§2.6.3.16)	259
28.6.17	Constraint 16: DC Line (§2.6.3.17)	260
28.6.18	Constraint 17: Hydro Water Level (§2.6.3.18)	260
28.6.19	Constraint 18: Hydro Energy Limits (§2.6.3.19)	261
28.6.20	Constraint 19: Renewable Output (§2.6.3.20)	261
28.6.21	Constraint 20: Inter-Provincial Priority Schedule (§2.6.3.21)	261
28.7	SCED – Security-Constrained Economic Dispatch (§2.6.4)	261
28.7.1	SCED Objective (§2.6.4)	261
28.7.2	SCED Key Differences from SCUC	261
28.8	LMP – Locational Marginal Price (§2.6.5–2.6.6)	262
28.8.1	LMP Objective (§2.6.5)	262
28.8.2	δ -Neighbourhood Bounds (§2.6.5.5)	262
28.8.3	LMP Decomposition (§2.6.6)	262
28.9	Market Clearing Pipeline	263
28.10	Summary of C++ Implementation Mapping	263
28.11	MILP Formulation Tightening: Problem-Specific Cutting Planes	264
28.11.1	LP Relaxation Gap Sources	264
28.11.2	Generator Cut Families (Families 1–6)	265
28.11.3	Storage Cut Families (Families 7–11)	266
28.11.4	Cut Properties and Separation Complexity	268
28.11.5	Layered Separation Strategy	268
28.12	Market-Model-Specific Cutting Planes (SCUC Extensions)	269
28.12.1	Family A: Bid-Segment Cumulative Coupling	269
28.12.2	Family B: Reserve–Commitment Cover Inequality	269

28.12.3 Family C: Monitored-Section Commitment Inequality	270
28.12.4 Family D: DC-Line Direction–Commitment Coupling	270
28.12.5 Family E: Storage Cycle-Limit Strengthening	270
28.12.6 Family F: Hydro–Thermal Substitution Inequality	271
28.12.7 Family G: Extended Time-Window Clique (96-Period)	271
28.12.8 Family H: Inter-Provincial Schedule–Commitment Coupling	272
28.12.9 Family I: Primary Frequency Regulation–Storage Joint Cut	272
28.12.10 Market-Cut Priority and Separation Strategy	272
28.13 Numerical Performance: Pre-Formulation Cuts on Five-Province SCUC	273
29 Stochastic Dispatch and Market-Boundary Optimization	273
29.1 Problem Form and Runtime Scope	273
29.2 Code Map	274
29.3 Pseudocode Map	274
29.4 Current Status	274
29.5 Canonical Problem Form	274
29.6 How This Chapter Relates to the Standalone Manuscript	274

1 Scope and Documentation Contract

This notebook is the implementation-facing handbook for the current `hacdcpf` C++ module. Its purpose is not only to list equations, but to connect each mathematical model to the actual public data structures, preprocessing steps, solver paths, and result objects used by the code base.

1.1 Covered Module Families

The current notebook covers the following runtime subsystems:

- the engine solver layer under `hacdcpf::engine`, including canonical problem classes, adapter dispatch, native kernels, and external backend wrappers,
- hybrid AC/DC power flow under `hacdcpf::solve_power_flow`,
- AC fast-decoupled, DC-only, linearized AC/DC, adaptive-islanding, and distributed-slack power flow variants,
- AC optimal power flow under `hacdcpf::opf::solve_ac_opf`, including the native primal-dual path and the parity-IPM path,
- DC optimal power flow under `hacdcpf::opf::solve_dc_opf`, with piecewise-linear or quadratic cost and LMP extraction,
- reactive power optimization under `hacdcpf::opf::solve_reactive_power_opt`, a MINLP with outer-approximation cutting planes for discrete tap/shunt control,
- short-circuit analysis under `hacdcpf::analysis::compute_short_circuit`,
- distribution power flow under `hacdcpf::analysis::solve_distribution_pf`,
- optimal network reconfiguration under `hacdcpf::analysis::solve_optimal_reconfiguration`,
- unit commitment and time-series power flow under `hacdcpf::analysis::solve_time_series_pf`,

- annual production simulation under `hacdcpf::analysis::solve_annual_production_simulation`, with hierarchical L0/L2/L3 decomposition,
- lifecycle simulation under `hacdcpf::analysis::run_lifecycle_simulation`, with multi-year degradation tracking and stratified carbon error bounds,
- data interchange and case-building support used to feed the above solvers.

1.2 Documentation Principle

This notebook follows four rules.

1. Equations are written in the same sign convention as the current C++ implementation.
2. Field explanations are tied to concrete structs, not generic textbook entities.
3. Each component is classified by how it enters a module: direct state, fixed injection, projected equivalent, or not yet consumed.
4. Extension points declared in headers are explicitly separated from functionality that is already wired in source files.

1.3 Canonical Modeling Pipeline

The steady-state solvers do not consume every rich component table directly. Instead, the dominant runtime path is:

1. start from a public `HybridPowerSystem`,
2. call `project_to_canonical_models`,
3. build `SolverData` through `make_solver_data`,
4. assemble residuals, Jacobians, or fault admittances for PF, OPF, or SC.

The projection stage currently performs the following transformations:

- `FlexibleLoad` $\rightarrow$ equivalent `Load`,
- `AsymmetricLoad` $\rightarrow$ equivalent aggregated `Load`,
- `Transformer2W` and `Transformer3W` $\rightarrow$ one or more equivalent `ACBranch` objects,
- `Switch` $\rightarrow$ equivalent `ACBranch` whose `in_service` flag follows the switch state,
- `Charger` $\rightarrow$ accumulation into `ChargingStation` demand,
- optional `ThreePhaseACSystem` $\rightarrow$ canonical single-phase-equivalent AC models when the explicit AC system is absent.

1.4 Important Boundary Between “Implemented” and “Declared”

The parity OPF formulation now instantiates **all** enhanced component decision blocks as runtime variables with full Jacobian and Hessian wiring:

- generator dispatch (P_g, Q_g) ,
- VSC converter power $(P^{\text{conv,ac}}, Q^{\text{conv,ac}}, P^{\text{conv,dc}})$,
- DCDC converter input power P^{ddc} with analytical I^2R loss derivatives,

- DC bus voltages v^{dc} ,
- per-bus load shedding $(\Delta P_d, \Delta Q_d)$ with VOLL penalty,
- curtailable renewable dispatch $(P^{\text{ren}}, Q^{\text{ren}})$ for `RenewableGen` with `curtailable=true` and `PVSystem` with `controllable=true`,
- storage dispatch $(P^{\text{stor}}, Q^{\text{stor}})$ for AC storage and $P^{\text{stor,dc}}$ for DC storage,
- flexible load adjustment P^{flex} .

These are no longer extension hooks: they are fully wired decision variable blocks with explicit mapping arrays in `parity_formulation.cpp`. The notebook marks residual I/O contract gaps (e.g. result export for DCDC/curtailment/storage dispatch fields) where applicable.

2 Nomenclature

2.1 Notation Policy

This notebook uses a strict notation policy consistent with the C++ module:

- scalar variables use italic math symbols, for example P_i, Q_i, V_i ,
- sets use calligraphic symbols, for example $\mathcal{N}_{ac}, \mathcal{E}_{dc}$,
- all semantic qualifiers use upright superscripts via $\cdot$, for example $P_i^{\text{spec}}, P_i^{\text{calc}}, V_k^{\text{dc}}$,
- for variable symbols, subscripts are reserved for ordered indices (bus/component/time) and not for semantic class labels,
- set symbols may use compact domain tags in subscript, e.g., $\mathcal{N}_{ac}, \mathcal{E}_{dc}$,
- model-family and semantic words must stay in superscripts: write $P^{\text{dcdc}}, P^{\text{flex}}, P_r^{\text{rated}}, c_r^{\text{curt}}$, and $\mathcal{R}^{\text{curt}}$.

2.2 Index and Qualifier Conventions

Token	Meaning
i, j	AC bus indices
k, m	DC bus indices
c	converter index (VSC / DCDC)
b	branch index
g	generator index
spec	specified (assembled) quantity from component injections/demands
calc	network-calculated quantity from current state
ac, dc	AC-side / DC-side quantity qualifier
conv	converter-side quantity qualifier
dcdc, flex	DCDC-dispatch / flexible-load variable class qualifier
rated, curt	rated-capacity / curtailment-related qualifier
min, max	lower/upper operating bounds

2.3 Canonical Naming Grammar

Use the canonical symbol template

$$X_{\text{indices}}^{\text{qualifiers}}, \quad (1)$$

with the following rules:

- index block (.): ordered indices only $(i, j, k, c, r, t, \dots)$,
- qualifier block ('): semantic tags only (spec, calc, conv, dcdc, flex, rated, curt, min, max, ...),
- when multiple qualifiers are needed, order them as: model/class $\rightarrow$ side or direction $\rightarrow$ status/bound/cost.

Examples:

$$P_i^{\text{conv,ac}}, \quad P_k^{\text{dcdc,in}}, \quad P_r^{\text{rated}}, \quad c_r^{\text{curt}}. \quad (2)$$

2.4 Superscript/Subscript Rule

Rule	Usage
Subscript (.)	for variable symbols: ordered indices only (i, j, k, c, r, t) ; set symbols may use domain tags $(\mathcal{N}_{ac}, \mathcal{E}_{dc})$.
Superscript (')	semantic labels: domain, role, operating mode, variable class, bounds, and status qualifiers.
Forbidden mixed style	avoid semantic words in subscript, such as $P_{dcdc}, P_{flex}, \mathcal{R}_{curt}, P_{rated}, c_{curt}$.
Preferred style	$P^{\text{dcdc}}, P^{\text{flex}}, \mathcal{R}^{\text{curt}}, P_r^{\text{rated}}, c_r^{\text{curt}}$.

2.5 Integration Status Codes

The integration matrix uses these codes:

- D: direct model consumption in the module,
- F: fixed-only consumption (no explicit variable),
- P: consumed after projection to canonical equivalents,
- N: not consumed by the module.

3 Current C++ Architecture

3.1 Layered View

The current module is organized around a small number of namespaces with clear runtime responsibilities.

Layer / namespace	Representative files	Current responsibility
model	system.hpp, components.hpp, options.hpp, results.hpp	Domain objects, enums, solver option bundles, result containers, validation helpers, and canonical-projection utilities.
io	json_io.hpp, excel_io.hpp, matpower_parser.hpp, case_builders.hpp	Case ingestion, serialization, canned benchmark systems, and workbook/JSON export of inputs and results.
powerflow	solver_data.hpp, admittance_builder.hpp, residual_evaluator.hpp, fdcpf_solver.hpp, dc_solver.hpp	Canonical steady-state assembly, AC/DC network matrices, branch-flow evaluation, converter coupling, and PF solver variants.
engine	newton_solver.hpp, linear_solver.hpp, dual_simplex.hpp, ipm_solver.hpp, branch_and_cut.hpp	Reusable numerical kernels: sparse linear solvers, Newton infrastructure, native NLP IPM, LP simplex, and MILP/MINLP branch-and-cut.
opf	ac_opf.hpp, parity_formulation.hpp, parity_ipm.hpp, dc_opf.hpp, reactive_power_opt.hpp	AC OPF orchestration, native reduced-space primal-dual path, parity formulation, parity-specific IPM, fallback selection logic, DC OPF (B- θ LP/QP), and reactive-power optimisation (MINLP with outer-approximation).
analysis	distribution_power_flow.hpp, short_circuit.hpp, topology_analysis.hpp, annual_production_sim.hpp, lifecycle_simulation.hpp, time_series_distribution.hpp	Secondary analytical modules that sit on top of the model layer and reuse parts of the power-flow or engine stack. Includes annual production simulation (hierarchical L0/L2/L3 decomposition), multi-year lifecycle simulation with degradation and NPV, and time-series distribution PF with topology reconfiguration.
api	include/hacdcpf/api/hacdcpf.hpp, src/api/hacdcpf.cpp	Stable top-level solve functions, solver-handle caching, public entry points, and derived-result postprocessing.
core / solver	compatibility re-export headers	Backward-compatible namespace aliases over the canonical powerflow and engine implementations.

3.2 Runtime Data Flow

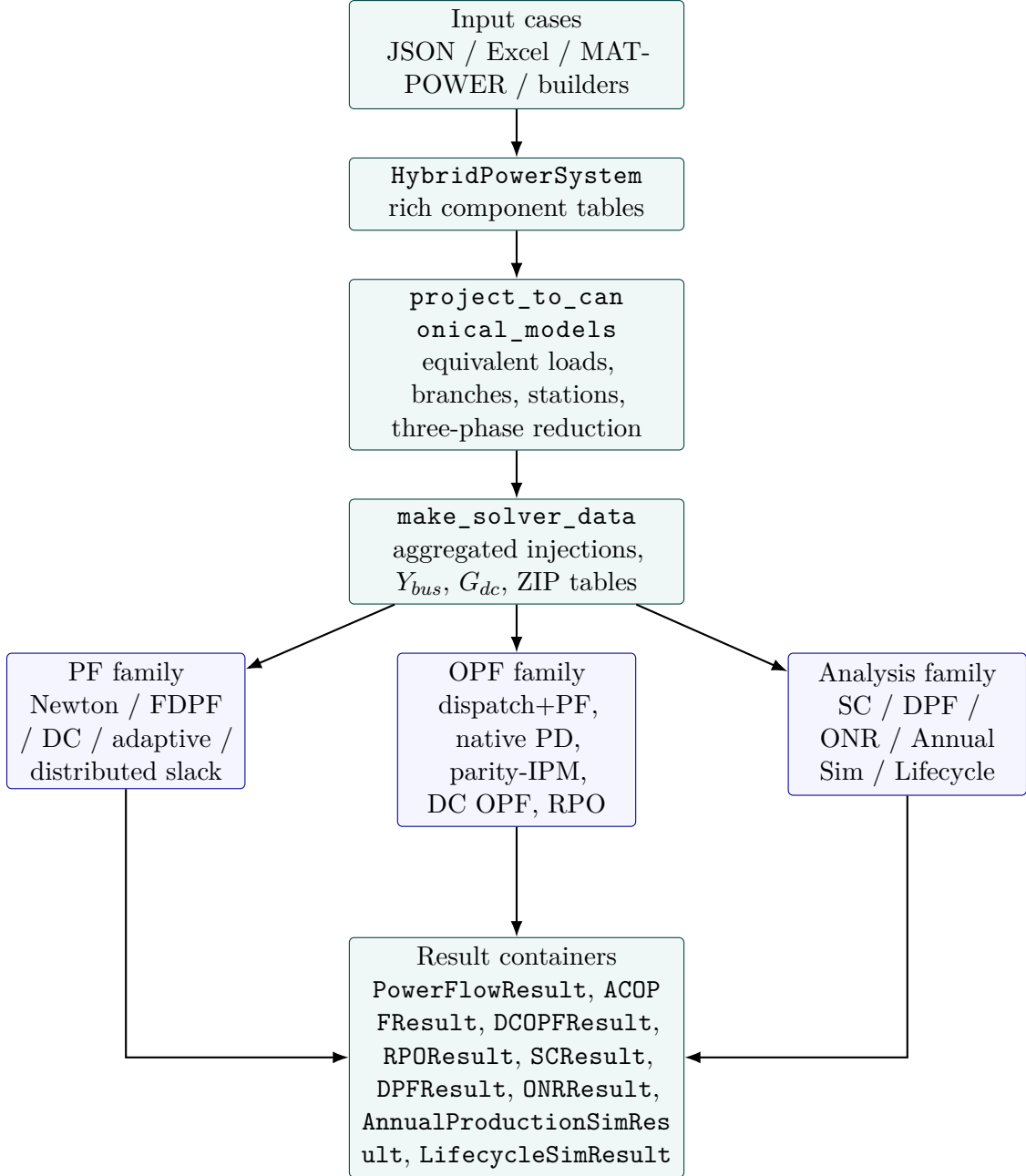


Figure 1: Primary runtime data path in the current C++ module.

3.3 Public API Surface

The public API layer exposes a deliberately small set of top-level entry points. The most important ones are:

- `solve_power_flow`, `solve_dc_power_flow`, `solve_power_flow_fdpf`,
- `solve_power_flow_adaptive`, `solve_power_flow_distributed_slack`,
- `solve_ac_dc_power_flow`,
- `opf::solve_ac_opf`,

- `opf::solve_dc_opf`, `opf::solve_reactive_power_opt`,
- `analysis::solve_annual_production_simulation`, `analysis::run_lifecycle_simulation`,
- solver-handle creation/reset/destruction functions for repeated PF runs on cached matrix structure.

3.4 Solver Families and Backends

Subsystem	Native numerical kernel	Optional backend or wrapper	Current use in module
AC/DC Newton PF	sparse linear solve through <code>engine::SparseLinearSolver</code>	Eigen SparseLU by default; UMF-PACK/KLU through SuiteSparse when available	Main PF path, adaptive PF, solver-handle reuse.
DC PF	dense Jacobian + Eigen <code>FullPivLU</code>	none	Small-to-medium DC subnetworks.
FDPF	decoupled sparse LU of B' and B''	Eigen SparseLU	Fast AC approximation when the full Newton path is unnecessary.
Native AC OPF	custom reduced-space primal-dual KKT assembly	same sparse linear-solver abstraction as Newton PF	Used when <code>enable_primal_dual=true</code> and <code>use_parity_ipm=false</code> .
Parity OPF	full parity formulation plus dedicated primal-dual IPM	dense or sparse KKT solves inside parity IPM implementation	Used when <code>enable_primal_dual=true</code> and <code>use_parity_ipm=true</code> .
LP / MILP engine	dual simplex and native branch-and-cut	HiGHS / Gurobi adapters available through <code>engine::SolverAdapter</code>	ONR uses the native MILP branch-and-cut path.
NLP / MINLP engine	native IPM and branch-and-cut	IPOPT / SCIP adapters available through <code>engine::SolverAdapter</code>	Infrastructure exists, but PF/OPF modules mostly use the native paths directly.
DC OPF	B- θ LP/QP via <code>NativeQP</code> , <code>LCQP</code> solver, or <code>DualSimplex</code>	HiGHS / Gurobi through adapter	Used by <code>opf::solve_dc_opf</code> ; extracts LMP from dual variables.
RPO (Reactive Power Optimisation)	4-phase outer-approximation MINLP over discrete taps and switchable shunts	none (self-contained)	Used by <code>opf::solve_reactive_power_opt</code> ; iterates until OA gap converges.
LCQP (Linear/Convex QP)	Mehrotra predictor-corrector IPM with symbolic factorisation caching	none	Back-end for DC OPF (NativeQP mode) and reusable for other QP sub-problems.

Subsystem	Native numerical kernel	Optional backend or wrapper	Current use in module
Annual Production Sim	Hierarchical L0/L2/L3 decomposition orchestrating LP (L0) and MILP (L2/L3 via UC)	Same MILP adapters as UC (Gurobi → HiGHS → Native)	Used by <code>analysis::solve_annual_production_simulation</code> .
Lifecycle Simulation	Multi-year loop calling annual production sim with degradation and stratified sampling	uses annual sim solvers internally	Used by <code>analysis::run_lifecycle_simulation</code> ; produces per-year and NPV metrics.

3.5 Current Architectural Strengths

- The `model` layer is richer than the steady-state kernels, which makes future expansion possible without redesigning the external data model.
- The `project_to_canonical_models` step gives PF, OPF, and SC a common data contract.
- The engine layer isolates linear algebra and mathematical-programming infrastructure from application modules.
- The compatibility namespaces `core` and `solver` reduce migration cost for older code.

3.6 Current Architectural Friction Points

- Canonical projection is now reused by DPF and ONR as well, but the repository still lacks explicit regression tests and durable consumer keys proving that projected and non-projected views remain equivalent.
- The parity OPF path already fills advanced in-memory dispatch vectors, but JSON / Excel result export still exposes only the older core subset.
- The detailed IEC short-circuit path is now executable, but the public result contract still carries legacy scaffold fields and does not yet expose an explicit external-grid contribution bucket.
- Public documentation must track both canonical namespaces and their compatibility aliases, which increases cognitive load for maintainers.

4 Engine Solver System

This section documents the solver infrastructure under `include/hacdcpf/engine` and `src/engine` as a coherent system. The goal is to separate three concerns that are easy to blur when reading the repository file-by-file:

- the *canonical mathematical problem forms* that every backend sees,
- the *adapter and registry layer* that dispatches a problem to a solver,
- the *actual kernels* that are fully implemented today versus declared extension points.

This chapter is intentionally code-aligned. When a header advertises a solver family but the current source file is only a placeholder, the notebook states that explicitly instead of describing the ideal future design.

4.1 Layering Inside `hacdcpf::engine`

The engine layer is organized in five contracts.

1. **Canonical problem layer:** `problem_types.hpp` defines the shared mathematical models such as `LPMoDel`, `QPMoDel`, and `MIPMoDel`.
2. **Validation layer:** `problem_validation.hpp/.cpp` checks structural consistency before the numerical solve begins.
3. **Adapter layer:** `solver_adapter.hpp` defines the uniform interface `solve_lp`, `solve_nlp`, `solve_milp`, and so on.
4. **Registry layer:** `adapter_registry.hpp/.cpp` stores candidate adapters and selects one by supported `ProblemClass` or by name.
5. **Kernel layer:** concrete native and external implementations live in files such as `dual_simplex.cpp`, `lcqp_solver.cpp`, `pdlp_solver.cpp`, and `external_adapters.cpp`.

Two different solve paths coexist inside this layer:

- **Canonical optimization path:** a caller builds a `ProblemClass` object and dispatches it through a `SolverAdapter`.
- **Direct application kernel path:** some modules call engine kernels directly without going through the adapter abstraction. The most important example is `NewtonSolver`, which consumes `powerflow::SolverData` rather than `NonlinearSystem`.

4.2 Canonical Problem Classes and Standard Forms

Table 5 is the central contract of the engine. Every solver family either consumes one of these objects directly or wraps a domain-specific model into one of these forms first.

Class	C++ struct	Standard mathematical form	Current engine role
LE	<code>SparseLinSys</code>	Solve $Ax = b$.	Small reusable sparse linear-system abstraction used by <code>NativeLinearAdapter</code> .
NLE	<code>NonlinearSystem</code>	Find $F(x) = 0$, with residual and Jacobian callbacks and optional projection. min / max $c^\top x$	Generic Newton-style nonlinear equation interface.
LP	<code>LPMoDel</code>	s.t. $Ax \leq b$, $A_{eq}x = b_{eq}$, $\ell \leq x \leq u$	Core relaxation form for simplex, IPM-LP, PDLP, HiGHS, and Gurobi.
QP	<code>QPMoDel</code>	min / max $\frac{1}{2}x^\top Qx + c^\top x$ s.t. $Ax \leq b$, $A_{eq}x = b_{eq}$, $\ell \leq x \leq u$	Convex LCQP interface for native QP IPM and Gurobi.

Class	C++ struct	Standard mathematical form	Current engine role
NLP	NLPModel	$\begin{aligned} &\min / \max \quad f(x) \\ &\text{s.t.} \quad g(x) = 0, \\ &\quad \quad h(x) \leq 0, \\ &\quad \quad \ell \leq x \leq u \end{aligned}$	Generic nonlinear optimization contract with callback and optional symbolic forms.
MILP	MIPModel	LP plus $x_j \in \mathbb{Z}$ or $x_j \in \{0, 1\}$.	Native branch-and-cut input and external MILP adapter input.
MINLP	MINLPModel	NLP plus integer restrictions.	Native spatial B&B shell and SCIP-facing model form.

Table 5: Canonical problem classes in the engine layer.

The canonical model metadata are deliberately richer than just matrices and vectors.

- `VariableMeta` attaches type, lower bound, upper bound, and name to each decision variable.
- `SymExpr` and `SymbolicConstraint` allow an NLP to expose a symbolic representation when an external solver prefers exported algebra over callback-only evaluation.
- `MIPModel` carries integer index sets, an optional warm start, unit-commitment structure hints, and branching priorities, because the branch-and-cut engine uses all of them operationally.

4.3 Validation and Dispatch Workflow

The adapter path is intentionally thin. Validation and numerical work are kept separate so that front-end modules can fail early on malformed dimensions or invalid bounds before any solver-specific data structure is allocated.

Algorithm E1: Canonical Adapter Dispatch

```

1: procedure DISPATCH_PROBLEM(cls, model, registry, preferred)
2:   report  $\leftarrow$  validate(model)
3:   if report.valid = false then
4:     return unsuccessful result with validation errors
5:   end if
6:   if preferred  $\neq \emptyset$  then
7:     adapter  $\leftarrow$  registry.find_by_name(preferred)
8:   else
9:     adapter  $\leftarrow$  registry.first_for(cls)
10:  end if
11:  if adapter =  $\emptyset$  then
12:    return unsuccessful result with “no adapter available”
13:  end if
14:  Dispatch to the type-specific entry point
    solve_le, solve_nle, solve_lp, solve_qp, solve_nlp, solve_milp, or solve_minlp
15:  return SolveResult
16: end procedure

```

At the code level, the division of responsibility is straightforward.

- `SolverAdapter` provides default “unsupported problem class” results.
- Each concrete adapter overrides only the methods it really supports.
- `AdapterRegistry` is just an ordered list. Selection is therefore deterministic: the first supporting adapter wins unless the caller explicitly asks for one by name.

This simple registry design is important for application code such as market simulation and time-series PF, which register Gurobi first, HiGHS second, and a tuned native fallback last.

4.4 Solver Inventory and Current Implementation Status

Table 6 separates the solver *interfaces* from the solver *implementations*. This is the part most likely to reduce maintainer confusion, because the engine now contains both fully operational solvers and placeholder interfaces meant for future completion.

Solver family	Main class or entry point	Problem class	Current status in source
PF Newton kernel	<code>NewtonSolver</code>	direct <code>SolverData</code> path	Fully implemented power-flow-specific Newton kernel with Jacobian-pattern caching, sparse factorization reuse, line search, and bus-type switching.
Generic linear solve	<code>NativeLinearAdapter</code>	LE	Fully implemented thin wrapper over the sparse linear solver abstraction.
Generic NLE solve	<code>NativeNewtonAdapter</code>	NLE	Fully implemented regularized Newton method with backtracking.
Generic NLP solve	<code>NativeNLPAdapter</code>	NLP	Fully implemented penalty-based line-search method using callback gradients and Hessians.
Native NLP IPM interface	<code>NativeIPMAdapter</code>	NLP	Fully implemented sparse primal-dual predictor-corrector IPM with reusable KKT factorizations, shared reduced-KKT Schur solves for small equality blocks, sparse secant or finite-difference Hessian recovery when <code>hess</code> is absent, startup curvature caching across repeated solves, and IPOPT fallback for unrecovered failures.
Convex QP / LCQP	<code>NativeLCQPAdapter</code>	QP, LP	Fully implemented Mehrotra-style primal-dual IPM for convex QP and LP special cases.

Solver family	Main class or entry point	Problem class	Current status in source
High-performance LP IPM	<code>NativeIPMLPAdapter</code>	LP	Fully implemented predictor-corrector IPM with scaling, correctors, cached node solves, and optional crossover support.
Dual simplex	<code>solve_lp_with_basis</code>	LP	Fully implemented bounded-form simplex with standard-form conversion, warm starts, sparse basis reuse, and dual-simplex reoptimization.
PDLP / PDHG	<code>NativePDLPAdapter</code>	LP	Fully implemented first-order LP solver for large sparse problems.
MILP / MINLP tree search	<code>solve_milp_bc, solve_minlp_bc, NativeBranchAndCutAdapter</code>	MILP, MINLP	MILP branch-and-cut is fully implemented and mature. The MINLP path now uses the native NLP relaxation, carries multiplier-informed branch scores into child nodes, and remains backed by SCIP/IPOPT-style external fallbacks where available.
HiGHS wrapper	<code>HighsAdapter</code>	LP, MILP	Fully implemented external file-based wrapper around the HiGHS executable.
IPOPT wrapper	<code>IpoptAdapter</code>	NLP	Available when IPOPT is built in. Used for external NLP backend selection.
SCIP wrapper	<code>ScipAdapter</code>	MINLP	Available when SCIP support is built in.
Gurobi wrapper	<code>GurobiAdapter</code>	LP, QP, MILP	Native C-API wrapper for high-end LP/QP/MILP solves.

Table 6: Engine solver inventory and actual implementation status.

A practical consequence for readers of the code When the repository mentions a “native IPM” for NLP, that phrase must be read carefully. The engine contains:

- a working native IPM for **LP** in `ipm_lp_solver.cpp`,
- a working native IPM for **convex QP** in `lcqp_solver.cpp`,
- a working native IPM for **general NLP** in `ipm_solver.cpp`, with reduced-KKT and missing-Hessian recovery paths,
- and a separate working `NativeNLPAdapter` that is not a barrier IPM, but a penalty-and-line-search Newton-type method.

Current performance status The most relevant benchmark result is now the structured sparse chain NLP in `benchmark/nlp_ipm_benchmark.cpp`. On the current implementation, the exact-Hessian native path beats IPOPT on all benchmarked sizes

$$\text{Native/IPOPT time ratio} = 0.419 \ (n = 64), \quad 0.518 \ (n = 128), \quad 0.773 \ (n = 256),$$

while the missing-Hessian path is now also faster than IPOPT across the same benchmark family. After the sparse secant plus finite-difference Hessian recovery pass, shared reduced-KKT solves, and startup curvature caching, the native missing-Hessian solve achieves

$$\text{Native/IPOPT time ratio} = 0.340 \ (n = 64), \quad 0.444 \ (n = 128), \quad 0.677 \ (n = 256).$$

So the main nonlinear performance story has changed materially: the engine is no longer merely “equipped with a native NLP IPM,” but already ahead of the external IPOPT baseline on this structured sparse benchmark in both the exact-Hessian and missing-Hessian regimes. The remaining engineering work is therefore about broadening that win across more difficult NLP families, not about filling a missing core implementation.

4.5 Key Algorithm Workflows

4.5.1 Power-Flow-Specific Newton Kernel

`NewtonSolver` is the main reusable PF engine kernel rather than a generic optimization adapter. It operates on `powerflow::SolverData`, builds a Jacobian context from AC/DC bus states and converter modes, and then reuses sparse-pattern analysis aggressively across repeated solves.

Algorithm E2: `NewtonSolver.solve`

```

1: procedure NEWTONSOLVER.SOLVE(data, opt, init)
2:   Initialize voltage magnitudes, angles, DC voltages, and control states
3:   Detect AC slack bus and one DC slack bus per DC island
4:   Build Jacobian context from current bus types and converter modes
5:   if cached Jacobian pattern matches data structure and context layout then
6:     Reuse analyzed sparse pattern and linear solver object
7:   else
8:     Rebuild Jacobian sparsity pattern and analyze it once
9:   end if
10:  for outer PV/PQ or converter-mode adjustment loop do
11:    for iter = 1 to max_iter do
12:      Evaluate mismatch vector and Jacobian
13:      if residual norm is below tolerance then
14:        Assemble final PF result and return
15:      end if
16:      Factorize Jacobian, solve for Newton step, and clip step components
17:      Run globalization / backtracking line search
18:      Apply PV/PQ switching and converter control-mode updates when enabled
19:    end for
20:  end for
21:  Return non-converged result with profiling counters
22: end procedure

```

4.5.2 Generic Nonlinear Equation Solver

Algorithm E3: NativeNewtonAdapter.solve_nle

```

1: procedure SOLVE_NLE( $F(x), J(x), x_0$ )
2:   Validate dimensions and callbacks
3:    $x \leftarrow x_0$ 
4:   for  $iter = 1$  to  $max\_iter$  do
5:     Evaluate residual  $f \leftarrow F(x)$ 
6:     if  $\|f\|_\infty \leq \epsilon$  then
7:       Return converged result
8:     end if
9:     Form Jacobian  $J(x)$ 
10:    for a small regularization ladder  $\lambda$  do
11:      Solve  $(J + \lambda I)\Delta x = -f$ 
12:      Try backtracking steps  $x + \alpha\Delta x$ 
13:      if a trial decreases residual norm then
14:        Accept step and continue outer loop
15:      end if
16:    end for
17:    if no acceptable step exists then
18:      Return failure: line search failed
19:    end if
20:  end for
21:  Return failure: maximum iterations reached
22: end procedure

```

4.5.3 Generic NLP Solver Used Today

Algorithm E4: NativeNLPAdapter.solve_nlp

```

1: procedure SOLVE_NLP( $f, \nabla f, \nabla^2 f, g, h, x_0$ )
2:   Validate callbacks and project  $x_0$  into variable bounds
3:   for  $iter = 1$  to  $max\_iter$  do
4:     Evaluate objective, gradient, and optional Hessian
5:     Evaluate equality and inequality violations
6:     Build penalized merit model
7:      $\phi(x) = f(x) + \frac{\rho}{2}\|g(x)\|_2^2 + \frac{\rho}{2}\|\max(h(x), 0)\|_2^2$ 
8:     Form penalized gradient and Hessian approximation
9:     if stationarity and feasibility are small enough then
10:      Return converged result
11:    end if
12:    Regularize Hessian and solve for a step  $\Delta x$ 
13:    Run backtracking on the penalized merit function
14:    Project trial point into variable bounds
15:    if no acceptable trial point exists then
16:      Return failure
17:    end if
18:  end for
19:  Return best iterate with status Maxiterationsreached

```

4.5.4 Convex QP and LCQP Solver

Algorithm E5: NativeLCQPAdapter.solve_qp

```
1: procedure SOLVE_LCQP( $Q, c, A_{eq}, b_{eq}, \ell, u$ )
2:   Apply scaling to variables, equalities, costs, and bounds
3:   Initialize primal variables at the center of finite bounds
4:   Initialize lower/upper slacks and dual multipliers to positive values
5:   Build the KKT sparsity pattern once and analyze it symbolically
6:   for  $iter = 1$  to effective_max_iter do
7:     Compute stationarity, equality, slack, and complementarity residuals
8:     Compute  $\mu$  from slack-dual products
9:     if primal feasibility, dual feasibility, and gap are below tolerance then
10:      Unscale solution and return
11:     end if
12:     Update only the diagonal-dependent part of the KKT matrix
13:     Factorize KKT numerically
14:     Choose centering parameter  $\sigma$ 
15:     Solve KKT system for primal, dual, slack, and multiplier directions
16:     Compute fraction-to-boundary step sizes
17:     Update primal and dual variables
18:   end for
19:   Return last iterate with non-converged status
20: end procedure
```

4.5.5 High-Performance LP Interior-Point Method

Algorithm E6: NativeIPMLPAdapter.solve_lp

```
1: procedure SOLVE_IPM_LP( $LP$ )
2:   Convert the LP to the engine's merged internal form with box bounds, equality rows,
   and slack variables
3:   Optionally presolve, scale, and cache repeated node-solve data structures
4:   Initialize primal variables, slacks, and duals strictly inside the feasible box
5:   for  $iter = 1$  to max_iter do
6:     Compute primal residual, dual residual, and complementarity gap
7:     if all three satisfy tolerance then
8:       Export primal solution, duals, and bound multipliers; return
9:     end if
10:    Form diagonal barrier scaling terms and factorize normal equations
11:    Compute affine predictor direction
12:    Compute affine step lengths and predicted  $\mu_{aff}$ 
13:    Choose centering parameter  $\sigma$  from the affine-gap ratio
14:    if the affine step is already nearly central then
15:      Skip expensive corrector and reuse affine direction
16:    else
17:      Form Mehrotra corrector right-hand side and solve corrector system
18:    end if
19:    Apply fraction-to-boundary step sizes and update primal/dual variables
20:  end for
21:  Return failure or best available iterate
22: end procedure
```

This solver is the native LP workhorse for repeated bound-change problems. The important engineering feature is not only the Mehrotra iteration itself, but also the `prepare_for_node_s`

`olves()` and cached-node-solve path used by branch-and-cut workers.

4.5.6 Dual Simplex and Standard-Form Reoptimization

Algorithm E7: `solve_lp_with_basis`

```

1: procedure SOLVE_DUAL_SIMPLEX(LP, basis_hint)
2:   Convert the original LP to bounded standard form
   add slack, surplus, and artificial columns as needed
3:   if a valid parent basis hint exists then
4:     Rebuild basis state, reduced costs, and basic solution from the hint
5:     if dual feasible but primal infeasible then
6:       Reoptimize by dual simplex
7:     else if warm start is numerically rejected and cold start is allowed then
8:       Fall back to cold-start Phase I / Phase II
9:     end if
10:  else
11:    Build crash basis and run cold-start Phase I / Phase II simplex
12:  end if
13:  Persist basis information for children
14:  Return primal solution, basis, reduced costs, and dual information
15: end procedure

```

The bounded-form conversion is central to the engine architecture. Child nodes usually change only bounds, so the matrix structure remains fixed while the basis and bound-shift terms are reused.

4.5.7 First-Order PDLP Solver

Algorithm E8: `NativePDLPAdapter.solve_lp`

```

1: procedure SOLVE_PDLP(LP)
2:   Convert the LP into a matrix-and-box form suitable for PDHG iterations
3:   Build both CSC and CSR sparse views of the matrix
4:   Apply Ruiz row/column equilibration
5:   Build Pock–Chambolle diagonal step sizes and optional IP-PMM regularization
6:   Initialize primal and dual iterates and their running averages
7:   for iter = 1 to max_iter do
8:     Optionally extrapolate the primal iterate
9:     Update dual variables by a row-wise sparse pass and box projection
10:    Update primal variables by a column-wise sparse pass and bound projection
11:    Update ergodic averages
12:    if it is a check iteration then
13:      Evaluate primal violation, dual stationarity, and relative gap
14:      Update the best current or averaged iterate
15:      Trigger adaptive restart if progress stagnates or periodic restart fires
16:      Adapt step gain, primal weight, and PMM regularization
17:      if current or averaged iterate satisfies tolerances then
18:        Return best iterate
19:      end if
20:    end if
21:  end for
22:  Unscale the best iterate and return
23: end procedure

```

4.5.8 Mixed-Integer Branch-and-Cut

Algorithm E9: `solve_milp_bc`

```
1: procedure SOLVE_MILP_BC(MIP, opt)
2:   Validate the model and run presolve / bound tightening
3:   Solve the root LP relaxation
4:   Run root cut rounds, primal heuristics, and incumbent discovery
5:   if the root solution is integral or the gap is closed then
6:     Return immediately
7:   end if
8:   if parallel mode is enabled then
9:     Launch heterogeneous worker threads and shared queues
10:  else
11:    Start sequential best-first / depth-first / hybrid tree loop
12:  end if
13:  while time, node, and gap limits are not hit and the node pool is not empty do
14:    Select a node and propagate local bounds
15:    Solve the node LP relaxation, preferably by simplex warm start
16:    if the node is infeasible or dominated by incumbent then
17:      Prune node
18:    else if the node solution is integral then
19:      Update incumbent and global bound
20:    else
21:      Choose branching variable using pseudocost / reliability / fallback rule
22:      Create up and down children
23:      Optionally add tree cuts or conflict information
24:      Push surviving children back to the search structure
25:    end if
26:  end while
27:  Return incumbent, best bound, gap, and detailed B&C statistics
28: end procedure
```

The full MILP machinery is larger than this summary. The detailed cut families, warm-start mechanics, presolve propagation, and parallel explorer design are documented further in Section 22.

4.5.9 External Backend Wrapper Pattern

Algorithm E10: External Adapter Shell

```
1: procedure SOLVE_EXTERNAL(model, executable)
2:   Validate the canonical model
3:   Export the model to an interchange format or backend API representation
4:   Launch the external solver or native C API call
5:   Parse objective, status, primal solution, and available dual data
6:   Map backend status to SolveStats
7:   Return SolveResult
8: end procedure
```

In the current code base, HiGHS mainly follows the file-export pattern, while Gurobi is linked through its C API. IPOPT and SCIP are enabled only when their build flags are available.

4.6 Assessment Against the Self-Contained Computation-Engine Target

Relative to the architecture goal in `ARCHITECTURE.md`, the current engine is best described as a *strong LP/MILP-centered computation engine with reusable nonlinear interfaces*, not yet as a fully self-contained solver stack across linear equation, nonlinear equation, MILP, and MINLP classes.

4.6.1 Robustness Assessment

Current strengths The framework is structurally sound in four important ways.

- **Canonical contracts are clean.** `problem_types.hpp`, `problem_validation.cpp`, and the adapter interface separate model definition, validation, dispatch, and numerical work in a maintainable way.
- **LP/MILP solve paths defend themselves well.** The MILP path includes validation, presolve, warm starts, fallback logic, detailed statistics, and multiple root/node strategies instead of a single brittle algorithm.
- **PF and linear algebra kernels are reusable.** `NewtonSolver`, sparse linear solves, and the LP/QP kernels have enough internal structure to serve as an actual computation-engine layer rather than one-off application code.
- **Compatibility wrappers reduce migration risk.** The canonical home is now `hacdcpf::engine`, while older `core/solver` include paths still exist as shims for the rest of the code base.

Current risks and gaps Three issues materially limit the claim of a self-contained engine.

- **Native MINLP is not currently complete.** The MINLP B&B driver calls `solve_nlp_relaxation()`, and that routine currently instantiates `NativeIPMAdapter`; however, `ipm_solver.cpp` still returns “not implemented” for the native NLP-IPM solve. In other words, the tree shell exists, but the native continuous relaxation underneath is incomplete.
- **The generic NLP path is usable but algorithmically modest.** `NativeNLPAdapter` is a penalty-plus-line-search Newton method, not a full primal-dual barrier solver with robust filter/merit updates, restoration, or strong multiplier handling. That is adequate for small smooth models, but it is a weaker foundation for large constrained NLP and for MINLP relaxations.
- **Regression protection is uneven on the hardest path.** The type-system and adapter tests are healthy, but the branch-and-cut test target is not currently a clean safety net for the full engine because it can abort before reaching later scenarios. That weakens confidence in day-to-day robustness of the mixed-integer core.

4.6.2 Performance Assessment

Where the engine is strong today The performance architecture is already credible in the LP/MILP center.

- **Root LP performance is strong.** The native LP-IPM path is heavily engineered with scaling, cached symbolic work, and node-solve preparation; internal benchmarks have shown it can outperform the native simplex root solve by a wide margin on large UC relaxations.
- **MILP tree engineering is real, not superficial.** The branch-and-cut stack contains presolve, cut pools, pseudocost and reliability branching, warm-started simplex reoptimization, heuristics, and a heterogeneous parallel mode.

- **Sparse factorization reuse is treated as a first-class concern.** This is visible across `newton_solver.cpp`, `dual_simplex.cpp`, `ipm_lp_solver.cpp`, and the sparse LU support files.

Main bottlenecks versus the target architecture The remaining bottlenecks are also clear.

- **Large-case MILP competitiveness is still uneven.** Internal benchmarking shows the native B&C path can be competitive on some UC families, but on harder large cases the external commercial or dedicated MILP solvers still retain a sizable advantage, mainly from stronger tree closure and LP reoptimization throughput.
- **External-wrapper overhead remains in practical workflows.** HiGHS and SCIP mainly use file export plus subprocess execution, which is fine for portability but weaker than an in-memory backend when low latency or high-frequency repeated solves are required.
- **Nonlinear performance headroom is large.** Because the native NLP-IPM path is missing, the engine does not yet have a high-performance native continuous relaxation for general MINLP. This is the main architectural hole between today's implementation and the intended computation-engine role.

4.6.3 Readiness Summary

For the four solver families named in the architecture document, the present readiness is:

Family	Current readiness	Comment
Linear equation	Strong	Ready as a reusable native engine kernel
Nonlinear equation	Strong	Newton-style solve path is available and robust for structured models
MILP	Strong-Moderate	Native path is mature, but still not uniformly best-in-class on large hard instances
MINLP	Partial	Interface and tree shell exist, but native continuous relaxation is not yet complete

Therefore, the most accurate near-term roadmap is not to redesign the engine abstraction, but to complete the missing native NLP/MINLP core under the existing abstraction: a real NLP-IPM or SQP-quality relaxation solver, then MINLP-specific reliability work on branching, node recovery, and regression testing.

4.7 Recommended Reading Order for Maintainers

For someone trying to understand or extend the engine, the lowest-friction order is:

1. `problem_types.hpp` and `problem_validation.cpp`,
2. `solver_adapter.hpp` and `adapter_registry.cpp`,
3. `native_adapters.cpp` to see which kernels are actually selected,
4. one kernel family in depth: `dual_simplex.cpp`, `ipm_lp_solver.cpp`, `lcqp_solver.cpp`, or `pdlp_solver.cpp`,
5. `branch_and_cut.cpp` and its helper files for mixed-integer logic,

6. `external_adapters.cpp` only after the canonical forms and native solve path are already clear.

That order mirrors the architecture itself: understand the problem contracts first, then the dispatch layer, then the heavy numerical kernels.

5 Native NLP Interior-Point Method: Theory, Algorithm, and Current Diagnostics

This section documents the nonlinear interior-point implementation under `include/hacdcp/engine/ipm_solver.hpp` and `src/engine/ipm_solver.cpp` with a narrower goal than Section 4: to explain what the native NLP IPM is mathematically solving, how the current code actually drives the iterations, what the present test evidence says, and which observed behaviors should be read as signs of a real algorithmic problem rather than mere tuning noise.

The discussion is intentionally diagnostic. The current code already contains multiple globalization paths, scaling hooks, restoration support, exact and inexact curvature modes, and IPOPT fallback. A reader who only sees the class name `NativeIPMAdapter` can therefore easily overestimate its maturity. The notebook below separates what is theoretically intended, what is already implemented, and what the latest large-case evidence still says is fragile.

5.1 Standard NLP Form and Barrier KKT System

The canonical engine nonlinear program is the `NLPModel` from `problem_types.hpp`:

$$\min_x f(x) \tag{3}$$

$$\text{s.t. } g(x) = 0, \tag{4}$$

$$h(x) \leq 0, \tag{5}$$

$$\ell \leq x \leq u. \tag{6}$$

The engine converts all nonlinear inequalities and finite variable bounds into one unified inequality block. Introducing slacks $s > 0$ such that

$$h(x) + s = 0, \tag{7}$$

and multipliers λ for equalities and $\mu > 0$ for inequalities, the barrier KKT conditions are

$$\nabla f(x) + J_g(x)^\top \lambda + J_h(x)^\top \mu = 0, \tag{8}$$

$$g(x) = 0, \tag{9}$$

$$h(x) + s = 0, \tag{10}$$

$$S\mu = \tau e, \tag{11}$$

where $S = \text{diag}(s)$.

The reduced Newton system used by both the parity-specific IPM and the generic engine IPM is based on the primal Schur complement

$$W = \nabla_{xx}^2 \mathcal{L}(x, \lambda, \mu) + J_h(x)^\top \text{diag}(\mu./s) J_h(x), \tag{12}$$

with the Lagrangian

$$\mathcal{L}(x, \lambda, \mu) = f(x) + \lambda^\top g(x) + \mu^\top h(x). \tag{13}$$

This identity is critical when reading the current code quality: if the solver only uses $\nabla^2 f$ and drops the constraint-curvature term inside $\nabla_{xx}^2 \mathcal{L}$, then the Newton direction is formally incorrect for nonlinear constrained problems. The recent engine changes fixed that part by adding an explicit `NLPModel::lagrangian_hess(...)` callback and by using it inside the native solver when available.

5.2 Current Driver Architecture in `ipm_solver.cpp`

The file now contains two distinct native drivers behind the same adapter.

1. Filter driver (current default) The public `IPMOptions` now expose `Globalization::Filter` as the default mode. In this path, the solver first optionally scales the problem, then runs a filter line-search method in the style of Wächter–Biegler, with optional restoration and second-order correction logic. If the filter path fails, it can restore feasibility and retry; if that still fails, it falls back to the Merit-backed native path and ultimately to IPOPT.

The important consequence is practical rather than aesthetic: “the native NLP IPM” is not one single algorithm anymore. It is a driver stack with this order:

1. scaled filter IPM,
2. restoration-driven filter retry,
3. merit-based native IPM,
4. IPOPT fallback.

2. Merit driver (fallback native path) The lower part of `ipm_solver.cpp` still contains the earlier Mehrotra-style merit-based driver. It has recently been upgraded with:

- exact Lagrangian Hessian support,
- residual-aware slack and multiplier initialization,
- normalized residual metrics,
- separate primal and dual step sizes, and
- best-iterate restoration.

These changes materially improved the behavior on large OPF-derived NLPs, but they did not eliminate the main robustness issue: the generic merit line search still breaks down on large parity OPF cases before the dedicated parity solver does.

5.3 Curvature Sources and Hessian Policy

The current engine supports three curvature regimes.

Exact Lagrangian curvature If `lagrangian_hess` is provided, the solver uses the exact $\nabla_{xx}^2 \mathcal{L}$ callback directly. This is now the preferred path for parity OPF and for any NLP with nonlinear constraint curvature.

Objective-only Hessian If only `hess` is present, the solver uses the objective Hessian $\nabla^2 f$. This is exact only when all constraints are linear; otherwise it is a model misspecification, not just a performance compromise.

Recovered curvature If neither callback is present, the solver constructs sparse secant and finite-difference approximations. This path is still operational and is well covered by the structured sparse-chain tests, but it is not the preferred path for physics-heavy OPF NLPs.

5.4 Algorithmic Workflow of the Merit Driver

The current merit path in `ipm_solver.cpp` can be summarized as follows.

Algorithm E5: Merit-backed Native NLP IPM (current implementation)

```

1: procedure NATIVEIPM.SOLVE_NLP_DETAIL(prob)
2:   Validate callbacks, dimensions, and bounds
3:   Interiorize the initial point with respect to box bounds
4:   Evaluate  $f, \nabla f, g, J_g, h, J_h$ 
5:   Initialize slacks and multipliers from inequality residuals
6:   Build exact or recovered Lagrangian Hessian
7:   for  $k = 0, 1, \dots$  until convergence or iteration limit do
8:     Form residuals  $(r_{dual}, r_{eq}, r_{ineq})$ 
9:     Compute normalized primal, dual, and complementarity metrics
10:    Build reduced primal system
11:    Solve affine predictor step
12:    Compute centering target and corrector step
13:    Compute separate primal and dual fraction-to-boundary steps
14:    Try merit-based backtracking on  $(x, s, \lambda, \mu)$ 
15:    if acceptable trial found then
16:      Accept step and continue
17:    else
18:      Recenter barrier state or terminate with line-search failure
19:    end if
20:  end for
21:  if best historical iterate satisfies tolerances then
22:    Restore best iterate and report convergence
23:  else
24:    Return failure or fall back to IPOPT
25:  end if
26: end procedure

```

The important diagnostic point is Step 10. The generic solver still judges trial steps with a scalar merit decrease condition. For large OPF NLPs this is currently weaker than the parity-specific direct primal/dual step acceptance, and it is the main remaining failure mode after the recent scaling and initialization fixes.

5.5 Algorithmic Workflow of the Filter Driver

The filter path is theoretically stronger and architecturally more ambitious. It adds four ingredients that the earlier merit-only solver did not have:

- problem scaling,
- filter-based globalization instead of one scalar merit test,
- optional second-order correction,
- one-shot restoration NLP construction and retry.

In theory this is the correct direction for difficult nonconvex NLPs. In practice the current source still has to be read as an implementation under active hardening rather than a finished IPOPT-equivalent engine. The notebook should therefore be interpreted conservatively: the filter driver is the intended production globalization path, while the merit driver remains the most transparent reference path for code reading and regression triage.

5.6 What the Unit Tests Currently Prove

The current dedicated native-IPM test coverage is concentrated in `tests/test_nlp_ipm_solve_r.cpp` and `tests/test_minlp_native_ipm.cpp`. The test evidence is summarized in Table 7.

Table 7: Native NLP IPM tests and what they actually verify

Test file / case	Status	What it proves
<code>test_nlp_ipm_solver</code> : small constrained quadratic	Passing	Basic correctness of the primal-dual solve path, multiplier output, and convergence on a smooth small NLP.
<code>test_nlp_ipm_solver</code> : sparse chain NLP	Passing	Exact-Hessian and missing-Hessian paths both work on a structured sparse benchmark family.
<code>test_nlp_ipm_solver</code> : exact Lagrangian Hessian callback	Passing	The solver now actually consumes <code>lagrangian_hess</code> rather than silently dropping nonlinear constraint curvature.
<code>test_minlp_native_ipm</code>	Passing	MINLP relaxations can call the native NLP IPM and return valid node relaxations for small cases.
Parity-specific Hessian finite-difference tests	Passing	The parity formulation’s analytic Lagrangian Hessian matches finite-difference checks.

These tests are important but not sufficient. They prove local algorithmic correctness on small and structured problems; they do *not* prove robust globalization on large OPF NLPs.

5.7 Structured Sparse Benchmark Results

The synthetic benchmark family in `benchmark/nlp_ipm_benchmark.cpp` still provides the strongest evidence that the engine’s linear algebra and Hessian handling are no longer the main bottleneck. On the structured sparse chain problems, the native exact Hessian path and the missing-Hessian recovery path both solve successfully, and the native solver remains competitive with or faster than IPOPT on this benchmark family.

That result is still technically relevant: it means the core KKT machinery, sparse Hessian handling, and callback contracts are not fundamentally broken. If large OPF cases fail, the most likely culprit is now globalization and problem scaling, not the absence of a Newton kernel.

5.8 Large Parity-OPF Comparison Results

The most informative recent evidence comes from the parity-OPF benchmark driver in `tests/benchmark_ac_opf_cases.cpp`, which now exposes three solve modes on the same parity formulation:

- `parity`: dedicated parity primal-dual IPM in `src/optimal_power_flow/parity_ipm.cpp`,
- `native`: generic `NativeIPMAdapter` applied to the same parity NLP callbacks,
- `ipopt`: IPOPT on the same parity NLP callbacks.

Table 8 summarizes the latest results after the recent merit-driver upgrades.

Table 8: Large-case parity OPF comparison on the same NLP formulation

Case	Mode	Con- verged	Mean time (ms)	Iterations	Status / interpreta- tion
IEEE118	native	No	2307.323	90	NativeIPMline searchfailed ; much better than the earlier 500-iteration stall, but still not robust.
IEEE118	ipopt	No	1475.001	500	Maxiterationse xceeded ; remains near the correct objective and near-feasible.
IEEE118	parity	Yes	15880.952	23	Dedicated parity solver converges and passes PF cross-validation.
PEGASE2869	native	No	76498.973	33	NativeIPMlines earchfailed ; materially improved over the earlier 500-iteration collapse, but still unusable as a final OPF solver.
PEGASE2869	ipopt	No	65131.865	500	Maxiteration sexceeded ; stays close to the parity objective.
PEGASE2869	parity	Yes	123052.538	61	Dedicated parity solver converges and passes PF cross-validation.

Two conclusions follow immediately.

1. The recent engine changes are real improvements: the generic native NLP IPM no longer simply burns the full iteration budget with catastrophic residuals on these OPF cases.
2. The generic native NLP IPM is still not production-robust on large OPF NLPs. The dominant failure is now a globalization failure, not missing curvature.

5.9 Current Diagnostic Interpretation: Is the Code Still Wrong?

For code review purposes, the current answer is nuanced.

What is already correct enough to trust

- The engine now has a mathematically correct interface for exact Lagrangian Hessians.
- Small and structured NLP tests pass on both exact and recovered curvature paths.
- The parity formulation’s Hessian and Jacobian contracts have dedicated finite-difference validation.

- The generic solver’s latest upgrades improved large OPF behavior by a large margin compared with the earlier 500-iteration stalls.

What still indicates a real algorithmic problem The remaining large-case failures are not benign. The following behaviors should be read as evidence that the generic NLP IPM is still incomplete for hard OPF-class problems:

- repeated `linesearchfailed` termination on large parity NLPs,
- native objective values far away from the parity and IPOPT trajectories,
- generic native failure on cases where the dedicated parity solver converges on the same formulation,
- dependence on fallback or parity-specific kernels for final robustness.

In other words, the code is no longer “wrong” in the narrow sense of missing constraint curvature or having no native nonlinear interior-point core. However, it is still incomplete in the stronger sense required for large-scale OPF robustness. The current bottleneck is algorithmic globalization rather than Hessian formation.

5.10 Most Likely Remaining Gap

The recent parity-derived changes fixed the easiest-to-identify deficiencies: better barrier-state initialization, normalized residuals, exact Lagrangian curvature, and separate primal/dual step sizes. The strongest remaining gap is that the generic merit path still uses a scalar merit backtracking test, while the dedicated parity solver accepts fraction-to-boundary primal and dual steps more directly and augments them with OPF-specific corrector logic.

So the most likely next successful improvement path is:

1. harden the filter-based driver until it reliably dominates the merit fallback on large OPF cases,
2. or, alternatively, import the parity solver’s direct primal/dual step acceptance and extra-corrector behavior more faithfully into the generic native path.

For a reviewer trying to judge the present source tree, the practical summary is simple: the native NLP IPM is now a real solver and not a placeholder, but its robustness envelope is still substantially narrower than the dedicated parity OPF IPM on large network NLPs.

5.11 Reviewer-Style Risk Checklist

For code review and regression triage, the current source can be reduced to the following seven risk points. These are the items most likely to explain why the native NLP IPM still underperforms on large OPF-class problems.

1. **Globalization mismatch remains the primary risk.** The generic native path still relies on merit or filter globalization logic that is materially more fragile than the direct fraction-to-boundary step acceptance used by the dedicated parity IPM. Current large-case failures are still dominated by `linesearchfailed` or earlier filter-stage rejection, not by Hessian unavailability.
2. **Filter driver maturity is still below production level for OPF.** The default path in `ipm_solver.cpp` is now the filter-based driver, but the current large-case evidence does not yet show that it dominates the merit fallback or IPOPT. A code reviewer should therefore treat the filter driver as an active development path rather than as a solved final design.

3. **The merit fallback is improved but still not robust enough.** The recent parity-derived upgrades fixed barrier initialization, residual normalization, exact Lagrangian curvature usage, separate primal and dual step lengths, and best-iterate restoration. Even so, the merit path still fails on IEEE118 and PEGASE2869 when applied to the parity NLP callbacks.
4. **Performance is still problem-class dependent rather than broadly solved.** The sparse structured chain benchmarks show that the linear algebra kernel and Hessian plumbing are not fundamentally broken. However, those wins do not yet transfer to large nonconvex OPF NLPs. A reviewer should therefore reject any blanket claim that the native NLP IPM is already competitive across realistic nonlinear planning workloads.
5. **The generic solver still trails the dedicated parity solver on the same formulation.** This is the strongest practical red flag in the current tree. When the same parity NLP callbacks are solved by `parity`, `native`, and `ipopt`, the dedicated parity solver converges while the generic native solver still fails. That means the remaining gap is algorithmic and not just an interface or model-building issue.
6. **Fallback layering can hide native weakness.** The current adapter stack can flow through filter solve, restoration retry, merit solve, and finally IPOPT fallback. This is a useful resilience feature, but it also means that a superficially successful solve may not actually prove native robustness unless the solver path is checked explicitly.
7. **The present bottleneck is not “missing Hessian” anymore.** That diagnosis was accurate earlier, but it is no longer the main one. The more serious current risks are globalization quality, scaling fidelity, and parity-to-generic algorithm mismatch in difficult OPF geometries.

In short, the present code base should be read as follows: the native NLP IPM is algorithmically real and test-backed, but its performance and robustness are still clearly insufficient for large OPF-class NLPs. This is now a globalization-and-driver-design problem more than a second-derivative problem.

6 Generic NativeIPM vs. Parity-Specific IPM: Algorithm Gap and Refactoring Roadmap

This section isolates one question that now matters more than raw feature completeness: why does the dedicated parity OPF interior-point solver converge on large cases where the generic `NativeIPMAdapter` still fails on the same nonlinear program callbacks?

The answer is no longer “because the generic solver has no exact Hessian.” That issue has already been addressed. The remaining gap is that the dedicated parity solver still has a better algorithmic envelope for OPF-class problems, especially in globalization, centrality control, and reduced-system handling.

6.1 What Is Actually Being Compared

The large-case comparison in `tests/benchmark_ac_opf_cases.cpp` now evaluates three modes on the same parity NLP model:

- **parity:** the dedicated solver in `src/optimal_power_flow/parity_ipm.cpp`,
- **native:** the generic `NativeIPMAdapter` with the same callback model and exact Lagrangian Hessian,

- `ipopt`: IPOPT applied to the same callback model.

This matters because it removes most modeling ambiguity. When `parity` converges and `native` does not, the remaining explanation is mostly in the solver algorithm, not in the OPF formulation itself.

6.2 Algorithmic Differences

Table 9 summarizes the most relevant current differences.

Table 9: Current algorithmic differences between generic NativeIPM and dedicated parity IPM

Aspect	Generic <code>NativeIPMAdapter</code>	Dedicated <code>parity_ipm.cpp</code>
Globalization	Default path is filter-based with restoration and second-order correction; fallback path is merit-based backtracking. In practice this stack is still fragile on large OPF NLPs.	Uses a more direct OPF-focused primal-dual step acceptance strategy with separate primal and dual fraction-to-boundary steps and no generic merit backtracking on each iteration.
Barrier-state initialization	Now improved with residual-aware slack and multiplier initialization in the merit path, but still embedded in a more generic driver.	Initializes slacks directly from inequality residuals to control μ_i/z_i spread and maintain a well-conditioned condensed system from the start.
Residual metrics	Generic path now uses normalized residuals, but its acceptance logic still depends on a more generic merit/filter driver architecture.	Uses OPF-focused normalized convergence tests that were tuned around the parity formulation and its scaling behavior.
Complementarity handling	Uses a generalized predictor-corrector centering target and restoration logic.	Uses condensed complementarity handling with OPF-tested centering and direct update rules tailored to the assembled parity inequalities.
Extra correctors	Filter path contains second-order correction; merit path has no parity-style Gondzio block.	Includes explicit Gondzio-type extra corrector passes, which help recover centrality without relying on a merit backtracking loop.
Reduced Newton system	Reuses generic KKT utilities and multiple solve paths, including reduced-KKT logic for some equality patterns.	Builds and solves the condensed parity-specific Schur system directly in the exact structure expected by the OPF formulation.
Failure behavior on large parity OPF	Still terminates with globalization failure on IEEE118 and PEGASE2869.	Converges on IEEE118 and PEGASE2869 and passes PF cross-validation.

6.3 Why the Dedicated Parity Solver Still Wins

The dedicated parity solver currently wins for three concrete reasons.

1. It solves the right condensed problem with less generic machinery The parity solver assembles the exact OPF inequality residuals, slack ratios, and reduced Hessian contributions in one coherent path. The generic solver can now represent the same mathematics, but it still routes that information through a broader abstraction that was designed to handle arbitrary NLPs rather than one specific network structure.

2. Its globalization is still closer to the OPF central path The parity solver takes separate primal and dual fraction-to-boundary steps and accepts them directly after corrector assembly. The generic native solver, especially in its merit-backed path, still has a stronger tendency to reject steps via generic globalization rules that are mathematically sensible but too conservative for the parity OPF trajectory.

3. It contains OPF-specific corrector behavior the generic solver still lacks The parity implementation includes Gondzio-style extra corrector passes and best-iterate restoration in a path already tuned around parity OPF residual geometry. The generic solver has partially moved in this direction, but the remaining gap is still visible on large cases.

6.4 Current Performance Interpretation

The present large-case evidence should be interpreted carefully.

- The generic native NLP IPM is no longer catastrophically incomplete. It now reaches much better neighborhoods on parity OPF than it did before the exact-Hessian and merit-path upgrades.
- However, the current performance is still not acceptable as a final OPF solver path. On realistic parity cases it either fails outright or only approaches the correct trajectory before globalization collapses.
- Therefore the remaining issue is not “can the solver form Newton systems?” but rather “can it keep and accept the right Newton trajectory on hard OPF landscapes?”

This is why the current Native IPM performance still looks problematic even after recent improvements: the solver core is now mathematically much closer to correct, but its driver logic is still not as OPF-effective as the parity solver’s dedicated update scheme.

6.5 Implementation Backlog: PR1 / PR2 / PR3

The next useful step is not another high-level roadmap but a concrete sequence of pull requests. The purpose of the following backlog is to bind the intended algorithm changes directly to the current source functions and acceptance rules.

PR1: unify globalization around a direct primal/dual accepted step Primary file: `src/engine/ipm_solver.cpp`

Functions to change.

- `solve_nlp_filter_impl`
- the merit fallback loop inside `NativeIPMAdapter::solve_nlp_detail`
- if needed, extract a shared helper for step acceptance so filter and merit paths do not diverge again

What should change.

- Replace the current mixed acceptance rule in which x, s, λ are accepted by filter logic while μ is still updated with an independent `alpha_max_dual`. The accepted step should become one coherent primal-dual update decided from the same accepted trial.
- Introduce a direct fraction-to-boundary accepted step for the predictor-corrector direction before falling back to repeated generic backtracking. In other words, the first-class path should be “accept the Newton step if positivity and normalized residual improvement are good enough,” not “always reduce α until the generic filter admits it.”
- Move the current acceptance decision away from pure (θ, ϕ) -based rejection and toward a parity-style gate that checks positivity, normalized primal feasibility, normalized dual feasibility, and complementarity together.

Acceptance criteria to replace or add.

- Replace “accepted trial point for x, s, λ plus separate dual multiplier step” with “one accepted (x, s, λ, μ) trial.”
- Add an explicit rule that a full or near-full step is kept when it improves the normalized residual summary produced by `summarize_residuals`, even if the generic filter would otherwise shrink it.
- Keep filter backtracking only as a secondary safeguard, not as the dominant accepted-step mechanism.

Minimum validation.

- Re-run `benchmark_ac_opf_cases` on IEEE118 and PEGASE2869 in `native` mode.
- Require that failure mode changes from early line-search collapse to a materially better trajectory, ideally either convergence or a much lower final normalized residual than the current baseline.

PR2: port parity-style extra correctors into the generic driver Primary files: `src/engine/ipm_solver.cpp` and, for reference, `src/optimal_power_flow/parity_ipm.cpp`

Functions to change.

- `solve_nlp_filter_impl`
- the merit fallback loop inside `NativeIPMAdapter::solve_nlp_detail`
- possibly a new helper dedicated to extra-corrector assembly so the code does not duplicate one-off second-order logic in multiple branches

What should change.

- Generalize the current one-shot second-order correction into a bounded parity-style extra-corrector loop. The present filter path already has a single SOC retry; PR2 should turn that into a structured “predictor, corrector, extra-corrector” sequence.
- Use the extra correctors specifically to improve centrality and recover a larger accepted step, rather than treating them only as a feasibility patch after filter rejection.
- Ensure the extra corrector is evaluated against the same normalized residual summary used by the main step so that centrality improvement is measured rather than guessed.

Acceptance criteria to replace or add.

- Replace “at most one SOC attempt on first rejection” with “up to a small fixed number of extra correctors that are accepted only if residuals and complementarity improve.”
- Add an explicit centrality-improvement test based on the normalized complementarity and aggregate merit terms, not only on filter admissibility.

Minimum validation.

- Re-run the same large-case parity benchmarks and compare accepted step lengths, iteration counts, and final residual summaries against PR1.
- Confirm that the generic solver is no longer losing obviously usable predictor-corrector steps that the parity solver would keep.

PR3: retune scaling, barrier-state initialization, and restoration triggers **Primary files:** `src/engine/ipm_solver.cpp`, `include/hacdcpf/engine/ipm_solver.hpp`, and any supporting scaling or restoration helpers already used by the filter path

Functions to change.

- `initialize_barrier_state`
- `summarize_residuals`
- `solve_nlp_filter_impl`
- restoration-trigger decisions inside the native driver stack

What should change.

- Tighten the connection between slack initialization, multiplier initialization, and the normalized residual scales actually used during step acceptance.
- Revisit the current filter constants and restoration triggers so that the solver does not declare globalization failure too early on OPF-class problems that still have a promising best iterate.
- Make best-iterate restoration a deliberate part of the native path’s acceptance policy rather than a final afterthought used only when the main loop is already exhausted.

Acceptance criteria to replace or add.

- Replace purely absolute or mixed-scale progress decisions with consistently normalized criteria derived from `summarize_residuals`.
- Add a restoration-entry rule based on repeated accepted-step collapse, not only on hard factorization or line-search failure.
- Keep the current best-iterate bookkeeping, but require restoration to be judged against parity-like feasibility and complementarity thresholds.

Minimum validation.

- Re-run IEEE118 and PEGASE2869 in `native`, `parity`, and `ipopt` modes on the same callback model.
- Only declare the generic path ready for default use on parity OPF when it converges without IPOPT fallback, matches parity objective and feasibility closely, and no longer fails primarily through globalization breakdown.

Why this PR order matters This order is intentional. PR1 attacks the most visible current defect, namely that the accepted step is still too generic and internally inconsistent. PR2 then adds the parity-style corrector machinery needed to keep that better step on hard iterations. PR3 only makes sense after PR1 and PR2, because scaling and restoration thresholds cannot be tuned meaningfully while the accepted-step logic itself is still unstable.

6.6 Practical Bottom Line

The current technical conclusion is blunt but useful.

- The generic NativeIPM is now a genuine interior-point implementation and no longer a placeholder.
- Its current performance is still problematic on large OPF-class NLPs.
- The parity-specific solver remains the stronger implementation for this problem family.
- The right next move is not more Hessian plumbing, but better globalization and closer transfer of parity’s accepted-step logic into the generic driver.

So if the review question is whether the current native NLP IPM still has major performance problems, the answer is yes. They are now concentrated in the driver and globalization layer, not in the existence of the Newton system or the availability of exact second derivatives.

7 Symbols, Indexing, and Sign Convention

7.1 Electrical Bases

$$S_{base} = \text{system apparent-power base (MVA)}, \quad (14)$$

$$V_{base} = \text{nominal line-line voltage base (kV)}, \quad (15)$$

$$Z_{base} = \frac{V_{base}^2}{S_{base}}, \quad (16)$$

$$I_{base} = \frac{S_{base}}{\sqrt{3}V_{base}}. \quad (17)$$

7.2 Sets and Indexing

$$\mathcal{N}_{ac} : \text{AC buses}, \quad \mathcal{E}_{ac} : \text{AC branches}, \quad (18)$$

$$\mathcal{N}_{dc} : \text{DC buses}, \quad \mathcal{E}_{dc} : \text{DC branches}, \quad (19)$$

$$\mathcal{N}_{sl} : \text{AC slack buses}, \quad \mathcal{N}_{PQ} : \text{AC PQ buses}, \quad (20)$$

$$\mathcal{N}_{PV} : \text{AC PV buses}, \quad \mathcal{N}_{sl}^{\text{dc}} : \text{DC voltage-controlled buses}. \quad (21)$$

The public model stores external bus identifiers in each component (`bus`, `from_bus`, `to_bus`). Most solver kernels then remap those identifiers into contiguous zero-based local indices.

7.3 Core AC and DC State Variables

$$V_i = v_i e^{j\theta_i}, \quad (22)$$

$$Y_{ij} = G_{ij} + jB_{ij}, \quad (23)$$

$$V_k^{\text{dc}} = v_k^{\text{dc}}, \quad (24)$$

$$P_k^{\text{dc,calc}} = v_k^{\text{dc}} (G_{dc} v^{\text{dc}})_k. \quad (25)$$

For the main Newton power-flow path, the state vector is

$$x_{PF} = \left[\theta_{\mathcal{N}_{ac} \setminus \mathcal{N}_{sl}} \quad v_{\mathcal{N}_{PQ}} \quad v_{\mathcal{N}_{dc} \setminus \mathcal{N}_{sl}}^{dc} \right]^\top. \quad (26)$$

For the native primal-dual AC OPF path, the decision vector is extended to include generator dispatch and converter operating points:

$$x_{OPF} = \left[\theta \quad v \quad P_g \quad Q_g \quad v^{dc} \quad P^{\text{conv},ac} \quad Q^{\text{conv},ac} \quad P^{\text{conv},dc} \right]^\top, \quad (27)$$

with additional load-shedding variables in the parity formulation.

7.4 Unified Sign Convention: Bus Injection Positive

The current C++ module uses one consistent notation in the PF kernels and the notebook adopts the same rule:

Positive active or reactive power at a bus means injection into the network from the component attached to that bus.

Loads therefore store positive demand values in their data model, but they enter power-balance equations with a minus sign.

Quantity / field	Positive value means	Consequence in equations
<code>Generator::pg_mw, qg_mvar</code>	generator injects active/reactive power into its AC bus	added to $P_i^{\text{spec}}, Q_i^{\text{spec}}$.
<code>Load::p_mw, q_mvar</code>	consumption requested from the network	subtracted from $P_i^{\text{spec}}, Q_i^{\text{spec}}$; ZIP dependence scales the subtraction.
<code>Storage::p_mw</code>	discharge to the grid; charging is negative P	treated as generation when positive, load when negative.
<code>Microgrid::p_exchange_mw</code>	export from the microgrid to the host AC grid	added as AC-bus injection in PF; ignored when the microgrid is islanded.
<code>VSCConverter</code> AC-side $P^{\text{conv},ac}$	converter injects active power into the AC bus	positive AC injection must be balanced by negative DC-bus injection plus losses.
<code>VSCConverter</code> DC-side $P^{\text{conv},dc}$	converter injects active power into the DC bus	the current code enforces $P^{\text{conv},ac} + P^{\text{conv},dc} + P^{\text{loss}} = 0$.

Quantity / field	Positive value means	Consequence in equations
<code>DCDCConverter::p_ref_mw</code>	positive transfer from <code>bus_in</code> toward <code>bus_out</code>	implemented as negative injection at <code>bus_in</code> and positive injection at <code>bus_out</code> .
<code>BranchFlow::pf_mw</code>	active power sent from <code>from_bus</code> to <code>to_bus</code>	reporting only; no sign reversal in stored result.

7.5 Power-Balance Template

With bus-injection-positive notation, the balance equations are written as

$$\Delta P_i = P_i^{\text{spec}} - P_i^{\text{calc}}, \quad (28)$$

$$\Delta Q_i = Q_i^{\text{spec}} - Q_i^{\text{calc}}, \quad (29)$$

$$\Delta P_k^{\text{dc}} = P_k^{\text{dc,spec}} - P_k^{\text{dc,calc}}. \quad (30)$$

The specified term is assembled by adding all component injections and then subtracting all demand terms. This is the convention followed by the PF residual evaluator, the native AC OPF assembly, and the parity formulation.

8 Information Models and Field Dictionaries

8.1 Modeling Layers Used by the Code

The code base uses three distinct model layers.

1. **Public information model:** the rich structs under `include/hacdcpf/model/` used by I/O, case builders, and external callers.
2. **Canonical steady-state model:** the projected subset generated by `project_to_canonical_models`, where rich components are converted into equivalent loads, branches, or aggregated injections.
3. **Solver assembly model:** `powerflow::SolverData`, which stores contiguous vectors, aggregated per-bus injections, ZIP coefficients, and sparse matrices needed by the PF and OPF kernels.

8.2 System Containers and Solver-Level Metadata

System containers

Struct / group	Fields and meaning	Primary consumers
ACSystem	buses, branches, generators, loads, shunts, storage, renewable_gens, pv_systems, transformers_2w, transformers_3w, switches, charging_stations, chargers, external_grids, motors; plus base_mva, freq_hz, name. It is the main AC container for PF, DPF, SC, and ONR.	PF, OPF, SC, DPF, ONR, I/O
DCSystem	buses, branches, loads, storage, static_generators, plus base_mva and name. Used only by hybrid AC/DC solvers.	PF, OPF, I/O
HybridPowerSystem	Holds ac, dc, vsc_converters, dcdc_converters, energy_routers, mobile_storage, vpps, microgrids, optional three_phase_ac, and global base_mva. This is the top-level public object for most APIs.	all top-level APIs
ThreePhaseACSystem	Optional unbalanced AC subsystem with three-phase buses, lines, transformers, loads, generators, and external grids. It is projected to canonical AC objects when the explicit ACSystem is empty.	projection, PF, SC, I/O

Power-flow options and results

Struct / group	Fields and meaning	Primary consumers
InitialState	vm, va, vdc provide optional warm starts for AC magnitude, AC angle, and DC voltage states.	Newton PF, DC PF
ReactiveLimit	qmin, qmax define reactive limits used by adaptive/PV-PQ logic.	adaptive PF extensions
DistributedSlack	participating_buses, participation_factors, reference_bus, and optional max_participation_p describe how slack mismatch is redistributed.	distributed-slack PF
PowerFlowOptions	Iteration and tolerance controls: max_iter, tol; robustness controls: line search, regularization, and step clipping; PV/PQ and converter hysteresis parameters; ZIP weights; solver profiling flag; loss_model; FDPF iteration cap.	PF family
SolverProfiling	Sparse-factorization counters, line-search and regularization statistics, PV/PQ switches, converter-mode switches, timing vectors, and cumulative timing totals.	PF result diagnostics
PowerFlowResult	Solved states vm, va, vdc; status fields converged, iterations, residual; and derived reports branch_flows, vsc_transfers, dcdc_transfers, er_port_transfers.	PF family outputs

Struct / group	Fields and meaning	Primary consumers
<code>AdaptiveSolverResult</code> , <code>IslandedSolverResult</code> , <code>DistributedSlackResult</code>	Variants of PF results that additionally expose detected <code>islands</code> , distributed-slack allocations, and buses hitting slack-sharing limits.	adaptive / islanded / distributed-slack PF

Analysis and OPF options/results

Struct / group	Fields and meaning	Primary consumers
<code>ACOPFOptions</code>	Inner/outer iteration caps, feasibility/stationarity tolerances, barrier schedule, merit-penalty and regularization terms, line-search settings, parallel AC evaluation threads, and path-selection flags <code>enable_primal_dual</code> , <code>use_parity_ipm</code> , <code>allow_fallback</code> .	AC OPF
<code>ACOPFResult</code>	Voltage solution, generator dispatch, optional bus-level load shedding, converter operating points, convergence metrics, status text, and profiling summary.	AC OPF output
<code>SCOptions</code> , <code>SCResult</code> , <code>BusFaultResult</code>	Short-circuit fault type, voltage correction factor, default machine reactance, per-bus Thevenin impedance, fault current, and short-circuit power.	short circuit
<code>DPFOptions</code> , <code>DPFResult</code>	BFS controls, optional shunt and Q-limit handling, radial PF voltages, branch powers, total losses, and residual.	distribution PF
<code>ONROptions</code> , <code>ONRRResult</code>	Voltage bounds, thermal defaults, big- M , MILP gap and time limit, resulting branch statuses, estimated loss, verification PF, and branch-and-cut diagnostics.	ONR

`powerflow::SolverData`

Group	Fields and meaning	Primary consumers
Canonical component tables	Copies of projected AC/DC buses, branches, generators, converters, DCDC converters, energy routers, loads, DERs, shunts, switches, charging stations, external grids, VPPs, microgrids, and mobile storage.	PF, native OPF
Aggregated steady-state injections	<code>pg</code> , <code>qg</code> hold per-bus generation after aggregation of generators, static generators, renewables, PV, storage, VPPs, microgrids, and mobile storage.	PF, native OPF

Group	Fields and meaning	Primary consumers
Demand model data	<code>has_component_loads</code> , <code>pd_pu</code> , <code>qd_pu</code> , and per-bus ZIP coefficient vectors describe the demand model after load-table aggregation.	PF, native OPF
Network matrices	<code>ybus</code> and <code>gdc</code> store the canonical AC admittance and DC conductance matrices; <code>build_id</code> is used to validate Jacobian-pattern caching.	PF, native OPF
Bookkeeping	<code>base_mva</code> , <code>loss_model</code> , and global ZIP weights copied from <code>PowerFlowOptions</code> .	PF, native OPF

8.3 Component Inventory

Table 15 enumerates the rich public models currently present in the repository.

Table 15: Public component-model inventory

Struct	Mathematical role	Current steady-state integration mode
ACBus	AC state host, voltage limits, embedded bus demand and shunt	direct
ACBranch	AC series/shunt admittance, tap and phase-shift branch	direct
Transformer2W	named two-winding transformer with short-circuit/tap data	projected to ACBranch
Transformer3W	explicit three-winding transformer with pairwise short-circuit data	projected to three ACBranch equivalents
ExternalGrid	slack source and short-circuit source equivalent	direct in PF/SC
AsynchronousMotor	motor fault contribution	SC only
Generator	synchronous generator dispatch and limits	direct
StaticGenerator	fixed/scaled distributed generation	direct fixed injection
RenewableGen	renewable injection with curtailment metadata	direct fixed injection; OPF curtailment slot not fully wired
PVSystem	PV inverter and array metadata	direct fixed injection with power-curve helper
Load	AC demand with ZIP and motor-fraction metadata	direct
FlexibleLoad	demand-response model	projected to Load ; direct OPF flexibility variable not yet wired
AsymmetricLoad	per-phase aggregate demand	projected to Load

Struct	Mathematical role	Current steady-state integration mode
Shunt	fixed or stepped shunt element	direct in PF; simplified treatment in DPF
Switch	switching device with electrical/protection data	projected to ACBranch for PF/OPF/SC
ChargingStation	aggregate EV charging demand	direct fixed load
Charger	individual EV charger	aggregated into ChargingStation on projection
Storage	stationary storage power and energy state	direct fixed injection in current steady-state solvers
MobileStorage	relocatable storage resource	direct fixed injection when stationary
VirtualPowerPlant	aggregated DER portfolio	direct fixed AC-bus injection
Microgrid	PCC exchange model	direct fixed AC-bus injection when grid-connected
DCBus	DC voltage host and DC demand host	direct
DCBranch	DC resistive branch	direct
DCLoad	DC active demand	direct
VSCConverter	AC/DC coupling with loss model and control mode	direct
DCDCConverter	DC/DC transfer device with efficiency and I^2R conduction loss	direct in PF and parity OPF (decision variable with Jacobian and Hessian)
EnergyRouterPort	AC or DC port injection for a multi-port router	direct fixed injection
EnergyRouter	multi-port converter container	direct through its ports
ThreePhaseACBus	optional unbalanced bus model	projected to ACBus
ThreePhaseACLine	optional unbalanced line model	projected to ACBranch using positive sequence
ThreePhaseTransformer	optional unbalanced transformer model	projected to Transformer2W equivalent
ThreePhaseLoad	optional unbalanced load model	projected to Load
ThreePhaseGenerator	optional unbalanced generator model	projected to Generator
ThreePhaseExternalGrid	optional unbalanced slack/grid equivalent	projected to ExternalGrid

8.4 AC Network Elements

ACBus

Field group	Detailed explanation	Used by
Identity and topology	<code>index</code> is the external bus identifier referenced by other components; <code>name</code> is purely descriptive; <code>area</code> and <code>zone</code> are retained for partitioning and reporting.	all modules
Steady-state electrical state	<code>bus_type</code> , <code>vm_pu</code> , <code>va_deg</code> , <code>base_kv</code> , <code>vmax_pu</code> , <code>vmin_pu</code> , <code>in_service</code> . These fields initialize the PF state, define slack/PV/PQ behavior, and bound voltages.	PF, OPF, DPF
Embedded demand and shunt	<code>pd_mw</code> , <code>qd_mvar</code> store fallback bus demand when no load table is present; <code>gs_mw</code> , <code>bs_mvar</code> store bus shunt terms.	PF, OPF, DPF, ONR
Resilience / geography	<code>n_customers</code> , <code>importance</code> , <code>latitude</code> , <code>longitude</code> . These are not yet decision variables, but they are useful for planning logic, GIS, or load-priority extensions.	planning / future use

ACBranch

Field group	Detailed explanation	Used by
Connectivity	<code>index</code> , <code>from_bus</code> , <code>to_bus</code> , <code>in_service</code> , <code>name</code> . Bus references use external bus IDs.	PF, OPF, SC, DPF, ONR
Positive-sequence electrical data	<code>r_pu</code> , <code>x_pu</code> , <code>b_pu</code> , <code>tap</code> , <code>shift_deg</code> . These define the AC π -model and branch-flow reporting.	PF, OPF, SC, DPF, ONR
Ratings and physical metadata	<code>rate_a_mva</code> , <code>rate_b_mva</code> , <code>rate_c_mva</code> , <code>length_km</code> . Only <code>rate_a_mva</code> is used directly by the current OPF/ONR limits.	OPF, ONR, reporting
Reliability and sequence data	<code>failure_rate</code> , <code>mttr_hr</code> , <code>r0_pu</code> , <code>x0_pu</code> , <code>b0_pu</code> , and transformer-nameplate fields <code>vn_hv_kv</code> , <code>vn_lv_kv</code> , <code>sn_mva</code> . These support SC correction factors and planning extensions.	SC, planning

Transformer2W, Transformer3W, and Switch

Struct	Field-level explanation	Current treatment
Transformer2W	Identity/connectivity fields <code>index</code> , <code>name</code> , <code>hv_bus</code> , <code>lv_bus</code> , <code>in_service</code> ; nameplate fields <code>sn_mva</code> , <code>vn_hv_kv</code> , <code>vn_lv_kv</code> ; short-circuit fields <code>vk_percent</code> , <code>vkr_percent</code> , <code>pk_kw</code> ; no-load fields <code>pf_e_kw</code> , <code>i0_percent</code> ; tap-changer fields <code>tap_side</code> , <code>tap_pos</code> , <code>tap_min</code> , <code>tap_max</code> , <code>tap_neutral</code> , <code>tap_step_percent</code> ; phase shift <code>shift_deg</code> ; zero-sequence fields <code>z0_percent</code> , <code>x0_r0</code> ; reliability fields <code>mtbf_hours</code> , <code>mttr_hours</code> .	projected to one equivalent ACBranch

Struct	Field-level explanation	Current treatment
Transformer3W	Connectivity <code>hv_bus</code> , <code>mv_bus</code> , <code>lv_bus</code> ; per-winding ratings <code>sn_hv_mva</code> , <code>sn_mv_mva</code> , <code>sn_lv_mva</code> ; pairwise short-circuit data <code>vk_hv_mv_percent</code> , <code>vk_hv_lv_percent</code> , <code>vk_mv_lv_percent</code> and the matching <code>vk_r</code> fields; tap data and phase shifts <code>shift_mv_deg</code> , <code>shift_lv_deg</code> .	projected to three equivalent ACBranch objects
Switch	Topology fields <code>bus_from</code> , <code>bus_to</code> , <code>closed</code> ; electrical fields <code>r_contact_ohm</code> , <code>z_ohm</code> , <code>i_rated_ka</code> , <code>i_breaking_ka</code> ; protection fields <code>element_type</code> , <code>element_id</code> ; automation fields <code>is_remote</code> , <code>is_automated</code> , <code>t_operation_s</code> ; reliability fields <code>p_sw_fail</code> , <code>mtbf_hours</code> , <code>mttr_hours</code> .	projected to equivalent branch with <code>in_service=closed</code>

ExternalGrid and AsynchronousMotor

Struct	Field-level explanation	Current treatment
ExternalGrid	<code>bus</code> , <code>vm_pu</code> , <code>va_deg</code> establish slack voltage targets; <code>s_sc_max_mva</code> , <code>s_sc_min_mva</code> , <code>rx_max</code> , <code>rx_min</code> , <code>r_pu</code> , <code>x_pu</code> , <code>r0_pu</code> , <code>x0_pu</code> , <code>vn_kv</code> , <code>ikq_ka</code> , <code>x_r</code> store fault-source data; <code>controllable</code> can support future dispatch logic.	PF slack/source and SC source equivalent
AsynchronousMotor	<code>bus</code> , <code>vn_kv</code> , <code>sn_mva</code> , <code>r_pu</code> , <code>x_pu</code> , <code>x_r</code> , <code>lrc</code> , <code>poles</code> , <code>cos_phi</code> , <code>efficiency</code> , <code>r0_pu</code> , <code>x0_pu</code> . These fields describe the motor contribution to short-circuit current, not a steady-state PF state.	SC only

8.5 Load Models and Demand-Side Components

Struct	Field-level explanation	Current treatment
Load	Identity fields <code>index</code> , <code>bus</code> , <code>name</code> , <code>in_service</code> ; steady-state demand <code>p_mw</code> , <code>q_mvar</code> ; model selector <code>model</code> ; ZIP coefficients <code>z_percent_p</code> , <code>i_percent_p</code> , <code>p_percent_p</code> , and the matching <code>q</code> fields; demand-response metadata <code>controllable</code> , <code>p_min_mw</code> , <code>cost_mw</code> ; load-priority metadata <code>priority</code> , <code>n_customers</code> , <code>profile_id</code> ; motor-fraction SC metadata <code>sn_mva</code> , <code>motor_percent</code> , <code>x_sub_pu</code> , <code>r_sc_pu</code> .	direct in PF/OPF/SC

Struct	Field-level explanation	Current treatment
FlexibleLoad	Baseline demand <code>p_mw</code> , <code>q_mvar</code> ; flexibility envelope <code>flex_up_mw</code> , <code>flex_down_mw</code> , <code>flex_duration_h</code> ; response timing <code>response_time_s</code> , <code>ramp_rate_mw_min</code> , <code>availability_pct</code> ; <code>control_area</code> .	projected to equivalent Load; direct OPF flexibility variable not yet active
AsymmetricLoad	Per-phase rated values <code>pa_rated_mw</code> , ..., <code>qc_rated_mvar</code> ; per-phase actual values <code>pa_mw</code> , ..., <code>qc_mvar</code> ; connection, grounded, scaling; ZIP shares <code>const_z_percent</code> , <code>const_i_percent</code> , <code>const_p_percent</code> .	projected to aggregated Load
DCLoad	<code>p_mw</code> is the direct active demand at a DC bus; <code>controllable</code> , <code>p_min_mw</code> , <code>cost_mw</code> , and <code>profile_id</code> are control metadata for future extensions.	direct in DC PF and hybrid OPF balances
ChargingStation	<code>bus</code> , <code>n_fast</code> , <code>n_slow</code> , <code>num_chargers</code> ; capacity fields <code>p_fast_max_kw</code> , <code>p_slow_max_kw</code> , <code>max_power_kw</code> ; utilization fields <code>simultaneity_factor</code> , <code>power_factor</code> , <code>utilization_rate</code> ; aggregated operating demand <code>p_total_kw</code> , <code>q_total_kvar</code> .	direct fixed load
Charger	<code>station_id</code> links the charger to its parent station; <code>charger_type</code> , <code>p_rated_kw</code> , <code>p_ch_max_kw</code> , <code>p_ch_min_kw</code> , <code>eta</code> , <code>v2g_capable</code> , <code>p_dis_max_kw</code> .	aggregated into ChargingStation during projection

8.6 Generation, Storage, and Aggregation Components

Struct	Field-level explanation	Current treatment
Generator	Dispatch/state fields <code>pg_mw</code> , <code>qg_mvar</code> , <code>vg_pu</code> , <code>is_slack</code> ; limits <code>pmax_mw</code> , <code>pmin_mw</code> , <code>qmax_mvar</code> , <code>qmin_mvar</code> ; quadratic cost coefficients <code>cost_c2</code> , <code>cost_c1</code> , <code>cost_c0</code> ; unit-commitment metadata <code>startup_cost</code> , <code>shutdown_cost</code> , <code>min_up_time_hr</code> , <code>min_dn_time_hr</code> , <code>ramp_up_mw_min</code> , <code>ramp_dn_mw_min</code> ; machine/stability data <code>mbase_mva</code> , <code>inertia_h</code> , <code>droop_r</code> , <code>xd_pu</code> , <code>xdp_pu</code> , <code>xdpp_pu</code> , <code>td0p_s</code> , <code>td0pp_s</code> ; SC and environmental metadata <code>ra_pu</code> , <code>vn_kv</code> , <code>cos_phi</code> , <code>xq_pu</code> , <code>xd_xq</code> , <code>x0_pu</code> , <code>r0_pu</code> , emission factors, outage rate, and repair time.	direct
StaticGenerator	Fixed or scaled DER injection <code>p_mw</code> , <code>q_mvar</code> , scaling; device rating/limits <code>p_rated_mw</code> , <code>sn_mva</code> , <code>pmax_mw</code> , <code>pmin_mw</code> , <code>qmax_mvar</code> , <code>qmin_mvar</code> ; control metadata <code>controllable</code> , <code>v_ref_pu</code> , <code>k_p</code> , <code>k_q</code> ; SC metadata <code>k</code> , <code>rx</code> .	direct fixed injection

Struct	Field-level explanation	Current treatment
RenewableGen	type, actual output p_mw, q_mvar, rated power p_rated_mw, reactive limits, curtailability flag, curtailment cost, capacity factor, profile pointer, and avoided-emission factor.	direct fixed injection; curtailability metadata not yet a top-level decision variable
PVSystem	Actual output p_mw, q_mvar; inverter limits and control sn_mva, pmax_mw, pmin_mw, qmax_mvar, qmin_mvar, control_mode, controllable, v_ac_set_pu, v_dc_set_pu; inverter efficiency inverter_eff, loss_percent; array data num_series, num_parallel, vmpp, impp, voc, isc, alpha_isc, beta_voc; environmental state irradiance, temperature.	direct fixed injection with power-curve helper
Storage	Snapshot dispatch p_mw, q_mvar; power limits p_rated_mw, pmax_mw, pmin_mw, qmax_mvar, qmin_mvar; energy state e_rated_mwh, soc_init, soc_min, soc_max; efficiency eta_charge, eta_discharge, self_discharge_pct; lifecycle max_cycles, current_cycles, soh, replacement_cost.	direct fixed injection
MobileStorage	Reuses storage-style power and energy fields and adds mobility fields is_mobile, status, current_location, target_bus, e_consumption_mwh_km, max_travel_distance_km.	direct fixed injection when not InTransit
VirtualPowerPlant	Fleet counts, aggregated capacities, output fields p_output_mw, q_output_mvar, limits, and ramp rates.	direct fixed AC-bus injection
Microgrid	PCC bus pcc_bus, internal bus list, operating-mode fields operating_mode, islanding_capability, auto_reconnection; exchange limits and current p_exchange_mw; aggregate capability metrics and droop/voltage setpoints.	direct fixed AC-bus injection when grid-connected

8.7 DC and Power-Electronic Components

Struct	Field-level explanation	Current treatment
DCBus	index, bus_type, vm_pu, vmax_pu, vmin_pu, pd_mw, in_service, name. pd_mw is used only when the dedicated DC load table is empty.	direct
DCBranch	from_bus, to_bus, r_pu, rate_a_mva, length_km, in_service.	direct

Struct	Field-level explanation	Current treatment
VSCConverter	Connectivity <code>bus_ac</code> , <code>bus_dc</code> ; control mode <code>control_mode</code> ; setpoints <code>p_set_mw</code> , <code>q_set_mvar</code> , <code>v_dc_set_pu</code> , <code>v_ac_set_pu</code> ; loss parameters <code>eta</code> , <code>loss_percent</code> , <code>loss_mw</code> ; control gain <code>k_vdc</code> ; limits <code>pmax_mw</code> , <code>pmin_mw</code> , <code>qmax_mvar</code> , <code>qmin_mvar</code> , <code>p_rated_mw</code> ; plus reliability fields.	direct in PF and OPF
DCDCConverter	Connectivity <code>bus_in</code> , <code>bus_out</code> ; operating mode <code>control_mode</code> ; references <code>p_ref_mw</code> (output-side referenced), <code>v_ref_pu</code> ; rating fields <code>sn_mva</code> , <code>vn_in_kv</code> , <code>vn_out_kv</code> ; efficiency <code>eta</code> ; conduction-loss resistance <code>r_eq_pu</code> ; droop gain <code>k_droop</code> ; limits <code>pmax_mw</code> , <code>pmin_mw</code> . Sign convention: positive direction is <code>bus_in</code> → <code>bus_out</code> ; PF uses output-referenced <code>p_ref_mw</code> , OPF uses input-referenced P^{in} .	direct in PF (with I^2R loss via <code>r_eq_pu</code>) and parity OPF (decision variable with analytical Jacobian and Hessian)
EnergyRouterPort	<code>bus</code> , <code>port_type</code> , <code>voltage_level_kv</code> , operating injection <code>p_mw</code> , <code>q_mvar</code> , <code>v_pu</code> ; limits <code>pmax_mw</code> , <code>pmin_mw</code> , <code>qmax_mvar</code> , <code>qmin_mvar</code> ; control metadata <code>control_mode</code> , <code>p_set_mw</code> , <code>q_set_mvar</code> , <code>v_set_pu</code> .	direct fixed port injection
EnergyRouter	Container fields <code>router_type</code> , <code>num_ports</code> , <code>ports</code> ; rating and loss fields <code>p_rated_mw</code> , <code>vn_ac_kv</code> , <code>vn_dc_kv</code> , <code>loss_percent</code> ; device-level limits and reliability.	direct through <code>ports</code>

8.8 Optional Three-Phase Models

Struct	Field-level explanation	Current treatment
ThreePhaseACBus	Per-phase voltages <code>vm_a_pu</code> , <code>va_a_deg</code> , <code>vm_b_pu</code> , ...; per-phase demand and shunts; common voltage limits, area, and zone.	projected to ACBus by phase summation / averaging
ThreePhaseACLine	Positive- and zero-sequence per-km parameters, optional precomputed per-unit values, parallel count, thermal rating, and reliability data.	projected to ACBranch using positive-sequence values
ThreePhaseTransformer	Two-winding transformer with vector group, positive- and zero-sequence short-circuit data, tap changer, and phase shift.	projected to Transformer2W equivalent before branch projection
ThreePhaseLoad	Connection type, grounding flag, per-phase powers, ZIP shares, and motor fraction.	projected to aggregated Load

Struct	Field-level explanation	Current treatment
ThreePhaseGenerator	Per-device dispatch and limits plus sequence impedances <code>xd_pu</code> , <code>xdpp_pu</code> , <code>x2_pu</code> , <code>x0_pu</code> , <code>r0_pu</code> .	projected to Generator
ThreePhaseExternalGrid	Per-sequence grid-equivalent impedances and short-circuit capacities.	projected to ExternalGrid
ThreePhaseBusResult	Per-phase solved voltage and injection report. The current one-line notebook path documents the model, but the main steady-state solvers still operate on projected single-phase equivalents.	reporting / future unbalanced workflows

9 I/O Contracts, Sheet Dictionaries, and JSON Keys

The model dictionaries in the previous section explain the physical meaning of each field. This section documents the external I/O contract actually exposed by the current C++ implementation: JSON path names, Excel sheet names, column names, defaulting rules, aggregated views, and the current serialization gaps.

9.1 Serialization Families

API entry point	Artifact	Direction	Practical behavior
<code>to_json</code> / <code>from_json</code>	one JSON document for <code>HybridPowerSystem</code>	bidirectional	now covers all major public model families attached to <code>HybridPowerSystem</code> , including <code>ac.motors</code> and optional <code>three_phase_ac</code> ; some legacy field-level gaps still remain inside individual structs.
<code>save_xlsx</code> / <code>load_xlsx</code>	engineering workbook with one table per supported component class	bidirectional	input-oriented Excel template plus derived aggregated views such as <code>ACLoad</code> and <code>DCLoad</code> .
<code>save_results_xlsx</code>	dedicated PF result workbook	export only	writes <code>Results_*</code> sheets only.
<code>save_results_compact_xlsx</code>	input-like workbook with appended PF result columns	export only	rewrites the input sheets, appends <code>result_*</code> columns to selected tables, and also writes <code>Results_Summary</code> .
<code>save_opf_results_xlsx</code>	dedicated OPF result workbook	export only	writes <code>OPF_*</code> sheets only.

API entry point	Artifact	Direction	Practical behavior
<code>save_opf_results_compact_xlsx</code>	input sheets plus dedicated OPF result sheets	export only	unlike compact PF export, OPF does not append <code>result_*</code> columns into the input tables; it ships separate <code>OPF_*</code> sheets beside the input data.
<code>opf_result_to_json / opf_result_from_json</code>	AC OPF result JSON	bidirectional	serializes solver status, primal solution vectors, load shedding vectors, converter vectors, and profiling summary.
<code>save_carbon_results_xlsx / carbon_result_to_json</code>	carbon-analysis workbook / JSON	export only for Excel, bidirectional for JSON	uses dedicated report-oriented structures, not the main system model.

9.2 Global Parsing and Workbook Rules

Rule	Current implementation behavior
JSON missing keys	Most JSON importers use <code>jget(..., default)</code> ; missing optional keys are silently defaulted. Whole arrays are optional as well.
JSON required keys	Identity and connectivity fields are usually mandatory: for example <code>index</code> , <code>bus</code> , <code>from_bus</code> , <code>to_bus</code> , <code>hv_bus</code> , <code>lv_bus</code> , <code>bus_ac</code> , and <code>bus_dc</code> .
JSON enums	Enum values are string-coded: for example <code>PQ/PV/SLACK</code> , <code>DC_V/DC_P</code> , <code>VDC_Q</code> , <code>Wind</code> , <code>FuelCell</code> , <code>Recloser</code> , <code>Islanded</code> .
Excel header row	All Excel tables use row 1 as the header row. Data rows start at row 2.
Excel row termination	Readers stop at the first row with an empty <code>index</code> cell. This means blank rows inside a table terminate import early.
Excel missing sheets	<code>load_xlsx</code> silently skips missing sheets; the corresponding component vector remains empty.
Excel boolean parser	<code>TRUE</code> , <code>T</code> , <code>YES</code> , <code>Y</code> , and <code>1</code> map to true; <code>FALSE</code> , <code>F</code> , <code>NO</code> , <code>N</code> , and <code>0</code> map to false.
Sheet precedence	For AC loads, <code>Load</code> is preferred; <code>ACLoad</code> is used only as a fallback. For DC loads, <code>DCLoadRaw</code> is preferred; <code>DCLoad</code> is only a fallback.
Bus-demand synchronization	After Excel import, <code>sync_bus_demands_from_load_tables(sys)</code> is called so the bus-level embedded demand reflects the explicit load tables.
Aggregated Excel views	<code>ACBus</code> and <code>DCBus</code> written by the Excel exporters are aggregated bus views, not always the raw struct contents; <code>ACLoad</code> and <code>DCLoad</code> are explicitly derived tables.

Rule	Current implementation behavior
Empty result sheets in templates	<code>save_xlsx</code> currently pre-creates the <code>Results_*</code> sheets even though it does not fill them. A freshly exported template therefore contains empty result tabs.

9.3 Hybrid System JSON Contract

Top-level JSON object

JSON path	Value type	Current meaning
<code>name</code>	scalar string	top-level <code>HybridPowerSystem::name</code> .
<code>base_mva</code>	scalar number	top-level system base.
<code>ac</code>	object	AC subsystem object with its own name, base, frequency, and AC component arrays.
<code>dc</code>	object	DC subsystem object with its own name, base, and DC component arrays.
<code>vsc_converters</code>	array	top-level VSC array.
<code>dc_dc_converters</code>	array	top-level DC/DC converter array.
<code>energy_routers</code>	array	top-level multi-port router array; ports are nested inside each router object.
<code>mobile_storage</code>	array	top-level mobile-storage array.
<code>vpps</code>	array	top-level virtual power plant array.
<code>microgrids</code>	array	top-level microgrid array.
<code>three_phase_ac</code>	object	optional unbalanced AC subsystem with its own base, frequency, and three-phase component arrays.

Current JSON caveats

Public model area	Current JSON status
Standalone energy-router port array	not used; ports appear only as nested objects under each <code>energy_routers[i].ports</code> .
Load short-circuit sub-fields	the embedded SC motor-fraction fields <code>sn_mva</code> , <code>motor_percent</code> , <code>x_sub_pu</code> , and <code>r_sc_pu</code> are still not part of the main <code>ac.loads[]</code> JSON contract.

AC-network and demand JSON objects

Model	JSON path	Required keys on import	Other exported keys
ACBus	ac.buses[]	index	bus_type, pd_mw, qd_mvar, vm_pu, va_deg, area, base_kv, vmax_pu, vmin_pu, gs_mw, bs_mvar, zone, in_service, name, n_customers, importance, latitude, longitude
ACBranch	ac.branches[]	index, from_bus, to_bus	r_pu, x_pu, b_pu, tap, shift_deg, rate_a_mva, rate_b_mva, rate_c_mva, in_service, name, length_km, failure_rate, mttr_hr
Load	ac.loads[]	index, bus	in_service, name, p_mw, q_mvar, model, z_percent_p, i_percent_p, p_percent_p, z_percent_q, i_percent_q, p_percent_q, controllable, p_min_mw, cost_mw, priority, n_customers, profile_id
FlexibleLoad	ac.flexible_loads[]	index, bus	in_service, name, p_mw, q_mvar, flex_up_mw, flex_down_mw, flex_duration_h, response_time_s, ramp_rate_mw_min, availability_pct, controllable, priority, control_area
AsymmetricLoad	ac.asymmetric_loads[]	index, bus	in_service, name, connection, grounded, pa_rated_mw, qa_rated_mvar, pb_rated_mw, qb_rated_mvar, pc_rated_mw, qc_rated_mvar, pa_mw, qa_mvar, pb_mw, qb_mvar, pc_mw, qc_mvar, scaling, const_z_percent, const_i_percent, const_p_percent, controllable, priority
Shunt	ac.shunts[]	index, bus	in_service, name, gs_mw, bs_mvar, switchable, n_steps, current_step, bs_per_step
Switch	ac.switches[]	index, bus_from, bus_to	name, in_service, switch_type, closed, r_contact_ohm, z_ohm, i_rated_ka, i_breaking_ka, element_type, element_id, is_remote, is_automated, t_operation_s, p_sw_fail, mtbf_hours, mttr_hours
ChargingStation	ac.charging_stations[]	index, bus	name, location, in_service, n_fast, n_slow, num_chargers, p_fast_max_kw, p_slow_max_kw, max_power_kw, simultaneity_factor, power_factor, utilization_rate, p_total_kw, q_total_kvar, mtbf_hours, mttr_hours
Charger	ac.chargers[]	index	name, station_id, charger_type, in_service, p_rated_kw, p_ch_max_kw, p_ch_min_kw, eta, v2g_capable, p_dis_max_kw, mtbf_hours, mttr_hours
Asynchronous Motor	ac.motors[]	index, bus	name, in_service, machine ratings, direct / zero-sequence parameters, locked-rotor data, and efficiency metadata

Model	JSON path	Required keys on import	Other exported keys
ExternalGrid	ac.external_grid_s[]	index, bus	name, in_service, vm_pu, va_deg, s_sc_max_mva, s_sc_min_mva, rx_max, rx_min, r_pu, x_pu, r0_pu, x0_pu, vn_kv, controllable
Transformer2W	ac.transformers_2w[]	index, hv_bus, lv_bus	name, std_type, in_service, sn_mva, vn_hv_kv, vn_lv_kv, vk_percent, vkr_percent, pk_kw, pfe_kw, i0_percent, tap_side, tap_pos, tap_min, tap_max, tap_neutral, tap_step_percent, shift_deg, vector_group, z0_percent, x0_r0, mtbf_hours, mtrr_hours
Transformer3W	ac.transformers_3w[]	index, hv_bus, mv_bus, lv_bus	name, std_type, in_service, sn_hv_mva, sn_mv_mva, sn_lv_mva, vn_hv_kv, vn_mv_kv, vn_lv_kv, vk_hv_mv_percent, vk_hv_lv_percent, vk_mv_lv_percent, vkr_hv_mv_percent, vkr_hv_lv_percent, vkr_mv_lv_percent, pfe_kw, i0_percent, tap_side, tap_pos, tap_step_percent, shift_mv_deg, shift_lv_deg, mtbf_hours, mtrr_hours

Generation, storage, and hybrid JSON objects

Model	JSON path	Required keys on import	Other exported keys
Generator	ac.generators[]	index, bus	in_service, pg_mw, qg_mvar, vg_pu, pmax_mw, pmin_mw, qmax_mvar, qmin_mvar, is_slack, cost_c2, cost_c1, cost_c0, name, fuel_type, startup_cost, shutdown_cost, min_up_time_hr, min_dn_time_hr, ramp_up_mw_min, ramp_dn_mw_min, mbase_mva, inertia_h, droop_r, xd_pu, xdp_pu, xdpp_pu, td0p_s, td0pp_s, emission_factor_tco2_mwh, nox_factor_kg_mwh, so2_factor_kg_mwh, forced_outage_rate, mtrr_hr
StaticGenerator	ac.static_generators[] or dc.static_generators[]	index, bus	in_service, name, sgen_type, p_mw, q_mvar, p_rated_mw, sn_mva, pmax_mw, pmin_mw, qmax_mvar, qmin_mvar, scaling, controllable, v_ref_pu, k_p, k_q, k, rx, co2_emission_rate, mtbf_hours, mtrr_hours
RenewableGen	ac.renewable_gen_s[]	index, bus	in_service, name, type, p_mw, q_mvar, p_rated_mw, qmax_mvar, qmin_mvar, curtailable, cost_curtail_mwh, capacity_factor, profile_id, emission_offset_tco2_mwh

Model	JSON path	Required keys on import	Other exported keys
PVSystem	ac.pv_systems[]	index, bus	in_service, name, p_mw, q_mvar, sn_mva, pmax_mw, pmin_mw, qmax_mvar, qmin_mvar, control_mode, controller, v_ac_set_pu, v_dc_set_pu, inverter_eff, loss_percent, num_series, num_parallel, vmpp, impp, voc, isc, alpha_isc, beta_voc, irradiance, temperature, mtbf_panel_hours, mttr_panel_hours, mtbf_inverter_hours, mttr_inverter_hours
Storage	ac.storage[] or dc.storage[]	index, bus	in_service, name, p_mw, q_mvar, p_rated_mw, pmax_mw, pmin_mw, qmax_mvar, qmin_mvar, e_rated_mwh, soc_init, soc_min, soc_max, soc_carbon_intensity_tco2_mwh, eta_charge, eta_discharge, self_discharge_pct, max_cycles, current_cycles, soh, replacement_cost, forced_outage_rate, mttr_hr
DCBus	dc.buses[]	index	bus_type, vm_pu, vmax_pu, vmin_pu, pd_mw, in_service, name
DCBranch	dc.branches[]	index, from_bus, to_bus	r_pu, rate_a_mva, in_service, name, length_km
VSCConverter	vsc_converters[]	index, bus_ac, bus_dc	in_service, control_mode, p_set_mw, q_set_mvar, v_dc_set_pu, v_ac_set_pu, eta, loss_percent, loss_mw, k_vdc, pmax_mw, pmin_mw, qmax_mvar, qmin_mvar, p_rated_mw, name, forced_outage_rate, mttr_hr
DCDCConverter	dcdc_converters[]	index, bus_in, bus_out	in_service, name, control_mode, p_ref_mw, v_ref_pu, sn_mva, vn_in_kv, vn_out_kv, eta, r_eq_pu, pmax_mw, pmin_mw, k_droop, mtbf_hours, mttr_hours
EnergyRouterPort	energy_routers[] . ports[]	index, bus	name, port_type, voltage_level_kv, p_mw, q_mvar, v_pu, pmax_mw, pmin_mw, qmax_mvar, qmin_mvar, control_mode, p_set_mw, q_set_mvar, v_set_pu, in_service
EnergyRouter	energy_routers[]	index	name, in_service, router_type, num_ports, ports, p_rated_mw, vn_ac_kv, vn_dc_kv, loss_percent, pmax_mw, pmin_mw, qmax_mvar, qmin_mvar, mtbf_hours, mttr_hours
MobileStorage	mobile_storage[]	index, bus	in_service, name, p_mw, q_mvar, p_rated_mw, pmax_mw, pmin_mw, qmax_mvar, qmin_mvar, e_rated_mwh, soc_init, soc_min, soc_max, eta_charge, eta_discharge, max_cycles, current_cycles, soh, is_mobile, status, current_location, target_bus, e_consumption_mwh_km, max_travel_distance_km, mtbf_hours, mttr_hours

Model	JSON path	Required keys on import	Other exported keys
VirtualPower Plant	vpps[]	index	name, description, aggregation_bus, resource-count fields, aggregate capability / output terms, ramp limits, reliability metadata, and in_service
Microgrid	microgrids[]	index	name, description, in_service, pcc_bus, internal_buses, operating_mode, islanding_capability, auto_reconnection, p_exchange_max_mw, p_exchange_min_mw, p_import_max_mw, p_export_max_mw, p_exchange_mw, total_generation_mw, total_storage_mwh, total_load_mw, capacity_mw, peak_load_mw, f_set_hz, v_set_pu, k_droop, area, mtbf_hours, mttr_hours
ThreePhase ACSystem	three_phase_ac	none	name, base_mva, base_freq_hz, and component arrays for buses, lines, transformers, loads, generators, and external grids
ThreePhaseAC Bus	three_phase_ac.buses[]	index	bus_type, name, base_kv, in_service, vm_a_pu, va_a_deg, vm_b_pu, va_b_deg, vm_c_pu, va_c_deg, pd_a_mw, qd_a_mvar, pd_b_mw, qd_b_mvar, pd_c_mw, qd_c_mvar, gs_a_mw, bs_a_mvar, gs_b_mw, bs_b_mvar, gs_c_mw, bs_c_mvar, vmin_pu, vmax_pu, area, zone
ThreePhaseAC Line	three_phase_ac.lines[]	index, from_bus, to_bus	name, in_service, length_km, parallel, r1_ohm_per_km, x1_ohm_per_km, c1_nf_per_km, r0_ohm_per_km, x0_ohm_per_km, c0_nf_per_km, r1_pu, x1_pu, b1_pu, r0_pu, x0_pu, b0_pu, max_i_ka, rate_a_mva, failure_rate, mttr_hr
ThreePhase Transformer	three_phase_ac.transformers[]	index, hv_bus, lv_bus	name, in_service, nameplate data, leakage and magnetizing terms, zero-sequence data, tap settings, vector group, and reliability metadata
ThreePhaseLoad	three_phase_ac.loads[]	index, bus	name, in_service, connection, grounded, p_a_mw, q_a_mvar, p_b_mw, q_b_mvar, p_c_mw, q_c_mvar, const_z_percent, const_i_percent, const_p_percent, motor_percent, lrc_pu, x_r_ratio
ThreePhase Generator	three_phase_ac.generators[]	index, bus	name, in_service, is_slack, dispatch setpoints / limits, voltage target, machine base, and sequence reactance terms
ThreePhase ExternalGrid	three_phase_ac.external_grids[]	index, bus	name, in_service, voltage target, short-circuit level bounds, R/X bounds, and sequence Thevenin terms

Result JSON objects

JSON artifact	Top-level keys	Current note
ACOPFResult JSON	converged, iterations, outer_iterations, objective, max_constraint_violation, max_stationarity, status, vm, va, pg_mw, qg_mvar, dpd_mw, dqd_mvar, pac_mw, qac_mvar, profiling	profiling is a nested object containing linear_solver_backend, analyze_calls, factorization_calls, linear_solve_calls, total_iterations, accepted_steps, rejected_steps, final_barrier_mu.
CarbonAnalysisResult JSON	tracing_verified, matrix_solved, matrix_residual, load_carbon, dc_load_carbon, branch_carbon, dc_branch_carbon, bus_carbon, dc_bus_carbon, vsc_carbon, dcdc_carbon, storage_carbon, energy_router_carbon, generator_loss_allocation, tracing_summary, matrix_summary	AC/DC load arrays carry the same scalar fields plus nested generator_supply_mw; branch / converter / router arrays expose loss and direction fields; storage_carbon adds charge/discharge-side storage semantics including traced charging intensity.

9.4 Excel Input Workbook Contract

Input-sheet catalog

Sheet	Primary model / load target	Columns	Current note
Info	global metadata	Key, Value	loader recognizes name, base_mva, ac_freq_hz, three_phase_name, three_phase_base_mva, and three_phase_base_freq_hz.
ACBus	ACBus	index, name, bus_type, pd_mw, qd_mvar, vm_pu, va_deg, base_kv, vmin_pu, vmax_pu, gs_mw, bs_mvar, area, zone, in_service, n_customers, importance, latitude, longitude	write path uses aggregated AC demand, not necessarily the raw bus fields.
ACBranch	ACBranch	index, name, from_bus, to_bus, r_pu, x_pu, b_pu, tap, shift_deg, rate_a_mva, rate_b_mva, rate_c_mva, length_km, in_service, failure_rate, mttr_hr	near-1:1 with JSON coverage.
Generator	Generator	index, name, bus, pg_mw, qg_mvar, vg_pu, pmax_mw, pmin_mw, qmax_mvar, qmin_mvar, is_slack, fuel_type, mbase_mva, cost_c2, cost_c1, cost_c0, in_service, inertia_h, droop_r, ramp_up_mw_min, ramp_dn_mw_min	Excel omits the deeper machine, emissions, and commitment fields that JSON keeps.
Load	Load	index, name, bus, p_mw, q_mvar, model, z_percent_p, i_percent_p, p_percent_p, z_percent_q, i_percent_q, p_percent_q, controllable, priority, cost_mw, in_service	raw explicit AC load table.

Sheet	Primary model / load target	Columns	Current note
ACLoad	fallback Load import; aggregated export view	same columns as Load	written as a derived table that aggregates Load, charging stations, charging storage, VPP imports, microgrid imports, and AC router ports.
FlexibleLoad	FlexibleLoad	index, name, bus, p_mw, q_mvar, flex_up_mw, flex_down_mw, flex_duration_h, response_time_s, ramp_rate_mw_min, availability_pct, controllable, priority, control_area, in_service	dedicated Excel input sheet now exists; these rows are still converted to canonical Load equivalents before PF/OPF/SC assembly.
AsymmetricLoad	AsymmetricLoad	index, name, bus, connection, grounded, paRated_mw, qaRated_mvar, pbRated_mw, qbRated_mvar, pcRated_mw, qcRated_mvar, pa_mw, qa_mvar, pb_mw, qb_mvar, pc_mw, qc_mvar, scaling, const_z_percent, const_i_percent, const_p_percent, controllable, priority, in_service	dedicated Excel input sheet now exists; solver-side use still relies on projection to a single Load equivalent.
StaticGenerator	AC StaticGenerator	index, name, bus, sgen_type, p_mw, q_mvar, pRated_mw, sn_mva, pmax_mw, pmin_mw, qmax_mvar, qmin_mvar, scaling, controllable, in_service	Excel omits control, SC, emissions, and reliability metadata.
Storage	AC Storage	index, name, bus, p_mw, q_mvar, pRated_mw, pmax_mw, pmin_mw, qmax_mvar, qmin_mvar, eRated_mwh, soc_init, soc_min, soc_max, eta_charge, eta_discharge, in_service	Excel omits life-cycle and outage fields.
RenewableGen	RenewableGen	index, name, bus, type, p_mw, q_mvar, pRated_mw, qmax_mvar, qmin_mvar, curtailable, capacity_factor, in_service	Excel omits cost_curtail_mwh, profile_id, and emission-offset metadata.
PVSystem	PVSystem	index, name, bus, p_mw, q_mvar, sn_mva, pmax_mw, pmin_mw, qmax_mvar, qmin_mvar, control_mode, inverter_eff, in_service	Excel keeps only a narrow operational subset; JSON is much richer.
Shunt	Shunt	index, name, bus, gs_mw, bs_mvar, switchable, n_steps, current_step, bs_per_step, in_service	near-1:1 with JSON coverage.
ExternalGrid	ExternalGrid	index, name, bus, vm_pu, va_deg, s_sc_max_mva, s_sc_min_mva, rx_max, rx_min, controllable, in_service	Excel omits explicit Thevenin impedance fields present in JSON.
Transformer2W	Transformer2W	index, name, hv_bus, lv_bus, sn_mva, vn_hv_kv, vn_lv_kv, vk_percent, vkr_percent, pfe_kw, i0_percent, tap_side, tap_pos, tap_min, tap_max, tap_neutral, tap_step_percent, shift_deg, vector_group, in_service	Excel omits std_type, pk_kw, zero-sequence, and reliability fields.

Sheet	Primary model / load target	Columns	Current note
Transformer3W	Transformer3W	index, name, std_type, hv_bus, mv_bus, lv_bus, sn_hv_mva, sn_mv_mva, sn_lv_mva, vn_hv_kv, vn_mv_kv, vn_lv_kv, vk_hv_mv_percent, vk_hv_lv_percent, vk_mv_lv_percent, vkr_hv_mv_percent, vkr_hv_lv_percent, vkr_mv_lv_percent, pfe_kw, i0_percent, tap_side, tap_pos, tap_step_percent, shift_mv_deg, shift_lv_deg, mtbf_hours, mttr_hours, in_service	dedicated Excel input sheet now exists; solver-side treatment still projects the device into equivalent branches.
ChargingStation	ChargingStation	index, name, bus, n_fast, n_slow, num_chargers, p_fast_max_kw, p_slow_max_kw, max_power_kw, simultaneity_factor, power_factor, p_total_kw, q_total_kvar, in_service	Excel omits location, utilization_rate, and reliability fields.
Charger	Charger	index, name, station_id, charger_type, in_service, p_rated_kw, p_ch_max_kw, p_ch_min_kw, eta, v2g_capable, p_dis_max_kw, mtbf_hours, mttr_hours	dedicated Excel input sheet now exists; charger rows are linked to their parent station by station_id.
Asynchronous Motor	Asynchronous Motor	index, name, bus, machine ratings, direct / zero-sequence parameters, SC motor data, and in_service	dedicated Excel input sheet now exists for short-circuit motor data.
Switch	Switch	index, name, bus_from, bus_to, switch_type, closed, in_service	Excel keeps only the topology/status subset.
DCBus	DCBus	index, name, bus_type, vm_pu, vmax_pu, vmin_pu, pd_mw, in_service	write path uses aggregated DC demand.
DCLoad	fallback DCLoad import; aggregated export view	index, name, bus, p_mw, controllable, p_min_mw, cost_mw, in_service	written as derived aggregated DC demand.
DCLoadRaw	raw DCLoad table	same columns as DCLoad	preserves the unaggregated DC load objects and is preferred on import.
DCBranch	DCBranch	index, name, from_bus, to_bus, r_pu, rate_a_mva, length_km, in_service	near-1:1 with JSON coverage.
DCStorage	DC Storage	same columns as Storage	same coverage limits as AC storage.
DCStaticGen	DC StaticGenerator	same columns as StaticGenerator	same coverage limits as AC static generators.
VSCConverter	VSCConverter	index, name, bus_ac, bus_dc, control_mode, p_set_mw, q_set_mvar, v_dc_set_pu, v_ac_set_pu, eta, loss_percent, pmax_mw, pmin_mw, qmax_mvar, qmin_mvar, p_rated_mw, in_service	Excel omits loss_mw, k_vdc, and outage metadata.
DCDCConverter	DCDCConverter	index, name, bus_in, bus_out, control_mode, p_ref_mw, v_ref_pu, sn_mva, eta, pmax_mw, pmin_mw, in_service	Excel omits voltage-rating, droop, resistance, and reliability fields.
EnergyRouter	EnergyRouter	index, name, router_type, num_ports, p_rated_mw, vn_ac_kv, vn_dc_kv, loss_percent, pmax_mw, pmin_mw, qmax_mvar, qmin_mvar, mtbf_hours, mttr_hours, in_service	router-level metadata now has a dedicated Excel sheet.

Sheet	Primary model / load target	Columns	Current note
EnergyRouter Port	EnergyRouter Port	router_index, index, name, bus, port_type, voltage_level_kv, p_mw, q_mvar, v_pu, pmax_mw, pmin_mw, qmax_mvar, qmin_mvar, control_mode, p_set_mw, q_set_mvar, v_set_pu, in_service	port rows are matched back to their parent router through router_index.
VPP	VirtualPower Plant	index, name, aggregation_bus, aggregate output / limit snapshot, selected resource counts, and in_service	Excel omits description, EV/load counts, aggregate capability, ramp rates, and reliability.
Microgrid	Microgrid	index, name, pcc_bus, operating_mode, islanding_capability, p_exchange_mw, p_exchange_max_mw, total_generation_mw, total_load_mw, in_service	Excel omits internal_buses and most planning/control metadata.
MobileStorage	MobileStorage	index, name, bus, status, p_mw, q_mvar, p_rated_mw, e_rated_mwh, soc_init, in_service	Excel keeps only a snapshot subset and omits most mobility/limit metadata.
ThreePhaseBus	ThreePhaseAC Bus	index, name, bus_type, base_kv, vm_a_pu, va_a_deg, vm_b_pu, va_b_deg, vm_c_pu, va_c_deg, pd_a_mw, qd_a_mvar, pd_b_mw, qd_b_mvar, pd_c_mw, qd_c_mvar, gs_a_mw, bs_a_mvar, gs_b_mw, bs_b_mvar, gs_c_mw, bs_c_mvar, vmin_pu, vmax_pu, area, zone, in_service	explicit unbalanced bus table for the optional three_phase_ac subsystem.
ThreePhaseLine	ThreePhaseAC Line	index, name, from_bus, to_bus, length_km, parallel, r1_ohm_per_km, x1_ohm_per_km, c1_nf_per_km, r0_ohm_per_km, x0_ohm_per_km, c0_nf_per_km, r1_pu, x1_pu, b1_pu, r0_pu, x0_pu, b0_pu, max_i_ka, rate_a_mva, failure_rate, mttr_hr, in_service	sequence-parameter line sheet for explicit unbalanced input.
ThreePhase Transformer	ThreePhase Transformer	index, name, buses, nameplate data, leakage / zero-sequence terms, tap settings, reliability metadata, and in_service	explicit three-phase transformer input sheet; solver path still projects it to Transformer2W.
ThreePhaseLoad	ThreePhaseLoad	index, name, bus, connection, grounded, p_a_mw, q_a_mvar, p_b_mw, q_b_mvar, p_c_mw, q_c_mvar, const_z_percent, const_i_percent, const_p_percent, motor_percent, lrc_pu, x_r_ratio, in_service	explicit unbalanced demand table; projected before main PF/OPF/SC assembly.
ThreePhase Generator	ThreePhase Generator	index, name, bus, slack flag, dispatch setpoints / limits, sequence reactance terms, and in_service	explicit unbalanced generator sheet with sequence impedance metadata.
ThreePhase ExternalGrid	ThreePhase ExternalGrid	index, name, bus, voltage target, SC level bounds, sequence impedance terms, and in_service	explicit three-phase grid-equivalent sheet.

Remaining field-level Excel caveats

Area	Current Excel status
Load short-circuit motor subfields	the embedded SC fields <code>sn_mva</code> , <code>motor_percent</code> , <code>x_sub_pu</code> , and <code>r_sc_pu</code> still do not have dedicated Load/ACLoad workbook columns.
<code>ThreePhaseACSystem</code> metadata container	subsystem metadata is carried in <code>Info</code> keys rather than a dedicated <code>ThreePhaseInfo</code> sheet.

JSON Key / Excel Column Dictionary for the Newly Added Contracts

The next tables are intentionally 1:1 with the current serializer code. The “JSON default” text below means the value used by `from_json` when a key is absent. Excel uses the exact sheet and column names shown here.

Optional three-phase subsystem metadata: `three_phase_ac` and `Info`

Field	Excel sheet / column	JSON default / import rule	Meaning
<code>name</code>	<code>Info[three_phase_name]</code>	default "Three-Phase ACSystem"	user-visible subsystem name for the optional explicit unbalanced AC network
<code>base_mva</code>	<code>Info[three_phase_base_mva]</code>	default 100.0	per-subsystem power base; independent from top-level <code>base_mva</code>
<code>base_freq_hz</code>	<code>Info[three_phase_base_freq_hz]</code>	default 50.0	nominal subsystem frequency in Hz
<code>buses[]</code>	sheet <code>ThreePhaseBus</code>	missing array / sheet keeps vector empty	explicit three-phase bus family
<code>lines[]</code>	sheet <code>ThreePhaseLine</code>	missing array / sheet keeps vector empty	explicit three-phase line family
<code>transformers[]</code>	sheet <code>ThreePhaseTransformer</code>	missing array / sheet keeps vector empty	explicit three-phase transformer family
<code>loads[]</code>	sheet <code>ThreePhaseLoad</code>	missing array / sheet keeps vector empty	explicit unbalanced load family
<code>generators[]</code>	sheet <code>ThreePhaseGenerator</code>	missing array / sheet keeps vector empty	explicit three-phase generator family
<code>external_grids[]</code>	sheet <code>ThreePhaseExternalGrid</code>	missing array / sheet keeps vector empty	explicit sequence-based grid-equivalent family

Asynchronous motor contract: `ac.motors[]` and sheet `AsynchronousMotor`

Field	Excel column	JSON default / import rule	Meaning
<code>index</code>	<code>index</code>	required on JSON import; blank Excel <code>index</code> stops table import	stable device identifier
<code>bus</code>	<code>bus</code>	required on JSON import	connected AC bus index
<code>name</code>	<code>name</code>	default empty string	display label only
<code>in_service</code>	<code>in_service</code>	default <code>true</code>	availability flag; not a solved PF state
<code>vn_kv</code>	<code>vn_kv</code>	default 0.0	rated terminal voltage in kV
<code>sn_mva</code>	<code>sn_mva</code>	default 0.0	rated apparent power used in motor fault-contribution scaling

Field	Excel column	JSON default / import rule	Meaning
r_pu	r_pu	default 0.0	motor equivalent positive-sequence resistance input
x_pu	x_pu	default 0.0	motor equivalent positive-sequence reactance input
x_r	x_r	default 0.0	X/R ratio used in short-circuit contribution calculations
lrc	lrc	default 0.0	locked-rotor current ratio
poles	poles	default 2	machine pole count
cos_phi	cos_phi	default 0.85	rated power factor
efficiency	efficiency	default 0.95	rated efficiency factor
r0_pu	r0_pu	default 0.0	zero-sequence resistance input for unsymmetrical faults
x0_pu	x0_pu	default 0.0	zero-sequence reactance input for unsymmetrical faults

Three-phase bus contract: `three_phase_ac.buses[]` and sheet `ThreePhaseBus`

Field	Excel column	JSON default / import rule	Meaning
index	index	required on JSON import; blank Excel <code>index</code> stops table import	stable bus identifier
name	name	default empty string	bus label
bus_type	bus_type	default PQ	bus category using the same string tokens as the AC bus model
base_kv	base_kv	default 0.0	nominal line-to-line bus voltage base
in_service	in_service	default <code>true</code>	availability flag
vm_a_pu, vm_b_pu, vm_c_pu	same columns	default 1.0 each	per-phase initial or specified voltage magnitudes
va_a_deg, va_b_deg, va_c_deg	same columns	defaults 0.0, -120.0, 120.0	phase-angle initialization in degrees for phase order a, b, c
pd_a_mw, pd_b_mw, pd_c_mw	same columns	default 0.0	explicit per-phase active demand before projection
qd_a_mvar, qd_b_mvar, qd_c_mvar	same columns	default 0.0	explicit per-phase reactive demand before projection
gs_a_mw, gs_b_mw, gs_c_mw	same columns	default 0.0	per-phase conductance shunt terms
bs_a_mvar, bs_b_mvar, bs_c_mvar	same columns	default 0.0	per-phase susceptance shunt terms
vmin_pu	vmin_pu	default 0.9	lower per-unit voltage bound
vmax_pu	vmax_pu	default 1.1	upper per-unit voltage bound
area	area	default 1	planning / reporting area tag
zone	zone	default 1	planning / reporting zone tag

Three-phase line contract: `three_phase_ac.lines[]` and sheet `ThreePhaseLine`

Field	Excel column	JSON default / import rule	Meaning
<code>index</code>	<code>index</code>	required on JSON import; blank Excel <code>index</code> stops table import	stable line identifier
<code>from_bus, to_bus</code>	same columns	required on JSON import	line terminal bus indices
<code>name</code>	<code>name</code>	default empty string	line label
<code>in_service</code>	<code>in_service</code>	default <code>true</code>	availability flag
<code>length_km</code>	<code>length_km</code>	default 0.0	physical route length
<code>parallel</code>	<code>parallel</code>	default 1	number of parallel circuits represented by the row
<code>r1_ohm_per_km, x1_ohm_per_km, c1_nf_per_km</code>	same columns	default 0.0	positive-sequence per-km impedance / capacitance data
<code>r0_ohm_per_km, x0_ohm_per_km, c0_nf_per_km</code>	same columns	default 0.0	zero-sequence per-km impedance / capacitance data
<code>r1_pu, x1_pu, b1_pu</code>	same columns	default 0.0	positive-sequence per-unit branch parameters
<code>r0_pu, x0_pu, b0_pu</code>	same columns	default 0.0	zero-sequence per-unit branch parameters
<code>max_i_ka</code>	<code>max_i_ka</code>	default 0.0	thermal current limit in kA
<code>rate_a_mva</code>	<code>rate_a_mva</code>	default 0.0	apparent-power rating used by planning / screening logic
<code>failure_rate</code>	<code>failure_rate</code>	default 0.0	reliability failure-rate input
<code>mttr_hr</code>	<code>mttr_hr</code>	default 0.0	repair-time input in hours

Three-phase transformer contract: `three_phase_ac.transformers[]` and sheet `ThreePhaseTransformer`

Field	Excel column	JSON default / import rule	Meaning
<code>index</code>	<code>index</code>	required on JSON import; blank Excel <code>index</code> stops table import	stable transformer identifier
<code>hv_bus, lv_bus</code>	same columns	required on JSON import	high-voltage and low-voltage terminal bus indices
<code>name</code>	<code>name</code>	default empty string	transformer label
<code>in_service</code>	<code>in_service</code>	default <code>true</code>	availability flag
<code>sn_mva</code>	<code>sn_mva</code>	default 0.0	rated apparent power
<code>vn_hv_kv, vn_lv_kv</code>	same columns	default 0.0	rated winding voltages
<code>vk_percent, vkr_percent</code>	same columns	default 0.0	short-circuit magnitude and resistive share in percent
<code>pfe_kw, i0_percent</code>	same columns	default 0.0	no-load loss and excitation-current inputs
<code>vector_group</code>	<code>vector_group</code>	default empty string	winding connection / clock notation for zero-sequence path interpretation
<code>vk0_percent, vkr0_percent, mag0_percent, mag0_rx, si0_hv_partial</code>	same columns	defaults 0.0 except <code>si0_hv_partial=0.5</code>	zero-sequence transformer data preserved for SC and future unbalanced workflows
<code>tap_side, tap_pos, tap_min, tap_max, tap_neutral</code>	same columns	default 0	tap-changer placement and discrete tap position meta-data

Field	Excel column	JSON default / import rule	Meaning
tap_step_percent	tap_step_percent	default 0.0	tap step size in percent
shift_deg	shift_deg	default 0.0	phase shift angle in degrees
mtbf_hours, mtrt_h ours	same columns	default 0.0	reliability inputs retained in the explicit contract

Three-phase load contract: `three_phase_ac.loads[]` and sheet `ThreePhaseLoad`

Field	Excel column	JSON default / import rule	Meaning
index	index	required on JSON import; blank Excel <code>index</code> stops table import	stable load identifier
bus	bus	required on JSON import	connected three-phase bus index
name	name	default empty string	load label
in_service	in_service	default <code>true</code>	availability flag
connection	connection	default <code>"wye"</code>	wiring model; current code expects free-form connection text such as <code>wye</code> or <code>delta</code>
grounded	grounded	default <code>true</code>	grounding-path assumption for the connection
p_a_mw, p_b_mw, p_c_ mw	same columns	default 0.0	per-phase active demand
q_a_mvar, q_b_mvar, q_c_mvar	same columns	default 0.0	per-phase reactive demand
const_z_percent, const_i_percent	same columns	default 0.0	ZIP-model constant-impedance and constant-current shares
const_p_percent	const_p_percent	default 100.0	ZIP-model constant-power share
motor_percent	motor_percent	default 0.0	embedded motor-fraction share inside the unbalanced load object
lrc_pu	lrc_pu	default 0.0	locked-rotor current ratio for the embedded motor fraction
x_r_ratio	x_r_ratio	default 0.0	X/R ratio for the embedded motor fraction

Three-phase generator contract: `three_phase_ac.generators[]` and sheet `ThreePhaseGenerator`

Field	Excel column	JSON default / import rule	Meaning
index	index	required on JSON import; blank Excel <code>index</code> stops table import	stable generator identifier
bus	bus	required on JSON import	connected three-phase bus index
name	name	default empty string	generator label
in_service	in_service	default <code>true</code>	availability flag
is_slack	is_slack	default <code>false</code>	slack-machine indicator for explicit three-phase studies

Field	Excel column	JSON default / import rule	Meaning
p_mw, q_mvar	same columns	default 0.0	current active / reactive injection setpoints
vm_pu	vm_pu	default 1.0	terminal voltage target or initialization
pmax_mw, pmin_mw, qmax_mvar, qmin_mvar	same columns	default 0.0	capability limits
mbase_mva	mbase_mva	default 0.0	machine MVA base for the sequence reactances
xd_pu, xdpp_pu, x2_pu, x0_pu, r0_pu	same columns	default 0.0	positive / subtransient / negative / zero-sequence machine impedance set

Three-phase external-grid contract: `three_phase_ac.external_grids[]` and sheet `ThreePhaseExternalGrid`

Field	Excel column	JSON default / import rule	Meaning
index	index	required on JSON import; blank Excel <code>index</code> stops table import	stable external-grid identifier
bus	bus	required on JSON import	connected three-phase bus index
name	name	default empty string	source label
in_service	in_service	default <code>true</code>	availability flag
vm_pu	vm_pu	default 1.0	Thevenin voltage magnitude target
va_deg	va_deg	default 0.0	Thevenin voltage angle in degrees
s_sc_max_mva, s_sc_min_mva	same columns	default 0.0	short-circuit strength limits
rx_max, rx_min	same columns	default 0.0	R/X ratio bounds used in equivalent-source parametrization
r1_pu, x1_pu	same columns	default 0.0	positive-sequence Thevenin impedance
r2_pu, x2_pu	same columns	default 0.0	negative-sequence Thevenin impedance
r0_pu, x0_pu	same columns	default 0.0	zero-sequence Thevenin impedance

9.5 Compact Power-Flow Workbook Additions

Sheet updated by <code>save_results_compact_xlsx</code>	Appended columns	Current note
ACBus	result_vm_pu, result_va_deg	solved AC bus state.
DCBus	result_vdc_pu	solved DC voltage.
ACBranch	result_pf_mw, result_qf_mvar, result_pt_mw, result_qt_mvar	branch terminal powers.

Sheet updated by save_results_c ompact_xlsx	Appended columns	Current note
DCBranch	result_v_from_pu, result_v_to_pu, result_i_pu, result_p_from_mw, result _p_to_mw, result_loss_mw	derived from solved DC bus voltages and branch resistance.
Storage	result_vm_pu, result_va_deg	bus-state annotation only; storage dispatch is not recomputed.
DCStorage	result_vdc_pu	bus-state annotation only.
Load	result_vm_pu, result_va_deg	bus-state annotation on the raw AC load table.
ACLoad	result_vm_pu, result_va_deg	bus-state annotation on the aggregated AC load view.
DCLoad	result_vdc_pu	bus-state annotation on the aggregated DC load view.
ChargingStation	result_vm_pu, result_va_deg	station bus state.
VSCConverter	result_p_ac_mw, result_q_ac_mvar, result_p_dc_mw, result_loss_mw, resu lt_v_ac_pu, result_v_dc_pu	solved converter trans- fer and terminal volt- ages.
DCDCConverter	result_p_in_mw, result_p_out_mw, result_loss_mw, result_v_in_pu, resu lt_v_out_pu	solved converter trans- fer and terminal volt- ages.
VPP	result_vm_pu, result_va_deg	aggregation-bus anno- tation only.
Microgrid	result_vm_pu, result_va_deg	PCC-bus annotation only.
MobileStorage	result_vm_pu, result_va_deg	current bus annota- tion only.
Results_Summary	Key, Value	key-value sheet with converged, iteratio ns, and residual.

9.6 Dedicated Result Workbooks

Dedicated power-flow result sheets

Sheet	Columns	Current note
Results_ACBus	index, name, vm_pu, va_deg	solved AC bus state.

Sheet	Columns	Current note
Results_DCBus	index, name, vm_pu	this sheet uses the AC-style column name <code>vm_pu</code> even though the value is the DC solved voltage.
Results_Branch Flow	index, from_bus, to_bus, pf_mw, qf_mvar, pt_mw, qt_mvar	branch flows reported in branch order.
Results_DCBranch	index, name, from_bus, to_bus, in_service, v_from_pu, v_to_pu, i_pu, p_from_mw, p_to_mw, loss_mw	computed from solved voltages and branch resistance.
Results_VSCConverter	index, name, bus_ac, bus_dc, in_service, control_mode, p_ac_mw, q_ac_mvar, p_dc_mw, loss_mw, p_set_mw, q_set_mvar, v_ac_pu, v_dc_pu, eta, loss_percent	includes both solved transfer and original setpoint metadata.
Results_DCDCCoconverter	index, name, bus_in, bus_out, in_service, control_mode, p_ref_mw, v_ref_pu, v_in_pu, v_out_pu, eta, p_in_mw, p_out_mw, loss_mw	includes both solved transfer and original reference metadata.
Results_Storage_AC	index, name, bus, in_service, p_mw, q_mvar, soc, soh, vm_pu, va_deg	uses <code>soc_init</code> as the reported <code>soc</code> .
Results_Storage_DC	index, name, bus, in_service, p_mw, q_mvar, soc, soh, vdc_pu	uses <code>soc_init</code> as the reported <code>soc</code> .
Results_ChargingStation	index, name, bus, in_service, num_chargers, p_total_kw, q_total_kvar, max_power_kw, utilization_rate, vm_pu, va_deg	station bus state plus current aggregate demand fields.
Results_Charger	index, name, station_id, station_bus, in_service, charger_type, p_rated_kw, p_ch_max_kw, p_dis_max_kw, eta, v2g_capable, vm_pu, va_deg	<code>charger_type</code> now uses the same string token family as JSON and workbook input sheets.
Results_EnergyRouter	index, name, in_service, router_type, num_ports, p_rated_mw, loss_percent, p_port_sum_mw, q_port_sum_mvar	device-level summary only; solved per-port values live in the port sheet.
Results_EnergyRouterPort	router_index, port_index, port_name, bus, port_type, control_mode, in_service, p_mw, q_mvar, v_solved_pu	flattens nested router ports into one table.
Results_VPP	index, name, aggregation_bus, in_service, p_output_mw, q_output_mvar, pmax_mw, pmin_mw, vm_pu, va_deg	aggregation-bus annotation only.
Results_Microgrid	index, name, pcc_bus, operating_mode, in_service, p_exchange_mw, total_generation_mw, total_load_mw, vm_pu, va_deg, internal_buses	<code>internal_buses</code> is written as one comma-separated string.

Sheet	Columns	Current note
Results_MobileStorage	index, name, bus, status, in_service, p_mw, q_mvar, soc, target_bus, vm_pu, va_deg	uses <code>soc_init</code> as the reported soc.
Results_Summary	Key, Value	keys are <code>converged</code> , <code>iterations</code> , and <code>residual</code> .

Result-Sheet Enum and Flag Conventions

Artifact	Field	Output representation	Field note
Results_VSCConverter	control_mode	string token: PQ, VDC_Q, VDC_VAC	copied from the input converter mode; not a solver-selected mode switch
Results_DCDCConverter	control_mode	string token: Voltage, Power, Droop	copied from the input converter mode
Results_Charger	charger_type	string token: AC_L1, AC_L2, DC_Fast	now aligned with JSON and workbook input instead of numeric enum codes
Results_EnergyRouterPort	port_type	string token: AC or DC	physical port side classification
Results_EnergyRouterPort	control_mode	string token: PQ, VF, Droop	copied from the input port control mode
Results_Microgrid	operating_mode	string token: Grid Connected, Islanded, Transition	reported operating mode metadata, not a PF state variable
Results_MobileStorage	status	string token: Stationary, InTransit, Deployed	reported mobility state metadata
Results_Charger	v2g_capable	boolean token: TRUE / FALSE	static capability flag, not current dispatch direction
Results_* device sheets	in_service	boolean token: TRUE / FALSE	copied device availability flag across result workbooks
Results_Summary, OPF_Summary	converged	boolean token: TRUE / FALSE	solver convergence outcome

Dedicated OPF and carbon result sheets

Sheet	Columns	Current note
OPF_Bus	index, name, base_kv, vm_pu, va_deg, dpd_mw, dqd_mvar	bus voltages plus bus-level load shedding vectors.
OPF_Generator	index, name, bus, pg_mw, qg_mvar, pmax_mw, pmin_mw, qmax_mvar, qmin_mvar, cost_c2, cost_c1, cost_c0, fuel_type	generator dispatch report.
OPF_Converter	index, name, bus_ac, bus_dc, pac_mw, qac_mvar	only VSC converters are reported here.
OPF_Summary	Key, Value	keys are converged, iterations, outer_iterations, objective, max_constraint_violation, max_stationarity, and status.
LoadCarbon	load_index, bus, demand_mw, carbon_intensity_tco2_mwh, total_emissions_tco2	dedicated carbon workbook, sheet 1.
DCLoadCarbon	load_index, bus, demand_mw, carbon_intensity_tco2_mwh, total_emissions_tco2	dedicated carbon workbook, sheet 2.
BusCarbon	bus_index, carbon_intensity_tco2_mwh	dedicated carbon workbook, sheet 3.
DCBusCarbon	bus_index, carbon_intensity_tco2_mwh	dedicated carbon workbook, sheet 4.
BranchCarbon	branch_index, from_bus, to_bus, loss_mw, carbon_intensity_tco2_mwh, total_emissions_tco2	dedicated carbon workbook, sheet 5.
DCBranchCarbon	branch_index, from_bus, to_bus, loss_mw, carbon_intensity_tco2_mwh, total_emissions_tco2	dedicated carbon workbook, sheet 6.
VSCCarbon	converter_index, bus_ac, bus_dc, ac_to_dc, input_power_mw, output_power_mw, loss_mw, carbon_intensity_tco2_mwh, total_emissions_tco2	dedicated carbon workbook, sheet 7.
DCDCCarbon	converter_index, bus_in, bus_out, input_to_output, input_power_mw, output_power_mw, loss_mw, carbon_intensity_tco2_mwh, total_emissions_tco2	dedicated carbon workbook, sheet 8.
StorageCarbon	storage_index, bus, is_dc, p_mw, soc, stored_energy_mwh, soc_carbon_intensity_tco2_mwh, carbon_intensity_tco2_mwh, total_emissions_tco2	dedicated carbon workbook sheet for explicit storage charge/discharge carbon semantics.

Sheet	Columns	Current note
EnergyRouterCarbon	<code>router_index</code> , <code>input_power_mw</code> , <code>output_power_mw</code> , <code>loss_mw</code> , <code>active_input_ports</code> , <code>active_output_ports</code> , <code>carbon_intensity_tco2_mwh</code> , <code>total_emissions_tco2</code>	dedicated carbon workbook sheet for router-internal loss allocation reconstructed from port transfers.
Summary	key-value blocks for tracing and matrix summaries	dedicated carbon workbook summary sheet; contains generation/load/loss totals, balance errors, and flags such as <code>tracing_verified</code> and <code>matrix_solved</code> .

9.7 Current I/O Coverage Gaps That Matter to Engineering Work

Public model area	JSON system file	Excel input workbook	Engineering consequence
ACBus, ACBranch, DCBus, DCBranch	yes	yes	core network objects are available in both I/O families.
Generator, Load, Shunt, ExternalGrid	yes	yes	Excel coverage is narrower than JSON for the richer machine / source metadata.
StaticGenerator, RenewableGen, PVSyst, Storage	yes	yes	Excel exposes only the operational subset; JSON is the better notebook-to-code contract for detailed studies.
Transformer2W	yes	yes	Excel omits zero-sequence and reliability fields.
Transformer3W	yes	yes	JSON and Excel now both carry the explicit 3-winding transformer data, although solver use still goes through projection.
FlexibleLoad, AsymmetricLoad	yes	yes	both I/O families now preserve the rich pre-projection load models.
ChargingStation	yes	yes	input support exists in both families.
Charger	yes	yes	both I/O families now preserve per-charger metadata; the parent station link remains <code>station_id</code> .
DCLoad	yes	yes	the main system JSON and Excel workbook are now aligned for the explicit DC load table.
VSCConverter, DCCDC Converter	yes	yes	Excel coverage is narrower than JSON.

Public model area	JSON system file	Excel input workbook	Engineering consequence
EnergyRouter , EnergyRouterPort	yes	yes	router studies can now use either JSON or Excel input, with ports flattened into a dedicated Excel table.
VPP, Microgrid, MobileStorage	yes	yes	Excel again keeps only a reduced operational subset.
AsynchronousMotor	yes	yes	the SC motor model now has a stable external contract in both families.
ThreePhaseACSystem family	yes	yes	the optional unbalanced subsystem can now round-trip through both JSON and Excel, although main PF/OPF/SC still consume it after projection.

9.8 Interpretation

The current I/O layer is already useful, but it is not a single canonical schema yet.

1. JSON is now the richest external representation for the full current **HybridPowerSystem** container, including **ac.motors** and optional **three_phase_ac**.
2. Excel is an engineering workbook interface with deliberate aggregation and reduced field coverage.
3. The JSON and Excel paths are now aligned for **DCLoad**, **Transformer3W**, **FlexibleLoad**, **AsymmetricLoad**, **Charger**, **EnergyRouter**, **AsynchronousMotor**, and the **ThreePhaseACSystem** family.
4. The remaining gaps are now field-level rather than family-level, especially the SC-specific motor-fraction fields embedded inside **Load** and the lack of dedicated result-workbook contracts for explicit three-phase solved states.

10 Detailed I/O Field Dictionaries

This section expands the per-field contract to the remaining public model families. Section 02a_io_contracts already gives detailed field maps for **AsynchronousMotor**, the optional **three_phase_ac** family, and the result-sheet enum conventions. The tables below complete the same JSON-key / Excel-column dictionary pattern for the other current input models.

When the “Excel sheet / column” entry says **none**, the engineering workbook does not preserve that JSON field directly. When two workbook sheet names are listed, the first is the preferred explicit input table and the second is a fallback or derived view with the same column family.

10.1 System Metadata

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
name	Info[name]	default "HybridAC/DC System"	top-level HybridPowerSystem display name

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
<code>base_mva</code>	<code>Info[base_mva]</code>	default 100.0	top-level system power base; workbook import also copies it into <code>ac.base_mva</code> and <code>dc.base_mva</code>
<code>ac.name</code>	none	default "ACSystem"	JSON-only AC subsystem label
<code>ac.base_mva</code>	none	default 100.0	JSON-only AC subsystem base; workbook uses the shared top-level <code>base_mva</code> instead
<code>ac.freq_hz</code>	<code>Info[ac_freq_hz]</code>	default 50.0	nominal AC frequency in Hz
<code>dc.name</code>	none	default "DCSystem"	JSON-only DC subsystem label
<code>dc.base_mva</code>	none	default 100.0	JSON-only DC subsystem base; workbook again uses the shared top-level <code>base_mva</code>
<code>three_phase_ac.*</code>	<code>Info[three_phase_*]</code> plus dedicated sheets	see Section 02a_io_contracts	optional explicit unbalanced subsystem documented in the earlier detailed dictionary

10.2 AC Network and Demand Models

ACBus : `ac.buses[]` / sheet **ACBus**

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
<code>index</code>	<code>ACBus.index</code>	required on JSON import; blank Excel <code>index</code> stops table import	stable bus identifier
<code>name</code>	<code>ACBus.name</code>	default empty string	bus label
<code>bus_type</code>	<code>ACBus.bus_type</code>	default PQ	bus category token PQ/PV/SLACK/ISOLATED
<code>pd_mw, qd_mvar</code>	same columns	default 0.0	embedded bus demand snapshot; workbook export may contain synchronized aggregated demand rather than original raw struct values
<code>vm_pu, va_deg</code>	same columns	defaults 1.0, 0.0	initial or specified bus voltage magnitude and angle
<code>base_kv</code>	<code>ACBus.base_kv</code>	default 110.0	nominal bus voltage base
<code>vmin_pu, vmax_pu</code>	same columns	defaults 0.9, 1.1	voltage operating bounds
<code>gs_mw, bs_mvar</code>	same columns	default 0.0	shunt conductance and susceptance embedded at the bus
<code>area, zone</code>	same columns	defaults 1, 1	planning / reporting partitions
<code>in_service</code>	<code>ACBus.in_service</code>	default true	availability flag
<code>n_customers</code>	<code>ACBus.n_customers</code>	default 0	customer-count metadata
<code>importance</code>	<code>ACBus.importance</code>	default 1.0	bus criticality / weighting metadata
<code>latitude, longitude</code>	same columns	default 0.0	geospatial metadata only

ACBranch : ac.branches[] / sheet ACBranch

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
index	ACBranch.index	required on JSON import; blank Excel index stops table import	stable branch identifier
from_bus, to_bus	same columns	required on JSON import	branch terminal bus indices
name	ACBranch.name	default empty string	branch label
r_pu, x_pu, b_pu	same columns	default 0.0	series resistance, series reactance, and total shunt charging in per unit
tap	ACBranch.tap	default 1.0	off-nominal tap ratio used in the branch model
shift_deg	ACBranch.shift_deg	default 0.0	phase-shifting angle in degrees
rate_a_mva, rate_b_mva, rate_c_mva	same columns	default 0.0	thermal / planning ratings
length_km	ACBranch.length_km	default 0.0	physical route length metadata
in_service	ACBranch.in_service	default true	availability flag
failure_rate, mttr_hr	same columns	default 0.0	reliability metadata
r0_pu, x0_pu, b0_pu, vn_hv_kv, vn_lv_kv, sn_mva	none	not part of the current JSON / Excel contract	struct-level short-circuit and transformer-nameplate helpers currently stay internal to the in-memory model

Load : ac.loads[] / sheet Load (fallback ACLoad)

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
index	Load.index or ACLoad.index	required on JSON import; blank Excel index stops table import	stable load identifier
bus	Load.bus or ACLoad.bus	required on JSON import	connected AC bus index
name	same column	default empty string	load label
in_service	same column	default true	availability flag
p_mw, q_mvar	same columns	default 0.0	active and reactive demand
model	same column	default ConstantPower	ZIP / exponential / constant-power load-model token
z_percent_p, i_percent_p, p_percent_p	same columns	defaults 0.0, 0.0, 100.0	active-power ZIP shares
z_percent_q, i_percent_q, p_percent_q	same columns	defaults 0.0, 0.0, 100.0	reactive-power ZIP shares
controllable	same column	default false	whether shedding / dispatch logic may treat the load as controllable

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
<code>p_min_mw</code>	<code>none</code>	default 0.0	JSON-only minimum retained active demand; workbook currently omits this field for AC loads
<code>cost_mw</code>	same column	default 0.0	marginal shedding or curtailment penalty
<code>priority</code>	same column	default <code>Medium</code>	service-priority token <code>Low/Medium/High/Critical</code>
<code>n_customers</code>	<code>none</code>	default 0	JSON-only customer-count metadata
<code>profile_id</code>	<code>none</code>	default -1	JSON-only profile linkage identifier
<code>sn_mva, motor_percent, x_sub_pu, r_sc_pu</code>	<code>none</code>	not part of the current main JSON contract	short-circuit helper fields remain outside the stable Load I/O contract

FlexibleLoad : `ac.flexible_loads[]` / sheet `FlexibleLoad`

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
<code>index, bus</code>	corresponding columns	required on JSON import; blank Excel <code>index</code> stops table import	stable identifier and connected AC bus
<code>name</code>	<code>FlexibleLoad.name</code>	default empty string	load label
<code>in_service</code>	<code>FlexibleLoad.in_service</code>	default <code>true</code>	availability flag
<code>p_mw, q_mvar</code>	same columns	default 0.0	baseline active and reactive demand
<code>flex_up_mw, flex_down_mw</code>	same columns	default 0.0	upward / downward flexibility envelope in MW
<code>flex_duration_h</code>	same column	default 0.0	duration for which the flex envelope can be sustained
<code>response_time_s</code>	same column	default 0.0	response latency in seconds
<code>ramp_rate_mw_min</code>	same column	default 0.0	adjustment ramp-rate limit
<code>availability_pct</code>	same column	default 100.0	expected availability of the flexibility service
<code>controllable</code>	same column	default <code>true</code>	whether the flexibility service is enabled
<code>priority</code>	same column	default <code>Medium</code>	load-service priority token
<code>control_area</code>	same column	default empty string	operator / aggregation area tag

AsymmetricLoad : `ac.asymmetric_loads[]` / sheet `AsymmetricLoad`

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
<code>index, bus</code>	corresponding columns	required on JSON import; blank Excel <code>index</code> stops table import	stable identifier and connected AC bus
<code>name</code>	<code>AsymmetricLoad.name</code>	default empty string	load label
<code>in_service</code>	<code>AsymmetricLoad.in_service</code>	default <code>true</code>	availability flag

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
connection	AsymmetricLoad.connection	default empty string	wiring model string such as wye or delta
grounded	AsymmetricLoad.grounded	default true	grounding-path assumption
pa_rated_mw, pb_rated_mw, pc_rated_mw	same columns	default 0.0	rated per-phase active demand
qa_rated_mvar, qb_rated_mvar, qc_rated_mvar	same columns	default 0.0	rated per-phase reactive demand
pa_mw, pb_mw, pc_mw	same columns	default 0.0	current per-phase active demand
qa_mvar, qb_mvar, qc_mvar	same columns	default 0.0	current per-phase reactive demand
scaling	AsymmetricLoad.scaling	default 1.0	overall scaling factor applied to the phase values
const_z_percent, const_i_percent	same columns	default 0.0	ZIP constant-impedance and constant-current shares
const_p_percent	same column	default 100.0	ZIP constant-power share
controllable	same column	default false	controllability flag
priority	same column	default Medium	load-service priority token

Shunt : `ac.shunts[]` / sheet **Shunt**

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
index, bus	corresponding columns	required on JSON import; blank Excel index stops table import	stable identifier and connected AC bus
name	Shunt.name	default empty string	shunt label
in_service	Shunt.in_service	default true	availability flag
gs_mw, bs_mvar	same columns	default 0.0	fixed shunt conductance and susceptance
switchable	Shunt.switchable	default false	whether the shunt is modeled as stepped / switchable
n_steps	Shunt.n_steps	default 1	number of available discrete shunt steps
current_step	Shunt.current_step	default 1	currently selected shunt step
bs_per_step	Shunt.bs_per_step	default 0.0	susceptance increment per step

ExternalGrid : `ac.external_grids[]` / sheet **ExternalGrid**

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
index, bus	corresponding columns	required on JSON import; blank Excel index stops table import	stable identifier and connected AC bus
name	ExternalGrid.name	default empty string	source label
in_service	ExternalGrid.in_service	default true	availability flag

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
vm_pu, va_deg	same columns	defaults 1.0, 0.0	voltage magnitude and angle target for the slack / Thevenin source
s_sc_max_mva, s_sc_min_mva	same columns	default 0.0	short-circuit strength range
rx_max, rx_min	same columns	default 0.0	R/X ratio bounds for equivalent-source construction
r_pu, x_pu	none	default 0.0	JSON-only explicit positive-sequence Thevenin impedance
r0_pu, x0_pu	none	default 0.0	JSON-only zero-sequence source impedance
vn_kv	none	default 0.0	JSON-only nominal source voltage rating
ikq_ka, x_r	none	not part of the current JSON / Excel contract	struct-level IEC 60909 helper fields currently stay internal and are not preserved by either import path
controllable	ExternalGrid.controllable	default true	whether optimization logic may vary the source dispatch

Transformer2W : ac.transformers_2w[] / sheet Transformer2W

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
index, hv_bus, lv_bus	corresponding columns	required on JSON import; blank Excel index stops table import	stable identifier and terminal bus indices
name	Transformer2W.name	default empty string	transformer label
std_type	none	default empty string	JSON-only library / standard type name
in_service	Transformer2W.in_service	default true	availability flag
sn_mva	Transformer2W.sn_mva	default 0.0	rated apparent power
vn_hv_kv, vn_lv_kv	same columns	default 0.0	winding rated voltages
vk_percent, vkr_percent	same columns	default 0.0	short-circuit magnitude and resistive share in percent
pk_kw	none	default 0.0	JSON-only load-loss field retained in the richer transformer contract
pfe_kw, i0_percent	same columns	default 0.0	no-load loss and excitation current
tap_side, tap_pos, tap_min, tap_max, tap_neutral	same columns	default 0	tap-changer placement and discrete tap metadata
tap_step_percent	Transformer2W.tap_step_percent	default 0.0	tap step size in percent
shift_deg	Transformer2W.shift_deg	default 0.0	phase-shift angle in degrees
vector_group	Transformer2W.vector_group	default empty string	winding connection / clock notation

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
z0_percent, x0_r0	none	default 0.0	JSON-only zero-sequence impedance parameters
mtbf_hours, mttr_hours	none	default 0.0	JSON-only reliability meta-data

Transformer3W : `ac.transformers_3w[]` / sheet **Transformer3W**

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
index, hv_bus, mv_bus, lv_bus	corresponding columns	required on JSON import; blank Excel index stops table import	stable identifier and three winding bus indices
name	Transformer3W.name	default empty string	transformer label
std_type	Transformer3W.std_type	default empty string	library / standard type name
in_service	Transformer3W.in_service	default true	availability flag
sn_hv_mva, sn_mv_mva, sn_lv_mva	same columns	default 0.0	winding ratings on the HV / MV / LV sides
vn_hv_kv, vn_mv_kv, vn_lv_kv	same columns	default 0.0	winding rated voltages
vk_hv_mv_percent, vk_hv_lv_percent, vk_mv_lv_percent	same columns	default 0.0	pairwise short-circuit magnitudes
vkr_hv_mv_percent, vkr_hv_lv_percent, vkr_mv_lv_percent	same columns	default 0.0	pairwise resistive shares
pfe_kw, i0_percent	same columns	default 0.0	no-load loss and excitation-current data
tap_side, tap_pos	same columns	default 0	tap-changer side and current position
tap_step_percent	Transformer3W.tap_step_percent	default 0.0	tap step size in percent
shift_mv_deg, shift_lv_deg	same columns	default 0.0	phase shifts from HV to MV / LV windings
mtbf_hours, mttr_hours	same columns	default 0.0	reliability metadata preserved in both JSON and workbook

Switch : `ac.switches[]` / sheet **Switch**

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
index, bus_from, bus_to	corresponding columns	required on JSON import; blank Excel index stops table import	stable identifier and terminal bus indices
name	Switch.name	default empty string	switch label
in_service	Switch.in_service	default true	availability flag
switch_type	Switch.switch_type	default CircuitBreaker	switching-device token such as Recloser or Fuse

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
<code>closed</code>	<code>Switch.closed</code>	default <code>true</code>	topology status flag
<code>r_contact_ohm</code> , <code>z_ohm</code>	<code>none</code>	default 0.0	JSON-only detailed contact / arc impedance inputs
<code>i_rated_ka</code> , <code>i_breaking_ka</code>	<code>none</code>	default 0.0	JSON-only current capability ratings
<code>element_type</code> , <code>element_id</code>	<code>none</code>	defaults 0, 0	JSON-only foreign-key reference to the protected element
<code>is_remote</code> , <code>is_automated</code>	<code>none</code>	default <code>false</code>	JSON-only automation metadata
<code>t_operation_s</code> , <code>p_sw_fail</code>	<code>none</code>	default 0.0	JSON-only operation timing and failure-probability metadata
<code>mtbf_hours</code> , <code>mttr_hours</code>	<code>none</code>	default 0.0	JSON-only reliability metadata

ChargingStation : `ac.charging_stations[]` / sheet **ChargingStation**

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
<code>index</code> , <code>bus</code>	corresponding columns	required on JSON import; blank Excel <code>index</code> stops table import	stable identifier and connected AC bus
<code>name</code>	<code>ChargingStation.name</code>	default empty string	station label
<code>location</code>	<code>none</code>	default empty string	JSON-only free-form location descriptor
<code>in_service</code>	<code>ChargingStation.in_service</code>	default <code>true</code>	availability flag
<code>n_fast</code> , <code>n_slow</code> , <code>num_chargers</code>	same columns	default 0	counts of charger assets by class and in total
<code>p_fast_max_kw</code> , <code>p_slow_max_kw</code>	same columns	default 0.0	per-charger maximum power by service class
<code>max_power_kw</code>	same column	default 0.0	aggregate station power ceiling
<code>simultaneity_factor</code>	same column	default 1.0	diversification factor used in the aggregate station model
<code>power_factor</code>	same column	default 0.95	station-level reactive-power assumption
<code>utilization_rate</code>	<code>none</code>	default 0.0	JSON-only expected utilization metadata
<code>p_total_kw</code> , <code>q_total_kvar</code>	same columns	default 0.0	aggregate present demand of the station
<code>mtbf_hours</code> , <code>mttr_hours</code>	<code>none</code>	default 0.0	JSON-only reliability metadata

Charger : `ac.chargers[]` / sheet **Charger**

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
<code>index</code>	<code>Charger.index</code>	required on JSON import; blank Excel <code>index</code> stops table import	stable charger identifier
<code>name</code>	<code>Charger.name</code>	default empty string	charger label
<code>station_id</code>	<code>Charger.station_id</code>	default 0	parent charging-station foreign key
<code>charger_type</code>	<code>Charger.charger_type</code>	default <code>AC_L2</code>	charger-class token <code>AC_L1/AC_L2/DC_Fast</code>
<code>in_service</code>	<code>Charger.in_service</code>	default <code>true</code>	availability flag
<code>p_rated_kw</code>	<code>Charger.p_rated_kw</code>	default 0.0	rated charger power
<code>p_ch_max_kw</code> , <code>p_ch_min_kw</code>	same columns	default 0.0	charging power limits
<code>eta</code>	<code>Charger.eta</code>	default 0.95	charger conversion efficiency
<code>v2g_capable</code>	<code>Charger.v2g_capable</code>	default <code>false</code>	whether reverse power flow is allowed
<code>p_dis_max_kw</code>	<code>Charger.p_dis_max_kw</code>	default 0.0	maximum discharging power when V2G is enabled
<code>mtbf_hours</code> , <code>mttr_hours</code>	same columns	default 0.0	reliability metadata preserved in both JSON and workbook

10.3 Generation and Storage Models

Generator : `ac.generators[]` / sheet `Generator`

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
<code>index</code> , <code>bus</code>	corresponding columns	required on JSON import; blank Excel <code>index</code> stops table import	stable identifier and connected AC bus
<code>name</code>	<code>Generator.name</code>	default empty string	generator label
<code>in_service</code>	<code>Generator.in_service</code>	default <code>true</code>	availability flag
<code>pg_mw</code> , <code>qg_mvar</code>	same columns	default 0.0	active and reactive generation setpoints
<code>vg_pu</code>	<code>Generator.vg_pu</code>	default 1.0	voltage setpoint on the connected bus
<code>pmax_mw</code> , <code>pmin_mw</code> , <code>qmax_mvar</code> , <code>qmin_mvar</code>	same columns	default 0.0	operating capability limits
<code>is_slack</code>	<code>Generator.is_slack</code>	default <code>false</code>	slack-machine flag
<code>fuel_type</code>	<code>Generator.fuel_type</code>	default <code>Unknown</code>	fuel-type token such as <code>Gas</code> , <code>Wind</code> , or <code>Solar</code>
<code>cost_c2</code> , <code>cost_c1</code> , <code>cost_c0</code>	same columns	default 0.0	quadratic production-cost coefficients
<code>startup_cost</code> , <code>shutdown_cost</code> , <code>min_up_time_hr</code> , <code>min_dn_time_hr</code>	none	default 0.0	JSON-only unit-commitment metadata
<code>ramp_up_mw_min</code> , <code>ramp_dn_mw_min</code>	same columns	default 0.0	ramp-rate limits
<code>mbase_mva</code> , <code>inertia_h</code> , <code>droop_r</code>	same columns	defaults 0.0, 0.0, 0.05	machine base, inertia, and governor droop metadata

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
<code>xd_pu, xdp_pu, xdp_pu, td0p_s, td0pp_s</code>	none	default 0.0	JSON-only machine dynamic / short-circuit parameters
<code>ra_pu, vn_kv, cos_phi, xq_pu, xd_xq, x0_pu, r0_pu</code>	none	not part of the current JSON / Excel contract	additional short-circuit parameters exist in the C++ struct but are not yet serialized or workbook-backed
<code>emission_factor_r_tco2_mwh, nox_factor_kg_mwh, so2_factor_kg_mwh</code>	none	default 0.0	JSON-only emissions metadata
<code>forced_outage_rate, mttr_hr</code>	none	default 0.0	JSON-only reliability metadata

StaticGenerator : `ac.static_generators[]` or `dc.static_generators[]` / sheets StaticGenerator and DCStaticGen

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
<code>index, bus</code>	corresponding columns	required on JSON import; blank Excel <code>index</code> stops table import	stable identifier and connected AC or DC bus
<code>name</code>	corresponding <code>name</code> column	default empty string	generator label
<code>in_service</code>	corresponding <code>in_service</code> column	default <code>true</code>	availability flag
<code>sgen_type</code>	corresponding <code>sgen_type</code> column	default <code>Other</code>	generator-type token such as <code>Wind</code> , <code>Battery</code> , or <code>FuelCell</code>
<code>p_mw, q_mvar</code>	same columns	default 0.0	present active / reactive injection
<code>p_rated_mw, sn_mva</code>	same columns	default 0.0	rated active power and apparent power
<code>pmax_mw, pmin_mw, qmax_mvar, qmin_mvar</code>	same columns	default 0.0	operating capability limits
<code>scaling</code>	same column	default 1.0	scalar multiplier for the static generation profile
<code>controllable</code>	same column	default <code>false</code>	controllability flag
<code>v_ref_pu, k_p, k_q, k_rx</code>	none	defaults 1.0, 0.0, 0.0, 1.0, 0.0	JSON-only control and equivalent-source tuning parameters
<code>co2_emission_rate</code>	none	default 0.0	JSON-only emissions metadata
<code>mtbf_hours, mttr_hours</code>	none	default 0.0	JSON-only reliability metadata

Storage : `ac.storage[]` or `dc.storage[]` / sheets Storage and DCStorage

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
<code>index, bus</code>	corresponding columns	required on JSON import; blank Excel <code>index</code> stops table import	stable identifier and connected bus
<code>name</code>	corresponding <code>name</code> column	default empty string	storage label
<code>in_service</code>	corresponding <code>in_service</code> column	default <code>true</code>	availability flag
<code>p_mw, q_mvar</code>	same columns	default 0.0	present active and reactive dispatch
<code>p_rated_mw</code>	same column	default 0.0	rated converter power
<code>pmax_mw, pmin_mw, qmax_mvar, qmin_mvar</code>	same columns	default 0.0	operating limits
<code>e_rated_mwh</code>	same column	default 0.0	energy capacity
<code>soc_init, soc_min, soc_max</code>	same columns	defaults 0.5, 0.1, 0.9	initial and admissible state-of-charge values
<code>soc_carbon_intensity_tco2_mwh</code>	same column	default 0.0	explicit carbon intensity of the stored energy state used by carbon-flow analysis during discharge
<code>eta_charge, eta_discharge</code>	same columns	default 0.95	charging and discharging efficiency
<code>self_discharge_pct</code>	<code>none</code>	default 0.0	JSON-only storage leakage metadata
<code>max_cycles, current_cycles</code>	<code>none</code>	defaults 5000, 0	JSON-only lifecycle counters
<code>soh</code>	<code>none</code>	default 1.0	JSON-only state-of-health value
<code>replacement_cost</code>	<code>none</code>	default 0.0	JSON-only replacement-cost metadata
<code>forced_outage_rate, mttr_hr</code>	<code>none</code>	default 0.0	JSON-only reliability metadata

RenewableGen : `ac.renewable_gens[]` / sheet RenewableGen

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
<code>index, bus</code>	corresponding columns	required on JSON import; blank Excel <code>index</code> stops table import	stable identifier and connected AC bus
<code>name</code>	<code>RenewableGen.name</code>	default empty string	generator label
<code>in_service</code>	<code>RenewableGen.in_service</code>	default <code>true</code>	availability flag
<code>type</code>	<code>RenewableGen.type</code>	default <code>Wind</code>	renewable-type token such as <code>Wind</code> , <code>SolarPV</code> , or <code>Hydro</code>
<code>p_mw, q_mvar</code>	same columns	default 0.0	present active and reactive output
<code>p_rated_mw</code>	same column	default 0.0	rated active power
<code>qmax_mvar, qmin_mvar</code>	same columns	default 0.0	reactive capability limits
<code>curtailable</code>	same column	default <code>true</code>	whether curtailment is allowed
<code>cost_curtail_mwh</code>	<code>none</code>	default 0.0	JSON-only curtailment penalty

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
capacity_factor	RenewableGen.capacity_factor	default 0.3	expected normalized availability
profile_id	none	default -1	JSON-only profile linkage identifier
emission_offset_tco2_mwh	none	default 0.0	JSON-only avoided-emissions metadata

PVSystem : ac.pv_systems[] / sheet PVSystem

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
index, bus	corresponding columns	required on JSON import; blank Excel index stops table import	stable identifier and connected AC bus
name	PVSystem.name	default empty string	PV system label
in_service	PVSystem.in_service	default true	availability flag
p_mw, q_mvar	same columns	default 0.0	present active and reactive output
sn_mva	PVSystem.sn_mva	default 0.0	apparent-power rating
pmax_mw, pmin_mw, qmax_mvar, qmin_mvar	same columns	default 0.0	operating capability limits
control_mode	PVSystem.control_mode	default MPPT	inverter control-mode token; one of MPPT, VoltVar, VoltWatt, FixedPF, or Curtailed
controllable	none	default false	JSON-only controllability flag
v_ac_set_pu, v_dc_set_pu	none	default 1.0	JSON-only AC/DC voltage reference targets
inverter_eff	PVSystem.inverter_eff	default 0.97	inverter efficiency
loss_percent	none	default 0.0	JSON-only miscellaneous loss factor
num_series, num_parallel, vmpp, impp, voc, isc, alpha_isc, beta_voc	none	default 0 or 0.0	JSON-only panel-array electrical metadata
irradiance, temperature	none	defaults 1000.0, 25.0	JSON-only environmental inputs
mtbf_panel_hours, mttr_panel_hours, mtbf_inverter_hours, mttr_inverter_hours	none	default 0.0	JSON-only reliability metadata

10.4 DC and Hybrid Coordination Models

DCBus : dc.buses[] / sheet DCBus

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
<code>index</code>	<code>DCBus.index</code>	required on JSON import; blank Excel <code>index</code> stops table import	stable DC bus identifier
<code>name</code>	<code>DCBus.name</code>	default empty string	bus label
<code>bus_type</code>	<code>DCBus.bus_type</code>	default <code>DC_P</code>	DC bus-type token <code>DC_P</code> or <code>DC_V</code>
<code>vm_pu</code>	<code>DCBus.vm_pu</code>	default 1.0	DC voltage magnitude in per unit
<code>vmax_pu</code> , <code>vmin_pu</code>	same columns	defaults 1.1, 0.9	DC voltage operating bounds
<code>pd_mw</code>	<code>DCBus.pd_mw</code>	default 0.0	embedded DC demand snapshot; workbook export may reflect synchronized aggregate demand
<code>in_service</code>	<code>DCBus.in_service</code>	default <code>true</code>	availability flag

DCBranch : `dc.branches[]` / sheet DCBranch

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
<code>index</code> , <code>from_bus</code> , <code>to_bus</code>	corresponding columns	required on JSON import; blank Excel <code>index</code> stops table import	stable identifier and terminal bus indices
<code>name</code>	<code>DCBranch.name</code>	default empty string	branch label
<code>r_pu</code>	<code>DCBranch.r_pu</code>	default 0.0	branch resistance in per unit
<code>rate_a_mva</code>	<code>DCBranch.rate_a_mva</code>	default 0.0	branch rating
<code>length_km</code>	<code>DCBranch.length_km</code>	default 0.0	physical route length meta-data
<code>in_service</code>	<code>DCBranch.in_service</code>	default <code>true</code>	availability flag

DCLoad : `dc.loads[]` / sheet DCLoadRaw (fallback DCLoad)

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
<code>index</code> , <code>bus</code>	corresponding columns	required on JSON import; blank Excel <code>index</code> stops table import	stable identifier and connected DC bus
<code>name</code>	same column	default empty string	load label
<code>in_service</code>	same column	default <code>true</code>	availability flag
<code>p_mw</code>	same column	default 0.0	active demand
<code>controllable</code>	same column	default <code>false</code>	controllability flag
<code>p_min_mw</code>	same column	default 0.0	minimum retained demand under curtailment
<code>cost_mw</code>	same column	default 0.0	marginal curtailment penalty
<code>profile_id</code>	<code>none</code>	default -1	JSON-only profile linkage identifier

VSCConverter : vsc_converters[] / sheet VSCConverter

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
index, bus_ac, bus_dc	corresponding columns	required on JSON import; blank Excel index stops table import	stable identifier and AC/DC terminal bus indices
name	VSCConverter.name	default empty string	converter label
in_service	VSCConverter.in_service	default true	availability flag
control_mode	VSCConverter.control_mode	default PQ	converter-mode token PQ/VD C_Q/VDC_VAC
p_set_mw, q_set_mvar	same columns	default 0.0	AC-side active and reactive setpoints
v_dc_set_pu, v_ac_set_pu	same columns	default 1.0	DC and AC voltage references
eta	VSCConverter.eta	default 0.99	conversion efficiency
loss_percent	VSCConverter.loss_percent	default 0.0	generic loss factor
loss_mw	none	default 0.0	JSON-only fixed loss term
k_vdc	none	default 0.1	JSON-only DC-voltage droop coefficient
pmax_mw, pmin_mw, qmax_mvar, qmin_mvar	same columns	default 0.0	operating capability limits
p Rated_mw	VSCConverter.p Rated_mw	default 0.0	converter rating
forced_outage_rate, mttr_hr	none	default 0.0	JSON-only reliability meta-data

DCDCConverter : dcdc_converters[] / sheet DCDCCConverter

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
index, bus_in, bus_out	corresponding columns	required on JSON import; blank Excel index stops table import	stable identifier and input / output DC bus indices
name	DCDCConverter.name	default empty string	converter label
in_service	DCDCConverter.in_service	default true	availability flag
control_mode	DCDCConverter.control_mode	default Voltage	control-mode token Voltage/Power/Droop
p_ref_mw	DCDCConverter.p_ref_mw	default 0.0	active-power reference
v_ref_pu	DCDCConverter.v_ref_pu	default 1.0	output-voltage reference
sn_mva	DCDCConverter.sn_mva	default 0.0	converter rating
vn_in_kv, vn_out_kv	none	default 0.0	JSON-only nominal voltage ratings
eta	DCDCConverter.eta	default 0.98	conversion efficiency
r_eq_pu	none	default 0.0	JSON-only equivalent resistance
pmax_mw, pmin_mw	same columns	default 0.0	power limits
k_droop	none	default 0.0	JSON-only droop coefficient
mtbf_hours, mttr_hours	none	default 0.0	JSON-only reliability meta-data

EnergyRouter : energy_routers[] / sheet EnergyRouter

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
index	EnergyRouter.index	required on JSON import; blank Excel index stops table import	stable router identifier
name	EnergyRouter.name	default empty string	router label
in_service	EnergyRouter.in_service	default true	availability flag
router_type	EnergyRouter.router_type	default empty string	free-form router classification token
num_ports	EnergyRouter.num_ports	default 0	expected number of ports; workbook import may later recompute it from the separate port table
ports[]	sheet EnergyRouterPort	missing JSON array keeps the vector empty	nested port objects are flattened into a dedicated workbook sheet
p_rated_mw	EnergyRouter.p_rated_mw	default 0.0	total router rating
vn_ac_kv, vn_dc_kv	same columns	default 0.0	nominal AC/DC voltage ratings
loss_percent	EnergyRouter.loss_percent	default 0.0	aggregate conversion loss factor
pmax_mw, pmin_mw, qmax_mvar, qmin_mvar	same columns	default 0.0	aggregate operating limits
mtbf_hours, mttr_hours	same columns	default 0.0	reliability metadata preserved in both JSON and workbook

EnergyRouterPort : nested energy_routers[].ports[] / sheet EnergyRouterPort

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
router_index	EnergyRouterPort.router_index	workbook-only join key	parent router identifier used because ports are nested in JSON but flattened in Excel
index	EnergyRouterPort.index	required on JSON import; workbook row also needs router_index	stable port identifier within its router
name	EnergyRouterPort.name	default empty string	port label
bus	EnergyRouterPort.bus	required on JSON import	connected AC or DC bus index
port_type	EnergyRouterPort.port_type	default AC	side-type token AC or DC
voltage_level_kv	EnergyRouterPort.voltage_level_kv	default 0.0	nominal port voltage level
p_mw, q_mvar	same columns	default 0.0	present active and reactive transfer assigned to the port
v_pu	EnergyRouterPort.v_pu	default 1.0	local port voltage state or initialization
pmax_mw, pmin_mw, qmax_mvar, qmin_mvar	same columns	default 0.0	operating limits

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
<code>control_mode</code>	<code>EnergyRouterPort.control_mode</code>	default PQ	control-mode token PQ/VF/Droop
<code>p_set_mw, q_set_mvar, v_set_pu</code>	same columns	defaults 0.0, 0.0, 1.0	reference setpoints for the local port controller
<code>in_service</code>	<code>EnergyRouterPort.in_service</code>	default true	availability flag

VirtualPowerPlant : `vpps[]` / sheet VPP

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
<code>index</code>	<code>VPP.index</code>	required on JSON import; blank Excel <code>index</code> stops table import	stable VPP identifier
<code>name</code>	<code>VPP.name</code>	default empty string	VPP label
<code>description</code>	<code>none</code>	default empty string	JSON-only free-form description
<code>aggregation_bus</code>	<code>VPP.aggregation_bus</code>	default 0	reporting / control aggregation bus
<code>in_service</code>	<code>VPP.in_service</code>	default true	availability flag
<code>n_pv_systems, n_wind_turbines, n_battery_systems</code>	same columns	default 0	asset counts preserved in both JSON and workbook
<code>n_ev_chargers, n_controllable_loads</code>	<code>none</code>	default 0	JSON-only aggregation metadata
<code>p_generation_sum_mw, e_storage_sum_mwh, p_load_controllable_mw</code>	<code>none</code>	default 0.0	JSON-only aggregate capability metadata
<code>p_output_mw, q_output_mvar</code>	same columns	default 0.0	present aggregated power output
<code>pmax_mw, pmin_mw</code>	same columns	default 0.0	aggregate operating limits
<code>ramp_up_max_mw_min, ramp_down_max_mw_min</code>	<code>none</code>	default 0.0	JSON-only ramping metadata
<code>mtbf_hours, mttr_hours</code>	<code>none</code>	default 0.0	JSON-only reliability metadata

Microgrid : `microgrids[]` / sheet Microgrid

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
<code>index</code>	<code>Microgrid.index</code>	required on JSON import; blank Excel <code>index</code> stops table import	stable microgrid identifier
<code>name</code>	<code>Microgrid.name</code>	default empty string	microgrid label
<code>description</code>	<code>none</code>	default empty string	JSON-only free-form description
<code>in_service</code>	<code>Microgrid.in_service</code>	default true	availability flag

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
pcc_bus	Microgrid.pcc_bus	default 0	point-of-common-coupling bus index
internal_buses	none	missing JSON array keeps the vector empty	JSON-only explicit internal bus membership list
operating_mode	Microgrid.operating_mode	default GridConnected	operating-mode token; one of GridConnected , Islanded , or Transition
islanding_capability	Microgrid.islanding_capability	default false	whether the microgrid may island
auto_reconnection	none	default false	JSON-only reconnection automation flag
p_exchange_max_mw,	Excel keeps only p_exchange_max_mw	default 0.0	aggregate exchange limits with the host grid
p_exchange_min_mw,		default 0.0	JSON-only directional exchange limits
p_import_max_mw,	none	default 0.0	JSON-only directional exchange limits
p_export_max_mw	none	default 0.0	JSON-only directional exchange limits
p_exchange_mw	Microgrid.p_exchange_mw	default 0.0	present net power exchange at the PCC
total_generation_mw, total_load_mw	same columns	default 0.0	current aggregate generation and demand
total_storage_mwh, capacity_mw, peak_load_mw	none	default 0.0	JSON-only planning metadata
f_set_hz, v_set_pu, k_droop	none	defaults 50.0, 1.0, 0.0	JSON-only islanded-control targets
area	none	default 0	JSON-only area tag
mtbf_hours, mttr_hours	none	default 0.0	JSON-only reliability metadata

MobileStorage : mobile_storage[] / sheet MobileStorage

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
index, bus	corresponding columns	required on JSON import; blank Excel index stops table import	stable identifier and current connected bus
name	MobileStorage.name	default empty string	unit label
in_service	MobileStorage.in_service	default true	availability flag
p_mw, q_mvar	same columns	default 0.0	present dispatch at the current location
p_rated_mw	MobileStorage.p_rated_mw	default 0.0	converter power rating
pmax_mw, pmin_mw, qmax_mvar, qmin_mvar	none	default 0.0	JSON-only operating limits
e_rated_mwh	MobileStorage.e_rated_mwh	default 0.0	energy capacity
soc_init	MobileStorage.soc_init	default 0.5	initial state of charge
soc_min, soc_max	none	defaults 0.1, 0.9	JSON-only admissible state-of-charge bounds
eta_charge, eta_discharge	none	default 0.95	JSON-only efficiency metadata
max_cycles, current_cycles, soh	none	defaults 5000, 0, 1.0	JSON-only lifecycle / health metadata

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
<code>is_mobile</code>	<code>none</code>	default <code>true</code>	JSON-only flag distinguishing mobile vs. stationary storage representation
<code>status</code>	<code>MobileStorage.status</code>	default <code>Stationary</code>	mobility-state token; one of <code>Stationary</code> , <code>InTransit</code> , or <code>Deployed</code>
<code>current_location</code>	<code>none</code>	default empty string	JSON-only free-form location string
<code>target_bus</code>	<code>none</code>	default <code>0</code>	JSON-only relocation target
<code>e_consumption_mwh_k m, max_travel_dis tance_k m</code>	<code>none</code>	default <code>0.0</code>	JSON-only transportation-energy metadata
<code>mtbf_hours, mtr_h ours</code>	<code>none</code>	default <code>0.0</code>	JSON-only reliability metadata

10.5 Cross-Reference

The only remaining detailed dictionaries not repeated in this section are the ones already written in Section 02a_io_contracts:

1. `AsynchronousMotor`
2. the optional `three_phase_ac` metadata container
3. `ThreePhaseBus`, `ThreePhaseLine`, `ThreePhaseTransformer`, `ThreePhaseLoad`, `ThreePhaseGenerator`, and `ThreePhaseExternalGrid`
4. result-sheet enum and flag conventions

11 Output and Report Field Dictionaries

This section applies the same field-level dictionary template to the current export artifacts. Section 02a_io_contracts already catalogs which workbooks and JSON serializers exist; the tables below make the output-side contract explicit at workbook-column and JSON-key level.

11.1 Export Families

Artifact	Export path	Direction	Current note
<code>PowerFlowResult</code> dedicated workbook	<code>save_results_xlsx</code>	export only	writes the <code>Results_*</code> sheets only; there is no dedicated PF result JSON serializer today.
<code>PowerFlowResult</code> compact workbook	<code>save_results_compact_xlsx</code>	export only	rewrites input sheets and appends <code>result_*</code> columns to selected tables, plus <code>Results_Summary</code> .
<code>ACOPFResult</code> dedicated workbook	<code>save_opf_results_xlsx</code>	export only	writes <code>OPF_*</code> sheets only.
<code>ACOPFResult</code> compact workbook	<code>save_opf_results_compact_xlsx</code>	export only	rewrites input sheets but still emits separate <code>OPF_*</code> sheets; no <code>opf_*</code> columns are appended into input tables.

Artifact	Export path	Direction	Current note
ACOPFResult JSON	opf_result_to_json / opf_result_from_json	bidirectional	serializes the scalar solver summary, core bus / generator / converter vectors, and OPF profiling block.
CarbonAnalysisResult workbook	save_carbon_results.xlsx	export only	writes AC/DC load and bus sheets, AC/DC branch-loss sheets, converter-loss sheets, storage / EnergyRouter sheets, and block-style Summary .
CarbonAnalysisResult JSON	carbon_result_to_json / carbon_result_from_json	bidirectional	preserves the richer tracing maps and the hybrid AC/DC carrier results that the workbook flattens.
SCResult / BusFaultResult API	compute_short_circuit / compute_fault_at_bus	in-memory only	no workbook / JSON serializer exists yet; batch results follow projected AC-bus order.
SCDetailedResult API	run_short_circuit_detailed / run_short_circuit_detailed_batch	in-memory only	detailed IEC-style current quantities are exposed only through the C++ API; the header still carries legacy scaffold flags.
ONRResult API	solve_optimal_reconfiguration	in-memory only	no workbook / JSON serializer exists yet; branch-status vectors currently report branch-table positions rather than original <code>ACBranch::index</code> values.

11.2 Result Semantic Conventions

Semantic aspect	Current rule	Main artifacts	Engineering note
Electrical units	Active power uses MW, reactive power uses Mvar, AC / DC voltage magnitudes use pu, AC voltage angle uses degree, and current stays in pu unless the field name explicitly says ka .	PF / OPF workbooks and JSON	Workbook and JSON field names usually encode the unit directly, for example <code>pg_mw</code> , <code>va_deg</code> , <code>loss_mw</code> , and <code>matrix_residual</code> .
Carbon units	Carbon intensity uses <code>tCO2/MWh</code> ; total emissions use <code>tCO2</code> .	Carbon workbook and JSON	The JSON / workbook field names use the suffixes <code>_tco2_mwh</code> and <code>_tco2</code> ; no implicit mass-unit conversion is applied by the exporter.

Semantic aspect	Current rule	Main artifacts	Engineering note
Vector order	Numerical result vectors follow solver-side component order, not sparse physical ids. <code>vm</code> , <code>va</code> , <code>dpcd_mw</code> , and <code>dpcd_mvar</code> follow AC-bus order; <code>vdc</code> follows DC-bus order; <code>pg_mw</code> , <code>qg_mvar</code> follow AC-generator order; <code>pac_mw</code> , <code>qac_mvar</code> follow VSC order.	PF / OPF JSON and workbook exports	Consumers should treat workbook rows and JSON vectors as aligned to the source component arrays unless a sheet explicitly says otherwise.
Row identity	Most workbook result sheets copy the source model <code>index</code> and <code>name</code> . The main exception is <code>Results_BranchFlow.index</code> , which is the export row ordinal <code>i+1</code> .	PF workbooks	For branch-flow joins, use source branch order or the <code>from_bus/to_bus</code> pair rather than assuming the result-sheet <code>index</code> is the original branch id.
Fallback values	When a solver vector entry is absent or out of range, exporters write deterministic defaults: typically 0.0 for powers, angles, counters, and residual-like scalars, or 1.0 for voltage magnitudes.	PF / OPF workbooks	This keeps workbook shapes stable, but it means a consumer must distinguish “solver wrote a fallback” from “the physical result is exactly zero” using convergence and artifact context.
JSON missing keys	JSON importers keep absent arrays or object maps empty and default absent scalar keys to false, zero, or empty string according to the target field type.	OPF / carbon JSON	The JSON contracts are intentionally sparse-friendly; round-trip consumers should not require every optional key to be present.
Echoed vs solved values	Some exported fields are metadata echoes or derived annotations rather than true solver states: storage <code>soc</code> comes from input <code>soc_init</code> ; router port sums come from stored port structs; <code>Results_DCBranch</code> is recomputed from solved voltages and branch resistance.	PF workbooks	Consumers should not infer that every result-sheet field belongs to the nonlinear state vector.

Semantic aspect	Current rule	Main artifacts	Engineering note
Boolean spelling	PF / OPF workbook booleans use the standard workbook token form produced by <code>bool_str</code> ; carbon <code>Summary</code> uses lowercase <code>true/false</code> .	Result workbooks	Downstream parsers should normalize workbook booleans instead of assuming one literal spelling.
Conditional sheets	Some result sheets are pre-created even when no rows are written. <code>OPF_Converter</code> is the main example.	PF / OPF workbooks	Empty sheets are part of the current export contract and do not by themselves indicate an error.
Object-map semantics	Carbon contribution maps are exported as JSON object maps keyed by internal carbon-source id rather than positional arrays.	Carbon JSON	Map-key order is not semantically meaningful; workbook export intentionally flattens these maps away.
Projected API order	<code>SCResult.bus_results</code> and <code>SCDetailedResult.bus_results</code> follow projected AC-bus order; <code>ONRResult.open_branch_ids</code> / <code>closed_branch_ids</code> contain stable <code>ACBranch::index</code> values from the projected AC system.	SC / ONR C++ API results	When a <code>HybridPowerSystem</code> overload expands transformers, switches, or chargers, the result identities belong to the projected equivalents rather than the original composite component tables. ONR branch-status lists now use durable component indices rather than positional offsets.
Fault-current units	<code>z_thevenin</code> and <code>i_fault_pu</code> stay in per-unit complex form, while <code>ikpp_ka</code> , <code>ikss_ka</code> , <code>ip_ka</code> , <code>ib_ka</code> , and <code>ik_ka</code> are reported in kA.	SC C++ API results	Consumers should not compare per-unit and kA fields directly without the documented base quantities.

11.3 Solver-Specific Result Notes

Solver path	Result fields with special meaning	Where visible	Current note
Hybrid PF Newton / FDPF / DC PF	<code>converged</code> , <code>iterations</code> , <code>residual</code>	<code>Results_Summary</code> and compact PF summary sheet	The public PF artifacts expose only the compact scalar summary; detailed profiling arrays stay in memory and are not serialized.
Hybrid PF converter transfer postprocessing	<code>p_ac_mw</code> , <code>q_ac_mvar</code> , <code>p_dc_mw</code> , <code>loss_mw</code> , plus DC/DC transfer fields	dedicated and compact PF workbooks	Converter rows are matched by converter <code>index</code> ; when a solved transfer is missing, the exporter falls back to setpoints and static metadata.
Native AC OPF IPM	<code>outer_iterations</code> , <code>profiling.final_barrier_mu</code> , profiling counters	<code>OPF_Summary</code> and <code>ACOPFResult</code> JSON	These fields are meaningful primarily for the native IPM path; they may remain zero-like on simpler fallback paths.
Parity OPF formulation	<code>dpd_mw</code> , <code>dqd_mvar</code>	<code>OPF_Bus</code> and <code>ACOPFResult</code> JSON	The parity path exposes explicit active / reactive load-shedding vectors. On runs without shedding, the vectors may still exist but stay numerically zero.
OPF fallback selection	<code>status</code> , convergence flags, and partial vectors	<code>OPF_Summary</code> , <code>ACOPFResult</code> JSON	Consumers should read <code>status</code> together with <code>converged</code> ; some fallback paths can return structurally valid but less detailed result blocks.
Carbon tracing method	<code>tracing_verified</code> and <code>tracing_summary.*</code>	carbon workbook and JSON	These fields describe graph-tracing consistency, not matrix-solver success.
Carbon matrix method	<code>matrix_solved</code> , <code>matrix_residual</code> , and <code>matrix_summary.*</code>	carbon workbook and JSON	<code>matrix_residual</code> is specific to the matrix solve; it is not directly comparable to PF or OPF residual metrics.
Simplified short-circuit path	<code>z_thevenin</code> , <code>i_fault_pu</code> , <code>sk_mva</code> , <code>ikpp_kA</code>	<code>SCResult</code> / <code>BusFaultResult</code>	The batch and single-bus APIs share the same projected fault-admittance model; they differ only in whether every bus or one selected bus is evaluated.

Solver path	Result fields with special meaning	Where visible	Current note
Detailed short-circuit path	<code>ikss_*</code> , <code>ip_ka</code> , <code>ib_ka</code> , <code>ik_ka</code> , legacy placeholders, <code>only</code>	SCDetailedResult API	The detailed path now produces populated current fields for all buses, but <code>placeholders_only</code> and <code>pending_functions</code> remain legacy compatibility fields and explicit external-grid contribution is still folded into <code>ikss_no_motor_ka</code> .
ONR MILP plus verification PF	<code>open_branch_ids</code> , <code>closed_branch_ids</code> , <code>milp_objective</code> , <code>estimated_loss_mw</code> , <code>verification_pf</code> , <code>bc_stats</code>	ONRResult API	The branch-status lists describe the MILP topology decision, while <code>verification_pf</code> re-runs the nonlinear PF solver on the snapped binary topology.

11.4 Consumer Contracts

Results_* workbook family

Consumer slice	Join key	Stable / optional fields	Compatibility constraint
Results_ACBus, Results_DCBus	<code>index</code>	voltage fields are stable; <code>name</code> is descriptive only	consumers should join by <code>index</code> rather than row number even though row order follows the source bus tables.
Entity sheets with explicit ids	<code>index</code> on converter, storage, charging-station, charger, router, VPP, micro-grid, and mobile-storage sheets	identity and meta-data columns are stable; solved voltages / transfer values may fall back to deterministic defaults	additive columns are expected to be backward compatible; consumers should read by header name instead of fixed workbook position.
Results_Energy RouterPort	pair (<code>router_index</code> , <code>port_index</code>)	solved transfer fields may fall back to stored port meta-data; <code>port_name</code> is descriptive only	the flattened port contract is stable at the key-field level, not at the row-order level.
Results_BranchFlow	exported row order aligned to the source branch table, plus <code>from_bus/to_bus</code> ; not <code>index</code>	flow fields are stable; <code>index</code> is only the export row ordinal	consumers must not persist <code>Results_BranchFlow.index</code> as a durable foreign key because it is not the original branch id.

Consumer slice	Join key	Stable / optional fields	Compatibility constraint
Results_Summary	Key	current scalar keys are stable; extra keys may appear additively	consumers should parse it as a name/value sheet and must not depend on fixed row numbers.

OPF_* workbook family and ACOPFResult JSON

Consumer slice	Join key	Stable / optional fields	Compatibility constraint
OPF_Bus	index	vm_pu, va_deg, dpd_mw, and dqd_mvar are the stable solved fields; name and base_kv are echoes	workbook rows align to <code>sys.ac.buses</code> , but consumers should still join by <code>index</code> .
OPF_Generator	index	dispatch fields are stable; cost and fuel_type columns are echoed metadata	unknown future columns should be ignored, and consumers should not assume the sheet contains only solved variables.
OPF_Converter	index	pac_mw and qac_mvar are stable when the sheet has rows	the sheet may be present but empty when no VSC converters exist, so emptiness is not itself an error condition.
OPF_Summary and JSON scalar block	workbook Key; JSON top-level scalar names	convergence, iteration, objective, residual-like, and status fields are stable	summary consumers should tolerate additive keys while continuing to treat existing keys as stable.
ACOPFResult vector blocks	AC-bus order for vm, va, dpd_mw, dqd_mvar; generator order for pg_mw, qg_mvar; VSC order for pac_mw, qac_mvar	the profiling object is optional on import; absent arrays stay empty	the current public JSON / workbook contract still omits <code>pren_mw</code> , <code>qren_mvar</code> , <code>pstor_mw</code> , <code>qstor_mvar</code> , <code>pdcdc_mw</code> , and <code>pflex_mw</code> even though the in-memory result struct already contains them.

CarbonAnalysisResult JSON

Consumer slice	Join key	Stable / optional fields	Compatibility constraint
top-level scalar block	top-level key names	<code>tracing_verified</code> , <code>matrix_solved</code> , and <code>matrix_residual</code> are stable; absent keys default to zero-like values on import	additive scalar keys are backward compatible because the current importer only reads known fields.
<code>load_carbon[]</code> and <code>dc_load_carbon[]</code>	<code>load_index</code>	the explicit scalar fields are stable; <code>generator_supply_mw</code> may be absent and then defaults to an empty map	array order follows producer-side load order, but consumers should join by <code>load_index</code> when combining with other artifacts.
<code>branch_carbon[]</code> and <code>dc_branch_carbon[]</code>	<code>branch_index</code>	the explicit scalar fields are stable; <code>generator_loss_mw</code> may be absent and then defaults to an empty map	object-map key order is not stable and must not be used as a semantic ordering.
<code>bus_carbon[]</code> and <code>dc_bus_carbon[]</code>	<code>bus_index</code>	the explicit scalar fields are stable; absent array keeps the vector empty	row order follows producer-side bus order, but <code>bus_index</code> is the durable join key.
<code>vsc_carbon[]</code> and <code>dc_dc_carbon[]</code>	<code>converter_index</code>	scalar direction and power fields are stable; <code>generator_loss_mw</code> may be absent and then defaults to an empty map	consumers should join on converter index and not rely on array order once projected-canonical filtering is enabled.
<code>storage_carbon[]</code>	pair (<code>storage_index</code> , <code>is_dc</code>)	scalar storage-state fields are stable; <code>source_supply_mw</code> may be absent and then defaults to an empty map	<code>p_mw</code> uses the bus-injection sign convention and determines whether the row should be interpreted as discharge or charging sink.
<code>energy_router_carbon[]</code>	<code>router_index</code>	scalar router loss fields are stable; <code>source_loss_mw</code> may be absent and then defaults to an empty map	consumers should join by router index and not infer semantics from workbook row order.

Consumer slice	Join key	Stable / optional fields	Compatibility constraint
generator-allocation maps and summary blocks	source-id string for maps; top-level object name for summaries	<code>generator_loss_allocation</code> , <code>tracing_summary</code> , and <code>matrix_summary</code> are stable object names	map keys are JSON strings of internal source ids, and consumers must normalize them back to integer identifiers if a typed downstream schema requires that.

11.5 Dedicated Power-Flow Result Sheets

Results_ACBus

Field	Workbook sheet / column	Population rule	Meaning / note
<code>index</code>	<code>Results_ACBus.index</code>	copied from <code>sys.ac.buses[i].index</code>	stable AC bus identifier
<code>name</code>	<code>Results_ACBus.name</code>	copied from <code>sys.ac.buses[i].name</code>	descriptive bus label
<code>vm_pu</code> , <code>va_deg</code>	same columns	copied from <code>result.v</code> <code>m[i]</code> and <code>result.va[i]</code> ; out-of-range fallback 0.0	solved AC voltage magnitude and angle in bus-vector order

Results_DCBus

Field	Workbook sheet / column	Population rule	Meaning / note
<code>index</code>	<code>Results_DCBus.index</code>	copied from <code>sys.dc.buses[i].index</code>	stable DC bus identifier
<code>name</code>	<code>Results_DCBus.name</code>	copied from <code>sys.dc.buses[i].name</code>	descriptive bus label
<code>vm_pu</code>	<code>Results_DCBus.vm_pu</code>	copied from <code>result.vdc[i]</code> ; out-of-range fallback 0.0	solved DC voltage; the sheet keeps the AC-style column name <code>vm_pu</code>

Results_BranchFlow

Field	Workbook sheet / column	Population rule	Meaning / note
<code>index</code>	<code>Results_BranchFlow.index</code>	row ordinal <code>i+1</code> , not the source AC branch <code>index</code> field	export-side branch-flow row id
<code>from_bus</code> , <code>to_bus</code>	same columns	copied from <code>sys.ac.branches[i]</code> when present; fallback 0	terminal buses of the reported AC branch row
<code>pf_mw</code> , <code>qf_mvar</code>	same columns	copied from <code>result.branch_flows[i]</code>	sending-end active and reactive power
<code>pt_mw</code> , <code>qt_mvar</code>	same columns	copied from <code>result.branch_flows[i]</code>	receiving-end active and reactive power

Results_DCBranch

Field	Workbook sheet / column	Population rule	Meaning / note
<code>index, name</code>	corresponding columns	copied from <code>sys.dc.branches[i]</code>	stable identifier and label
<code>from_bus, to_bus</code>	same columns	copied from <code>sys.dc.branches[i]</code>	terminal DC bus indices
<code>in_service</code>	same column	copied from <code>sys.dc.branches[i].in_service</code> via <code>bool_str</code>	availability flag in workbook boolean-token form
<code>v_from_pu, v_to_pu</code>	same columns	looked up from solved <code>result.vdc</code> by branch terminal bus; fallback 1.0	solved DC terminal voltages
<code>i_pu</code>	<code>Results_DCBranch.i_pu</code>	computed as $(v_{\text{from}} - v_{\text{to}})/r_{\text{pu}}$ when in service and <code>r_pu</code> >0; else 0.0	derived branch current on system base
<code>p_from_mw, p_to_mw</code>	same columns	computed from solved terminal voltages, derived current, and system base MVA	terminal powers implied by the solved DC branch state
<code>loss_mw</code>	<code>Results_DCBranch.loss_mw</code>	computed as <code>p_from_mw+p_to_mw</code>	exported branch-loss figure

Results_VSCConverter

Field	Workbook sheet / column	Population rule	Meaning / note
<code>index, name, bus_ac, bus_dc</code>	corresponding columns	copied from <code>sys.vsc_converters[i]</code>	stable converter identity and terminal buses
<code>in_service, control_mode</code>	same columns	copied from the input converter object	static converter metadata, not a solver-selected mode switch
<code>p_ac_mw, q_ac_mvar, p_dc_mw, loss_mw</code>	same columns	taken from matching <code>result.vsc_transfers[index]</code> when available; fallback uses <code>p_set_mw</code> , <code>q_set_mvar</code> , and stored <code>loss_mw</code>	solved transfer report with deterministic metadata fallback
<code>p_set_mw, q_set_mvar</code>	same columns	copied from <code>sys.vsc_converters[i]</code>	original setpoint metadata retained beside solved transfer values
<code>v_ac_pu, v_dc_pu</code>	same columns	solved voltages looked up by terminal buses; fallback 1.0	terminal AC and DC voltage annotations
<code>eta, loss_percent</code>	same columns	copied from the input converter object	static efficiency / loss metadata

Results_DCDCConverter

Field	Workbook sheet / column	Population rule	Meaning / note
<code>index, name, bus_in, bus_out</code>	corresponding columns	copied from <code>sys.dcdc_converters[i]</code>	stable converter identity and terminal buses
<code>in_service, control_mode</code>	same columns	copied from the input converter object	static converter metadata
<code>p_ref_mw, v_ref_pu, eta</code>	same columns	copied from <code>sys.dcdc_converters[i]</code>	original reference and efficiency metadata

Field	Workbook sheet / column	Population rule	Meaning / note
v_in_pu, v_out_pu	same columns	solved voltages looked up by terminal buses; fallback 1.0	terminal DC voltage annotations
p_in_mw, p_out_mw, loss_mw	same columns	taken from matching <code>result.dcdc_transfers[index]</code> when available; fallback uses <code>p_ref_mw</code> and derived zero-loss default	solved DC/DC transfer report with metadata fallback

Results_Storage_AC

Field	Workbook sheet / column	Population rule	Meaning / note
index, name, bus, in_service	corresponding columns	copied from <code>sys.ac.storage[i]</code>	stable identity, connected bus, and availability
p_mw, q_mvar	same columns	copied from <code>sys.ac.storage[i]</code>	current storage dispatch snapshot; not solved from a storage state estimator
soc	Results_Storage_AC.soc	copied from <code>sys.ac.storage[i].soc_init</code>	reported state of charge is the input initial SOC
soh	Results_Storage_AC.soh	copied from <code>sys.ac.storage[i].soh</code>	stored state-of-health metadata
vm_pu, va_deg	same columns	solved AC bus voltage looked up by <code>bus</code> ; fallback 1.0/0.0	bus-state annotation for the storage location

Results_Storage_DC

Field	Workbook sheet / column	Population rule	Meaning / note
index, name, bus, in_service	corresponding columns	copied from <code>sys.dc.storage[i]</code>	stable identity, connected bus, and availability
p_mw, q_mvar	same columns	copied from <code>sys.dc.storage[i]</code>	current storage dispatch snapshot
soc	Results_Storage_DC.soc	copied from <code>sys.dc.storage[i].soc_init</code>	reported state of charge is the input initial SOC
soh	Results_Storage_DC.soh	copied from <code>sys.dc.storage[i].soh</code>	stored state-of-health metadata
vdc_pu	Results_Storage_DC.vdc_pu	solved DC voltage looked up by <code>bus</code> ; fallback 1.0	bus-state annotation for the storage location

Results_ChargingStation

Field	Workbook sheet / column	Population rule	Meaning / note
index, name, bus, in_service	corresponding columns	copied from <code>sys.ac.charging_stations[i]</code>	stable station identity, connected bus, and availability
num_chargers, p_total_kw, q_total_kvar, max_power_kw, utilization_rate	same columns	copied from <code>sys.ac.charging_stations[i]</code>	current aggregate station metadata; no per-solver charging dispatch reconstruction is performed here

Field	Workbook sheet / column	Population rule	Meaning / note
vm_pu, va_deg	same columns	solved AC bus voltage looked up by <code>bus</code> ; fallback 1.0/0.0	bus-state annotation for the station location

Results_Charger

Field	Workbook sheet / column	Population rule	Meaning / note
index, name, station_id	corresponding columns	copied from <code>sys.ac.chargers[i]</code>	stable charger identity and parent-station link
station_bus	<code>Results_Charger.station_bus</code>	joined from the parent charging-station map by <code>station_id</code> ; fallback 0	resolved AC bus of the parent charging station
in_service, charger_type, p_rate_d_kw, p_ch_max_kw, p_dis_max_kw, eta, v2g_capable	same columns	copied from <code>sys.ac.chargers[i]</code>	static charger metadata repeated in the result workbook
vm_pu, va_deg	same columns	solved AC bus voltage looked up by the resolved <code>station_bus</code> ; fallback 1.0/0.0	station-bus voltage annotation, not per-charger internal electrical state

Results_EnergyRouter

Field	Workbook sheet / column	Population rule	Meaning / note
index, name, in_service, router_type, num_ports, p_rated_mw, loss_percent	corresponding columns	copied from <code>sys.energy_routers[i]</code>	router metadata preserved in the result workbook
p_port_sum_mw, q_port_sum_mvar	same columns	computed by summing the stored router-port <code>p_mw</code> / <code>q_mvar</code> values	router-level summary over the current port structs; it does not re-sum the solved transfer table

Results_EnergyRouterPort

Field	Workbook sheet / column	Population rule	Meaning / note
router_index, port_index, port_name, bus	corresponding columns	copied from each nested port row under <code>sys.energy_routers[*].ports[*]</code>	flattened router / port identity and connected bus
port_type, control_mode, in_service	same columns	copied from the nested port object	static port metadata
p_mw, q_mvar, v_solved_pu	same columns	taken from the matching solved router-port transfer when available; fallback uses the port's stored <code>p_mw</code> , <code>q_mvar</code> , and solved bus voltage	solved per-port transfer report with deterministic metadata fallback

Results_VPP

Field	Workbook sheet / column	Population rule	Meaning / note
index, name, aggregation_bus, in_service	corresponding columns	copied from <code>sys.vpps[i]</code>	stable VPP identity and aggregation location
p_output_mw, q_output_mvar, pmax_mw, pmin_mw	same columns	copied from <code>sys.vpps[i]</code>	current VPP aggregate output / limits metadata
vm_pu, va_deg	same columns	solved AC bus voltage looked up by <code>aggregation_bus</code> ; fallback 1.0/0.0	aggregation-bus state annotation

Results_Microgrid

Field	Workbook sheet / column	Population rule	Meaning / note
index, name, pcc_bus, operating_mode, in_service	corresponding columns	copied from <code>sys.microgrids[i]</code>	microgrid identity and current mode metadata
p_exchange_mw, total_generation_mw, total_load_mw	same columns	copied from <code>sys.microgrids[i]</code>	current aggregate exchange / generation / demand snapshot
vm_pu, va_deg	same columns	solved AC bus voltage looked up by <code>pcc_bus</code> ; fallback 1.0/0.0	PCC bus-state annotation
internal_buses	<code>Results_Microgrid.internal_buses</code>	comma-joined string formed from the stored microgrid membership vector	flattened membership list; workbook does not preserve the vector structure directly

Results_MobileStorage

Field	Workbook sheet / column	Population rule	Meaning / note
index, name, bus, status, in_service	corresponding columns	copied from <code>sys.mobile_storage[i]</code>	mobile-storage identity, current location, and mobility state metadata
p_mw, q_mvar	same columns	copied from <code>sys.mobile_storage[i]</code>	current dispatch snapshot
soc	<code>Results_MobileStorage.soc</code>	copied from <code>sys.mobile_storage[i].soc_init</code>	reported state of charge is the input initial SOC
target_bus	<code>Results_MobileStorage.target_bus</code>	copied from <code>sys.mobile_storage[i].target_buses</code>	relocation target metadata
vm_pu, va_deg	same columns	solved AC bus voltage looked up by the current bus; fallback 1.0/0.0	current-bus state annotation

Results_Summary

Field	Workbook sheet / column	Population rule	Meaning / note
converged	Results_Summary row with Key=converged	written from <code>result.c</code> onverged via <code>bool_str</code>	PF convergence flag using workbook boolean-token form
iterations	Results_Summary row with Key=iterations	written from <code>result.iterations</code>	total PF iteration count
residual	Results_Summary row with Key=residual	written from <code>result.residual</code>	final PF residual metric

11.6 Compact Power-Flow Result Columns

Field	Workbook sheet / column	Population rule	Meaning / note
<code>result_vm_pu</code> , <code>result_va_deg</code>	ACBus appended columns	copied from <code>result.v</code> <code>m[i]</code> and <code>result.va[i]</code> in AC bus order	compact PF solved AC bus state
<code>result_vdc_pu</code>	DCBus appended column	copied from <code>result.vdc</code> <code>[i]</code> in DC bus order	compact PF solved DC bus voltage
<code>result_pf_mw</code> , <code>result_qf_mvar</code> , <code>result_pt_mw</code> , <code>result_qt_mvar</code>	ACBranch appended columns	copied from <code>result.branch_flows[i]</code>	compact PF AC branch-flow report
<code>result_v_from_pu</code> , <code>result_v_to_pu</code> , <code>result_i_pu</code> , <code>result_p_from_mw</code> , <code>result_p_to_mw</code> , <code>result_loss_mw</code>	DCBranch appended columns	recomputed from solved DC voltages and branch resistance, same as <code>Results_DCBranch</code>	compact PF DC branch report
<code>result_vm_pu</code> , <code>result_va_deg</code>	Storage, Load, ACLoad, ChargingStation, VPP, Microgrid, MobileStorage appended columns	solved AC voltage looked up by each row's connected bus or aggregation bus	compact PF AC-side bus-state annotation on input tables
<code>result_vdc_pu</code>	DCStorage, DCLoad appended column	solved DC voltage looked up by each row's connected bus	compact PF DC-side bus-state annotation on input tables
<code>result_p_ac_mw</code> , <code>result_q_ac_mvar</code> , <code>result_p_dc_mw</code> , <code>result_loss_mw</code> , <code>result_v_ac_pu</code> , <code>result_v_dc_pu</code>	VSCConverter appended columns	populated from matching <code>result.vsc_transfers[index]</code> plus solved terminal voltages	compact PF converter transfer report
<code>result_p_in_mw</code> , <code>result_p_out_mw</code> , <code>result_loss_mw</code> , <code>result_v_in_pu</code> , <code>result_v_out_pu</code>	DCDCConverter appended columns	populated from matching <code>result.dcdc_transfers[index]</code> plus solved terminal voltages	compact PF DC/DC converter report
Results_Summary	separate sheet still written in compact PF export	same key-value rows as the dedicated PF result workbook	compact PF export still carries a standalone summary sheet

11.7 Dedicated OPF Sheets

OPF_Bus

Field	Workbook sheet / column	Population rule	Meaning / note
<code>index</code> , <code>name</code> , <code>base_kv</code>	corresponding columns	copied from <code>sys.ac.buses[i]</code>	stable bus identity and base-voltage metadata

Field	Workbook sheet / column	Population rule	Meaning / note
vm_pu, va_deg	same columns	copied from <code>result.v</code> <code>m[i]</code> and <code>result.va[i]</code> ; out-of-range fallback 0.0	optimal AC bus voltage solution
dpd_mw, dqd_mvar	same columns	copied from <code>result.dpd</code> <code>_mw[i]</code> and <code>result.dqd</code> <code>_mvar[i]</code> ; out-of-range fallback 0.0	parity-formulation bus-level active / reactive load shedding

OPF_Generator

Field	Workbook sheet / column	Population rule	Meaning / note
index, name, bus	corresponding columns	copied from <code>sys.ac.generators[i]</code>	stable generator identity and connected bus
pg_mw, qg_mvar	same columns	copied from <code>result.pg_mw[i]</code> and <code>result.qg_mvar[i]</code> ; fallback to input <code>pg_mw/qg_mvar</code>	optimal generator dispatch
pmax_mw, pmin_mw, qmax_mvar, qmin_mvar	same columns	copied from <code>sys.ac.generators[i]</code>	generator capability metadata retained beside the optimal point
cost_c2, cost_c1, cost_c0, fuel_type	same columns	copied from <code>sys.ac.generators[i]</code>	cost-model and fuel metadata used for interpretation

OPF_Converter

Field	Workbook sheet / column	Population rule	Meaning / note
index, name, bus_ac, bus_dc	corresponding columns	copied from <code>sys.vsc_converters[i]</code>	stable VSC identity and terminal buses
pac_mw, qac_mvar	same columns	copied from <code>result.pac_mw[i]</code> and <code>result.qac_mvar[i]</code> ; out-of-range fallback 0.0	optimal AC-side converter operating point

OPF_Summary

Field	Workbook sheet / column	Population rule	Meaning / note
converged	OPF_Summary row with Key=converged	written from <code>result.converged</code> via <code>bool_str</code>	OPF convergence flag
iterations	OPF_Summary row with Key=iterations	written from <code>result.iterations</code>	total inner-iteration count
outer_iterations	OPF_Summary row with Key=outer_iterations	written from <code>result.outer_iterations</code>	barrier / outer-loop iteration count
objective	OPF_Summary row with Key=objective	written from <code>result.objective</code>	final objective value
max_constraint_violation	OPF_Summary row with Key= max_constraint_violation	written from <code>result.max_constraint_violation</code>	worst final feasibility residual
max_stationarity	OPF_Summary row with Key=max_stationarity	written from <code>result.max_stationarity</code>	worst final first-order optimality metric

Field	Workbook sheet / column	Population rule	Meaning / note
status	OPF_Summary row with Key=status	written from result.status	textual solver termination status

ACOPFResult JSON

Field	JSON key / path	Export / import rule	Meaning / note
converged, iterations, outer_iterations, objective, max_constraint_violation, max_stationarity, status	top-level scalar keys	bidirectional; missing keys default to false, zero, or empty string	OPF scalar summary block
vm, va	top-level arrays	bidirectional; absent array keeps the vector empty	bus-voltage vectors in bus order
pg_mw, qg_mvar	top-level arrays	bidirectional; absent array keeps the vector empty	generator dispatch vectors in generator order
dpd_mw, dqd_mvar	top-level arrays	bidirectional; absent array keeps the vector empty	bus-level parity load-shedding vectors
pac_mw, qac_mvar	top-level arrays	bidirectional; absent array keeps the vector empty	VSC AC-side operating-point vectors
profiling.linear_solver_backend	profiling[linear_solver_backend]	bidirectional; default empty string	linear solver backend name recorded by the OPF run
profiling.analyze_calls, profiling.linear_solve_calls	same nested keys	bidirectional; default 0	OPF linear-solver call counters
factorization_calls	same nested key	bidirectional; default 0	OPF sparse factorization counter
profiling.total_iterations	profiling[total_iterations]	bidirectional; default 0	total OPF inner iterations reported by the solver
profiling.accepted_steps	same nested key	bidirectional; default 0	accepted line-search steps
profiling.rejected_steps	same nested key	bidirectional; default 0	rejected line-search steps
profiling.final_barrier_mu	profiling[final_barrier_mu]	bidirectional; default 0.0	final barrier parameter reported by the IPM path
pren_mw, qren_mvar, pstor_mw, qstor_mvar, pdcdc_mw, pflex_mw	none	not part of the current JSON or workbook contract	enhanced component-dispatch vectors exist in ACOPFResult but are not serialized today

11.8 Carbon Result JSON and Workbook Dictionaries

CarbonAnalysisResult JSON

Field	JSON key / path	Export / import rule	Meaning / note
tracing_verified, matrix_solved	top-level scalar booleans	bidirectional; default false	method-level status flags for tracing and matrix solutions

Field	JSON key / path	Export / import rule	Meaning / note
<code>matrix_residual</code>	top-level scalar number	bidirectional; default 0.0	residual metric from the matrix method
<code>load_carbon[]</code> and <code>dc_load_carbon[]</code>	top-level arrays	bidirectional; absent array keeps the vector empty	per-load tracing results for AC loads and DC loads respectively
<code>*.load_index,</code> <code>bus,</code> <code>demand_mw,</code> <code>carbon_intensity</code> <code>_tco2_mwh,</code> <code>total_</code> <code>emissions_tco2</code>	nested keys on each AC/DC load row	bidirectional; defaults 0 or 0.0	flat load-level carbon report fields
<code>per-load generator_supply_mw</code> map	nested object map keyed by source id	bidirectional; absent map keeps the per-load map empty	source-contribution map preserved only in JSON form
<code>branch_carbon[]</code> and <code>dc_branch_carbon[]</code>	top-level arrays	bidirectional; absent array keeps the vector empty	per-branch loss-allocation results for AC branches and DC branches respectively
<code>*.branch_index,</code> <code>from_bus,</code> <code>to_bus,</code> <code>loss_mw,</code> <code>carbon_intensity_tco2_mwh,</code> <code>total_emissions_tco2</code>	nested keys on each AC/DC branch row	bidirectional; defaults 0 or 0.0	flat branch-loss carbon report fields
<code>per-branch generator_loss_mw</code> map	nested object map keyed by source id	bidirectional; absent map keeps the per-branch map empty	source loss-allocation map preserved only in JSON form
<code>bus_carbon[]</code> and <code>dc_bus_carbon[]</code>	top-level arrays	bidirectional; absent array keeps the vector empty	matrix-based AC-bus and DC-bus carbon intensities
<code>*.bus_index,</code> <code>carbon_intensity</code> <code>_tco2_mwh</code>	nested keys on each AC/DC bus row	bidirectional; defaults 0 or 0.0	explicit bus-level carbon report fields
<code>vsc_carbon[]</code>	top-level array	bidirectional; absent array keeps the vector empty	converter-loss report for VSC links in projected-converter order
<code>vsc_carbon[].converter_index,</code> <code>bus_ac,</code> <code>bus_dc,</code> <code>ac_to_dc,</code> <code>input_power_mw,</code> <code>output_power_mw,</code> <code>loss_mw,</code> <code>carbon_intensity_tco2_mwh,</code> <code>total_emissions_tco2</code>	nested keys on each VSC row	bidirectional; defaults 0, false , or 0.0	reports actual hybrid transfer direction plus converter loss carbon intensity
<code>dcdc_carbon[]</code>	top-level array	bidirectional; absent array keeps the vector empty	converter-loss report for DCDC links in projected-converter order
<code>dcdc_carbon[].converter_index,</code> <code>bus_in,</code> <code>bus_out,</code> <code>input_to_output,</code> <code>input_power_mw,</code> <code>output_power_mw,</code> <code>loss_mw,</code> <code>carbon_intensity_tco2_mwh,</code> <code>total_emissions_tco2</code>	nested keys on each DCDC row	bidirectional; defaults 0, false , or 0.0	reports actual DC/DC transfer direction plus converter loss carbon intensity

Field	JSON key / path	Export / import rule	Meaning / note
<code>storage_carbon[]</code>	top-level array	bidirectional; absent array keeps the vector empty	per-storage carbon report covering both explicit discharge-side sources and traced charging-side sinks
<code>storage_carbon[[] .storage_index, bus, is_dc, p_mw, soc, stored_energ y_mwh, soc_carbon _intensity_tco2_ mwh, carbon_int ensity_tco2_mwh, total_emissions_ tco2</code>	nested keys on each storage row	bidirectional; defaults 0, false , or 0.0	p_mw keeps the bus-injection sign convention; charging rows additionally preserve a source-attribution map in JSON
<code>energy_router_carbon[]</code>	top-level array	bidirectional; absent array keeps the vector empty	per-router internal-loss report reconstructed from port transfer signs and magnitudes
<code>energy_router_carbon[[] .router_index, in put_power_mw, output_power_mw, loss_mw, active_i nput_ports, acti ve_output_ports, carbon_intensity _tco2_mwh, total_ emissions_tco2</code>	nested keys on each router row	bidirectional; defaults 0 or 0.0	router loss is derived from actual port mismatch; JSON preserves the source-loss map
<code>generator_loss_allocation</code> entries	top-level object map keyed by source id	bidirectional; absent map keeps the allocation map empty	total MW of branch and converter losses allocated to each internal source id
<code>tracing_summary.* matrix_summary.*</code>	nested summary objects	bidirectional; absent object keeps defaults	system-level emissions summary blocks for the tracing and matrix methods
<code>tracing_summary. total_generation_ emissions_tco2, total_load_emiss ions_tco2, total_loss_emiss ions_tco2, balanc e_error_tco2, ba lance_error_pct</code>	same nested keys under both summary objects	bidirectional; default 0.0	emissions-balance totals and consistency metrics

LoadCarbon

Field	Workbook sheet / column	Population rule	Meaning / note
<code>load_index, bus, demand_mw, carbon _intensity_tco2_ mwh, total_emissi ons_tco2</code>	corresponding columns	copied from each result.load_carbon[i] row	flat workbook view of the per-load carbon result without the generator-contribution map

DCLoadCarbon

Field	Workbook sheet / column	Population rule	Meaning / note
load_index, bus, demand_mw, carbon _intensity_tco2_ mwh, total_emissi ons_tco2	corresponding columns	copied from each resu lt.dc_load_carbon[i] row	flat workbook view of the per- DC-load carbon result without the source-contribution map

BusCarbon

Field	Workbook sheet / column	Population rule	Meaning / note
bus_index, carbon _intensity_tco2_ mwh	corresponding columns	copied from each resu lt.bus_carbon[i] row	flat workbook view of the per- bus carbon intensity

DCBusCarbon

Field	Workbook sheet / column	Population rule	Meaning / note
bus_index, carbon _intensity_tco2_ mwh	corresponding columns	copied from each resu lt.dc_bus_carbon[i] row	flat workbook view of the per- DC-bus carbon intensity

BranchCarbon

Field	Workbook sheet / column	Population rule	Meaning / note
branch_index, from_bus, to_bus, loss_mw, carbon_i ntensity_tco2_mw h, total_emission s_tco2	corresponding columns	copied from each resu lt.branch_carbon[i] row	flat workbook view of the per- branch carbon-loss result with- out the generator-loss map

DCBranchCarbon

Field	Workbook sheet / column	Population rule	Meaning / note
branch_index, from_bus, to_bus, loss_mw, carbon_i ntensity_tco2_mw h, total_emission s_tco2	corresponding columns	copied from each resu lt.dc_branch_carbo n[i] row	flat workbook view of the per- DC-branch carbon-loss result without the source-loss map

VSCCarbon

Field	Workbook sheet / column	Population rule	Meaning / note
converter_index, bus_ac, bus_dc	corresponding columns	copied from each resu lt.vsc_carbon[i] row	stable converter identifier and terminal buses
ac_to_dc	corresponding column	exported as lowercase workbook string tr ue/false	actual solved transfer direction, not merely the input setpoint sign

Field	Workbook sheet / column	Population rule	Meaning / note
<code>input_power_mw</code> , <code>output_power_mw</code> , <code>loss_mw</code>	corresponding columns	copied from each <code>result.vsc_carbon[i]</code> row	carrier input, carrier output, and converter loss in MW
<code>carbon_intensity_tco2_mwh</code> , <code>total_emissions_tco2</code>	corresponding columns	copied from each <code>result.vsc_carbon[i]</code> row	loss-side carbon intensity and the corresponding converter-loss emissions

DCDCCarbon

Field	Workbook sheet / column	Population rule	Meaning / note
<code>converter_index</code> , <code>bus_in</code> , <code>bus_out</code>	corresponding columns	copied from each <code>result.dcdc_carbon[i]</code> row	stable converter identifier and declared input/output buses
<code>input_to_output</code>	corresponding column	exported as lowercase workbook string <code>true/false</code>	actual solved transfer direction relative to the declared <code>bus_in/bus_out</code> pair
<code>input_power_mw</code> , <code>output_power_mw</code> , <code>loss_mw</code>	corresponding columns	copied from each <code>result.dcdc_carbon[i]</code> row	converter input, converter output, and DCDC loss in MW
<code>carbon_intensity_tco2_mwh</code> , <code>total_emissions_tco2</code>	corresponding columns	copied from each <code>result.dcdc_carbon[i]</code> row	loss-side carbon intensity and the corresponding converter-loss emissions

StorageCarbon

Field	Workbook sheet / column	Population rule	Meaning / note
<code>storage_index</code> , <code>bus</code>	corresponding columns	copied from each <code>result.storage_carbon[i]</code> row	stable storage identifier and connected bus
<code>is_dc</code>	corresponding column	exported as lowercase workbook string <code>true/false</code>	distinguishes AC storage from DC storage when indices overlap across subsystems
<code>p_mw</code>	corresponding column	copied from <code>result.storage_carbon[i].p_mw</code>	signed storage dispatch in bus-injection convention; positive means discharge, negative means charging sink
<code>soc</code> , <code>stored_energy_mwh</code>	corresponding columns	copied from the storage carbon result row	reported storage state snapshot used by the carbon model
<code>soc_carbon_intensity_tco2_mwh</code>	corresponding column	copied from the storage carbon result row	explicit carbon intensity of stored energy used when the unit discharges
<code>carbon_intensity_tco2_mwh</code> , <code>total_emissions_tco2</code>	corresponding columns	copied from the storage carbon result row	current-mode carbon intensity and emissions; charging uses traced incoming intensity, discharging uses SOC intensity

EnergyRouterCarbon

Field	Workbook sheet / column	Population rule	Meaning / note
<code>router_index</code>	corresponding column	copied from each <code>result.energy_router_carbon[i]</code> row	stable energy-router identifier

Field	Workbook sheet / column	Population rule	Meaning / note
input_power_mw, output_power_mw, loss_mw	corresponding columns	copied from the router carbon result row	router-wide input, output, and internally allocated loss reconstructed from solved port transfers
active_input_ports, active_output_ports	corresponding columns	copied from the router carbon result row	number of non-zero sending and receiving ports participating in the router carbon graph
carbon_intensity _tco2_mwh, total_ emissions_tco2	corresponding columns	copied from the router carbon result row	loss-side carbon intensity and the corresponding router-loss emissions

Summary in the carbon workbook

Field	Workbook sheet / cell block	Population rule	Meaning / note
TracingMethod	Summary [A1]	literal label written at row 1	heading for the tracing summary block
tracing_summary. total_generation_ emissions_tco2	Summary [A2:B2]	key literal plus result .tracing_summary.tot al_generation_emissi ons_tco2	tracing-method generation emissions total
tracing_summary. total_load_emissi ons_tco2	Summary [A3:B3]	key literal plus result .tracing_summary.tot al_load_emissions_tc o2	tracing-method load emis sions total
tracing_summary. total_loss_emissi ons_tco2	Summary [A4:B4]	key literal plus result .tracing_summary.tot al_loss_emissions_tc o2	tracing-method loss emis sions total
tracing_summary. balance_error_tco 2	Summary [A5:B5]	key literal plus result .tracing_summary.bal ance_error_tco2	tracing-method absolute emissions-balance error
tracing_summary. balance_error_pct	Summary [A6:B6]	key literal plus result .tracing_summary.bal ance_error_pct	tracing-method relative emissions-balance error
tracing_verified	Summary [A7:B7]	key literal plus lower- case string true/false	tracing verification flag; note the lowercase boolean spelling differs from PF / OPF workbooks
MatrixMethod	Summary [A9]	literal label written at row 9	heading for the matrix- summary block
matrix_summary.t otal_generation_ emissions_tco2	Summary [A10:B10]	key literal plus result .matrix_summary.tota l_generation_emissio ns_tco2	matrix-method generation emissions total
matrix_summary.t otal_load_emissi ons_tco2	Summary [A11:B11]	key literal plus result .matrix_summary.tota l_load_emissions_tco2	matrix-method load emis sions total
matrix_summary.t otal_loss_emissi ons_tco2	Summary [A12:B12]	key literal plus result .matrix_summary.tota l_loss_emissions_tco2	matrix-method loss emis sions total
matrix_summary.b alance_error_tco 2	Summary [A13:B13]	key literal plus result .matrix_summary.bala nce_error_tco2	matrix-method absolute emissions-balance error
matrix_summary.b alance_error_pct	Summary [A14:B14]	key literal plus result .matrix_summary.bala nce_error_pct	matrix-method relative emissions-balance error

Field	Workbook sheet / cell block	Population rule	Meaning / note
<code>matrix_solved</code>	Summary[A15:B15]	key literal plus lower-case string true/false	matrix-solver success flag
<code>matrix_residual</code>	Summary[A16:B16]	key literal plus result <code>.matrix_residual</code>	matrix-method residual metric

11.9 Short-Circuit and Network-Reconfiguration API Results

BusFaultResult and SCResult

Field	API path	Population rule	Meaning / note
<code>base_mva</code>	<code>SCResult.base_mva</code>	copied from the projected study base MVA	system base used when converting per-unit short-circuit quantities to engineering units
<code>bus_results[]</code>	<code>SCResult.bus_results</code>	one row per projected AC bus in projected bus order	batch short-circuit output vector for the simplified solver path
<code>bus_id</code>	<code>bus_results[k].bus_id</code>	copied from <code>projected.ac.buses[k].index</code>	stable projected-bus join key
<code>z_thevenin</code>	<code>bus_results[k].z_thevenin</code>	diagonal Thevenin term recovered from the factored fault network	complex Thevenin impedance seen at the faulted bus in per-unit
<code>i_fault_pu</code>	<code>bus_results[k].i_fault_pu</code>	derived from the selected fault-type equation and the computed Thevenin impedance	complex fault current in per-unit on the study base
<code>sk_mva, ikpp_ka</code>	corresponding fields	converted from the per-unit fault result using base MVA and base kV	reported three-phase short-circuit power and initial symmetrical current in engineering units

SCDetailedResult and SCDetailedBusResult

Field	API path	Population rule	Meaning / note
<code>fault_bus_id</code>	<code>SCDetailedResult.fault_bus_id</code>	copied from the user-selected fault bus index	stable identifier of the studied fault location
<code>placeholders_only, pending_functions</code>	top-level legacy fields	current implementation sets them to false and empty respectively	compatibility-only scaffold markers retained in the public header; they no longer describe the current runtime behavior accurately
<code>bus_results[]</code>	<code>SCDetailedResult.bus_results</code>	one row per projected AC bus in projected bus order	detailed IEC-style current report for the selected fault case
<code>bus_id, ikss_ka, ikss_1_ka, ikss_2_ka</code>	nested fields on each bus row	computed from the dense positive- / zero-sequence matrices and transfer terms	bus identity plus initial short-circuit current totals in kA
<code>ikss_gen_contrib_ka, ikss_motor_contrib_ka, ikss_load_contrib_ka, ikss_sgen_contrib_ka</code>	nested fields on each bus row	derived from the current implementation's explicit per-source buckets	explicit contribution buckets currently cover generators, motors, load-motor fractions, and static generators only

Field	API path	Population rule	Meaning / note
<code>ikss_no_motor_ka</code>	nested field on each bus row	computed as the residual after subtracting the explicit contribution buckets from total <code>ikss</code>	currently absorbs external-grid contribution and any remaining network-side residual, so it is not a pure “network only” channel
<code>ip_ka</code> , <code>ib_ka</code> , <code>ik_ka</code>	nested fields on each bus row	derived from the detailed κ -factor, breaking-current, and steady-state-current calculations	detailed IEC-style peak, breaking, and steady-state short-circuit currents in kA

ONRRResult

Field	API path	Population rule	Meaning / note
<code>open_branch_ids</code> , <code>closed_branch_ids</code>	top-level vectors	built from the rounded α_b topology decision after the MILP solve	branch-status lists contain stable <code>ACBranch::index</code> values from the projected AC system, enabling consumers to join results back to the original branch table by component identity
<code>milp_objective</code>	<code>ONRRResult.milp_objective</code>	copied from the native branch-and-cut solve statistics	linearized MILP objective value before the nonlinear verification PF
<code>estimated_loss_mw</code>	<code>ONRRResult.estimated_loss_mw</code>	recomputed from closed-branch $\sum r_b P_b $ on system base	engineering-unit approximation of the MILP loss proxy
<code>verification_pf</code>	<code>ONRRResult.verification_pf</code>	produced by rerunning the nonlinear PF solver on the snapped binary topology	full <code>PowerFlowResult</code> verification artifact for the chosen reconfiguration plan
<code>feasible</code> , <code>optimal</code>	corresponding flags	copied from branch-and-cut success / gap checks	high-level solve-status flags; <code>optimal</code> is tied to the MILP gap criterion, not to the nonlinear verification PF result
<code>bc_stats</code>	<code>ONRRResult.bc_stats</code>	copied from the native branch-and-cut engine result	node counts, cut counts, gap, runtime, and other MILP diagnostics

11.10 Current Output-Side Gaps

Area	Artifact	Status	Engineering consequence
<code>PowerFlowResult</code> JSON	none	not implemented	PF profiling, branch flows, and converter transfer vectors cannot currently round-trip through a dedicated JSON result file.
PF profiling block	PF workbooks	not exported	the in-memory <code>PowerFlowResult.profiling</code> structure is not written to workbook output.

Area	Artifact	Status	Engineering consequence
ACOPFResult enhanced dispatch vectors	OPF JSON / workbook	exported with packed semantics	Renewable, storage, DC/DC, and flexible-load dispatch vectors are exported as packed variable arrays with companion <code>ComponentRef</code> identity maps. JSON uses structured <code>renewable_dispatch</code> / <code>storage_dispatch</code> / <code>dc_dc_dispatch</code> / <code>flex_dispatch</code> objects; workbook sheets use <code>var_idx</code> , <code>source_type</code> , and <code>component_index</code> columns.
carbon contribution maps	carbon workbook	not exported	<code>generator_supply_mw</code> , <code>generator_loss_mw</code> , <code>source_supply_mw</code> , <code>source_loss_mw</code> , and <code>generator_loss_allocation</code> are available only in JSON.
OPF_Converter	OPF workbook	conditional population	the sheet is only populated when VSC converters exist; otherwise the sheet may stay empty even though the workbook pre-creates it.
short-circuit result serialization	SC outputs	not implemented	<code>SCResult</code> and <code>SCDetailedResult</code> can only be consumed through the C++ API today.
ONR result serialization	ONR outputs	not implemented	<code>ONRResult</code> has no workbook / JSON export, so reconfiguration studies do not yet have a stable file-level output contract.
ONR branch identity	<code>ONRResult</code>	resolved	branch-status vectors now emit stable <code>ACBranch::index</code> values from the projected system, matching the component identity contract.
<code>SCDetailedResult</code> legacy fields	detailed SC API	stale public contract	<code>placeholders_only</code> and <code>pending_functions</code> remain in the public header even though the implementation now fills detailed results.

12 Power-Flow Models

12.1 Scope and Architecture Overview

This chapter documents the steady-state hybrid AC/DC power-flow (PF) module in `src/power_flow/` and `src/engine/solver/native/nle/`. The module is production-quality and ships as the primary numerical engine for planning, optimization warm-starts, and annual simulation.

The module exposes seven public solver paths:

Public API	Primary file	Scope
<code>solve_power_flow</code>	<code>hybrid.cpp</code>	Hybrid AC/DC Newton; full coupling
<code>solve_dc_power_flow</code>	<code>dc.cpp</code>	DC-only Newton
<code>solve_ac_power_flow</code> (FDPF)	<code>fdpf_solver.cpp</code>	Fast-decoupled AC-only
<code>solve_ac_dc_power_flow</code>	<code>ac_linearized_pf.cpp</code>	Linearized DC-load-flow angle + DC
<code>solve_adaptive_power_flow</code>	<code>adaptive_solver.cpp</code>	Island-aware with fallback chain
<code>solve_islanded_power_flow</code>	<code>adaptive_solver.cpp</code>	Explicit per-island solve

Public API	Primary file	Scope
<code>solve_distributed_slack_pf</code>	<code>distributed_slack_solver.cpp</code>	Multi-source slack participation

Table 118: Public power-flow API family.

All paths share the same canonical preprocessing pipeline (`solver_data.cpp`, `admittance_builder.cpp`) and differ only in their iteration loops or post-processing. The main Newton kernel lives in `src/engine/solver/native/nle/newton_solver.cpp` and is reused by the hybrid solver, the OPF warm-start path, and the adaptive solver.

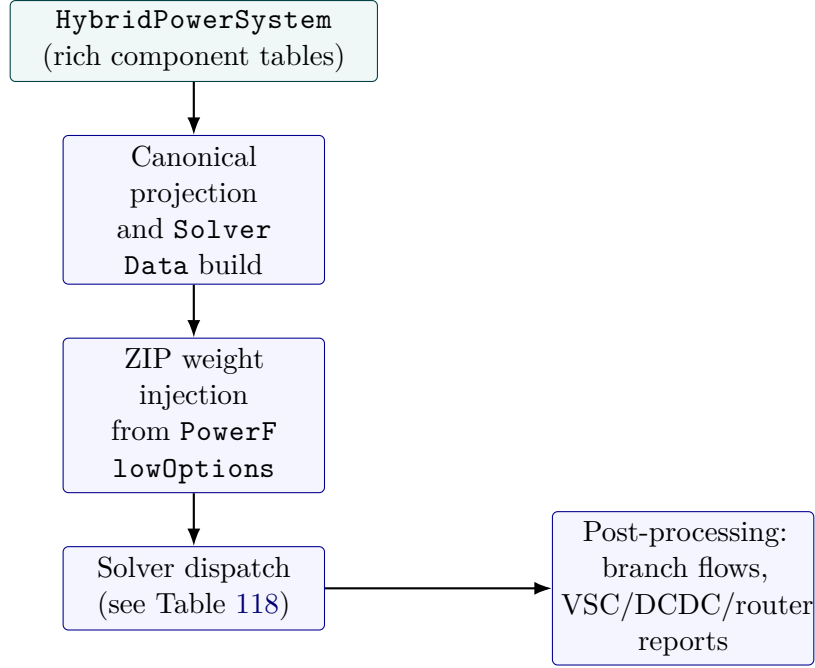


Figure 2: Shared execution pipeline for all power-flow API variants.

12.2 Mathematical Problem Form

The Newton kernel solves the hybrid residual system

$$F(x) = 0, \quad x = \begin{bmatrix} \theta_{\mathcal{N}_{ac} \setminus \mathcal{N}_{sl}} \\ v_{\mathcal{N}_{PQ}} \\ v_{\mathcal{N}_{dc} \setminus \mathcal{N}_{sl}}^{\text{dc}} \end{bmatrix}, \quad (31)$$

where $\mathcal{N}_{sl}$ and $\mathcal{N}_{sl}^{\text{dc}}$ are the sets of AC and DC slack buses, and $\mathcal{N}_{PQ}$ is the set of PQ buses.

AC bus mismatch. For each non-slack AC bus i ,

$$\Delta P_i = P_i^{\text{spec}} - P_i^{\text{calc}}, \quad i \in \mathcal{N}_{ac} \setminus \mathcal{N}_{sl}, \quad (32)$$

$$\Delta Q_i = Q_i^{\text{spec}} - Q_i^{\text{calc}}, \quad i \in \mathcal{N}_{PQ}, \quad (33)$$

where the calculated nodal injections are

$$P_i^{\text{calc}} = \sum_{j \in \mathcal{N}_{ac}} v_i v_j (G_{ij} \cos \theta_{ij} + B_{ij} \sin \theta_{ij}), \quad (34)$$

$$Q_i^{\text{calc}} = \sum_{j \in \mathcal{N}_{ac}} v_i v_j (G_{ij} \sin \theta_{ij} - B_{ij} \cos \theta_{ij}), \quad (35)$$

with $\theta_{ij} = \theta_i - \theta_j$ and $(G_{ij} + jB_{ij})$ the (i, j) entry of the sparse bus admittance matrix Y_{bus} .

DC bus mismatch. For each non-DC-slack bus k ,

$$\Delta P_k^{\text{dc}} = P_k^{\text{dc,spec}} - P_k^{\text{dc,calc}}, \quad (36)$$

$$P_k^{\text{dc,calc}} = v_k^{\text{dc}} \left(G_{dc} v^{\text{dc}} \right)_k. \quad (37)$$

Specified-injection assembly.

$$P_i^{\text{spec}} = P_i^{\text{g,agg}} - P_i^{\text{load}}(v_i) + P_i^{\text{vsc}}, \quad (38)$$

$$Q_i^{\text{spec}} = Q_i^{\text{g,agg}} - Q_i^{\text{load}}(v_i) + Q_i^{\text{vsc}}, \quad (39)$$

where the aggregated generation vectors are

$$P_i^{\text{g,agg}} = P_i^{\text{gen}} + P_i^{\text{sg}} + P_i^{\text{ren}} + P_i^{\text{pv}} + P_i^{\text{stor}} + P_i^{\text{vpp}} + P_i^{\text{mg}} + P_i^{\text{ms}}, \quad (40)$$

$$Q_i^{\text{g,agg}} = Q_i^{\text{gen}} + Q_i^{\text{sg}} + Q_i^{\text{ren}} + Q_i^{\text{pv}} + Q_i^{\text{stor}} + Q_i^{\text{vpp}} + Q_i^{\text{ms}}. \quad (41)$$

Here P_i^{mg} is microgrid exchange at the PCC bus in `GridConnected` mode.

ZIP demand model. The runtime voltage-dependent load model is

$$P_i^{\text{load}}(v_i) = P_{i,0} (w_{P,0,i} + w_{P,1,i} v_i + w_{P,2,i} v_i^2), \quad (42)$$

$$Q_i^{\text{load}}(v_i) = Q_{i,0} (w_{Q,0,i} + w_{Q,1,i} v_i + w_{Q,2,i} v_i^2). \quad (43)$$

The ZIP weights are populated in one of three assembly cases: (1) explicit Load/charging-station tables present, (2) charging-station tables only, (3) neither table — global `zip_pw/zip_qw` from `PowerFlowOptions` are used.

12.3 VSC Converter Model

The converter model uses bus-injection-positive signs on both AC and DC sides. Three control modes are supported.

Loss model. With active transfer p (p.u.), AC-side proxy current $I^{\text{ac}} = |p|/v^{\text{dc,safe}}$, $v^{\text{dc,safe}} = \max(v^{\text{dc}}, 0.1)$:

$$P^{\text{loss,lin}} = (1 - \eta) |p|, \quad (44)$$

$$P^{\text{loss,cur}} = a + b I^{\text{ac}} + c (I^{\text{ac}})^2, \quad a = \frac{\text{loss_mw}}{S_{\text{base}}}, \quad b = \frac{\text{loss_pct}}{100}, \quad c = 1 - \eta. \quad (45)$$

PQ_MODE.

$$P^{\text{conv,ac}} = P^{\text{set}}, \quad Q^{\text{conv,ac}} = Q^{\text{set}}, \quad P^{\text{conv,dc}} = -(P^{\text{set}} + P^{\text{loss}}). \quad (46)$$

VDC_Q and VDC_VAC.

$$P^{\text{tr}} = k^{\text{vdc}} \left[(v^{\text{dc}})^2 - (v^{\text{dc, set}})^2 \right], \quad P^{\text{conv, ac}} = P^{\text{tr}} - P^{\text{loss}}, \quad P^{\text{conv, dc}} = -P^{\text{tr}}. \quad (47)$$

In **VDC_VAC** mode the Newton loop enforces $v^{\text{ac}} = v^{\text{ac, set}}$ for the converter AC bus via an augmented setpoint equation.

12.4 DCDC Converter Model

The DCDC converter connects **bus_in** to **bus_out**. Power **p_ref_mw** is output-side referenced.

$$P^{\text{in}} = \begin{cases} P^{\text{ref}}/\eta, & P^{\text{ref}} \geq 0, \\ P^{\text{ref}} \cdot \eta, & P^{\text{ref}} < 0, \end{cases} \quad P^{\text{out}} = P^{\text{ref}} - r_{eq} \frac{(P^{\text{in}})^2}{(v^{\text{dc, in}})^2}. \quad (48)$$

12.5 Jacobian Structure

The linearized Newton system is

$$J \Delta x = F(x), \quad J = \begin{bmatrix} J_{\theta\theta} & J_{\theta V} & J_{\theta V^{\text{dc}}} \\ J_{V\theta} & J_{VV} & J_{VV^{\text{dc}}} \\ 0 & J_{V^{\text{dc}}V} & J_{V^{\text{dc}}V^{\text{dc}}} \end{bmatrix}. \quad (49)$$

The sparse Jacobian pattern is cached in a **PatternCache** keyed by matrix storage addresses and a monotone **build_id**, and is rebuilt only on topology changes.

AC diagonal entries (with ZIP load corrections).

$$\frac{\partial \Delta P_i}{\partial v_i} = \frac{P_i^{\text{calc}}}{v_i} + G_{ii} v_i - P_{i,0} (w_{P,1,i} + 2w_{P,2,i} v_i), \quad (50)$$

$$\frac{\partial \Delta Q_i}{\partial v_i} = \frac{Q_i^{\text{calc}}}{v_i} - B_{ii} v_i - Q_{i,0} (w_{Q,1,i} + 2w_{Q,2,i} v_i). \quad (51)$$

In the code, v_i is floored at **min_vm_pu** (configurable via **RobustNonlinearOptions**) to prevent zero-division.

DC entries.

$$\frac{\partial P_k^{\text{dc, calc}}}{\partial v_m^{\text{dc}}} = \begin{cases} (G_{dc} v^{\text{dc}})_k + G_{kk}^{\text{dc}} v_k^{\text{dc}}, & m = k, \\ G_{km}^{\text{dc}} v_k^{\text{dc}}, & m \neq k. \end{cases} \quad (52)$$

AC/DC coupling. When **enable_coupled_jacobian** is set, the converter Jacobian blocks $\partial P^{\text{vsc}}/\partial v^{\text{dc}}$, $\partial Q^{\text{vsc}}/\partial v^{\text{dc}}$, and $\partial P^{\text{dc}}/\partial v^{\text{ac}}$ are inserted analytically. The coupling entries are pre-registered as **CouplingEntry** records in **JacobianPattern** and updated in-place each iteration without re-analyzing the pattern.

12.6 Hybrid Newton Algorithm

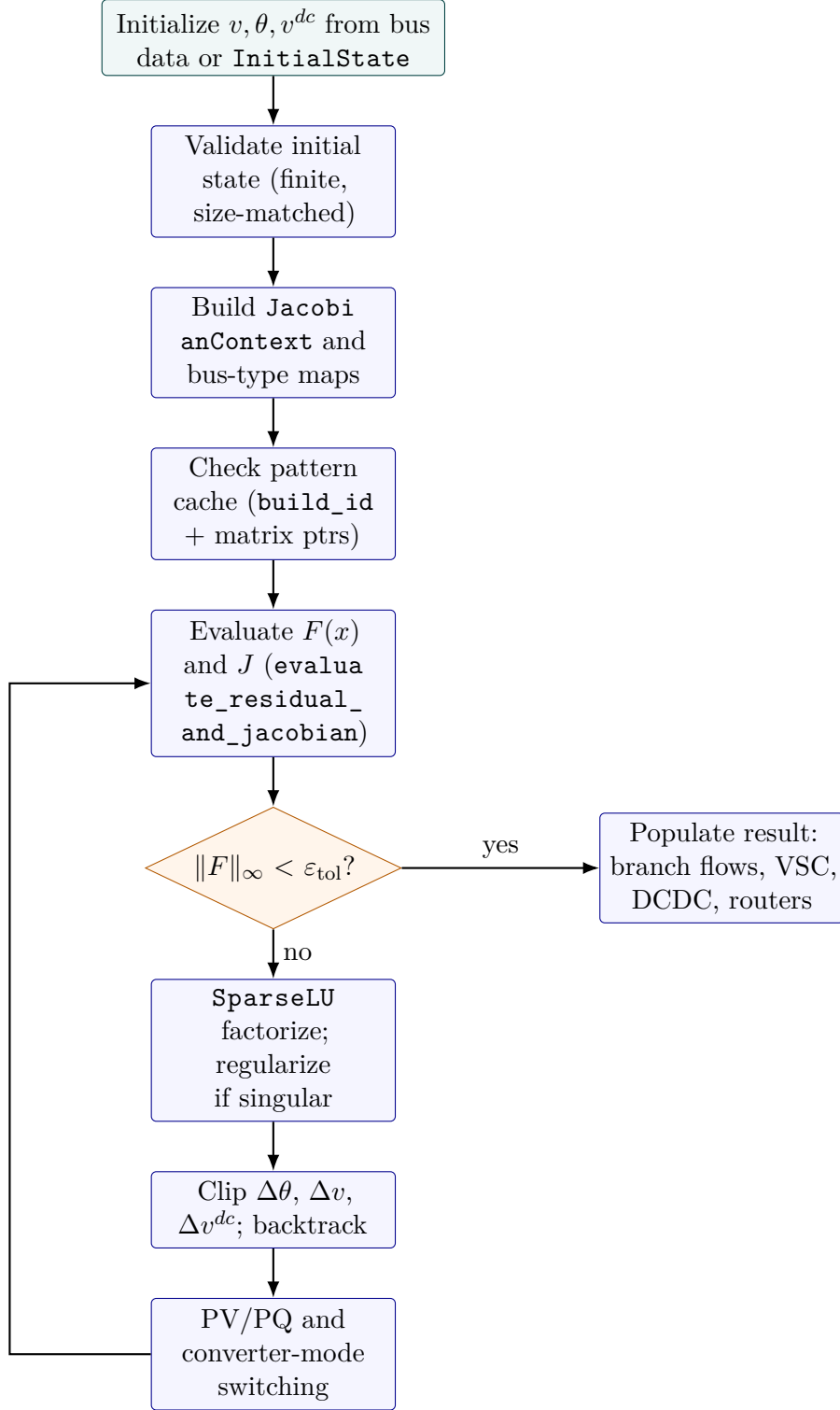


Figure 3: Hybrid AC/DC Newton power-flow algorithm (`NewtonSolver::solve`).

Algorithm PF-1: Hybrid Newton PF

- 1: **procedure** `NEWTONSOLVER::SOLVE`(*data, opt, init*)
- 2: Validate *init*: check size, NaN, Inf; **throw** `invalid_argument` on failure
- 3: $x \leftarrow$ flat start or *init* values; build `JacobianContext`

```

4:    $x_{ctx}.min\_vm\_pu \leftarrow opt.robust\_nonlinear.min\_vm\_pu$ 
5:   for  $k = 1, \dots, opt.max\_iter$  do
6:      $F, J \leftarrow evaluate\_residual\_and\_jacobian(x, ctx, pattern)$ 
7:     if  $\|F\|_\infty < opt.tol$  then break
8:     end if
9:      $SparseLU(J) \Delta x = F$ ; if singular then regularize and retry
10:    Clip  $\Delta x$ :  $|\Delta \theta| \leq \Delta \theta_{max}, |\Delta v| \leq \Delta v_{max}$ 
11:    Backtracking line search; accept best step
12:    PV/PQ hysteresis switching; converter mode switching
13:  end for
14:  if not converged then set failure_reason
15:  Post-process branch, VSC, DCDC, router flows; return result
16: end procedure

```

Robust nonlinear pipeline. The solver implements a five-phase robustification pipeline controlled by `RobustNonlinearOptions`:

1. **Phase 1 – Scaling and diagnostics:** residual-equation scaling, variable scaling, and Jacobian row/column equilibration; condition-number proxy monitor.
2. **Phase 2 – Nonmonotone line search and smooth NCP:** Grippo–Lampariello–Lucidi nonmonotone Armijo search; optional μ -perturbed Fischer–Burmeister complementarity for PV/PQ switching.
3. **Phase 3 – LM trust-region fallback:** Levenberg–Marquardt step $(J^\top J + \lambda I) \Delta x = -J^\top F$ when Newton line search fails; SER-based pseudo-transient continuation.
4. **Phase 4 – Homotopy continuation:** ramp loads and converter setpoints from flat start to target in adaptive λ steps.
5. **Phase 5 – Krylov fallback:** Newton–GMRES with Schur-complement AC/DC block preconditioner (opt-in; not active by default).

12.7 Fast-Decoupled Power Flow (FDPF)

The FDPF solver builds two constant susceptance matrices from Y_{bus} :

$$B' = -\Im \left\{ Y_{bus}^{(R=0, b=0, \text{tap}=1)} \right\}, \quad (53)$$

$$B'' = -\Im \left\{ Y_{bus}^{(\text{full shunts and charging, zero phase shift})} \right\}, \quad (54)$$

and then alternates P- θ and Q-V decoupled solves:

$$B' \Delta \theta = \Delta P / v, \quad (55)$$

$$B'' \Delta v = \Delta Q / v. \quad (56)$$

Both B' and B'' are factorized once via `Eigen::SparseLU`; only the right-hand sides change across iterations. The solver is AC-only (no DC or converter coupling).

Algorithm PF-2: Fast-Decoupled PF

- ```

1: procedure FDPFSOLVER::SOLVE(data, opt)
2: Build reduced $B'_{\mathcal{N} \setminus \{sl\}}, B''_{\mathcal{N}_{PQ}}$; factorize
3: if factorization failed then set SingularJacobian; return
4: end if

```

```

5: for $k = 1, \dots, opt.fdpf_max_iter$ do
6: Compute $\Delta P/v$, $\Delta Q/v$ using sparse Y_{bus} iteration
7: if $\|\Delta P/v\|_\infty < \varepsilon$ and $\|\Delta Q/v\|_\infty < \varepsilon$ then break
8: end if
9: $\Delta \theta \leftarrow (B')^{-1}(\Delta P/v)$; update θ
10: $\Delta v \leftarrow (B'')^{-1}(\Delta Q/v)$; update v
11: end for
12: end procedure

```

## 12.8 DC-Only Power Flow

The DC solver iterates on non-slack DC voltages using dense LU on the small reduced Jacobian:

$$J_{dc} \Delta v^{dc} = \Delta P^{dc}, \quad (57)$$

$$J_{dc,kl} = \begin{cases} (G_{dc} v^{dc})_k + G_{kk}^{dc} v_k^{dc}, & l = k, \\ G_{kl}^{dc} v_k^{dc}, & l \neq k. \end{cases} \quad (58)$$

DC bus injections are assembled from DCLoad, DC storage, DC static generators, DC PV arrays, VSC DC-side injections, DCDC transfers, and DC EnergyRouter ports. A backtracking line search over  $\alpha \in \{1, \frac{1}{2}, \dots\}$  is applied with voltages floored at `kMinVdc` to prevent collapse.

## 12.9 Adaptive Islanded Solver

The adaptive solver handles multi-island AC/DC topologies that a single global solve cannot address:

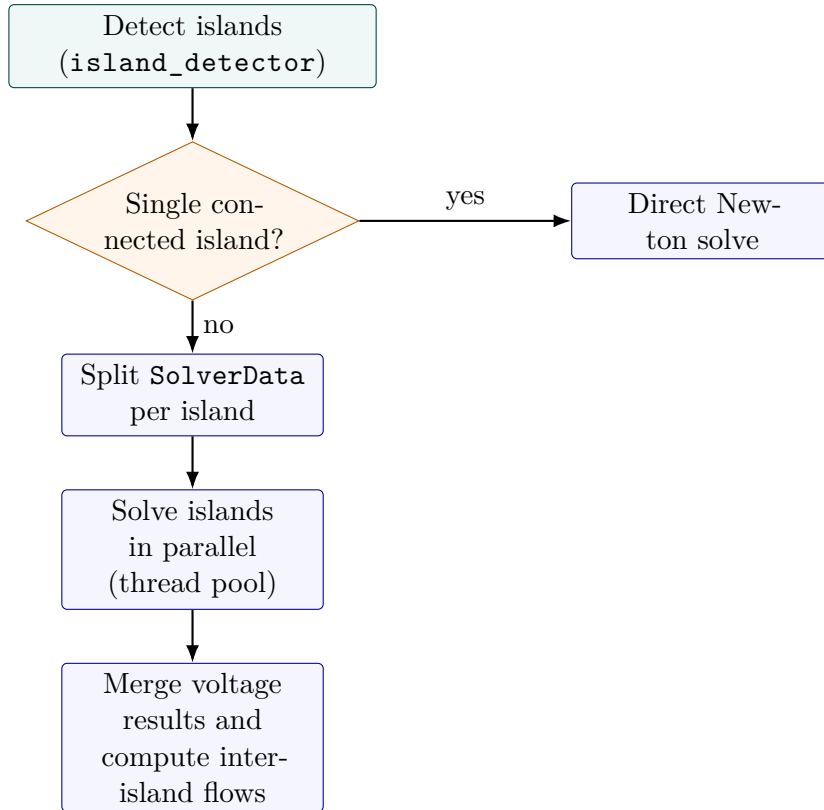


Figure 4: Adaptive islanded power-flow workflow.

## 12.10 HELM Solver

The HELM (Holomorphic Embedding Load-flow Method) solver constructs a power series in a complex embedding parameter  $s \in \mathbb{C}$ :

$$V_i(s) = \sum_{n=0}^N a_n^{(i)} s^n. \quad (59)$$

The recursion for order  $n$  coefficients follows from the holomorphic embedding of the network equations. Convergence is tested via Padé approximants  $[M/M]$  evaluated at  $s = 1$ :

$$V_i(1) \approx \frac{P_M(1)}{Q_M(1)}. \quad (60)$$

The HELM solver is used as a DC warm-start supplier for the hybrid Newton path (`HelmWarmStart::DCPF`) and as a standalone fallback in the adaptive solver chain.

## 12.11 Error Handling and Diagnostics

All solvers return a result struct with a `SolverFailureReason` field (added in the May 2026 hardening pass):

- `None` — converged successfully,
- `SingularJacobian` — `SparseLU` or dense LU factorization reported failure ( $B'$ ,  $B''$ , or DC Jacobian),
- `NonFiniteSolution` — Newton step or intermediate state contained NaN or Inf,
- `InvalidInitialState` — provided `InitialState` contained non-finite values (guarded by `validate_initial_state`),
- `MaxIterationsReached` — tolerance not met within the iteration limit.

The solver **throws** `std::invalid_argument` for structural API violations (e.g., size mismatch, NaN in initial state) and **returns a non-converged result** for numerical failures. This two-tier policy separates programming errors from expected numerical difficulties.

## 12.12 Maintainability Roadmap: Code-Duplication Inventory

All six items D1–D6 from the original inventory were resolved across two refactoring passes (*code-dedup-D1-D5* and *code-dedup-D3-D6*).

**D1 — `inf_norm` (DONE).** The anonymous-namespace duplicate removed from `jacobian_builder.cpp` and `dc_solver.cpp`. The canonical definition now lives in `include/hacdcpf/power_flow/pf_utils.hpp` as an inline function. Test files `test_powerflow_performance.cpp` updated to use `hacdcpf::powerflow::inf_norm` via a `using` declaration.

**D2 — DC injection assembly (DONE).** The 80-line DC injection loop extracted into `assemble_dc_injections(data, vm, va, vdc, pdc_spec)`, declared in a new header `include/hacdcpf/power_flow/pf_injection_assembly.hpp` and implemented in `src/power_flow/pf_injection_assembly.cpp`. Both `dc_solver.cpp` and `residual_evaluator.cpp` now call this single canonical function.

**D3 — Duplicated B' matrix construction (DONE).** The anonymous-namespace functions `build_bp` and `build_bpp` were removed from `fdpf_solver.cpp` and replaced with calls to a new utility `build_susceptance_matrix(data, flags)` implemented in `admittance_builder.cpp`. A `BpBuildFlags` struct (declared in `admittance_builder.hpp`) controls all assembly variants: zero resistance, zero line charging, unit tap, zero phase shift, and optional bus shunt susceptance. Convenience factories `make_bp_flags(min_x_pu)` and `make_bpp_flags()` cover the two FDPF use cases.

**D4 — DC initial-state warm-start (DONE).** The size-guarded copy of `init->vdc` with NaN/Inf validation extracted into `load_dc_initial_state(init, dc_buses, vdc)` in `pf_utils.hpp`. Applied to both `dc_solver.cpp` and `newton_solver.cpp`.

**D5 — Bus-type classification loop (DONE).** The manual AC bus classification into SLACK/PV/PQ sets extracted into `classify_ac_buses(data)` returning an `AcBusSets` struct (all in `pf_utils.hpp`). Applied to `fdpf_solver.cpp`, `residual_evaluator.cpp`, and the helper in `test_unified_pf_upgrade.cpp`.

**D6 — jacobian\_builder.cpp file split (DONE).** The monolithic `jacobian_builder.cpp` was split into three focused compilation units:

- `jacobian_pattern.cpp` — `build_jacobian_pattern` and its anonymous-namespace helpers (`rc_key`, `add_pattern_position`, `build_entry_to_nz`, `lookup_nz`);
- `ac_kernel.cpp` — `evaluate_ac_kernel_serial`, `evaluate_ac_kernel_parallel`, and `effective_ac_threads`;
- `jacobian_builder.cpp` (retained) — `build_power_spec`, `evaluate_residual_impl`, `evaluate_residual_and_jacobian`, and `evaluate_residual_only`.

The cross-file linkage between `jacobian_builder.cpp` and `ac_kernel.cpp` is provided by an internal header `src/power_flow/ac_kernel_impl.hpp` (not installed).

## 12.13 Numerical Performance (Release Mode)

Benchmarks were run on 84 MATPOWER cases (`build_rel`, Apple M4, `-O3-DNDEBUG`, single thread) using `build_rel/helm_powerflow_benchmark../data`. Four paths are compared: NR = hybrid Newton, FD = fast-decoupled, HE = HELM standalone, HE+DC = HELM with DC warm start.

| Case           | <i>N</i> | Branches | NR ms | FD ms | HE ms | HE+DC<br>ms |
|----------------|----------|----------|-------|-------|-------|-------------|
| case14         | 14       | 20       | 0.0   | 0.3   | 0.1   | 0.1         |
| case57         | 57       | 80       | 0.1   | 2.9   | 0.6   | 0.6         |
| case118        | 118      | 186      | 0.4   | 11.5  | 1.8   | 1.9         |
| case300        | 300      | 411      | 0.9   | 59.5  | div   | div         |
| case1354pegase | 1354     | 1991     | 15.2  | skip  | div   | div         |
| case1888rte    | 1888     | 2531     | 10.8  | skip  | 135.8 | 136.5       |
| case2383wp     | 2383     | 2896     | 3.8   | skip  | 117.6 | 109.3       |
| case2736sp     | 2736     | 3504     | 5.6   | skip  | 75.2  | 76.6        |
| case3012wp     | 3012     | 3572     | 35.0  | skip  | 124.3 | 126.9       |

| Case              | $N$   | Branches | NR ms  | FD ms | HE ms  | HE+DC<br>ms |
|-------------------|-------|----------|--------|-------|--------|-------------|
| case6468rte       | 6468  | 9000     | 31.7   | skip  | 297.7  | 296.2       |
| case8387pegase    | 8387  | 14561    | 46.7   | skip  | 1776.5 | 1786.7      |
| case13659pegase   | 13659 | 20467    | 95.9   | skip  | skip   | skip        |
|                   | 10000 | 12706    | 21.8   | skip  | 4327.9 | 4389.7      |
| case_ACTIVSg10k   | 25000 | 32230    | 64.3   | skip  | skip   | skip        |
| case_ACTIVSg25k   | 82000 | 104121   | 1745.2 | skip  | skip   | skip        |
| case_SyntheticUSA |       |          |        |       |        |             |

Table 119: Power-flow solver timing (release mode, Apple M4, single thread, post D1–D5 refactor). “div” = diverged, “skip” = case excluded from path at this size.

Key observations:

- The hybrid Newton path converges on the 82000-bus USA synthetic system in 1.75 s with 17 iterations after the D1–D5 refactoring pass (no regression vs. pre-refactoring baseline).
- Newton solve time scales approximately as  $O(N^{1.2})$  in practice for typical sparse transmission systems.
- FDPF is 10–200 $\times$  slower than Newton on small-to-medium cases due to the current  $O(N^2)$  dense Ybus iteration in the power-injection loop; replacing it with a sparse row iteration remains an open performance item.
- HELM converges on most well-conditioned cases under 2000 buses but diverges on stressed systems (case300, case1354pegase); it remains useful as a warm-start provider for Newton on difficult cases.
- The D1–D6 refactoring passes introduced no performance regression: Newton solve times match the pre-refactoring baseline on all benchmark cases.

## 13 Optimal Power Flow Models

### 13.1 Problem Form and Runtime Scope

This chapter covers the OPF family built on top of canonical AC/DC network projection. The implemented problem classes are:

- nonlinear AC/DC OPF for the native primal-dual and parity-IPM routes,
- convex DC OPF in LP or QP form,
- mixed-integer reactive-power optimization over discrete taps and shunts.

The runtime solver family is selected from option flags and DC-network presence.

## 13.2 Code Map

The current implementation is concentrated in:

- `src/optimal_power_flow/ac_opf.cpp` for top-level AC OPF routing and native reduced-space solve orchestration,
- `src/optimal_power_flow/parity_formulation.cpp` and `src/optimal_power_flow/parity_ipm.cpp` for the parity full-space NLP route,
- `src/optimal_power_flow/dc_opf.cpp` for DC OPF model build and backend selection,
- `src/optimal_power_flow/reactive_power_opt.cpp` for the four-phase outer-approximation MINLP,
- `src/engine/lcqp_solver.cpp`, `src/engine/dual_simplex.cpp`, and the adapter layer for native and external backend execution.

## 13.3 Pseudocode Map

The relevant code-aligned workflows are:

- Algorithms A8–A11 in Section 21 for AC OPF routing, fast dispatch fallback, native primal-dual OPF, and parity-IPM build/solve,
- Algorithm A16 for DC OPF,
- Algorithm A17 for reactive-power optimization,
- Algorithms E5–E8 in Section 4 for the native LCQP, LP IPM, dual simplex, and PDLP backends used underneath those routes.

## 13.4 Current Status

This is an active production chapter with multiple live backends. The AC OPF family is fully implemented, but backend coverage differs by path: parity and native nonlinear routes are the main AC/DC engines, while DC OPF and RPO reuse the solver infrastructure through specialized model builders.

## 13.5 Top-Level Solver Routing

The public `solve_ac_opf` path selects among three solver families. A key implementation rule is automatic DC routing: if the case has DC buses or VSC converters and `enable_primal_dual=false`, the options are internally overridden to `enable_primal_dual=true, use_parity_ipm=true`.

| Path                                  | Trigger                                                               | Current behavior                                                                                          |
|---------------------------------------|-----------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| Fast dispatch + PF fallback (AC-only) | <code>enable_primal_dual=false</code> and no forced DC parity routing | Drops DC and converter subsystems, solves economic dispatch, then runs AC PF projection.                  |
| Native primal-dual OPF                | <code>enable_primal_dual=true, use_parity_ipm=false</code>            | Uses a custom sparse KKT loop with AC equations, DC-bus equations, and converter power-balance equations. |

| Path                      | Trigger                                                                                                | Current behavior                                                                                                             |
|---------------------------|--------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| Parity primal-dual<br>IPM | <code>enable_primal_dual=true</code> , <code>use_parity_ipm=true</code> or auto-enabled by DC presence | Builds the parity full-space NLP and solves by Mehrotra-type primal-dual IPM. Optional fallback to native path when enabled. |

### 13.6 Fast Dispatch + PF Fallback (AC-Only Path)

This path is used for fastest feasible dispatch recovery in AC-only operation. It solves a clipped economic dispatch first, then projects via AC PF.

**Economic dispatch stage.**

$$\min_{P_g} \sum_{g \in \mathcal{G}} (c_{2g} P_g^2 + c_{1g} P_g + c_{0g}), \quad (61)$$

subject to generator bounds and  $\sum_g P_g = P_D$  (clamped to feasible range). For quadratic units, the code uses incremental-cost clipping  $P_g = \text{clip}((\lambda - c_1)/(2c_2), P_g^{\min}, P_g^{\max})$ .

**Feasibility projection stage.** The dispatch is passed to AC Newton PF; resulting bus injections are split back across generators at each bus and clamped to individual bounds. If direct projection fails, one reshaped economic-dispatch attempt is executed before declaring failure.

### 13.7 Native Primal-Dual OPF Formulation

#### 13.7.1 Decision Variables

For the native path, variables are

$$x_{nat} = \left[ \theta_{\mathcal{N}_{ac} \setminus \mathcal{N}_{sl}} \quad v_{\mathcal{N}_{PQ}} \quad P_g \quad Q_g \quad v^{dc} \quad P^{\text{conv},ac} \quad Q^{\text{conv},ac} \quad P^{\text{conv},dc} \right]^\top. \quad (62)$$

The implementation includes all DC bus voltages in  $v^{dc}$ ; for DC\_V buses, tight bounds are built around the setpoint.

#### 13.7.2 Objective

$$\min_x f(x) = \sum_{g \in \mathcal{G}} (c_{2g} P_g^2 + c_{1g} P_g + c_{0g}). \quad (63)$$

#### 13.7.3 Equality Constraints

**AC equations.** AC rows reuse the PF residual kernel (with converter table disabled in that kernel call), then subtract explicit converter variables:

$$g_{P,i} = \left( P_i^{\text{spec},\text{PF}} - P_i^{\text{calc}} \right) - \sum_{c \in \mathcal{C}_i} P_c^{\text{ac}} = 0, \quad i \in \mathcal{N}_{ac} \setminus \mathcal{N}_{sl}, \quad (64)$$

$$g_{Q,i} = \left( Q_i^{\text{spec},\text{PF}} - Q_i^{\text{calc}} \right) - \sum_{c \in \mathcal{C}_i} Q_c^{\text{ac}} = 0, \quad i \in \mathcal{N}_{PQ}. \quad (65)$$

**DC equations.** For each DC bus  $k$ :

$$g_k^{\text{dc}} = P_{d,k}^{\text{dc}} + V_k^{\text{dc}} (G_{dc} V^{\text{dc}})_k - \sum_{c \in \mathcal{C}_k^{\text{dc}}} P_c^{\text{dc}} = 0. \quad (66)$$

**Converter energy balance.** For converter  $c$ :

$$g_c^{\text{conv}} = P_c^{\text{ac}} + P_c^{\text{dc}} + P_c^{\text{loss}} = 0, \quad (67)$$

$$P_c^{\text{loss}} = a_c + b_c I_c^{\text{ac}} + (1 - \eta_c)(I_c^{\text{ac}})^2, \quad (68)$$

$$I_c^{\text{ac}} = \frac{\sqrt{(P_c^{\text{ac}})^2 + (Q_c^{\text{ac}})^2} + \epsilon}{V_c^{\text{ac}}}, \quad \epsilon = 10^{-6}. \quad (69)$$

### 13.7.4 Bounds and Nonlinear Limits

Variable bounds are enforced for  $\theta, v, P_g, Q_g, v^{\text{dc}}, P^{\text{conv,ac}}, Q^{\text{conv,ac}}, P^{\text{conv,dc}}$ .

Nonlinear engineering limits are modeled as  $h(x) \leq 0$  and handled by exterior quadratic penalty terms inside the merit model:

$$h_{ij}^{\text{from}} = P_{f,ij}^2 + Q_{f,ij}^2 - (S_{ij}^{\text{max}})^2 \leq 0, \quad (70)$$

$$h_{ij}^{\text{to}} = P_{t,ij}^2 + Q_{t,ij}^2 - (S_{ij}^{\text{max}})^2 \leq 0, \quad (71)$$

$$h_c^{\text{conv}} = (P_c^{\text{ac}})^2 + (Q_c^{\text{ac}})^2 - (S_c^{\text{max}})^2 \leq 0. \quad (72)$$

### 13.7.5 KKT System and Globalization

At each inner iteration, the solver builds

$$\begin{bmatrix} H + \delta I & J_g^\top \\ J_g & -\delta I \end{bmatrix} \begin{bmatrix} \Delta x \\ \Delta \lambda \end{bmatrix} = - \begin{bmatrix} \nabla_x L \\ g(x) \end{bmatrix}, \quad (73)$$

with dynamic regularization  $\delta$ , line search, trust/restore logic, and bounded step clipping.

A barrier term is applied to variable bounds; nonlinear branch/converter limits enter the merit model through adaptive exterior penalties.

## 13.8 Parity Full-Space Formulation

### 13.8.1 Decision Variables

The parity NLP includes AC, DC, converter, shedding, and enhanced DER variables:

$$x_{\text{par}} = \begin{bmatrix} \theta, v^{\text{ac}}, P_g, Q_g, v^{\text{dc}}, P^{\text{conv,ac}}, Q^{\text{conv,ac}}, P^{\text{conv,dc}}, \\ \Delta P_d, \Delta Q_d, P^{\text{ren}}, Q^{\text{ren}}, P^{\text{stor}}, Q^{\text{stor}}, \\ P^{\text{dc,dc}}, P^{\text{flex}} \end{bmatrix}^\top. \quad (74)$$

### 13.8.2 Objective

$$\min_x f_{\text{par}}(x) = \sum_{g \in \mathcal{G}} (c_{2g} P_g^2 + c_{1g} P_g + c_{0g}) + \text{VOLLEff} S_{\text{base}} \sum_i (\Delta P_{d,i} + \Delta Q_{d,i}) \quad (75)$$

$$+ \sum_{r \in \mathcal{R}^{\text{curt}}} \max(0, P_r^{\text{rated}} - P_r) c_r^{\text{curt}}. \quad (76)$$

### 13.8.3 Equality Constraints

**AC full-bus balance (scaled).** For each AC bus  $i$ :

$$g_{P,i} = \left( P_i^{\text{calc}} - P_i^{\text{gen}} - P_i^{\text{fixed}} - P_i^{\text{ren}} - P_i^{\text{stor}} + P_i^{\text{load}} - \Delta P_{d,i} + P_i^{\text{flex}} - P_i^{\text{conv,ac}} \right) \alpha_P = 0, \quad (77)$$

$$g_{Q,i} = \left( Q_i^{\text{calc}} - Q_i^{\text{gen}} - Q_i^{\text{fixed}} - Q_i^{\text{ren}} - Q_i^{\text{stor}} + Q_i^{\text{load}} - \Delta Q_{d,i} - Q_i^{\text{conv,ac}} \right) \alpha_Q = 0. \quad (78)$$

Here  $P_i^{\text{fixed}}, Q_i^{\text{fixed}}$  aggregate non-variable injections (static generators, non-curtailable renewables/PV, VPP, microgrid exchange, mobile storage).

**DC balance with DCDC coupling.** For each DC bus  $k$ :

$$g_k^{\text{dc}} = P_{d,k}^{\text{dc}} + \sum_{m \neq k} g_{km} \left( (V_k^{\text{dc}})^2 - V_k^{\text{dc}} V_m^{\text{dc}} \right) - P_k^{\text{conv,dc}} + P_k^{\text{dcdc,in}} - P_k^{\text{dcdc,out}} = 0, \quad (79)$$

where the OPF decision variable is  $P^{\text{in}}$  (*input-side referenced*, i.e. the power absorbed from **bus\_in**). The output power is a deterministic function of  $P^{\text{in}}$  including both efficiency and  $I^2R$  conduction loss:

$$P^{\text{out}} = \begin{cases} \eta P^{\text{in}} - r_{eq}(P^{\text{in}})^2/(V^{\text{dc}})^2, & P^{\text{in}} \geq 0, \\ P^{\text{in}}/\eta - r_{eq}(P^{\text{in}})^2/(V^{\text{dc}})^2, & P^{\text{in}} < 0. \end{cases} \quad (80)$$

There is no separate converter-balance constraint row for DCDC converters; their power contributions are folded directly into the two DC bus balance equations:

$$g_{\text{bus\_in}} += P^{\text{in}}, \quad g_{\text{bus\_out}} -= P^{\text{out}}. \quad (81)$$

**DCDC Jacobian and Hessian.** The first-order derivatives of the DCDC coupling terms are:

$$\frac{\partial g_{\text{bus\_in}}}{\partial P^{\text{in}}} = 1, \quad \frac{\partial g_{\text{bus\_out}}}{\partial P^{\text{in}}} = -\frac{\partial P^{\text{out}}}{\partial P^{\text{in}}}, \quad (82)$$

$$\frac{\partial P^{\text{out}}}{\partial P^{\text{in}}} = \begin{cases} \eta - 2r_{eq}P^{\text{in}}/(V^{\text{dc}})^2, & P^{\text{in}} \geq 0, \\ 1/\eta - 2r_{eq}P^{\text{in}}/(V^{\text{dc}})^2, & P^{\text{in}} < 0. \end{cases} \quad (83)$$

When  $r_{eq} > 0$ , there is also a cross-derivative with respect to the input-side DC voltage:

$$\frac{\partial g_{\text{bus\_out}}}{\partial V_{\text{in}}^{\text{dc}}} = \frac{2r_{eq}(P^{\text{in}})^2}{(V_{\text{in}}^{\text{dc}})^3}. \quad (84)$$

The Hessian contributions from the  $I^2R$  loss term are:

$$\frac{\partial^2 g}{\partial (P^{\text{in}})^2} = \frac{2r_{eq}}{(V^{\text{dc}})^2}, \quad \frac{\partial^2 g}{\partial P^{\text{in}} \partial V^{\text{dc}}} = \frac{-4r_{eq}P^{\text{in}}}{(V^{\text{dc}})^3}, \quad \frac{\partial^2 g}{\partial (V^{\text{dc}})^2} = \frac{6r_{eq}(P^{\text{in}})^2}{(V^{\text{dc}})^4}. \quad (85)$$

**OPF warm-start convention.** Note that the PF uses output-referenced  $P^{\text{ref}}$  while the OPF uses input-referenced  $P^{\text{in}}$ . The warm-start path converts between these reference frames when seeding the OPF initial point from a PF result.

**Converter balance.** For each VSC converter:

$$P_c^{\text{ac}} + P_c^{\text{dc}} + P_c^{\text{loss}} = 0, \quad (86)$$

with the same loss model as in the native formulation.

#### 13.8.4 Inequality Constraints

Nonlinear inequalities:

$$P_{f,ij}^2 + Q_{f,ij}^2 - (S_{ij}^{\text{max}})^2 \leq 0, \quad (87)$$

$$P_{t,ij}^2 + Q_{t,ij}^2 - (S_{ij}^{\text{max}})^2 \leq 0, \quad (88)$$

$$(P_c^{\text{ac}})^2 + (Q_c^{\text{ac}})^2 - (S_c^{\text{max}})^2 \leq 0, \quad (89)$$

$$\left( g_{km} V_k^{\text{dc}} (V_k^{\text{dc}} - V_m^{\text{dc}}) \right)^2 - (P_{km}^{\text{max}})^2 \leq 0. \quad (90)$$

Variable bounds are converted to inequality rows:

$$x^{\text{min}} - x \leq 0, \quad x - x^{\text{max}} \leq 0. \quad (91)$$

### 13.8.5 Primal-Dual IPM System

The parity solver introduces slack  $z > 0$  and multipliers  $\mu > 0$ :

$$g(x) = 0, \quad (92)$$

$$h(x) + z = 0, \quad (93)$$

$$\nabla f(x) + J_g^\top \lambda + J_h^\top \mu = 0, \quad (94)$$

$$z \circ \mu = \gamma \mathbf{1}. \quad (95)$$

A Mehrotra predictor-corrector method is used, with:

1. affine predictor step,
2. adaptive centering parameter  $\sigma = \min(0.3, 100\mu)$ ,
3. corrector step with second-order compensation,
4. iterative refinement (2 steps) for conditioning of the KKT solve,
5. fraction-to-boundary step lengths for primal and dual updates.

**Implementation note.** The current `parity_ipm.cpp` uses the Mehrotra predictor-corrector scheme. Gondzio extra corrections are **not** implemented. The KKT system is solved using either Eigen `SparseLU` for sparse or `PartialPivLU` for dense factorizations.

Convergence monitors primal infeasibility, dual stationarity, and average complementarity. A best-iterate restoration is applied before final failure.

### 13.9 Component Coverage by Solver Path

| Component / feature                                                                     | Native primal-dual          | Parity IPM                          |
|-----------------------------------------------------------------------------------------|-----------------------------|-------------------------------------|
| <b>Generator</b> ( $P_g, Q_g$ )                                                         | decision variable           | decision variable                   |
| <b>VSCConverter</b><br>( $P^{\text{conv,ac}}, Q^{\text{conv,ac}}, P^{\text{conv,dc}}$ ) | decision variable           | decision variable                   |
| DCBus voltage                                                                           | decision variable           | decision variable                   |
| AC branch MVA limits                                                                    | nonlinear penalty term      | nonlinear inequality + slack        |
| DC branch power limits                                                                  | not explicit in native path | nonlinear inequality + slack        |
| Load shedding ( $\Delta P_d, \Delta Q_d$ )                                              | not modeled                 | decision variable with VOLL penalty |
| Curtailable renewable / controllable PV                                                 | not modeled as variable     | decision variable                   |
| Storage dispatch                                                                        | not modeled as variable     | decision variable                   |
| DCDC dispatch                                                                           | not modeled as variable     | decision variable + DC coupling     |
| Flexible load dispatch                                                                  | not modeled as variable     | decision variable                   |

| Component / feature              | Native primal-dual                              | Parity IPM                   |
|----------------------------------|-------------------------------------------------|------------------------------|
| StaticGen / fixed DER aggregates | not explicit variable in native objective model | included as fixed injections |

### 13.10 Fallback Behavior

1. If parity IPM is selected and fails, and `allow_fallback=true`, the solver retries with `use_parity_ipm=false`.
2. If native primal-dual fails and fallback is allowed, it attempts the fast dispatch + AC PF path.
3. Status strings in `ACOPFResult::status` record which fallback path was taken.

### 13.11 DC Optimal Power Flow (`solve_dc_opf`)

The DC OPF model solves a linearized economic dispatch problem using the DC power flow approximation. It is independent of the AC OPF paths above and is invoked through `hacdcopf::opf::solve_dc_opf`.

#### 13.11.1 Decision Variables

$$x_{dc} = \begin{bmatrix} \theta_{\mathcal{N} \setminus \{sl\}} & P_g & P_f & \Delta P_d \end{bmatrix}^\top, \quad (96)$$

where  $P_{f,ij}$  are branch flow variables (optional, depending on formulation mode) and  $\Delta P_{d,i}$  are per-bus load-shedding variables.

#### 13.11.2 Network Model

The DC power flow relationship is:

$$P_{f,ij} = \frac{\theta_i - \theta_j}{x_{ij}}, \quad (97)$$

with the bus susceptance matrix  $B$  assembled from branch reactances  $x_{ij}$ . The slack bus angle is fixed at  $\theta_{sl} = 0$ .

#### 13.11.3 Objective Function

The DC OPF supports two cost modes:

##### Linear cost (default).

$$\min \sum_{g \in \mathcal{G}} c_g^{\text{lin}} P_g + \sum_i \text{VOLL}_{\text{eff}} \cdot \Delta P_{d,i} \cdot S_{\text{base}}, \quad (98)$$

where the linearized coefficient is evaluated at the generator midpoint:

$$c_g^{\text{lin}} = c_{1g} + 2c_{2g} \frac{P_g^{\text{min}} + P_g^{\text{max}}}{2}. \quad (99)$$

**Quadratic cost (optional).**

$$\min \frac{1}{2} x^\top Q x + c^\top x, \quad (100)$$

where  $Q_{gg} = 2c_{2g}S_{\text{base}}^2$  and  $c_g = c_{1g}S_{\text{base}}$ . This mode requires a QP-capable backend (LCQP or Gurobi).

**VOLL auto-tuning.** The load-shedding penalty is auto-tuned as  $1.1 \times$  the maximum generator marginal cost to guarantee load serving priority.

#### 13.11.4 Constraints

**Power balance (per bus).**

$$\sum_{g \in \mathcal{G}_i} P_g - P_{d,i} + \Delta P_{d,i} = \sum_{j \in \mathcal{N}_i} P_{f,ij}, \quad \forall i. \quad (101)$$

**Flow definition.**

$$P_{f,ij} - b_{ij}(\theta_i - \theta_j) = 0, \quad \forall (i, j) \in \mathcal{E}. \quad (102)$$

**Variable bounds.**

$$\theta_{sl} = 0, \quad -\pi \leq \theta_i \leq \pi, \quad i \neq sl, \quad (103)$$

$$P_g^{\min} \leq P_g \leq P_g^{\max}, \quad (104)$$

$$-r_a^{\max} \leq P_{f,ij} \leq r_a^{\max}, \quad (105)$$

$$0 \leq \Delta P_{d,i} \leq \max(P_{d,i}, 0.01/S_{\text{base}}). \quad (106)$$

#### 13.11.5 Solver Backend Selection

The DC OPF solver selects among backends via `DCOPFSolverBackend`:

1. **NativeQP** (LCQP solver): true quadratic costs via primal-dual IPM.
2. **Gurobi**: commercial QP solver.
3. **HiGHS**: open-source LP solver (linearized costs).
4. **Native Dual Simplex**: via `solve_lp_with_basis()`.
5. **Auto**: tries each in sequence until success.

#### 13.11.6 Locational Marginal Prices

LMPs are extracted from the dual variables on the per-bus power balance equality constraints. The dual on bus  $i$  balance yields the locational marginal price at bus  $i$ .

### 13.12 Reactive Power Optimization (`solve_reactive_power_opt`)

The reactive power optimization (RPO) module solves a mixed-integer nonlinear program for optimal voltage/reactive power control by coordinating discrete control devices (transformer on-load tap changers and switchable shunt steps) with continuous AC OPF.

### 13.12.1 Problem Structure

**Discrete variables.**

- Transformer OLTC tap positions:  $\text{tap ratio} = 1 + (\text{tap\_pos} - \text{tap\_neutral}) \times \text{step\_pct}/100$ .
- Switchable shunt steps:  $B_s = \text{step} \times b_{\text{per\_step}}$  (susceptance per step).

**Continuous variables.** Bus voltages and generator reactive outputs, solved via AC OPF at each discrete configuration.

### 13.12.2 Objective Functions

Selectable objectives:

$$\text{MinVoltageDeviation: } \sum_i w_{\text{vdev}} (V_i - V_{\text{tgt}})^2, \quad (107)$$

$$\text{MinActiveLoss: } \sum_g P_g - \sum_d P_d, \quad (108)$$

$$\text{Combined: } w_{\text{vdev}} \sum_i (V_i - V_{\text{tgt}})^2 + \left( \sum_g P_g - \sum_d P_d \right). \quad (109)$$

### 13.12.3 Algorithm: Four-Phase Outer Approximation

**Phase 0: Sensitivity ranking.** Perturb each discrete variable by  $\pm 1$  from the baseline (two NLP solves per variable). Compute  $|\Delta \text{obj}|$  for each variable and order by descending sensitivity.

**Phase 1: Ternary-search sweep.** For each variable in sensitivity order, exploit approximate unimodality of the RPO objective using integer ternary search ( $O(\log R)$  evaluations per variable instead of  $O(R)$ ). At each evaluation, generate an outer-approximation cutting plane:

$$f(y) \geq f_k + g_k^\top (y - y_k), \quad (110)$$

where  $g_k$  is a finite-difference gradient estimate.

**Phase 2: Coordinate descent with OA pruning (up to 3 cycles).** Sweep each variable and evaluate only candidates where the OA lower bound  $<$  incumbent. Add OA cuts after each NLP evaluation. Terminate if no improvement this cycle. The OA lower bound is:

$$\max_k f_k + g_k^\top (y - y_k). \quad (111)$$

**Phase 3: Pairwise neighborhood polishing.** Check  $\pm 1$  perturbations of every pair of discrete variables. OA cuts prune many candidates before expensive NLP evaluation.

### 13.12.4 Inner NLP Solver

At each discrete configuration, the AC OPF solver (IPM by default, fallback enabled) is called to solve the continuous sub-problem. Solution caching by discrete configuration key avoids redundant NLP evaluations.

### 13.12.5 Result Structure

The result includes per-device tap/shunt adjustments, the number of NLP solves, OA cuts added, nodes explored, and the final status ("Optimal(OA)", "Timelimit", or "Nodelimit").

## 14 Short-Circuit Models

### 14.1 Problem Form and Runtime Scope

This chapter covers the short-circuit analysis layer built on a projected canonical AC network. The core computation is a fault-network Thevenin solve for one or more faulted buses, with both simplified and detailed IEC-style reporting routes exposed by the current API.

### 14.2 Code Map

The current implementation is concentrated in:

- `src/analysis/short_circuit.cpp` for batch and single-bus fault execution,
- canonical network projection and admittance assembly reused from the main power-flow preprocessing stack,
- the sparse linear-solver abstraction in the engine layer for repeated fault-network solves.

### 14.3 Pseudocode Map

Section 21 provides the code-aligned workflow summary. Algorithm A12 is the direct pseudocode reference for this chapter.

### 14.4 Current Status

The detailed short-circuit route is now executable rather than placeholder-only. The remaining gap is mostly in result-contract cleanup and richer external-grid reporting, not in the existence of a working solver path.

### 14.5 Implemented Solver Paths

The short-circuit module currently exposes two public result families:

- `compute_short_circuit`: batch computation over all buses or a user-selected subset,
- `compute_fault_at_bus`: single-bus query using the same underlying fault-admittance model.

In addition, the module now exposes an executable detailed IEC-style route under `SCDetailedOptions` / `SCDetailedResult` through `run_short_circuit_detailed` and `run_short_circuit_detailed_batch`. That detailed path computes  $I_k''$ ,  $i_p$ ,  $I_b$ , and  $I_k$  for all four supported fault types. The public struct still carries legacy scaffold fields `placeholders_only` and `pending_functions`, but the current implementation sets them to `false` and empty and fills the detailed per-bus current table.

### 14.6 Fault-Network Construction

The short-circuit entry point first projects the rich information model to the canonical AC network, then constructs a fault admittance matrix from the following components:

$$Y_{fault} = Y_{branch} + Y_{gen}'' + Y_{motor} + Y_{load,motor} + Y_{ext}. \quad (112)$$

**Branch contribution.** All in-service `ACBranch` objects, including projected transformers and switches, contribute their positive-sequence admittance to  $Y_{branch}$ . Tap ratios and phase shifts are handled through the complex off-nominal transformer model.

**Generator subtransient contribution.** Each in-service generator contributes a shunt term approximately equal to

$$Y_{gen}'' \approx \frac{1}{jx_d''}, \quad (113)$$

using `Generator::xdpp_pu` when present and a fallback value from `SCOptions::default_xdpp` otherwise.

**Motor contribution.** The solver adds subtransient motor admittances from two sources:

- explicit `AsynchronousMotor` objects in `ACSystem::motors`,
- the motor fraction embedded in `Load` objects through `motor_percent`, `x_sub_pu`, and `r_sc_pu`.

**External grid contribution.** External-grid data is used as a source equivalent in the fault network using available impedance or short-circuit-strength fields.

## 14.7 Z-Bus Evaluation

Once  $Y_{fault}$  is assembled, the solver factorizes it and obtains the Thevenin impedance at a faulted bus  $k$  from the  $k$ -th diagonal entry of the inverse:

$$Z_{kk} = \left( Y_{fault}^{-1} \right)_{kk}. \quad (114)$$

Rather than building the full inverse explicitly, the implementation solves

$$Y_{fault} z = e_k \quad (115)$$

for each faulted bus and reads the  $k$ -th component of the solution vector.

## 14.8 Current Implemented Fault Equations

The current simplified solver assumes a prefault voltage of

$$V_0 = c \angle 0, \quad c = \text{SCOptions::c\_factor}. \quad (116)$$

**Three-phase fault.**

$$I_f^{3\phi} = \frac{V_0}{Z_{kk}}. \quad (117)$$

**Single-line-to-ground, line-to-line, and double-line-to-ground faults.** The current code uses simplified sequence-network approximations when explicit negative- and zero-sequence detail is unavailable. In effect,

- the negative-sequence network is approximated by the positive-sequence network,
- the zero-sequence network is approximated or simplified rather than fully assembled from every device model.

Therefore, the unsymmetrical-fault results are best interpreted as engineering approximations rather than a complete IEC 60909 sequence implementation.

## 14.9 Reported Quantities

For each bus, the primary result object stores:

$$S_k^{sc} = \frac{S_{base}|V_0|^2}{|Z_{kk}|}, \quad (118)$$

$$I_k'' = |I_f|I_{base}, \quad (119)$$

where the current is reported in kA via the bus voltage base.

## 14.10 Detailed IEC Helper Functions

The detailed SC header also defines helper routines for:

- voltage-factor selection,
- $\kappa$ -factor estimation from  $R/X$ ,
- generator and transformer correction factors,
- zero-sequence correction factors,
- motor-impedance reconstruction.

These helpers are now used by the detailed solver path directly. The remaining gap is no longer “can the detailed route run?” but rather whether the public result contract exposes every contribution channel explicitly enough for downstream consumers.

## 14.11 Practical Interpretation

For the current code base, the short-circuit module should be interpreted as:

- **Implemented** for positive-sequence Z-bus short-circuit studies with generator, motor, and external-grid contributions,
- **Implemented** for approximate unsymmetrical-fault calculations,
- **Implemented** for detailed IEC-style current calculations  $\{I_k'', i_p, I_b, I_k\}$  across  $3\phi$ , SLG, LL, and DLG faults,
- **Planned** for a cleaner detailed public contract with explicit external-grid contribution fields and removal of the leftover scaffold markers.

# 15 Component-to-Module Integration Matrix

## 15.1 Status Codes

The following codes are used in the integration tables.

- D: direct model consumption in the target module,
- F: fixed injection or fixed parameter only,
- P: used after projection into a canonical equivalent,
- N: not currently consumed by that module.

## 15.2 Module Preprocessing Differences

The most important architectural difference between modules is whether they reuse the canonical-projection path.

| Module family       | Uses <code>project_to_canonical_models</code> ? | Practical consequence                                                                                                                                                    |
|---------------------|-------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Main PF family      | yes                                             | Flexible loads, asymmetric loads, transformers, switches, chargers, and optional three-phase models are converted before assembly.                                       |
| Native / parity OPF | yes                                             | Same canonical equivalents as PF are available to the OPF balances.                                                                                                      |
| Short circuit       | yes                                             | Projected branches and loads are visible to the fault-network builder.                                                                                                   |
| Distribution PF     | no                                              | Works directly on <code>ACSystem</code> / <code>HybridPowerSystem</code> ; transformer, switch, flexible-load, asymmetric-load, and charger projection is not automatic. |
| ONR                 | no                                              | Works directly on <code>ACSystem</code> ; hybrid-only components and projection-only equivalents are not inserted automatically.                                         |

## 15.3 Detailed Integration Matrix

For quick reading, the table legend is repeated here: D = direct, F = fixed-only, P = projection-based, N = not consumed.

Table 123: Current component integration across solver modules

| Component model   | PF | OPF fixed | OPF var | DCOPF | RPO | SC | DPF | ONR |
|-------------------|----|-----------|---------|-------|-----|----|-----|-----|
| ACBus             | D  | D         | N       | D     | D   | D  | D   | D   |
| ACBranch          | D  | D         | N       | D     | D   | D  | D   | D   |
| Transformer2W     | P  | P         | N       | N     | P   | P  | N   | N   |
| Transformer3W     | P  | P         | N       | N     | P   | P  | N   | N   |
| Switch            | P  | P         | N       | N     | P   | P  | N   | N   |
| ExternalGrid      | D  | F         | N       | N     | F   | D  | N   | N   |
| Generator         | D  | D         | D       | D     | D   | D  | D   | F   |
| StaticGenerator   | D  | D         | N       | F     | D   | N  | D   | F   |
| RenewableGen      | D  | D         | D       | F     | D   | N  | D   | F   |
| PVSystem          | D  | D         | D       | N     | D   | N  | D   | F   |
| Storage (AC)      | D  | D         | D       | N     | N   | N  | D   | F   |
| Load              | D  | D         | F       | D     | D   | D  | D   | F   |
| FlexibleLoad      | P  | P         | D       | N     | N   | P  | N   | N   |
| AsymmetricLoad    | P  | P         | N       | N     | P   | P  | N   | N   |
| Shunt             | D  | D         | N       | N     | D   | F  | F   | N   |
| ChargingStation   | D  | D         | N       | N     | N   | N  | D   | F   |
| Charger           | P  | P         | N       | N     | N   | N  | N   | N   |
| AsynchronousMotor | N  | N         | N       | N     | N   | D  | N   | N   |

| Component model        | PF | OPF fixed | OPF var | DCOPF | RPO | SC | DPF | ONR |
|------------------------|----|-----------|---------|-------|-----|----|-----|-----|
| DCBus                  | D  | D         | N       | N     | N   | N  | N   | N   |
| DCBranch               | D  | D         | N       | N     | N   | N  | N   | N   |
| DCLoad                 | D  | D         | N       | N     | N   | N  | N   | N   |
| Storage (DC)           | D  | D         | D       | N     | N   | N  | N   | N   |
| StaticGenerator (DC)   | D  | D         | N       | N     | N   | N  | N   | N   |
| VSCConverter           | D  | D         | D       | N     | N   | N  | N   | N   |
| DCDCConverter          | D  | D         | D       | N     | N   | N  | N   | N   |
| EnergyRouterPort       | D  | D         | N       | N     | N   | N  | N   | N   |
| EnergyRouter           | D  | D         | N       | N     | N   | N  | N   | N   |
| VirtualPowerPlant      | D  | D         | N       | N     | N   | N  | D   | N   |
| Microgrid              | D  | D         | N       | N     | N   | N  | D   | N   |
| MobileStorage          | D  | D         | N       | N     | N   | N  | D   | N   |
| ThreePhaseACBus        | P  | P         | N       | N     | P   | P  | N   | N   |
| ThreePhaseACLine       | P  | P         | N       | N     | P   | P  | N   | N   |
| ThreePhaseTransformer  | P  | P         | N       | N     | P   | P  | N   | N   |
| ThreePhaseLoad         | P  | P         | N       | N     | P   | P  | N   | N   |
| ThreePhaseGenerator    | P  | P         | N       | N     | P   | P  | N   | N   |
| ThreePhaseExternalGrid | P  | P         | N       | N     | P   | P  | N   | N   |

## 15.4 Notable Row-Level Notes

- **Load** is direct in OPF, but the explicit variable is currently bus-level load shedding in the parity path rather than per-load flexibility.
- **Shunt** is direct in the main PF/OPF path; in DPF it is simplified to a fixed power contribution instead of a true  $V^2$ -dependent admittance. In RPO, shunts are discrete decision variables (switchable step count).
- **Transformer2W**, **Transformer3W**, **Switch**, **FlexibleLoad**, **AsymmetricLoad**, and **Charger** rely on projection and therefore are invisible to DPF and ONR unless the caller has already converted them.
- **RenewableGen** and **PVSystem** are now parity-IPM decision variables ( $P^{\text{ren}}, Q^{\text{ren}}$ ) for curtailable renewables and controllable PV. In DC OPF they appear as fixed injections.
- **Storage** (AC and DC) and **DCDCConverter** are now parity-IPM decision variables. DCDC dispatch includes analytical  $I^2R$  Jacobian and Hessian.
- **FlexibleLoad** is now a parity-IPM decision variable  $P^{\text{flex}}$ , after projection to canonical form.
- DC OPF uses a  $B$ - $\theta$  network model with linearized or quadratic generator costs and branch flow limits.
- RPO uses AC OPF as the inner NLP and treats transformer taps and switchable shunts as discrete variables.
- **VirtualPowerPlant**, **Microgrid**, and **MobileStorage** are seen by the main PF/OPF path and by the hybrid overload of DPF, but not by the current AC-only ONR or DC OPF interfaces.

## 15.5 Interpretation

The table shows two important facts about the current code base.

1. The main PF, native OPF, parity OPF, and SC paths already share a strong canonical-equivalent layer.
2. The DPF and ONR paths have narrower direct component coverage and should be treated as AC-distribution-specialized modules rather than fully general consumers of every public model struct.

## 16 Optimal Network Reconfiguration

### 16.1 Problem Form and Runtime Scope

This chapter documents the current ONR mixed-integer model for radial feeder reconfiguration. The runtime problem is a LinDistFlow-style MILP with binary branch-status variables, feeder-flow variables, squared-voltage variables, and connectivity-flow variables, solved through the native branch-and-cut engine.

### 16.2 Code Map

The current implementation is concentrated in:

- `src/analysis/topology_analysis.cpp` for ONR model build, solve, and post-processing,
- `src/analysis/distribution_power_flow.cpp` for radial-feeder checks and distribution-side electrical support,
- `src/engine/branch_and_cut.cpp` and its helper files for MILP solution.

### 16.3 Pseudocode Map

The code-aligned workflow summary lives in two places:

- Algorithm A13 in Section 21 for the application-level ONR solve flow,
- Section 22 for the solver-kernel mechanics used underneath the ONR MILP.

### 16.4 Current Status

The ONR route is a live MILP application using the native branch-and-cut stack. Its main current boundary is modeling scope: projected canonical AC components are supported, but higher-level hybrid overlays such as VPP, microgrid, and mobile-storage behavior are not yet injected directly into the ONR MILP.

### 16.5 Scope of the Current ONR Module

The ONR solver now exposes both `ACSystem` and `HybridPowerSystem` overloads. The `HybridPowerSystem` overload first projects the rich model through `project_to_canonical_models()`, so the current production path can see

- direct AC buses and branches,
- direct AC loads and generators,
- direct static generators, renewable generators, PV systems, storage, and charging stations,

- projected transformers, switches, flexible loads, asymmetric loads, chargers, and optional three-phase reductions,
- still no automatic injection of **VPP**, **Microgrid**, or **MobileStorage** overlays into the ONR MILP.

## 16.6 Decision Variables

For an AC network with  $m = |\mathcal{E}_{ac}|$  branches and  $n = |\mathcal{N}_{ac}|$  buses, the MILP uses the variable blocks

$$x = \begin{bmatrix} \alpha & P & Q & v & f & t \end{bmatrix}^\top, \quad (120)$$

where

$$\alpha_b \in \{0, 1\} \quad \text{branch status,} \quad (121)$$

$$P_b, Q_b \in \mathbb{R} \quad \text{branch active/reactive flow,} \quad (122)$$

$$v_i \in \mathbb{R} \quad \text{squared bus voltage magnitude,} \quad (123)$$

$$f_b \in \mathbb{R} \quad \text{single-commodity connectivity flow,} \quad (124)$$

$$t_b \geq 0 \quad \text{auxiliary variable for } |P_b|. \quad (125)$$

## 16.7 Net-Injection Model

The ONR builder computes per-bus net injections directly from the AC component tables as

$$P_i^{net} = \frac{P_i^{load} + P_i^{cs} - P_i^{gen} - P_i^{sg} - P_i^{ren} - P_i^{pv} - P_i^{stor}}{S_{base}}, \quad (126)$$

$$Q_i^{net} = \frac{Q_i^{load} + Q_i^{cs} - Q_i^{gen} - Q_i^{sg} - Q_i^{ren} - Q_i^{pv} - Q_i^{stor}}{S_{base}}. \quad (127)$$

Storage discharge is therefore treated as generation, and charging stations are additional fixed loads.

## 16.8 LinDistFlow MILP

**Objective.** The objective minimizes a linearized active-loss proxy:

$$\min_x \sum_{b=1}^m r_b t_b. \quad (128)$$

**Tree cardinality.**

$$\sum_{b=1}^m \alpha_b = n - 1. \quad (129)$$

**Commodity-flow connectivity.** The solver uses a single-commodity-flow spanning-tree formulation. For the root bus,

$$\sum_{b \in \delta^-(root)} f_b - \sum_{b \in \delta^+(root)} f_b = -(n - 1), \quad (130)$$

and for every non-root bus,

$$\sum_{b \in \delta^-(i)} f_b - \sum_{b \in \delta^+(i)} f_b = 1. \quad (131)$$

Capacity of  $f_b$  is tied to  $\alpha_b$  using big- $M$  logic.

**Power-balance constraints.** For each non-root bus,

$$\sum_{b \in \delta^-(i)} P_b - \sum_{b \in \delta^+(i)} P_b = P_i^{net}, \quad (132)$$

$$\sum_{b \in \delta^-(i)} Q_b - \sum_{b \in \delta^+(i)} Q_b = Q_i^{net}. \quad (133)$$

**Voltage-drop constraints.** The current implementation uses the LinDistFlow relation relaxed by big- $M$ :

$$v_j - v_i + 2r_b P_b + 2x_b Q_b \leq M(1 - \alpha_b), \quad (134)$$

$$-(v_j - v_i + 2r_b P_b + 2x_b Q_b) \leq M(1 - \alpha_b). \quad (135)$$

**Thermal and absolute-value constraints.**

$$|P_b| \leq P_b^{max} \alpha_b, \quad (136)$$

$$|Q_b| \leq Q_b^{max} \alpha_b, \quad (137)$$

$$P_b \leq t_b, \quad (138)$$

$$-P_b \leq t_b. \quad (139)$$

## 16.9 Branch-and-Cut Configuration

The ONR path solves the MILP through the native branch-and-cut engine with the following current tuning:

- cut family: MIR cuts,
- root cut rounds: 5,
- cuts per round: 20,
- branching: pseudocost,
- node selection: hybrid,
- LP node solver: simplex enabled,
- feasibility pump: enabled.

## 16.10 Postprocessing and Verification

The branch-status variables returned by branch-and-cut are snapped to a binary assignment with exactly  $n - 1$  closed branches. The resulting switching plan is then verified by running the full Newton PF solver on a modified `ACSystem` where branch `in_service` flags follow the optimal topology.

## 16.11 Current Modeling Boundary

The ONR module is already useful for radial feeder reconfiguration, but its current mathematical scope is intentionally narrower than the main PF/OPF stack. The main structural limitations are:

- no hybrid-side injections from `VPP`, `Microgrid`, or `MobileStorage`,
- no integer co-optimization of DER dispatch and topology in the current public API.

## 17 Distribution Power Flow

### 17.1 Problem Form and Runtime Scope

This chapter covers the distribution backward-forward-sweep solver for radial AC feeders. The implemented mathematical model is a current-injection radial load flow with outer reactive-limit enforcement, distinct from the transmission-side hybrid Newton formulation documented in Section 12.

### 17.2 Code Map

The current implementation is concentrated in:

- `src/analysis/distribution_power_flow.cpp` for BFS execution and reporting,
- `src/analysis/topology_analysis.cpp` for radiality and connectivity checks,
- shared model structs in the main information-model layer for buses, branches, DERs, and loads.

### 17.3 Pseudocode Map

The relevant code-aligned workflow is Algorithm A14 in Section 21. The feeder-topology assumptions discussed here also align with the ONR workflow in Algorithm A13.

### 17.4 Current Status

The BFS solver is implemented and usable for radial feeders, but its model scope is intentionally narrower than the canonical projection used by the main PF stack. It is therefore best read as a specialized feeder-analysis solver rather than a universal hybrid AC/DC load-flow engine.

### 17.5 Scope of the Current DPF Module

The distribution solver implements a backward-forward sweep (BFS) for radial AC networks. Unlike the main PF stack, it does *not* automatically invoke the canonical projection layer. Its practical input coverage is therefore:

- direct ACBus, ACBranch, Generator, Load, Shunt, StaticGenerator, RenewableGen, PVSystem, Storage, and ChargingStation,
- hybrid overload only for fixed VirtualPowerPlant, Microgrid, and MobileStorage injections,
- no automatic use of Transformer2W, Transformer3W, Switch, FlexibleLoad, Asymmetric Load, Charger, or optional three-phase models.

### 17.6 Radial Network Requirement

The solver assumes a single slack bus and a radial branch topology. The helper functions `is_connected()` and `is_radial()` in `topology_analysis.hpp` are intended to be used before calling the BFS solver on a candidate feeder model.

## 17.7 Net-Injection Model

For each non-root bus, the BFS routine constructs a complex net demand

$$S_i^{net} = P_i^{net} + jQ_i^{net}, \quad (140)$$

with the sign convention “load minus generation” rather than “bus injection positive.” In current code form,

$$S_i^{net} = S_i^{load} + S_i^{cs} + S_i^{shunt} - S_i^{gen} - S_i^{sg} - S_i^{ren} - S_i^{pv} - S_i^{stor}, \quad (141)$$

plus optional hybrid injections from VPP, microgrid export, and stationary mobile storage when the `HybridPowerSystem` overload is used.

**Important implementation detail.** Even though `Load` stores ZIP coefficients, the current BFS routine uses `Load::p_mw` and `Load::q_mvar` directly as a constant-power baseline and does not iterate the full ZIP model against bus voltage. The same simplification applies to shunts: when included, they are added as fixed complex power contributions rather than true  $V^2$ -dependent admittance terms.

## 17.8 Backward-Forward Sweep Equations

For branch current and bus voltage updates, the current implementation follows standard radial BFS logic.

**Backward sweep.** At bus  $j$ , the injected current is approximated as

$$I_j^{inj} = \frac{\overline{S_j^{net}}}{V_j}, \quad (142)$$

and branch currents are accumulated from children toward the root.

**Forward sweep.** If branch  $b$  connects parent bus  $i$  to child bus  $j$ , then

$$V_j = V_i - Z_b I_b, \quad Z_b = r_b + jx_b. \quad (143)$$

**Convergence metric.** The stopping criterion is the maximum voltage change:

$$\varepsilon = \max_i |V_i^{(k)} - V_i^{(k-1)}|. \quad (144)$$

## 17.9 Outer Loop for Reactive Limits

When `DPFOptions::enforce_q_limits` is enabled, the solver runs an outer loop that:

1. solves the BFS problem,
2. estimates generator reactive output at PV buses,
3. clamps generators that exceed  $Q$  limits,
4. converts affected buses from PV behavior to PQ behavior,
5. repeats until no new violations are found or the outer limit is hit.

## 17.10 Reported Branch Power and Losses

After convergence, the module reports sending-end branch power

$$S_b^{from} = V_{from(b)} \overline{I_b}, \quad (145)$$

and network loss totals

$$P_{loss}^{tot} = \sum_b |I_b|^2 r_b S_{base}, \quad (146)$$

$$Q_{loss}^{tot} = \sum_b |I_b|^2 x_b S_{base}. \quad (147)$$

## 17.11 Relationship to the Main Newton PF

The BFS implementation is best viewed as a feeder-oriented fast solver for radial networks. It is *not* a drop-in replacement for the main projected hybrid PF path because:

- it does not consume the same projected component set,
- it does not solve the DC subsystem,
- it approximates load and shunt voltage dependence more aggressively,
- it uses a radial tree traversal rather than the sparse Jacobian machinery.

## 17.12 Three-Phase Power Flow (abc Domain)

Two three-phase solvers are provided, both operating in the full abc phase domain:

1. **Newton–Raphson (NR)** — the primary solver, suitable for general (meshed, multi-source, and hybrid AC/DC) network topologies. It builds a  $3n \times 3n$  admittance matrix and solves polar-form mismatch equations via Newton’s method.
2. **Backward–Forward Sweep (BFS)** — a legacy solver retained for radial single-source distribution feeders. See Section 17.12.5.

### 17.12.1 Phase Domain and API

An enumeration `PhaseDomain` selects between positive-sequence (default single-phase) and abc (per-phase) analysis.

The NR solver is invoked through `solve_three_phase_nr()`, and the convenience wrapper `solve_three_phase()` (in `powerflow/three_phase.hpp`) delegates to it. Both accept a `ThreePhaseACSystem` or the `HybridPowerSystem` wrapper (which checks for a populated `three_phase_ac` subsystem).

The legacy BFS solver remains available via `solve_three_phase_distribution_pf()`.

### 17.12.2 $3 \times 3$ Series Impedance from Sequence Parameters

Each `ThreePhaseACLine` carries positive-sequence  $Z_1 = r_1 + jx_1$  and zero-sequence  $Z_0 = r_0 + jx_0$  impedance. The  $3 \times 3$  abc-domain series impedance matrix is built via the Fortescue-to-abc transformation:

$$\mathbf{Z}_{abc} = \mathbf{A} \operatorname{diag}(Z_0, Z_1, Z_1) \mathbf{A}^{-1}, \quad (148)$$

where  $\mathbf{A}$  is the symmetrical component transformation matrix ( $a = e^{j2\pi/3}$ ):

$$\mathbf{A} = \frac{1}{3} \begin{bmatrix} 1 & 1 & 1 \\ 1 & a^2 & a \\ 1 & a & a^2 \end{bmatrix}. \quad (149)$$

The resulting impedance is a circulant matrix:

$$\mathbf{Z}_{abc} = \begin{bmatrix} Z_s & Z_m & Z_m \\ Z_m & Z_s & Z_m \\ Z_m & Z_m & Z_s \end{bmatrix}, \quad Z_s = \frac{Z_0 + 2Z_1}{3}, \quad Z_m = \frac{Z_0 - Z_1}{3}. \quad (150)$$

When the line is balanced ( $Z_0 = Z_1$ ), the mutual impedance  $Z_m = 0$  and the three phases decouple into independent single-phase circuits, each equivalent to the positive-sequence model.

### 17.12.3 Per-Phase Net Injection

At each non-slack bus  $i$ , the three-phase net demand vector is

$$\mathbf{S}_i^{net} = \begin{bmatrix} S_i^a \\ S_i^b \\ S_i^c \end{bmatrix} = \begin{bmatrix} P_i^{d,a} + jQ_i^{d,a} \\ P_i^{d,b} + jQ_i^{d,b} \\ P_i^{d,c} + jQ_i^{d,c} \end{bmatrix} - \mathbf{S}_i^{gen}, \quad (151)$$

where bus-level demand  $P_i^{d,\phi}$ ,  $Q_i^{d,\phi}$  and three-phase load table entries are summed per phase. Generators with per-phase output are subtracted, using equal  $P/3$ ,  $Q/3$  splits across phases when only aggregate ratings are available.

### 17.12.4 Three-Phase Newton–Raphson Solver

The three-phase Newton–Raphson solver generalises the standard single-phase polar-form NR power flow to a  $3n \times 3n$  phase-node system, where  $n$  is the number of buses.

**Phase-node indexing.** Each bus  $i \in \{0, \dots, n-1\}$  contributes three phase nodes:

$$r = 3i + \phi, \quad \phi \in \{0(a), 1(b), 2(c)\}. \quad (152)$$

**$3n \times 3n$  admittance matrix  $\mathbf{Y}^{abc}$ .** For each three-phase line connecting bus  $i$  to bus  $j$ , the  $3 \times 3$  abc admittance block is obtained by inverting the circulant impedance:

$$Y_s = \frac{1/Z_0 + 2/Z_1}{3}, \quad Y_m = \frac{1/Z_0 - 1/Z_1}{3}, \quad (153)$$

$$\mathbf{Y}_{abc} = \begin{bmatrix} Y_s & Y_m & Y_m \\ Y_m & Y_s & Y_m \\ Y_m & Y_m & Y_s \end{bmatrix}. \quad (154)$$

The global admittance matrix is assembled from the standard bus-admittance construction:

$$\mathbf{Y}_{[3i+p, 3i+q]}^{abc} += Y_{pq}^{(b)} + \frac{1}{2} j B_1^{(b)} \delta_{pq}, \quad (155)$$

$$\mathbf{Y}_{[3j+p, 3j+q]}^{abc} += Y_{pq}^{(b)} + \frac{1}{2} j B_1^{(b)} \delta_{pq}, \quad (156)$$

$$\mathbf{Y}_{[3i+p, 3j+q]}^{abc} -= Y_{pq}^{(b)}, \quad (157)$$

$$\mathbf{Y}_{[3j+p, 3i+q]}^{abc} -= Y_{pq}^{(b)}, \quad (158)$$

where  $B_1^{(b)}$  is the positive-sequence line charging susceptance of branch  $b$ , applied identically to each phase.

Per-phase bus shunt admittance is added on the diagonal when `include_shunts` is enabled:  $\mathbf{Y}_{[3i+\phi, 3i+\phi]}^{abc} += g_\phi + jb_\phi$ .

**State vector and unknowns.** The NR state vector is

$$\mathbf{x} = [\theta_{a,1}, \theta_{b,1}, \theta_{c,1}, \dots, V_{m,a,1}, V_{m,b,1}, V_{m,c,1}, \dots]^T. \quad (159)$$

- **Slack bus:** All 3 phase angles and magnitudes are fixed (excluded from  $\mathbf{x}$ ).
- **PQ bus:** Contributes  $3\theta$  and  $3V_m$  unknowns (6 per bus).
- **PV bus:** Contributes  $3\theta$  unknowns (3 per bus);  $V_m$  is fixed.

Denote the number of angle unknowns as  $n_p$  and the number of magnitude unknowns as  $n_q$ ; the total system dimension is  $n_p + n_q$ .

**Mismatch equations.** At each non-slack phase node  $r = 3i + \phi$ :

$$P_r^{calc} = \sum_{s=0}^{3n-1} |V_r||V_s|(G_{rs} \cos \theta_{rs} + B_{rs} \sin \theta_{rs}), \quad (160)$$

$$Q_r^{calc} = \sum_{s=0}^{3n-1} |V_r||V_s|(G_{rs} \sin \theta_{rs} - B_{rs} \cos \theta_{rs}), \quad (161)$$

with  $\theta_{rs} = \theta_r - \theta_s$  and  $Y_{rs} = G_{rs} + jB_{rs}$ . The mismatch vector is

$$\Delta \mathbf{f} = \begin{bmatrix} P_r^{spec} - P_r^{calc} \\ Q_r^{spec} - Q_r^{calc} \end{bmatrix}, \quad r \in (\text{angle or Vm unknowns}). \quad (162)$$

**Jacobian.** The standard polar-form Jacobian entries for  $r \neq s$  are:

$$\frac{\partial P_r}{\partial \theta_s} = |V_r||V_s|(G_{rs} \sin \theta_{rs} - B_{rs} \cos \theta_{rs}), \quad (163)$$

$$\frac{\partial P_r}{\partial |V_s|} = |V_r|(G_{rs} \cos \theta_{rs} + B_{rs} \sin \theta_{rs}), \quad (164)$$

$$\frac{\partial Q_r}{\partial \theta_s} = -|V_r||V_s|(G_{rs} \cos \theta_{rs} + B_{rs} \sin \theta_{rs}), \quad (165)$$

$$\frac{\partial Q_r}{\partial |V_s|} = |V_r|(G_{rs} \sin \theta_{rs} - B_{rs} \cos \theta_{rs}). \quad (166)$$

Diagonal (self) terms use the compact form:

$$\frac{\partial P_r}{\partial \theta_r} = -Q_r^{calc} - |V_r|^2 B_{rr}, \quad \frac{\partial P_r}{\partial |V_r|} = \frac{P_r^{calc}}{|V_r|} + |V_r| G_{rr}, \quad (167)$$

$$\frac{\partial Q_r}{\partial \theta_r} = P_r^{calc} - |V_r|^2 G_{rr}, \quad \frac{\partial Q_r}{\partial |V_r|} = \frac{Q_r^{calc}}{|V_r|} - |V_r| B_{rr}. \quad (168)$$

**Update step.** At each iteration the Newton correction is obtained by solving

$$\mathbf{J} \Delta \mathbf{x} = \Delta \mathbf{f}, \quad (169)$$

using the shared sparse linear solver infrastructure (`SparseLinearSolver`: KLU > UMFPACK > SparseLU fallback). The update is

$$\theta_r^{(k+1)} = \theta_r^{(k)} + \Delta \theta_r, \quad (170)$$

$$|V_r|^{(k+1)} = \max(|V_r|^{(k)} + \Delta |V_r|, 0.05), \quad (171)$$

with voltage clamping at 0.05 pu to prevent divergence.

**Convergence criterion.**

$$\varepsilon = \max_r |\Delta f_r| < \text{tol}. \quad (172)$$

Symbolic pattern analysis is performed once (iteration 0); subsequent iterations reuse the sparsity pattern.

**Post-convergence outputs.** Per-phase branch current is recovered from the abc admittance:

$$I_b^{(\phi)} = \sum_{q=0}^2 Y_{\phi q}^{(b)} (V_{from}^{(q)} - V_{to}^{(q)}), \quad (173)$$

producing sending-end power  $S_b^\phi = V_{from}^\phi \overline{I_b^\phi}$  and per-phase losses  $P_{loss}^\phi = |I_b^\phi|^2 \text{Re}(Z_s)$ ,  $Q_{loss}^\phi = |I_b^\phi|^2 \text{Im}(Z_s)$ .

### 17.12.5 Three-Phase Backward-Forward Sweep

**Note.** The BFS solver is retained as a legacy option for radial single-source feeders. The NR solver (Section 17.12.4) is the recommended primary solver because it supports meshed topologies, multiple generators, and hybrid AC/DC networks.

**Backward sweep.** At each non-root bus  $j$  in reverse BFS order, the per-phase injection current is

$$I_j^\phi = \frac{\overline{S_j^{met, \phi}}}{V_j^\phi}, \quad \phi \in \{a, b, c\}. \quad (174)$$

Branch currents are accumulated from children toward the root, preserving three-phase coupling:

$$\mathbf{I}_{b(\text{parent})} += \mathbf{I}_{b(\text{child})}. \quad (175)$$

**Forward sweep.** For branch  $b$  connecting parent bus  $i$  to child bus  $j$ :

$$\mathbf{V}_j = \mathbf{V}_i - \mathbf{Z}_{abc}^{(b)} \mathbf{I}_b, \quad (176)$$

where  $\mathbf{V}$ ,  $\mathbf{I}$  are 3-vectors and  $\mathbf{Z}_{abc}^{(b)}$  is the  $3 \times 3$  branch impedance.

**Convergence.** The stopping criterion extends to all phases:

$$\varepsilon = \max_{i, \phi} |V_i^{\phi, (k)} - V_i^{\phi, (k-1)}|. \quad (177)$$

### 17.12.6 Per-Phase Branch Power and Losses

After convergence, per-phase branch currents are recovered via the inverse of the circulant impedance matrix:

$$\mathbf{Z}_{abc}^{-1} = \begin{bmatrix} Y_s & Y_m & Y_m \\ Y_m & Y_s & Y_m \\ Y_m & Y_m & Y_s \end{bmatrix}, \quad Y_s = \frac{1/Z_0 + 2/Z_1}{3}, \quad Y_m = \frac{1/Z_0 - 1/Z_1}{3}, \quad (178)$$

and per-phase sending-end power:

$$S_b^\phi = V_{from(b)}^\phi \overline{I_b^\phi}. \quad (179)$$

Per-phase losses use the self-impedance component of  $\mathbf{Z}_{abc}$ :

$$P_{loss}^\phi = |I_b^\phi|^2 \operatorname{Re}(Z_s) S_{base}, \quad Q_{loss}^\phi = |I_b^\phi|^2 \operatorname{Im}(Z_s) S_{base}. \quad (180)$$

Total network losses are the sum across all branches and phases:

$$P_{loss}^{tot} = \sum_{\phi} P_{loss}^\phi, \quad Q_{loss}^{tot} = \sum_{\phi} Q_{loss}^\phi. \quad (181)$$

### 17.12.7 Voltage Unbalance Factor (VUF)

The maximum voltage unbalance factor across all buses is reported as a quality metric. VUF is defined via the symmetrical-component decomposition of bus voltages:

$$V_i^+ = \frac{1}{3}(V_i^a + a V_i^b + a^2 V_i^c), \quad (182)$$

$$V_i^- = \frac{1}{3}(V_i^a + a^2 V_i^b + a V_i^c), \quad (183)$$

$$\text{VUF}_i = \frac{|V_i^-|}{|V_i^+|} \times 100\%. \quad (184)$$

A VUF of 0% indicates a perfectly balanced system; the IEC 61000-2-2 standard recommends  $\text{VUF} \leq 2\%$  for low-voltage networks.

### 17.12.8 Balanced-System Reduction

When all three-phase loads are balanced ( $S^a = S^b = S^c$ ) and all lines have  $Z_0 = Z_1$ , both the NR and BFS abc-domain solvers produce per-phase results identical to 1/3 of the single-phase positive-sequence solution. This property is used for cross-validation of the three-phase implementations against the existing single-phase solver.

## 18 Distribution Reliability Assessment with Failure-Mode Enumeration

### 18.1 Problem Form and Runtime Scope

This chapter documents the reliability-assessment formulation for enumerated contingency analysis on hybrid distribution systems. The core mathematical object is a frequency-weighted restoration or service-loss evaluation over a set of explicit component-failure scenarios rather than pure Monte Carlo sampling.

## 18.2 Code Map

The current implementation and supporting infrastructure are concentrated in:

- `src/analysis/reliability_assessment.cpp` for reliability-index computation and scenario aggregation,
- `src/analysis/topology_analysis.cpp` for feeder switching and topology support,
- `src/analysis/distribution_power_flow.cpp` for electrical evaluation of restored feeder states.

## 18.3 Pseudocode Map

The notebook’s detailed algorithm section does not yet carry a dedicated FMEA pseudocode block. The closest operational references are the ONR and distribution-PF workflows in Algorithms A13 and A14 in Section 21. This chapter therefore serves as the primary problem-form reference for reliability enumeration itself.

## 18.4 Current Status

This chapter is partly descriptive and partly implementation-facing. The C++ code base contains the reliability-analysis module, but the documentation here still plays the role of the canonical formulation for contingency weighting, duration modeling, and service-restoration metrics.

This section defines a distribution-system reliability framework for hybrid AC/DC networks that is aligned with the topology-reconfiguration formulation in `DistributionPowerFlow/TopologyAnalysis/src/topology_reconfiguration.jl` and the current C++ component models.

The target use case is to replace pure Monte Carlo state sampling for distribution studies with an explicit  $N - 1$  failure-mode analysis (FMEA) that:

1. enumerates single-component contingencies across AC and DC assets,
2. models switching and repair windows separately,
3. co-optimizes restoration dispatch and feeder topology,
4. quantifies the resilience value of microgrids, network reconfiguration, and grid-forming BESS.

## 18.5 Sets and Indices

$$\mathcal{N} = \mathcal{N}^{ac} \cup \mathcal{N}^{dc} \quad \text{buses,} \quad (185)$$

$$\mathcal{E} = \mathcal{E}^{ac} \cup \mathcal{E}^{dc} \cup \mathcal{E}^{vsc} \quad \text{switchable edges,} \quad (186)$$

$$\mathcal{G} = \text{dispatchable generators,} \quad (187)$$

$$\mathcal{M} = \text{microgrids (with optional islanding),} \quad (188)$$

$$\mathcal{B} = \text{storage units (including grid-forming BESS),} \quad (189)$$

$$\mathcal{L} = \text{served load points,} \quad (190)$$

$$\mathcal{C} = \text{all reliability-tracked components,} \quad (191)$$

$$\Omega = \{0\} \cup \{k : k \in \mathcal{C}\} \quad \text{base state and } N - 1 \text{ contingencies,} \quad (192)$$

$$\mathcal{S} = \{\text{sw, rep}\} \quad \text{switching and repair stages.} \quad (193)$$

For each contingency  $k$ , stage  $s \in \mathcal{S}$ , and load  $i \in \mathcal{L}$ :

$$P_{i,k,s}^{\text{shed}} \geq 0 \quad \text{unsupplied active power (MW)}, \quad (194)$$

$$E_{i,k,s}^{\text{ens}} = P_{i,k,s}^{\text{shed}} \tau_{k,s} \quad \text{unsupplied energy (MWh)}. \quad (195)$$

## 18.6 Failure and Duration Parameterization

Each component  $k \in \mathcal{C}$  is represented by annual failure frequency  $\lambda_k$  (occ/yr) and stage durations  $\tau_{k,\text{sw}}, \tau_{k,\text{rep}}$  (hours).

**Field-to-parameter mapping.** The component models expose either forced outage rate (FOR), failure rate, or MTBF/MTTR fields. Use the following conversions:

$$\lambda_k = \frac{1}{\text{MTBF}_k}, \quad (196)$$

$$\mu_k = \frac{1}{\text{MTTR}_k}, \quad (197)$$

$$\text{FOR}_k = \frac{\lambda_k}{\lambda_k + \mu_k}, \quad (198)$$

$$\lambda_k = \frac{\text{FOR}_k}{(1 - \text{FOR}_k) \text{MTTR}_k}, \quad (199)$$

$$\lambda_k^{\text{line}} = \frac{\lambda_k^{\text{per km}} \ell_k}{\max(1, n_k^{\text{parallel}})}. \quad (200)$$

If a component has scheduled unavailability  $t_k^{\text{sched}}$  (hr/yr), include an equivalent availability derating  $A_k^{\text{sched}} = 1 - t_k^{\text{sched}}/8760$ , or represent scheduled outages as deterministic scenarios in  $\Omega$ .

**Switching duration model.** For feeder restoration after failure  $k$ , define

$$\tau_{k,\text{sw}} = \tau_k^{\text{detect}} + \tau_k^{\text{isolate}} + \tau_k^{\text{reconf}}, \quad (201)$$

where  $\tau_k^{\text{reconf}}$  may use switch operation data  $t^{\text{operation}}$  and switching-failure probability  $p^{\text{sw-fail}}$ :

$$\tau_k^{\text{reconf}} = \sum_{r \in \mathcal{R}_k} \left[ (1 - p_r^{\text{sw-fail}}) t_r^{\text{auto}} + p_r^{\text{sw-fail}} t_r^{\text{manual}} \right]. \quad (202)$$

## 18.7 Contingency Enumeration and Weights

For strict  $N-1$  FMEA, solve one restoration problem per failed component  $k \in \mathcal{C}$ . The expected annual reliability index of any severity term  $Z_{k,s}$  is frequency-weighted:

$$\mathbb{E}[Z] = \sum_{k \in \mathcal{C}} \lambda_k \sum_{s \in \mathcal{S}} Z_{k,s}. \quad (203)$$

This avoids Monte Carlo variance and is accurate in the rare-event regime used in distribution reliability planning.

## 18.8 Reconfiguration-Aware Restoration Optimization

For each contingency  $k$  and stage  $s$ , solve an operational restoration problem with topology and dispatch variables. A linearized mixed-integer model consistent with the Julia topology formulation is:

**Decision variables.**

$$\beta_{e,k,s} \in \{0, 1\} \quad \text{edge energized status,} \quad (204)$$

$$F_{e,k,s} \in \mathbb{R} \quad \text{fictitious/routing flow,} \quad (205)$$

$$P_{g,k,s}, Q_{g,k,s} \quad \text{generator dispatch,} \quad (206)$$

$$P_{m,k,s}, Q_{m,k,s} \quad \text{microgrid injection,} \quad (207)$$

$$P_{b,k,s}^{\text{dis}}, P_{b,k,s}^{\text{ch}}, Q_{b,k,s} \quad \text{BESS outputs,} \quad (208)$$

$$P_{i,k,s}^{\text{shed}} \geq 0 \quad \text{load shedding.} \quad (209)$$

**Objective (per contingency-stage).**

$$\min J_{k,s} = \sum_{g \in \mathcal{G}} c_g P_{g,k,s} + \sum_{b \in \mathcal{B}} c_b^{\text{cyc}} P_{b,k,s}^{\text{dis}} + \sum_{i \in \mathcal{L}} v_i P_{i,k,s}^{\text{shed}} + c^{\text{sw}} N_{k,s}^{\text{sw}}. \quad (210)$$

**Topology and radiality constraints.**

$$|F_{e,k,s}| \leq M_e \beta_{e,k,s}, \quad \forall e \in \mathcal{E}, \quad (211)$$

$$\sum_{e \in \mathcal{E}} \beta_{e,k,s} = |\mathcal{N}| - n_{k,s}^{\text{source}}, \quad (212)$$

with connectivity-flow equations equivalent to the single-commodity constraints in the network-reconfiguration section.

**Failure consistency and reconfiguration logic.** Let  $\zeta_{e,k} \in \{0, 1\}$  denote edge availability under contingency  $k$  (failed element forced open), and  $\alpha_{e,k,s} \in [0, 1]$  the restoration policy coefficient used in the Julia model:

$$\beta_{e,k,s} \leq \zeta_{e,k}, \quad (213)$$

$$\beta_{e,k,s} \geq \alpha_{e,k,s} + \zeta_{e,k} - 1. \quad (214)$$

**Nodal power balance with shedding.** For each bus  $n \in \mathcal{N}$ :

$$\sum_{g \in \mathcal{G}_n} P_{g,k,s} + \sum_{m \in \mathcal{M}_n} P_{m,k,s} + \sum_{b \in \mathcal{B}_n} (P_{b,k,s}^{\text{dis}} - P_{b,k,s}^{\text{ch}}) + P_{n,k,s}^{\text{import}} = P_n^{\text{load}} - P_{n,k,s}^{\text{shed}}. \quad (215)$$

Reactive-power and voltage/security constraints follow the same linearized model family as the ONR and DPF layers.

## 18.9 Microgrid and Grid-Forming BESS Formulation

### 18.9.1 Microgrid islanding states

Define  $z_{m,k,s} \in \{0, 1\}$ :

$$|P_{m,k,s}^{\text{pcc}}| \leq \bar{P}_m^{\text{pcc}} (1 - z_{m,k,s}), \quad (216)$$

$$P_{m,k,s}^{\text{island,gen}} + P_{m,k,s}^{\text{island,bess}} \geq P_{m,k,s}^{\text{island,load}} - P_{m,k,s}^{\text{shed}} - M(1 - z_{m,k,s}). \quad (217)$$

When  $z_{m,k,s} = 1$ , the microgrid supplies internal demand locally; when  $z_{m,k,s} = 0$ , PCC exchange is allowed.

### 18.9.2 Grid-forming BESS support constraints

For each BESS  $b$ :

$$0 \leq P_{b,k,s}^{\text{dis}} \leq \bar{P}_b^{\text{dis}} u_{b,k,s}, \quad (218)$$

$$0 \leq P_{b,k,s}^{\text{ch}} \leq \bar{P}_b^{\text{ch}} (1 - u_{b,k,s}), \quad (219)$$

$$\underline{Q}_b \leq Q_{b,k,s} \leq \bar{Q}_b, \quad (220)$$

$$\text{SOC}_{b,k,s}^+ = \text{SOC}_{b,k,s}^- - \frac{\tau_{k,s}}{E_b^{\text{rated}}} \left( \frac{P_{b,k,s}^{\text{dis}}}{\eta_b^{\text{dis}}} - \eta_b^{\text{ch}} P_{b,k,s}^{\text{ch}} \right), \quad (221)$$

$$\underline{\text{SOC}}_b \leq \text{SOC}_{b,k,s}^\pm \leq \overline{\text{SOC}}_b. \quad (222)$$

For grid-forming mode, enforce reserve for voltage/frequency support:

$$P_{b,k,s}^{\text{dis}} + R_{b,k,s}^{\text{up}} \leq \bar{P}_b^{\text{gfm}}, \quad (223)$$

and optionally impose a droop-compatible linear envelope around nominal frequency/voltage operating points.

## 18.10 Reliability Metrics for Distribution Assessment

### 18.10.1 Energy and adequacy indices

$$\text{EENS} = \sum_{k \in \mathcal{C}} \lambda_k \sum_{s \in \mathcal{S}} \sum_{i \in \mathcal{L}} E_{i,k,s}^{\text{ens}}, \quad (224)$$

$$\text{EDNS} = \frac{\text{EENS}}{8760}, \quad (225)$$

$$\text{LOLE} = \sum_{k \in \mathcal{C}} \lambda_k \sum_{s \in \mathcal{S}} \tau_{k,s} \mathbb{I} \left\{ \sum_i P_{i,k,s}^{\text{shed}} > 0 \right\}, \quad (226)$$

$$\text{LOLF} = \sum_{k \in \mathcal{C}} \lambda_k \mathbb{I} \left\{ \exists s : \sum_i P_{i,k,s}^{\text{shed}} > 0 \right\}. \quad (227)$$

### 18.10.2 Customer-based distribution indices

Let  $N_i$  be customers at load point  $i$ , and  $I_{i,k,s} \in \{0, 1\}$  indicate interruption at  $(i, k, s)$ . Define nodal interruption frequency and duration:

$$\text{CIF}_i = \sum_k \lambda_k \sum_s I_{i,k,s}, \quad (228)$$

$$\text{CID}_i = \sum_k \lambda_k \sum_s \tau_{k,s} I_{i,k,s}. \quad (229)$$

Then IEEE 1366 indices are

$$\text{SAIFI} = \frac{\sum_i N_i \text{CIF}_i}{\sum_i N_i}, \quad (230)$$

$$\text{SAIDI} = \frac{\sum_i N_i \text{CID}_i}{\sum_i N_i}, \quad (231)$$

$$\text{CAIDI} = \frac{\text{SAIDI}}{\text{SAIFI}}, \quad (232)$$

$$\text{ASAI} = 1 - \frac{\text{SAIDI}}{8760}, \quad (233)$$

$$\text{ASUI} = 1 - \text{ASAI}. \quad (234)$$

### 18.10.3 Priority-weighted resilience metric

For critical-service weighting  $w_i$ :

$$\text{WEENS} = \sum_k \lambda_k \sum_s \sum_i w_i E_{i,k,s}^{\text{ens}}. \quad (235)$$

### 18.11 Modeling 30+ AC/DC Component Types in One Framework

The framework uses a unified component-to-contingency mapping:

$$\Phi : \mathcal{C} \rightarrow (\lambda_k, \tau_{k,\text{sw}}, \tau_{k,\text{rep}}, \mathcal{U}_k), \quad (236)$$

where  $\mathcal{U}_k$  is the set of model constraints modified by failure  $k$  (edge outage, generator derating, converter outage, switch lockout, etc.).

The complete field-level catalog is provided in Section 18.14.

### 18.12 Algorithmic Procedure (Deterministic FMEA)

- 1: Build component set  $\mathcal{C}$  from AC/DC model tables.
- 2: Convert reliability fields to  $\lambda_k, \tau_{k,\text{sw}}, \tau_{k,\text{rep}}$ .
- 3: **for** each contingency  $k \in \mathcal{C}$  **do**
- 4:     **for** each stage  $s \in \{\text{sw}, \text{rep}\}$  **do**
- 5:         Apply failure state to availability vector  $\zeta_k$ .
- 6:         Solve restoration MILP/LP (Eq. 210-215).
- 7:         Record  $P_{i,k,s}^{\text{shed}}$ , switching count, and dispatch.
- 8:     **end for**
- 9: **end for**
- 10: Aggregate EENS/LOLE/LOLF/SAIFI/SAIDI using Eq. 203.
- 11: Optionally compare against NSQ/SEQ Monte Carlo as cross-validation.

### 18.13 Implementation Notes for the C++ Reliability Module

To integrate this formulation into `src/analysis/reliability_assessment.cpp`:

1. add a deterministic FMEA execution path alongside NSQ/SEQ Monte Carlo,
2. add a reconfiguration-aware contingency evaluator (LP/MILP) for distribution operation,
3. include microgrid islanding and BESS SOC update for  $\tau_{k,\text{sw}}$  and  $\tau_{k,\text{rep}}$ ,
4. compute customer-based indices from `Load::n_customers` and `Bus::importance`.

This gives a complete bridge from field-level component reliability parameters to planning-grade reliability metrics under feeder reconfiguration.

### 18.14 Component Reliability Catalog for Hybrid AC/DC FMEA

Table 124 lists the reliability-relevant component classes used to build contingency set  $\mathcal{C}$ . The catalog is organized to cover more than 30 AC/DC component types in a single modeling workflow.

Table 124: Reliability field mapping for hybrid AC/DC components

| Component class                                     | Typical contingency action            | Reliability and resilience fields                                                            |
|-----------------------------------------------------|---------------------------------------|----------------------------------------------------------------------------------------------|
| ACBus, DCBus                                        | bus unavailable / de-energized island | in_service, n_customers, importance                                                          |
| ACBranch, DCBranch, ThreePhaseACLine                | edge outage                           | failure_rate, mttr_hr, t_scheduled_hr, n_parallel, length_km                                 |
| Switch                                              | switching path blocked / delayed      | mtbf_hours, mttr_hours, t_scheduled_hr, p_sw_fail, t_operation_s, t_tp_hr                    |
| CircuitBreaker, DCCircuitBreaker                    | breaker unavailable                   | in_service (extendable with MTBF/MTTR if needed)                                             |
| Transformer2W, Transformer3W, ThreePhaseTransformer | transformer outage                    | mtbf_hours, mttr_hours, t_scheduled_hr, n_parallel                                           |
| Generator                                           | generation outage / derating          | forced_outage_rate, mttr_hr, t_scheduled_hr, in_service                                      |
| StaticGenerator, RenewableGen, PVSystem             | DER outage / derating                 | mtbf_hours, mttr_hours, t_scheduled_hr, in_service                                           |
| Storage                                             | storage subsystem outage              | forced_outage_rate, mttr_hr, subsystem MTBF/MTTR fields (battery, PCS, BMS), soh, max_cycles |
| MobileStorage                                       | mobile BESS unavailable / delayed     | system and subsystem MTBF/MTTR, status, location, arrival_time, departure_time, soh          |
| VSCConverter                                        | AC/DC interface outage                | forced_outage_rate, mttr_hr, t_scheduled_hr, control_mode                                    |
| DCDCConverter                                       | DC/DC transfer outage                 | mtbf_hours, mttr_hours, t_scheduled_hr                                                       |
| EnergyRouter and ports                              | multi-port converter outage           | mtbf_hours, mttr_hours, in_service                                                           |
| Microgrid                                           | PCC loss, island operation            | mtbf_hours, mttr_hours, t_scheduled_hr, islanding_capability, auto_reconnection              |
| VirtualPowerPlant                                   | aggregated DER outage                 | mtbf_hours, mttr_hours, t_scheduled_hr, asset count fields                                   |
| Load, DCLoad                                        | interruption accounting node          | priority, n_customers, controllable, cost_mw                                                 |
| FlexibleLoad, AsymmetricLoad                        | response-limited interruption         | availability_pct (flex load), phase and ZIP data (asymmetric load)                           |
| ChargingStation, Charger                            | EV service interruption               | mtbf_hours, mttr_hours, t_scheduled_hr, n_cars, utilization_rate, v2g_capable                |
| ExternalGrid                                        | upstream source unavailability        | in_service, source strength and equivalent impedance fields                                  |

| Component class          | Typical contingency action        | Reliability and resilience fields                     |
|--------------------------|-----------------------------------|-------------------------------------------------------|
| <b>AsynchronousMotor</b> | fault-current contribution change | SC equivalent parameters for protection-aware studies |

**Contingency construction rule.** Each in-service instance of the above classes contributes one candidate contingency  $k$  if it can change served load or restoration feasibility. Components without explicit reliability parameters can still be included with user-defined default  $(\lambda_k, \tau_{k,\text{rep}})$ .

**Priority and customer impact mapping.** Customer and priority fields map directly to interruption metrics:

$$N_i \leftarrow \text{Load}::\text{n\_customers} \text{ (or bus fallback),} \quad (237)$$

$$w_i \leftarrow f(\text{Load}::\text{priority}, \text{Bus}::\text{importance}). \quad (238)$$

This enables both IEEE indices (SAIFI/SAIDI/ASAI) and weighted resilience metrics (WEENS).

## 19 Distribution Resilience Assessment with Mobile Energy Storage Systems

### 19.1 Problem Form and Runtime Scope

This chapter documents the resilience-restoration layer for multi-period outage recovery with mobile energy storage systems. The mathematical model couples feeder availability, reconfiguration, served-load restoration, and inter-temporal MESS movement and discharge decisions over a discrete restoration horizon.

### 19.2 Code Map

The current implementation is concentrated in:

- `src/analysis/resilience_assessment.cpp` for restoration simulation and resilience KPI aggregation,
- `src/analysis/topology_analysis.cpp` for feeder-switching support,
- distribution and reliability-side result structures used by the GUI and regression tests.

### 19.3 Pseudocode Map

The detailed algorithm chapter does not yet include a dedicated MESS-resilience pseudocode block. The closest reusable workflow references are the ONR and distribution-PF algorithms (A13 and A14) in Section 21. This chapter therefore remains the main code-aligned statement of the restoration model itself.

### 19.4 Current Status

The C++ port now has two layers. The default runtime path remains the practical heuristic restoration simulator used by the GUI and validation demo, while a new strict multi-period

MIP/LinDistFlow restoration skeleton has been added as the replacement path for unexpected-fault studies. The current C++ scope is therefore broader than before, but still narrower than the original full stochastic two-stage decomposition design.

This section documents the resilience-restoration model used for multi-period distribution-service recovery with mobile energy storage systems (MESS). The formulation is aligned with the original MATLAB package in `Multi-stageRestoration/` and with the current C++ implementation in `resilience_assessment.cpp`.

The original MATLAB workflow is a two-stage stochastic mixed-integer recourse model with Benders-style decomposition. The current C++ port implements the core operational layer needed for practical resilience studies in this codebase:

1. time-indexed feeder outage and repair simulation,
2. radial reconfiguration through healthy tie-line closure,
3. island-level load restoration prioritization,
4. inter-temporal MESS routing and discharge scheduling,
5. resilience KPI aggregation for GUI and regression tests.

In addition, the latest C++ implementation now contains a strict restoration builder that formulates a single multi-period MIP with binary branch-status, energized-bus, and root-selection variables; active-power LinDistFlow voltage envelopes; fixed-storage inter-temporal energy balance; and time-space MESS movement arcs. This strict model is currently a first skeleton, intended to replace the heuristic unexpected-fault restoration path incrementally.

## 19.5 Sets and Indices

$$\mathcal{N} = \text{distribution buses}, \quad (239)$$

$$\mathcal{E} = \text{AC feeder branches}, \quad (240)$$

$$\mathcal{E}^{\text{tie}} \subseteq \mathcal{E} \quad \text{normally open tie branches}, \quad (241)$$

$$\mathcal{G} = \text{dispatchable local generators}, \quad (242)$$

$$\mathcal{R} = \text{static and renewable distributed resources}, \quad (243)$$

$$\mathcal{S} = \text{fixed stationary storage units}, \quad (244)$$

$$\mathcal{M} = \text{mobile energy storage systems (MESS)}, \quad (245)$$

$$\mathcal{L} = \text{loads}, \quad (246)$$

$$\mathcal{T} = \{0, 1, \dots, T - 1\} \quad \text{discrete restoration periods}, \quad (247)$$

$$\mathcal{F} = \text{faulted feeder branches}. \quad (248)$$

Each load  $\ell \in \mathcal{L}$  is attached to a bus  $n(\ell) \in \mathcal{N}$ , with active demand  $P_{\ell,t}^d$ , priority weight  $w_\ell$ , and customer count  $N_\ell^{\text{cust}}$ .

## 19.6 State Variables

For each time period  $t \in \mathcal{T}$ :

$$\alpha_{e,t} \in \{0, 1\} \quad \text{branch energization state,} \quad (249)$$

$$P_{\ell,t}^{\text{serv}} \geq 0 \quad \text{served load,} \quad (250)$$

$$P_{\ell,t}^{\text{shed}} \geq 0 \quad \text{unsupplied load,} \quad (251)$$

$$E_{m,t}^{\text{mess}} \geq 0 \quad \text{MESS stored energy,} \quad (252)$$

$$P_{m,t}^{\text{mess}} \geq 0 \quad \text{MESS discharge power,} \quad (253)$$

$$\chi_{m,n,t}^{\text{stay}} \in \{0, 1\} \quad \text{MESS located at bus } n, \quad (254)$$

$$\chi_{m,a,t}^{\text{move}} \in \{0, 1\} \quad \text{MESS traverses transport arc } a. \quad (255)$$

The C++ implementation reports these decisions in aggregated form through the per-step result objects:

$$P_t^{\text{demand}}, P_t^{\text{served}}, P_t^{\text{shed}}, R_t, \mathcal{E}_t^{\text{open}}, \mathcal{M}_t^{\text{dispatch}}. \quad (256)$$

## 19.7 Fault and Repair Process

A faulted branch  $f \in \mathcal{F}$  is unavailable during its outage window:

$$\alpha_{f,t} = 0, \quad \forall t \in \left[ t_f^{\text{out}}, t_f^{\text{out}} + \tau_f^{\text{rep}} \right). \quad (257)$$

Outside this interval, the branch may either return to its original state or be closed by reconfiguration if it is a healthy tie branch. In the deterministic C++ implementation, repair times are explicit inputs and branch failures are simulated chronologically over the horizon.

## 19.8 Load Restoration and Prioritization

For every load  $\ell$  and time  $t$ :

$$P_{\ell,t}^{\text{serv}} + P_{\ell,t}^{\text{shed}} = P_{\ell,t}^d, \quad 0 \leq P_{\ell,t}^{\text{serv}} \leq P_{\ell,t}^d. \quad (258)$$

Within each island, available supply is allocated to higher-priority loads first. If  $\pi(1), \pi(2), \dots$  denotes the ordering of island loads by descending  $w_\ell$ , then the restoration rule is

$$P_{\pi(k),t}^{\text{serv}} = \min \left( P_{\pi(k),t}^d, P_t^{\text{avail}} - \sum_{j=1}^{k-1} P_{\pi(j),t}^{\text{serv}} \right)_+. \quad (259)$$

The weighted unserved load is

$$P_t^{\text{wshed}} = \sum_{\ell \in \mathcal{L}} w_\ell P_{\ell,t}^{\text{shed}}. \quad (260)$$

## 19.9 Island Supply Model

For each electrical island  $\mathcal{I} \subseteq \mathcal{N}$  at time  $t$ , the available active supply is approximated as

$$P_{\mathcal{I},t}^{\text{avail}} = P_{\mathcal{I},t}^{\text{grid}} + \sum_{g \in \mathcal{G}(\mathcal{I})} P_g^{\text{max}} + \sum_{r \in \mathcal{R}(\mathcal{I})} P_{r,t} + \sum_{s \in \mathcal{S}(\mathcal{I})} P_{s,t}^{\text{dis}} + \sum_{m \in \mathcal{M}(\mathcal{I})} P_{m,t}^{\text{mess}}. \quad (261)$$

If an island contains a slack or external-grid source, then  $P_{\mathcal{I},t}^{\text{grid}}$  is treated as nonbinding and the island can serve its full demand. Otherwise restoration is limited by local DER and storage capability.

## 19.10 MESS Energy Dynamics

For each mobile unit  $m \in \mathcal{M}$ :

$$E_m^{\min} \leq E_{m,t}^{\text{mess}} \leq E_m^{\max}, \quad (262)$$

and during a discharge period,

$$E_{m,t+1}^{\text{mess}} = E_{m,t}^{\text{mess}} - \frac{\Delta t}{\eta_m^{\text{dis}}} P_{m,t}^{\text{mess}}. \quad (263)$$

Travel consumes energy according to the traversed road distance:

$$E_{m,t+}^{\text{mess}} = E_{m,t}^{\text{mess}} - \rho_m^{\text{travel}} d_{m,t}, \quad (264)$$

where  $\rho_m^{\text{travel}}$  is the transport energy cost in MWh/km and  $d_{m,t}$  is the routing distance during the dispatch decision.

The discharge capability is limited by both power and stored energy:

$$0 \leq P_{m,t}^{\text{mess}} \leq \min \left( P_m^{\max}, \frac{\eta_m^{\text{dis}}}{\Delta t} (E_{m,t}^{\text{mess}} - E_m^{\min}) \right). \quad (265)$$

## 19.11 Transportation-Network Coupling

The original MATLAB model defines a time-space transportation network with binary path-flow variables. The corresponding canonical constraints are:

$$\sum_{a \in \delta^+(v)} \chi_{m,a,t}^{\text{move}} - \sum_{a \in \delta^-(v)} \chi_{m,a,t}^{\text{move}} = b_{m,v,t}, \quad (266)$$

where  $b_{m,v,t}$  is the source/sink balance induced by the current MESS location and the desired destination.

In the current C++ implementation, this is approximated by a shortest-path assignment over a transport graph. If no explicit road network is provided, the AC feeder graph is used as a transportation proxy with branch length as edge weight. This keeps the data contract lightweight while preserving the essential spatial coupling between restoration need and MESS mobility.

## 19.12 Radial Reconfiguration

Healthy open ties may be closed to reconnect unsupplied islands, subject to a radial forest policy. In the simplified implementation, branch closures are accepted only when they connect two different components. This enforces cycle-free restoration and mimics the spanning-tree intent of the MATLAB master problem:

$$\alpha_{e,t} = 1 \implies e \text{ joins two distinct connected components at time } t. \quad (267)$$

This heuristic is consistent with single-commodity radial restoration logic and remains the default GUI path. In parallel, the code base now also contains a strict mixed-integer LinDistFlow restoration skeleton, so future unexpected- fault studies no longer need to remain confined to heuristic topology updates.

### 19.13 Objective Function

The stochastic MATLAB model minimizes expected restoration cost across scenarios. Its operational core is represented by

$$\min \sum_{t \in \mathcal{T}} \left[ C^{\text{shed}} \sum_{\ell \in \mathcal{L}} P_{\ell,t}^{\text{shed}} + C^{\text{sw}} \sum_{e \in \mathcal{E}^{\text{tie}}} |\alpha_{e,t} - \alpha_{e,t-1}| + C^{\text{travel}} \sum_{m \in \mathcal{M}} d_{m,t} \right]. \quad (268)$$

The implemented C++ routine uses the same operational priorities, but solves them sequentially by:

1. enforcing feeder outages and completed repairs,
2. greedily reconnecting unsourced islands through healthy ties,
3. allocating local generation and storage to maximize served load,
4. dispatching available MESS to the most critical unsupplied islands.

### 19.14 Resilience Metrics

The primary resilience outputs are:

$$E^{\text{demand}} = \sum_{t \in \mathcal{T}} P_t^{\text{demand}} \Delta t, \quad (269)$$

$$E^{\text{served}} = \sum_{t \in \mathcal{T}} P_t^{\text{served}} \Delta t, \quad (270)$$

$$E^{\text{uns}} = \sum_{t \in \mathcal{T}} P_t^{\text{shed}} \Delta t, \quad (271)$$

$$R_t = \frac{P_t^{\text{served}}}{P_t^{\text{demand}}}, \quad (272)$$

$$\text{RI} = \frac{E^{\text{served}}}{E^{\text{demand}}}, \quad (273)$$

$$E^{\text{MESS}} = \sum_{t \in \mathcal{T}} \sum_{m \in \mathcal{M}} P_{m,t}^{\text{mess}} \Delta t. \quad (274)$$

The GUI reports these quantities as:

- total served energy,
- total shed energy,
- resilience index,
- average and final restoration ratio,
- peak shed power,
- MESS-delivered energy,
- MESS travel distance,
- switching actions and repaired-fault count.

### 19.15 Implementation Notes

The present C++ port is intentionally conservative in scope:

1. it runs directly on the canonical `HybridPowerSystem` data model,
2. it supports arbitrary user-specified feeder faults and repair windows,
3. it uses explicit mobile-storage state tracking over time,
4. it returns lightweight JSON-friendly outputs for the GUI,
5. it remains extensible toward the full stochastic Benders architecture in the original MATLAB package.

This design is appropriate for interactive resilience screening and regression validation, while preserving a clean upgrade path toward a full mixed-integer recourse implementation.

### 19.16 Validation Snapshot

The new regression test `tests/test_resilience_assessment.cpp` uses the `case33mg_acdc` feeder with a single automatically selected feeder fault, a six-hour repair window, and an eight-hour restoration horizon. To isolate the marginal contribution of MESS, stationary DER is removed from the test scenario and one mobile unit is placed on the tail bus.

The measured results are:

$$E^{\text{demand}} = 29.72 \text{ MWh}, \quad (275)$$

$$E^{\text{shed,base}} = 0.48 \text{ MWh}, \quad (276)$$

$$E^{\text{shed,MESS}} = 0.00 \text{ MWh}, \quad (277)$$

$$RI^{\text{base}} = 0.983849, \quad (278)$$

$$RI^{\text{MESS}} = 1.000000, \quad (279)$$

$$E^{\text{MESS}} = 0.48 \text{ MWh}. \quad (280)$$

In other words, the mobile storage unit fully eliminates the residual outage energy left by the feeder fault in this validation case. This is the expected behavior for a tail-island restoration event and provides a useful sanity check for both the operational logic and the GUI integration.

## 20 Carbon Flow Analysis

### 20.1 Problem Form and Runtime Scope

This chapter documents the post-power-flow carbon tracing layer. It does not solve a new electrical operating point. Instead, it maps a converged canonical hybrid network solution into directed carrier flows, source intensities, and bus-level carbon intensity metrics.

### 20.2 Code Map

The current implementation is concentrated in:

- `src/analysis/carbon_analysis.cpp` for graph construction, tracing, and matrix-based intensity calculations,
- canonical PF result structures from the power-flow layer for branch, converter, and storage transfers,
- shared component emission-factor fields from the information model.

## 20.3 Pseudocode Map

The code-aligned workflow reference is Algorithm A15 in Section 21. This chapter then expands the algebra used in that workflow.

## 20.4 Current Status

The carbon-analysis route is implemented as a reporting layer over the canonical hybrid network. The main remaining gaps are source normalization for some higher-level hybrid overlays and richer explicit reporting for auxiliary participation paths.

## 20.5 Purpose and Required Inputs

The carbon-analysis module is a post-power-flow reporting layer. It does not solve a new electrical operating point. Instead, it consumes

- a `HybridPowerSystem`,
- a converged `PowerFlowResult`,
- component emission-factor data already stored in the information model,

and produces source-to-load carbon tracing reports and matrix-based bus carbon intensities on the canonical hybrid AC/DC network.

The public entry points are

- `compute_carbon_analysis(sys, pf_result, opt)`,
- `compute_carbon_analysis(sys, pf_opt, ca_opt)`,

where the second overload simply runs the main PF solver first and then invokes the carbon-analysis routine.

## 20.6 Implemented Scope and Current Boundary

The implementation now operates on the same canonical projected network used by the steady-state solver. In particular, the carbon model includes

- canonical AC buses and AC branches,
- canonical DC buses and DC branches,
- VSC AC/DC converters through `PowerFlowResult::vsc_transfers`,
- DCDC converters through `PowerFlowResult::dcdc_transfers`,
- energy routers through `PowerFlowResult::er_port_transfers`,
- AC and DC storage as explicit discharge-side carbon sources and charge-side traced sinks.

The current boundary is still narrower than the full information-model catalog:

- explicit carbon-source normalization now includes `Generator`, `StaticGenerator` on AC and DC, `RenewableGen`, `PVSystem`, and discharging `Storage`,
- charging `Storage` is represented as an explicit traced sink, but `MobileStorage` still participates only through auxiliary slack-balance bookkeeping,
- VPP and microgrid exchange are still treated as auxiliary net-injection terms in slack reconstruction; they are not yet emitted as standalone carbon sources,
- if PF did not converge or both AC and DC bus sets are empty, the routine returns an empty result object.

## 20.7 Canonical Hybrid Graph

Let the projected AC and DC bus sets be

$$\mathcal{N}^{ac}, \quad \mathcal{N}^{dc}, \quad \mathcal{N} = \mathcal{N}^{ac} \cup \mathcal{N}^{dc}. \quad (281)$$

The carbon module builds a directed hybrid carrier graph on  $\mathcal{N}$ . Each directed carrier edge is represented as

$$e = (u, v, F_e, R_e, L_e), \quad (282)$$

where

- $u$  is the sending node,
- $v$  is the receiving node,
- $F_e$  is the active power entering the carrier at the sending side,
- $R_e$  is the active power received at the downstream side,
- $L_e = F_e - R_e$  is the carrier loss.

The carrier set is

$$\mathcal{E} = \mathcal{E}^{ac} \cup \mathcal{E}^{dc} \cup \mathcal{E}^{vsc} \cup \mathcal{E}^{dcdc} \cup \mathcal{E}^{er}. \quad (283)$$

## 20.8 Source and Load Normalization

**Unified source model.** Each explicit source is normalized to

$$s = (\text{id}_s, i_s, P_s, e_s), \quad (284)$$

where  $i_s \in \mathcal{N}$  is the attached hybrid node,  $P_s$  is the actual active injection in MW, and  $e_s$  is the emission factor in tCO<sub>2</sub>/MWh.

The implemented source families are

$$\mathcal{S} = \mathcal{G}^{sync} \cup \mathcal{G}^{static,ac} \cup \mathcal{G}^{static,dc} \cup \mathcal{G}^{ren} \cup \mathcal{G}^{pv} \cup \mathcal{G}^{stor,ac,dis} \cup \mathcal{G}^{stor,dc,dis}. \quad (285)$$

The source emission factors are read from

$$e_s \in \left\{ \begin{array}{l} \text{Generator::emission_factor_tco2_mwh,} \\ \text{StaticGenerator::co2\_emission\_rate,} \\ 0.0, \\ \text{Storage::soc\_carbon\_intensity\_tco2\_mwh} \end{array} \right\}, \quad (286)$$

so `RenewableGen` and `PVSystem` are currently treated as zero-emission injections.

**Storage as explicit source / sink.** For storage unit  $s$ , the PF sign convention is bus injection:

$$p_s > 0 \Rightarrow \text{discharge / inject}, \quad p_s < 0 \Rightarrow \text{charge / withdraw}. \quad (287)$$

The implemented carbon normalization is

$$p_s > 0 : \text{explicit source with } P_s = p_s, \quad e_s = w_s^{soc}, \quad (288)$$

$$p_s < 0 : \text{explicit sink with } P_s^{ch} = -p_s, \quad (289)$$

where

$$w_s^{soc} = \text{Storage::soc\_carbon\_intensity\_tco2\_mwh}. \quad (290)$$

Therefore discharge-side carbon leaves the storage at the current stored-energy carbon intensity, while charging-side carbon is traced from the upstream source mix and exported in `storage_carbon`.

**SOC carbon-intensity propagation rule.** For a multi-period storage interpretation with interval  $\Delta t$ , stored energy  $E_{s,t}$ , stored carbon inventory  $C_{s,t}$ , self-discharge  $\sigma_s$ , charge efficiency  $\eta_s^{ch}$ , discharge power  $P_{s,t}^{dis}$ , and charging power  $P_{s,t}^{ch}$ , the recommended update rule is

$$E_{s,t+1} = (1 - \sigma_s)E_{s,t} + \eta_s^{ch} P_{s,t}^{ch} \Delta t - P_{s,t}^{dis} \Delta t, \quad (291)$$

$$C_{s,t+1} = (1 - \sigma_s)C_{s,t} + w_{s,t}^{in} \eta_s^{ch} P_{s,t}^{ch} \Delta t - w_{s,t}^{soc} P_{s,t}^{dis} \Delta t, \quad (292)$$

$$w_{s,t+1}^{soc} = \frac{C_{s,t+1}}{E_{s,t+1}}. \quad (293)$$

The current C++ carbon-analysis module exports the instantaneous field `soc_carbon_intensity_tco2_mwh` and uses it directly during discharge; the dynamic update above is the governing rule for any future time-coupled simulation or optimization layer.

**Slack-source reconstruction.** Non-slack synchronous generators keep their scheduled `pg_mw`. The slack source is reconstructed from hybrid active-power balance:

$$P_{slack} = \max \left( \sum_{\ell \in \mathcal{L}^{ac,u} \cup \mathcal{L}^{dc,u} \cup \mathcal{L}^{stor,ch}} P_\ell + \sum_{e \in \mathcal{E}} L_e + \sum_{i \in \mathcal{N}^{ac}} G_i^{sh} - \sum_{s \in \mathcal{S}_{ns}} P_s - P_{net}^{aux}, 0 \right), \quad (294)$$

where  $G_i^{sh}$  is the active shunt demand on AC bus  $i$ ,  $\mathcal{S}_{ns}$  is the explicit non-slack source set, and  $P_{net}^{aux}$  is the residual net injection from auxiliary components not yet promoted to explicit carbon sources or sinks, currently mobile storage, VPP, and microgrid exchange terms.

**Unified load model.** The module constructs

$$\mathcal{L}^{ac,u}, \quad \mathcal{L}^{dc,u}, \quad \mathcal{L}^{stor,ch}, \quad (295)$$

by combining first-class load rows with synthetic bus-demand rows created from `ACBus::pd_mw` or `DCBus::pd_mw` when no explicit component row covers that bus, plus one charge-side sink row per in-service storage unit with  $p_s < 0$ . Each unified load is represented as

$$\ell = (\text{id}_\ell, i_\ell, P_\ell, Q_\ell). \quad (296)$$

## 20.9 Directed Carrier Construction

**AC branches.** For AC branch  $k = (i, j)$ , let the solved terminal injections be  $P_{f,k}$  and  $P_{t,k}$ . The branch loss is

$$L_k = P_{f,k} + P_{t,k}. \quad (297)$$

The directed-edge rule is

$$P_{f,k} > 0 : i \rightarrow j, \quad F_k = P_{f,k}, \quad R_k = \max(-P_{t,k}, 0), \quad (298)$$

$$P_{t,k} > 0 : j \rightarrow i, \quad F_k = P_{t,k}, \quad R_k = \max(-P_{f,k}, 0). \quad (299)$$

Zero-flow AC branches are omitted from the directed carrier graph.

**DC branches.** For DC branch  $k = (i, j)$  with resistance  $r_k$ , solved endpoint voltages  $V_i$  and  $V_j$ , and system base  $S_{base}$ , the implementation reconstructs the DC current and terminal powers as

$$I_k = \frac{V_i - V_j}{r_k}, \quad (300)$$

$$P_{i \rightarrow k} = S_{base} V_i I_k, \quad (301)$$

$$P_{j \rightarrow k} = -S_{base} V_j I_k. \quad (302)$$

If  $P_{i \rightarrow k} > 0$ , then the edge direction is  $i \rightarrow j$  with

$$F_k = P_{i \rightarrow k}, \quad R_k = \max(-P_{j \rightarrow k}, 0), \quad L_k = P_{i \rightarrow k} + P_{j \rightarrow k}. \quad (303)$$

The opposite direction is used when  $P_{j \rightarrow k} > 0$ .

**VSC converters.** The PF sign convention is bus injection on both sides:

$$p_c^{ac} + p_c^{dc} + L_c = 0. \quad (304)$$

If the AC side withdraws power from the AC bus,

$$p_c^{ac} < 0, \quad p_c^{dc} > 0, \quad (305)$$

then the converter transports carbon from AC to DC with

$$F_c = -p_c^{ac}, \quad R_c = p_c^{dc}, \quad L_c = -(p_c^{ac} + p_c^{dc}). \quad (306)$$

If instead  $p_c^{dc} < 0$  and  $p_c^{ac} > 0$ , the same rule is applied in DC-to-AC direction.

**DCDC converters.** For DCDC transfer  $(p_d^{in}, p_d^{out}, L_d)$ , the bus injections are

$$P_{bus\_in}^{inj} = -p_d^{in}, \quad P_{bus\_out}^{inj} = p_d^{out}, \quad L_d = p_d^{in} - p_d^{out}. \quad (307)$$

When  $p_d^{in} > 0$  and  $p_d^{out} > 0$ , the carbon carrier direction is  $bus\_in \rightarrow bus\_out$  with

$$F_d = p_d^{in}, \quad R_d = p_d^{out}. \quad (308)$$

If the signs reverse, the directed carrier is flipped accordingly.

**Energy routers.** Each router port uses the same bus-injection convention:

$$p_{r,p} < 0 \Rightarrow \text{bus} \rightarrow \text{router input}, \quad p_{r,p} > 0 \Rightarrow \text{router} \rightarrow \text{bus output}. \quad (309)$$

For router  $r$ , define the active input and output port sets

$$\mathcal{P}_r^{in} = \{p \mid p_{r,p} < 0\}, \quad \mathcal{P}_r^{out} = \{p \mid p_{r,p} > 0\}, \quad (310)$$

and the aggregate powers

$$P_r^{in} = \sum_{p \in \mathcal{P}_r^{in}} (-p_{r,p}), \quad P_r^{out} = \sum_{p \in \mathcal{P}_r^{out}} p_{r,p}, \quad L_r = \max(P_r^{in} - P_r^{out}, 0). \quad (311)$$

The carbon module reconstructs a proportional internal carrier graph between all input/output port pairs. For input port  $i$  and output port  $o$ ,

$$\beta_o = \frac{p_{r,o}}{P_r^{out}}, \quad (312)$$

$$F_{io} = (-p_{r,i})\beta_o, \quad (313)$$

$$R_{io} = F_{io} \cdot \min\left(\frac{P_r^{out}}{P_r^{in}}, 1\right), \quad (314)$$

$$L_{io} = F_{io} - R_{io}. \quad (315)$$

This construction preserves the solved port-sign convention, allocates router losses from actual port mismatch, and avoids inventing additional loss from the metadata field `loss_percent`.

## 20.10 Proportional Tracing on the Hybrid Graph

**Node inflow composition.** For each hybrid node  $n \in \mathcal{N}$ , the total inflow used for tracing is

$$I_n = \sum_{s \in \mathcal{S}_n} P_s + \sum_{e \in \delta^-(n)} R_e, \quad (316)$$

where  $\mathcal{S}_n$  is the set of explicit sources attached to node  $n$ .

**Load tracing.** For each load  $\ell$  attached to node  $n$ , the initial traced fraction is

$$\phi_{\ell,n}^{(0)} = \frac{P_\ell}{I_n}. \quad (317)$$

During the upstream breadth-first traversal, every visited node  $m$  allocates its local explicit sources by

$$C_{\ell,s} += \phi_{\ell,m} P_s, \quad s \in \mathcal{S}_m, \quad (318)$$

and each incoming carrier edge  $e : u \rightarrow m$  propagates the traced fraction in proportion to the received-flow share

$$\phi_{\ell,u} \leftarrow \phi_{\ell,m} \frac{R_e}{I_m}. \quad (319)$$

The resulting source-attribution map  $C_{\ell,s}$  is exported as `LoadCarbonResult::generator_supply_mw` for both AC and DC load reports.

**Carrier-loss tracing.** For each lossy carrier edge  $e$ , the implementation traces the source composition at the sending node and allocates the loss proportionally:

$$\Lambda_{e,s} \approx \gamma_{e,s} L_e, \quad (320)$$

where  $\gamma_{e,s}$  is the traced source share at the carrier input. These allocations populate

- `branch_carbon` for AC branches,
- `dc_branch_carbon` for DC branches,
- `vsc_carbon` for VSC losses,
- `dcdc_carbon` for DCDC losses,
- `energy_router_carbon` for router internal losses,

as well as the aggregated `generator_loss_allocation` map.

**Tracing-derived carbon intensity.** For any traced demand or lossy carrier item  $x$ , the weighted-average carbon intensity is

$$w_x^{trace} = \frac{\sum_s e_s C_{x,s}}{\sum_s C_{x,s}}, \quad (321)$$

and the reported emissions are

$$E_x^{trace} = w_x^{trace} P_x, \quad (322)$$

with  $P_x$  equal to demand MW for loads or loss MW for carriers.

## 20.11 Matrix-Based Hybrid Bus Carbon Intensity

The second path solves one carbon-intensity value per hybrid node. Define

$$w = \begin{bmatrix} w^{ac} \\ w^{dc} \end{bmatrix}, \quad (323)$$

and let  $\alpha \in [0, 1]$  control how carrier losses are split between the sending and receiving sides.

**Node-level bookkeeping.** For each directed carrier edge  $e : u \rightarrow v$ , the implementation applies

$$P_{in,v} += F_e - \alpha L_e, \quad (324)$$

$$P_{out,u} += F_e, \quad (325)$$

$$P_{out,v} += (1 - \alpha) L_e. \quad (326)$$

Explicit source injections are added to  $P_{in}$ , while unified AC/DC loads and AC shunt conductance demand are added to  $P_{out}$ .

**Linear system.** The upstream transfer matrix is

$$A_{in}(i, j) = \begin{cases} F_e - \alpha L_e, & \text{if there is a directed carrier } j \rightarrow i, \\ 0, & \text{otherwise,} \end{cases} \quad (327)$$

and the hybrid system matrix is

$$A = \text{diag}(P_{out}) - A_{in}, \quad b_i = \sum_{s \in S_i} e_s P_s. \quad (328)$$

The node carbon-intensity vector is obtained from

$$Aw = b. \quad (329)$$

**End-node handling.** When a node has neither local generation nor local load and its system-matrix row is nearly singular, the implementation replaces that row with an averaging constraint

$$w_i = \frac{1}{|\mathcal{N}(i)|} \sum_{j \in \mathcal{N}(i)} w_j, \quad (330)$$

so electrically passive end-nodes inherit the intensity of adjacent hybrid nodes. The current linear solver then converts the repaired sparse system to a dense matrix and solves it with column-pivoted Householder QR. The reported `matrix_residual` is

$$\|Aw - b\|_2. \quad (331)$$

## 20.12 Emissions Summaries

The tracing path reports a generation-vs-load-vs-loss balance:

$$E_{gen} = \sum_{s \in \mathcal{S}} e_s P_s, \quad (332)$$

$$E_{load}^{trace} = \sum_{\ell \in \mathcal{L}^{ac,u}} E_{\ell}^{trace} + \sum_{\ell \in \mathcal{L}^{dc,u}} E_{\ell}^{trace} + \sum_{\ell \in \mathcal{L}^{stor,ch}} E_{\ell}^{trace}, \quad (333)$$

$$E_{loss}^{trace} = \sum_{e \in \mathcal{E}} E_e^{trace}. \quad (334)$$

The balance error is

$$\Delta E^{trace} = E_{gen} - E_{load}^{trace} - E_{loss}^{trace}, \quad (335)$$

and the verification flag `tracing_verified` checks whether the relative balance error is below `verification_tol`.

The matrix path reports an analogous summary using solved hybrid node intensities:

$$E_{load}^{mat} = \sum_{\ell \in \mathcal{L}^{ac,u} \cup \mathcal{L}^{dc,u} \cup \mathcal{L}^{stor,ch}} w_{i(\ell)} P_{\ell}, \quad (336)$$

$$E_{loss}^{mat} = \sum_{e=(u,v) \in \mathcal{E}} w_u L_e. \quad (337)$$

The matrix loss summary therefore uses the sending-node intensity of each carrier rather than the average of the two terminal nodes.

### 20.13 Result Artifacts

The dedicated carbon result object now contains

- `load_carbon` and `dc_load_carbon`: per-load tracing allocations and carbon intensity,
- `branch_carbon` and `dc_branch_carbon`: per-branch loss allocations and carbon intensity,
- `vsc_carbon` and `dc_dc_carbon`: converter-loss allocations and hybrid transfer direction,
- `storage_carbon`: per-storage charge/discharge carbon summary, including traced charging intensity and explicit discharge-side SOC intensity,
- `energy_router_carbon`: per-router internal-loss carbon allocation reconstructed from port transfers,
- `bus_carbon` and `dc_bus_carbon`: matrix-based AC/DC node carbon intensities,
- `generator_loss_allocation`: aggregated traced loss contribution by source id,
- `tracing_summary` and `matrix_summary`,
- `tracing_verified`, `matrix_solved`, and `matrix_residual`.

The field-level JSON / workbook contracts for these artifacts are documented in the I/O-contract and output-dictionary sections of this notebook; this chapter focuses on the analytical model and the meaning of the reported quantities.

### 20.14 Sankey Visualization of Carbon/SO<sub>2</sub> Source-Sink Relations

For engineering review, the repository includes a lightweight post-processing workflow to render source-to-sink Sankey charts:

1. run PF + carbon analysis and export a carbon-result JSON using `run_carbon_analysis_cli`,
2. render Sankey HTML using `tools/plot_emission_sankey.py`.

The standard commands are:

```
cmake --build build --target run_carbon_analysis_cli -j4
./build/run_carbon_analysis_cli \
 --input-system-json <system.json> \
 --output-carbon-json <carbon_result.json>
```

```
python3 tools/plot_emission_sankey.py \
 --system-json <system.json> \
 --carbon-json <carbon_result.json> \
 --output-html <emission_sankey.html> \
 --aggregate-view bus \
 --top-n-source-nodes 20 \
 --top-n-sink-nodes 30 \
 --top-n-links 200 \
 --load-granularity bus \
 --loss-granularity type
```

By default, the generated HTML contains two Sankey graphs:

- carbon flow (tCO<sub>2</sub>/h),
- sulfur-dioxide flow (kg/h).

The SO<sub>2</sub> graph currently reads source factors from `Generator::so2_factor_kg_mwh`; non-generator source families are treated as zero-SO<sub>2</sub> unless optional extension keys are provided in the input JSON.

## 21 Algorithm Flowcharts and Solver Workflows

### 21.1 Canonical Preprocessing Workflow

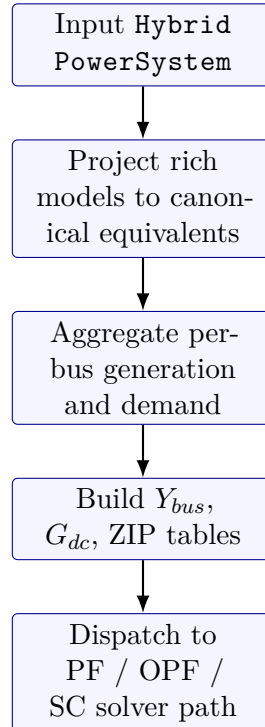


Figure 5: Shared preprocessing path used by the main PF, native OPF, parity OPF, and SC modules.

## 21.2 Hybrid Newton Power Flow

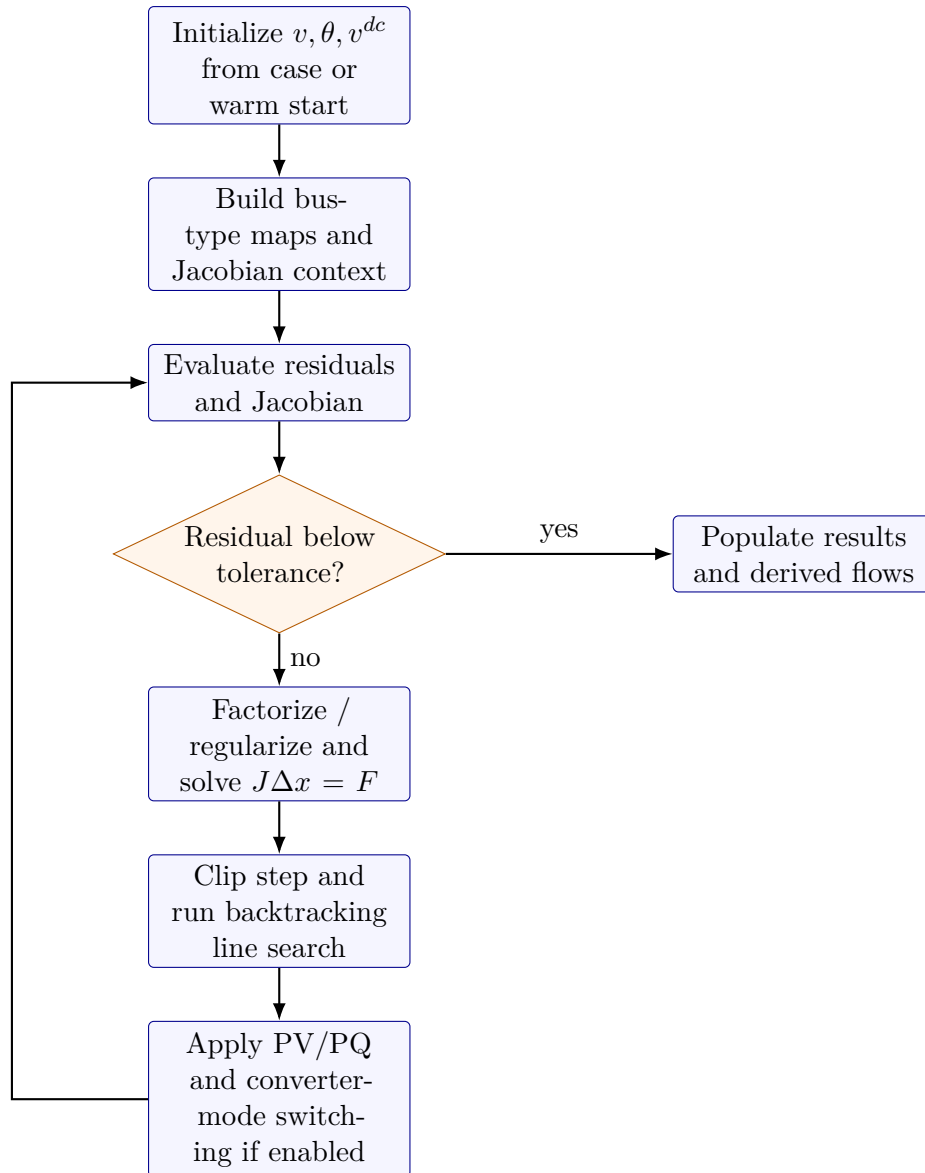


Figure 6: Main Newton-Raphson workflow for hybrid AC/DC power flow.

### 21.3 AC OPF Path Selection

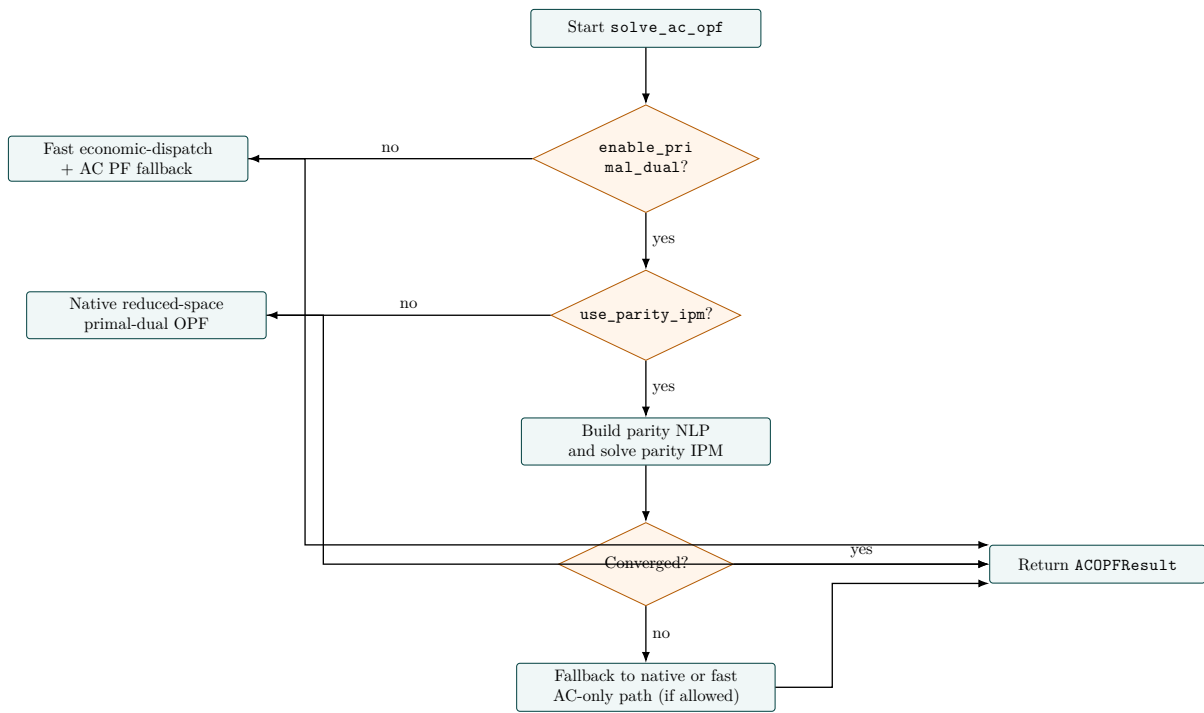


Figure 7: Current top-level solver-selection logic for AC OPF.

## 21.4 Hybrid Carbon-Analysis Workflow

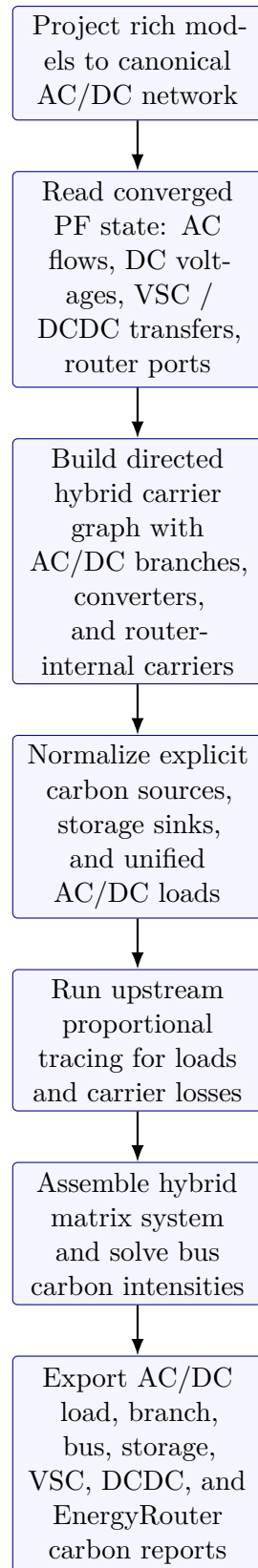


Figure 8: Current hybrid carbon-analysis workflow.

## 21.5 Distribution PF Workflow

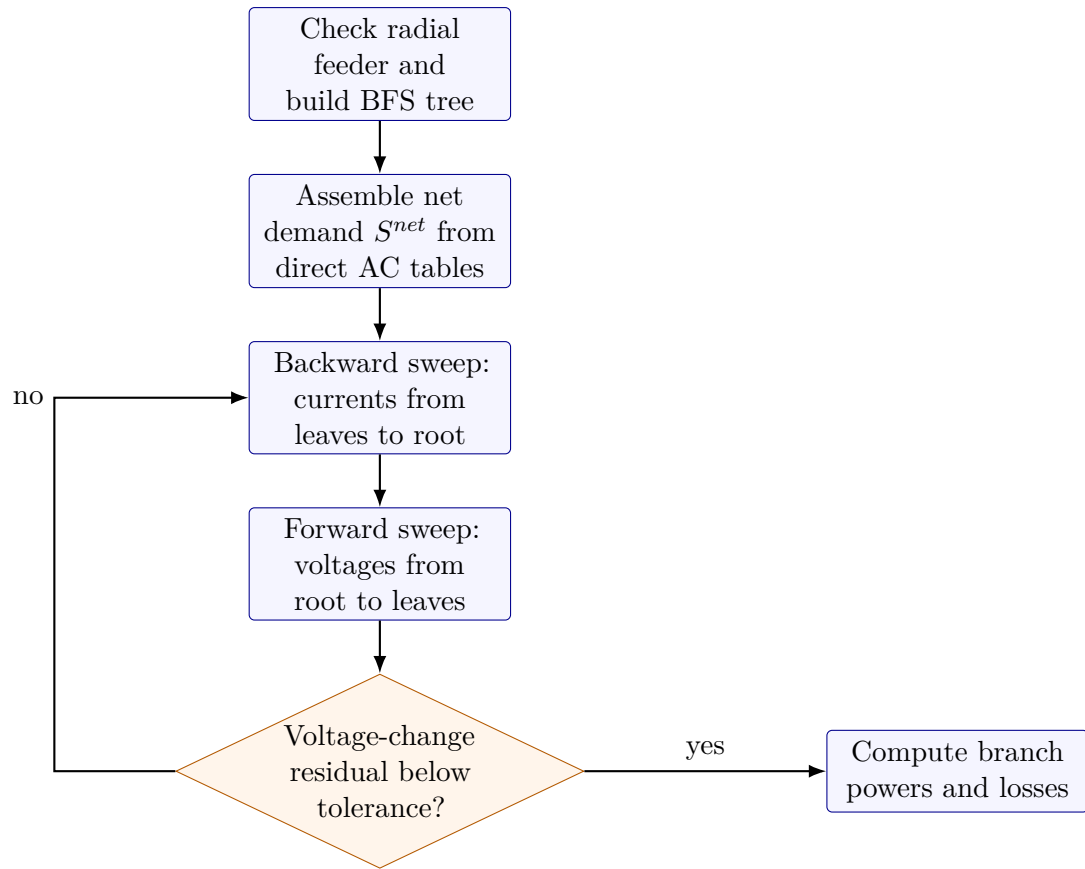


Figure 9: Backward-forward sweep workflow for radial distribution PF.

## 21.6 Short-Circuit Workflow

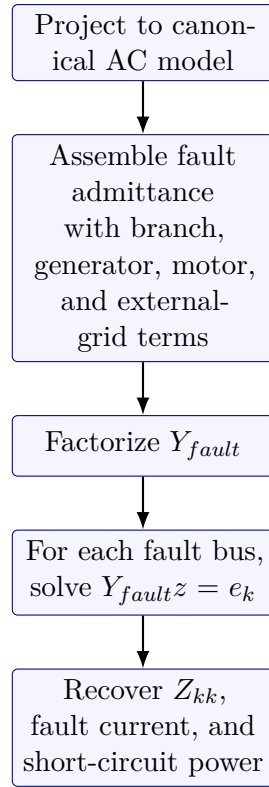


Figure 10: Primary short-circuit computation path in the current module.

## 21.7 ONR Workflow

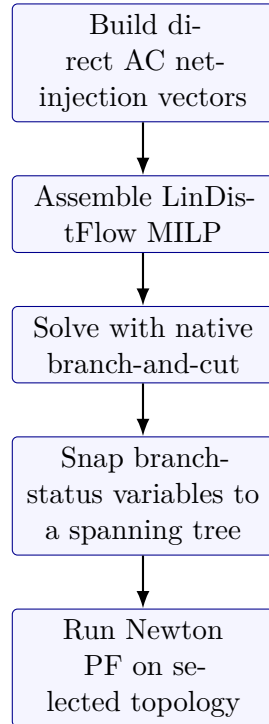


Figure 11: Optimal network reconfiguration workflow.

## 21.8 Solver-Family Summary

| Workflow                  | Numerical core                                   | Main reason to use it                                                                                                                                          |
|---------------------------|--------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Hybrid Newton PF          | sparse Newton + line search + regularization     | highest-fidelity steady-state solve in the current module.                                                                                                     |
| DC-only PF                | dense Newton on DC voltages                      | fast solution for purely resistive DC subnetworks.                                                                                                             |
| FDPF                      | sparse decoupled angle/voltage solves            | faster AC approximation when converter coupling is irrelevant.                                                                                                 |
| Adaptive PF               | island detection + per-island Newton solves      | robust handling of disconnected or partially energized systems.                                                                                                |
| Distributed slack         | post-PF mismatch redistribution                  | studies with multi-source slack participation.                                                                                                                 |
| Native AC OPF             | custom KKT system                                | detailed nonlinear dispatch with direct generator/converter variables.                                                                                         |
| Parity OPF                | parity-form NLP + dedicated IPM                  | full-space formulation with generator, VSC, DCDC, load-shedding, and DER decision variables; analytical Jacobian and Hessian including $I^2R$ DCDC loss terms. |
| DC OPF                    | LP/QP on $B-\theta$ network                      | fast linearized economic dispatch with LMP extraction; supports piecewise-linear and quadratic cost models.                                                    |
| Reactive power opt. (RPO) | MINLP with 4-phase OA                            | discrete tap/shunt optimization with AC OPF as inner NLP.                                                                                                      |
| LCQP solver               | Mehrotra predictor-corrector IPM                 | native convex QP/LP solver used as DC OPF backbone and RPO sub-solver.                                                                                         |
| Dual simplex              | two-phase simplex with basis caching             | LP for DC OPF and branch-and-cut MILP sub-problems; warm-start support.                                                                                        |
| Hybrid carbon analysis    | graph tracing + dense QR on hybrid carrier graph | post-PF carbon propagation across AC branches, DC branches, storage charge/discharge states, VSC links, DCDC links, and EnergyRouter carriers.                 |
| BFS DPF                   | tree-based current/voltage sweep                 | radial feeder studies where a full Jacobian solve is unnecessary.                                                                                              |
| ONR branch-and-cut        | MILP B&C with MIR cuts                           | topology optimization over switch states.                                                                                                                      |
| Z-bus SC                  | sparse factorization of fault admittance         | per-bus fault-level studies on the canonical AC model.                                                                                                         |
| Annual production sim     | hierarchical L0/L2/L3 decomposition              | year-ahead scheduling with UC + OPF/PF replay.                                                                                                                 |
| Lifecycle simulation      | multi-year degradation + stratified carbon       | techno-economic planning with battery replacement and error bounds.                                                                                            |

## 21.9 Code-Aligned Pseudocode (All Primary Algorithms)

### Algorithm A0: Runtime API Dispatch

```
1: procedure RUN_MODULE(sys, task, opt)
2: if task = "power_flow" then
3: return solve_power_flow(sys, opt.pf)
4: else if task = "dc_power_flow" then
5: return solve_dc_power_flow(sys, opt.pf)
6: else if task = "power_flow_adaptive" then
7: return solve_power_flow_adaptive(sys, opt.pf)
8: else if task = "opf" then
9: return solve_ac_opf(sys, opt.opf)
10: else if task = "short_circuit" then
11: return compute_short_circuit(sys, opt.sc)
12: else if task = "network_reconfiguration" then
13: return solve_optimal_reconfiguration(sys, opt.onr)
14: else if task = "distribution_pf" then
15: return solve_distribution_pf(sys, opt.dpf)
16: else if task = "carbon_analysis" then
17: pf $\leftarrow$ solve_power_flow(sys, opt.pf)
18: return compute_carbon_analysis(sys, pf, opt.carbon)
19: else
20: return UnsupportedTask
21: end if
22: end procedure
```

### Algorithm A1: Canonical Projection + SolverData Build

```
1: procedure MAKE_SOLVER_DATA(sys, loss_model)
2: projected $\leftarrow$ project_to_canonical_models(sys)
 $\triangleright$ Includes rich-model expansion and optional zero-Z bus merging
3: data $\leftarrow$ copy canonical tables from projected
4: data.loss_model $\leftarrow$ loss_model
5: Apply external-grid voltage/slack settings
6: Aggregate generation and demand
7: Build Y_{bus} , G_{dc} , and ZIP tables
8: return data
9: end procedure
```

### Algorithm A2: solve\_power\_flow (Top Level)

```
1: procedure SOLVE_POWER_FLOW(sys, opt)
2: data $\leftarrow$ cached SolverData for (sys, opt.loss_model)
3: Apply ZIP option weights to data
4: result $\leftarrow$ NewtonSolver.solve(data, opt)
5: Populate branch/converter/DCDC/router derived results
6: if data.bus_merge_map exists then
7: Unproject v_m, θ back to the original AC-bus count
8: end if
9: return result
10: end procedure
```

### Algorithm A3: Hybrid Newton PF Core

```

1: procedure NEWTONSOLVER.SOLVE(data, opt, init)
2: Initialize v, θ, v^{dc} , slack sets, and control modes
3: for outer = 1 to max_outer do
4: Build Jacobian context and sparse pattern caches
5: for iter = 1 to opt.max_iter do
6: Evaluate residual and Jacobian
7: if $\|F\|_{\infty} < \text{tol}$ then
8: break
9: end if
10: Solve $J\Delta x = F$ with regularization fallback
11: Clip step, then run backtracking line search
12: if no acceptable step then
13: mark failure; break
14: end if
15: Update state with accepted step
16: Apply PV→PQ and converter-mode switching rules
17: end for
18: if converged and PQ→PV restoration is enabled then
19: Restore eligible PV buses and continue outer loop
20: else
21: break
22: end if
23: end for
24: Export v, θ, v^{dc} , residual, and status
25: return result
26: end procedure

```

#### Algorithm A4: DC-Only Newton PF

```

1: procedure DCSOLVER.SOLVE(data, opt, init)
2: Initialize v^{dc} , detect DC slack, build reduced index set
3: for iter = 1 to max_iter do
4: Assemble $P^{\text{dc,spec}}$ from bus/load/storage/gen/VSC/DCDC/router
5: Compute $P^{\text{dc,calc}} = v^{\text{dc}} \odot (G_{\text{dc}} v^{\text{dc}})$
6: Form mismatch on non-slack rows
7: if $\|mismatch\|_{\infty} < \text{tol}$ then
8: converged; break
9: end if
10: Build dense reduced Jacobian and solve for Δv^{dc}
11: Apply positivity-safe line search
12: end for
13: return v^{dc} , residual, convergence flag
14: end procedure

```

#### Algorithm A5: Fast-Decoupled PF (FDPF)

```

1: procedure FDPFSOLVER.SOLVE(data, opt)
2: Build reduced B' and B'' ; factorize once
3: Initialize v, θ
4: for iter = 1 to max_iter do
5: Compute $(P^{\text{calc}}, Q^{\text{calc}})$ and ZIP-adjusted specs
6: Form P - and Q -mismatch vectors
7: if both mismatch norms $< \text{tol}$ then

```

```

8: converged; break
9: end if
10: Solve $B'\Delta\theta = -\Delta P$, $B''\Delta v = -\Delta Q$
11: Update non-slack θ and PQ-bus v
12: end for
13: return v, θ , residual, convergence flag
14: end procedure

```

#### Algorithm A6: Adaptive/Islanded PF

```

1: procedure ADAPTIVESOLVER.SOLVE(sys, opt)
2: $islands \leftarrow \text{detect_islands}(sys)$
3: if $islands$ is empty then
4: Solve full system once with Newton PF and return
5: end if
6: Initialize global output vectors
7: for each island do
8: if island has no generation then
9: Set island voltages to zero and continue
10: end if
11: Auto-select swing bus if required
12: Extract subsystem and run Newton PF (with one flat-start retry)
13: Unproject merged AC results if bus merging occurred
14: Map local results back to global vectors
15: end for
16: Aggregate iteration count and residual
17: return global adaptive result
18: end procedure

```

#### Algorithm A7: Distributed Slack

```

1: procedure SOLVE_DISTRIBUTED_SLACK($sys, slack_cfg, opt$)
2: Build/validate participation configuration
3: Solve base PF
4: if base PF not converged then
5: return base result
6: end if
7: Compute total active-power mismatch at participating buses
8: Allocate mismatch by participation factors and optional limits
9: Apply allocation and export adjusted injections/summary
10: return distributed-slack result
11: end procedure

```

#### Algorithm A8: AC OPF Top-Level Router

```

1: procedure SOLVE_AC_OPF(sys, opt)
2: Normalize default options
3: Detect hybrid case (DC buses or VSC converters present)
4: if hybrid case and $\text{enable_primal_dual}=\text{false}$ then
5: Force $\text{enable_primal_dual}=\text{true}$, $\text{use_parity_ipm}=\text{true}$
6: end if
7: if $\text{enable_primal_dual}=\text{true}$ and $\text{use_parity_ipm}=\text{true}$ then
8: Solve parity IPM path
9: if failed and allow_fallback then

```

```

10: retry native path
11: end if
12: return result
13: end if
14: if enable_primal_dual=false then
15: return fast dispatch + AC PF fallback path
16: end if
17: return native primal-dual OPF path
18: end procedure

```

**Algorithm A9: Fast Dispatch + AC PF Fallback (OPF)**

```

1: procedure TRY_DISPATCH_PF_FALLBACK(ac_sys, opt)
2: Try direct PF projection using current generator settings
3: if PF converged then
4: Back-calculate bus injections and split by generator limits
5: Compute objective and return converged OPF result
6: end if
7: Solve clipped economic dispatch (λ -search)
8: Rerun AC PF projection with dispatched P_g
9: if PF converged then
10: return converged result
11: end if
12: return not-converged result
13: end procedure

```

**Algorithm A10: Native Primal-Dual AC/DC OPF**

```

1: procedure SOLVE_NATIVE_PRIMAL_DUAL(sys, opt)
2: Build x -map, bounds, initial interior point, and sparse workspaces
3: for outer = 1 to max_outer do
4: for inner = 1 to max_inner do
5: Unpack $x \rightarrow (v, \theta, v^{\text{dc}}, P_g, Q_g, P^{\text{conv,ac}}, Q^{\text{conv,ac}}, P^{\text{conv,dc}})$
6: Evaluate equality constraints and Jacobian
7: Evaluate objective gradient and diagonal Hessian terms
8: Add bound barriers and nonlinear limit penalties
9: Solve regularized KKT step
10: Globalize by line search/restoration
11: if acceptance fails then
12: mark failure; break
13: end if
14: Update interior point and penalties
15: if primal/dual residuals below tolerance then
16: converged; break
17: end if
18: end for
19: if converged then
20: break
21: end if
22: Update barrier/penalty schedule
23: end for
24: if failed and fallback allowed then
25: call fast dispatch fallback

```

```

26: end if
27: return OPF result
28: end procedure

```

**Algorithm A11: Parity NLP Build + Primal-Dual IPM**

```

1: procedure SOLVE_WITH_PARITY_IPM(sys, opt)
2: Build full-space parity problem (AC, DC, converters, DER, shedding)
3: Build bounds/scaling and initial point
4: Initialize slacks/multipliers ($z > 0, \mu > 0$)
5: for $iter = 1$ to max_iter do
6: Form condensed KKT (Mehrotra predictor-corrector)
7: Solve affine predictor, then centering-corrector steps
8: Optionally run Gondzio extra corrections
9: Apply fraction-to-boundary step lengths
10: Update x, λ, z, μ ; re-evaluate residual metrics
11: Track best iterate
12: if primal, dual, and complementarity tolerances met then
13: converged; break
14: end if
15: end for
16: Restore best iterate when configured and needed
17: Map x to ACOPFResult fields and return
18: end procedure

```

**Algorithm A12: Short-Circuit (All-Bus IEC-Style)**

```

1: procedure COMPUTE_SHORT_CIRCUIT(sys, sc_opt)
2: Project to canonical AC model
3: Build/factorize fault admittance matrix once
4: for each faulted bus k do
5: Solve $Y^{fault} z = e_k$, recover Z_{kk}
6: Apply fault type and voltage-factor rule
7: Compute I_k'' , short-circuit power, and R/X
8: Append per-bus detailed result row
9: end for
10: return short-circuit result set
11: end procedure

```

**Algorithm A13: Optimal Network Reconfiguration (ONR)**

```

1: procedure SOLVE_OPTIMAL_RECONFIGURATION(sys, opt)
2: Project rich models to canonical AC network
3: Build bus map, root selection, and net P/Q injections
4: Build LinDistFlow MILP with α, P, Q, v, f, t variables
5: Solve MILP using native branch-and-cut + MIR cuts
6: Round/repair α to a radial spanning tree if needed
7: Export open/closed stable branch indices and objective
8: Run Newton PF on selected topology for verification
9: return ONR result
10: end procedure

```

**Algorithm A14: Distribution PF (BFS)**

```

1: procedure SOLVE_DISTRIBUTION_PF(system, opt)
2: if system is HybridPowerSystem then
3: Project to canonical AC system
4: else
5: Use provided AC system directly
6: end if
7: Build bus map and BFS tree from in-service branches
8: Assemble net demand S^{net}
9: Initialize complex voltage vector
10: for iter = 1 to max_iter do
11: Backward sweep (currents: leaves $\rightarrow$ root)
12: Forward sweep (voltages: root $\rightarrow$ leaves)
13: if max voltage update < tol then
14: converged; break
15: end if
16: end for
17: Compute branch powers and total losses
18: return DPF result
19: end procedure

```

#### Algorithm A15: Carbon Analysis (Tracing + Matrix)

```

1: procedure COMPUTE_CARBON_ANALYSIS(sys, pf, opt)
2: Require converged PF state
3: Project to canonical hybrid model and build AC/DC id maps
4: Build unified load sinks and explicit storage carbon-source states
5: Build directed AC/DC/converter/router flow graph
6: Build carbon sources and reconstruct slack contribution
7: Run proportional tracing and export per-device allocations
8: Solve hybrid matrix carbon-intensity system for bus intensities
9: Export summaries, balance checks, and solver-specific notes
10: return carbon-analysis result
11: end procedure

```

#### Algorithm A16: DC Optimal Power Flow

```

1: procedure SOLVE_DC_OPF(sys, opt)
2: Build bus/branch/generator index sets and susceptance matrix B
3: Assemble cost vector (linear or quadratic depending on backend)
4: Add per-bus load-shedding variables with auto-tuned VOLL penalty
5: Build power-balance equality constraints (per bus)
6: Add flow-definition equalities: $P_{f,ij} - b_{ij}(\theta_i - \theta_j) = 0$
7: Set variable bounds ($\theta, P_g, P_f, \Delta P_d$)
8: Select backend: NativeQP $\rightarrow$ Gurobi $\rightarrow$ HiGHS $\rightarrow$ Dual Simplex
9: Solve LP or QP
10: Extract LMPs from power-balance dual variables
11: return DC OPF result with dispatch, flows, and LMPs
12: end procedure

```

#### Algorithm A17: Reactive Power Optimization (OA)

```

1: procedure SOLVE_REACTIVE_POWER_OPT(sys, opt)
2: Enumerate discrete variables (transformer taps, switchable shunt steps)
3: Solve baseline AC OPF to obtain incumbent objective

```

- 4:   **Phase 0 (Sensitivity):** For each discrete variable, perturb  $\pm 1$ , solve NLP, rank by  $|\Delta \text{obj}|$
- 5:   **Phase 1 (Ternary search):** For each variable in sensitivity order, run integer ternary search; add OA cuts  $f(y) \geq f_k + g_k^T(y - y_k)$  at each evaluation
- 6:   **Phase 2 (Coordinate descent, up to 3 cycles):** Sweep each variable; skip candidates where OA lower bound  $\geq$  incumbent; add cuts; terminate if no improvement
- 7:   **Phase 3 (Pairwise polishing):** Check  $\pm 1$  pairs of discrete variables; OA prunes before NLP evaluation
- 8:   **return** best tap/shunt settings, OA cuts, NLP solve count
- 9: **end procedure**

**Algorithm A18: LCQP Interior-Point Solver**

- 1: **procedure** SOLVE\_LCQP( $Q, c, A_{eq}, b_{eq}, \ell, u$ )
- 2:   Initialize  $x$  at bound midpoints;  $s_\ell, s_u \leftarrow$  slacks;  $z_\ell, z_u \leftarrow 1$ ;  $y \leftarrow 0$
- 3:   Cache symbolic factorization of KKT sparsity pattern
- 4:   **for**  $iter = 1$  to  $\text{max\_iter}$  **do**
- 5:     Compute complementarity gap  $\mu = (s_\ell^T z_\ell + s_u^T z_u) / (2n_{\text{bounded}})$
- 6:     **if** primal, dual feasibility, and gap  $< \epsilon$  **then**
- 7:       converged; **break**
- 8:     **end if**
- 9:     Form KKT residuals (stationarity, primal feasibility, complementarity)
- 10:    Solve predictor step via sparse LU with  $\epsilon$ -regularization
- 11:    Compute centering parameter  $\sigma = \min(0.3, 100\mu)$
- 12:    Solve corrector step with second-order compensation
- 13:    Compute step lengths  $\alpha_p, \alpha_d$  via fraction-to-boundary ( $\tau = 0.995$ )
- 14:    Update  $(x, s_\ell, s_u, y, z_\ell, z_u)$
- 15:   **end for**
- 16:   **return**  $x$ , objective, convergence status
- 17: **end procedure**

**Algorithm A19: Dual Simplex LP Solver**

- 1: **procedure** SOLVE\_DUAL\_SIMPLEX( $c, A, b, \ell, u$ )
- 2:   Convert to standard form: add slacks, surplus, artificial variables
- 3:   **Crash basis:** For each artificial, find original column with largest  $|A_{ij}|$  and swap
- 4:   **Phase I:** Minimize sum of artificials
- 5:   **for**  $iter = 1$  to  $\text{max\_iter}$  **do**
- 6:     Compute reduced costs; select entering variable (most-negative)
- 7:     Compute step direction and leaving variable (ratio test)
- 8:     **if** all artificials non-basic **then**
- 9:       Phase I complete; **break**
- 10:    **end if**
- 11:    Pivot basis; refactorize every 15–50 iterations
- 12:   **end for**
- 13:   **if** artificials remain in basis **then**
- 14:     Extract Farkas certificate; **return** infeasible
- 15:   **end if**
- 16:   **Phase II:** Minimize original  $c^T x$
- 17:   **for**  $iter = 1$  to  $\text{max\_iter}$  **do**
- 18:     Compute reduced costs
- 19:     **if** all non-negative (dual feasible) **then**
- 20:       optimal; **break**

```

21: end if
22: Select entering/leaving variables and pivot
23: end for
24: return x , objective, basis (cached for warm-start in branch-and-cut)
25: end procedure

```

#### Algorithm A20: Annual Production Simulation

```

1: procedure SOLVE_ANNUAL_PRODUCTION_SIMULATION($sys, ts_data, opts$)
2: L0 (Annual planning): Build coarse monthly/weekly blocks
 Generate energy budgets $\bar{E}_{g,k}$, SOC trajectory, maintenance flags
3: for each block k do
4: L2 (Weekly UC): Extract sub-horizon and solve rolling UC
 Binding window: $T^{(2)}$ steps; look-ahead: T^{la} steps
 Call solve_unit_commitment(sub_sys, sub_ts, ...)
5: if not skip_replay then
6: for each day in block do
7: L3 (Daily replay): Slice UC schedule for the day
8: Call solve_time_series_pf(sub_sys, ...) for OPF/PF validation
9: Store per-step voltages, losses, flows, costs
10: end for
11: else
12: Fill step results from UC + load profiles (no OPF/PF)
13: end if
14: end for
15: Aggregate generator/renewable/storage annual statistics
 Capacity factors, curtailment rates, cycle counts, startups
16: return annual result with step/block/component summaries
17: end procedure

```

#### Algorithm A21: Lifecycle Simulation

```

1: procedure RUN_LIFECYCLE_SIMULATION($sys, opts$)
2: for year = 1 to num_years do
3: Apply load growth: $P_d^{(y)} = P_d^{(0)}(1 + r_g)^{y-1}$
4: Apply PV derating: $P_{max}^{(y)} = P_{max}^{(0)}(1 - d_{PV})^{y-1}$
5: Compute battery SOH: $\min(\text{SOH}_{cal}, \text{SOH}_{cyc})$
6: if $\text{SOH} \leq \text{SOH}_{EOL}$ then
7: Trigger replacement; reset SOH and accumulate replacement cost
8: end if
9: Tier 1: Run annual chronological dispatch
10: Tier 2: Stratified sampling into 7 operating regimes
 (peak load, low renewable, high curtailment, heavy charge/discharge, normal, low load)
11: Compute Neyman-allocated sampling error bounds
12: Compute dispatch, sampling, and storage carbon error bounds
13: Accumulate annual OpEx and carbon metrics
14: end for
15: Compute NPV: $\sum_y (\text{OpEx}_y + \text{ReplaceEx}_y) / (1 + r)^{y-1}$
16: return lifecycle result with per-year metrics and error bounds
17: end procedure

```

## 22 Native Branch-and-Cut Solver

This section documents the toolbox’s native branch-and-cut solver from three angles at once: mathematical model, algorithmic workflow, and code-level realization. The implementation is centered on `include/hacdcpf/engine/branch_and_cut.hpp`, `src/engine/branch_and_cut.cpp`, `src/engine/bc_utils.cpp`, `src/engine/bc_branching.cpp`, `src/engine/bc_cuts.cpp`, and `src/engine/bc_relaxation.cpp`, while the LP kernel relies on `include/hacdcpf/engine/dual_simplex.hpp` and `src/engine/dual_simplex.cpp`. The public namespace is `hacdcpf::engine`; `include/hacdcpf/solver/branch_and_cut.hpp` re-exports the same solver for the adapter layer.

The discussion below is intentionally code-aligned. When the implementation differs from a textbook branch-and-cut design, this section follows the current C++ code rather than the idealized label.

### 22.1 Scope and Code Map

The module currently serves two problem classes:

- MILP: the full branch-and-cut path with root cuts, pseudo-cost / reliability branching, cut pool, solution pool, primal heuristics, sequential tree search, and heterogeneous parallel tree search.
- MINLP: a lighter spatial branch-and-bound path with NLP relaxations, branching, and pruning, but without the MILP path’s cut generation, cut pool, or dedicated cut worker.

The main source files are summarized in Table 126.

Table 126: Branch-and-cut code map

| File                                                         | Main responsibility                                                                                                                    |
|--------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| <code>include/hacdcpf/engine/branch_and_cut.hpp</code>       | Public API, <code>BCOptions</code> , <code>BCStats</code> , <code>BCResult</code> , and solver enums.                                  |
| <code>include/hacdcpf/engine/det ail/bc_types.hpp</code>     | Core data structures: nodes, pseudo-costs, cut requests, LP relaxation records, and thread-side payloads.                              |
| <code>include/hacdcpf/engine/det ail/bc_pools.hpp</code>     | Global <code>CutPool</code> and <code>SolutionPool</code> .                                                                            |
| <code>include/hacdcpf/engine/det ail/bc_threading.hpp</code> | Thread-safe queues, shared incumbent, shared cut/solution pools, and atomic parallel statistics.                                       |
| <code>src/engine/bc_utils.cpp</code>                         | Objective normalization, fractional-variable detection, MILP presolve, node bound propagation, reduced-cost fixing, and row insertion. |
| <code>src/engine/bc_branching.cpp</code>                     | Most-infeasible branching, pseudo-cost branching, and reliability branching probes.                                                    |
| <code>src/engine/bc_cuts.cpp</code>                          | Tableau GMI/Gomory cuts, cover cuts, legacy integer-rounding cuts, and cut dispatch.                                                   |
| <code>src/engine/bc_relaxation.cpp</code>                    | LP and NLP relaxation solves plus auxiliary rounding logic.                                                                            |
| <code>src/engine/branch_and_cut.cpp</code>                   | Root strengthening, sequential B&C, heterogeneous parallel B&C, and MILP/MINLP top-level orchestration.                                |
| <code>src/engine/dual_simplex.cpp</code>                     | Standard-form construction, sparse and dense simplex, dual-simplex reoptimization, and incremental bound updates in standard form.     |

The most direct solver entry points in the current toolbox are:

- `solve_milp_bc(prob, opt)` for MILP.
- `solve_minlp_bc(prob, opt)` for MINLP.
- `NativeBranchAndCutAdapter` for integration into the solver-adapter interface.
- `analysis/topology_analysis.cpp` for ONR-type MILP usage.
- `analysis/time_series_pf.cpp`, `tests/test_unit_commitment.cpp`, and `tests/test_uc_performance.cpp` for UC and scheduling-style MILP integration and tuning.

## 22.2 Active and Prepared Tree Data Structures

The active MILP and MINLP solve paths still use the legacy `Node` structure:

- full node bounds `lb/ub`,
- relaxation solution `x_relax`,
- optional seed `x_seed`,
- simplex warm-start hint `basis_hint`,
- node bound `bound`, and
- tree depth.

A recent implementation change made `Node::branch_child()` explicitly lightweight: it copies only bounds, basis hint, bound, and depth, but skips copying `x_relax` and `x_seed`. This matters because branching creates children frequently, and the solver now avoids unnecessary vector copies on the hot path.

At the same time, `bc_types.hpp` now also contains a prepared diff-based representation:

- `BoundChange`,
- `TreeNode`, and
- `TreeDomainManager`.

These structures support LCA-based switching between nodes by replaying bound changes rather than copying full bound vectors. As of the current implementation, they are present in the type layer but are not yet wired into the active branch-and-cut search loop. Therefore the operational solver still uses the legacy full-bound `Node` path.

## 22.3 Problem Class and Outer Mathematical Model

For MILP, the input object is `MIPModel`, whose linear part is `LPMModel`:

$$\min / \max \quad c^\top x \tag{338}$$

$$\text{s.t.} \quad Ax \leq b, \tag{339}$$

$$A_{eq}x = b_{eq}, \tag{340}$$

$$\ell \leq x \leq u, \tag{341}$$

$$x_j \in \mathbb{Z} \text{ or } \{0, 1\}, \quad j \in \mathcal{I}. \tag{342}$$

The outer B&C logic compares all bounds in a unified minimization metric. The helper `objective_value(c, x, sense)` returns  $c^\top x$  for minimization and  $-c^\top x$  for maximization, so pruning, pseudo-cost updates, and gap computation can all be written as

$$z_{LP}(N) \leq z^* \leq z_{inc}, \quad (343)$$

$$\text{gap} = \max\left(0, \frac{z_{inc} - z_{LB}}{\max(1, |z_{inc}|)}\right). \quad (344)$$

For a fractional integer variable  $x_j^*$ , the two children are

$$\text{down child : } x_j \leq \lfloor x_j^* \rfloor, \quad (345)$$

$$\text{up child : } x_j \geq \lceil x_j^* \rceil. \quad (346)$$

A node is pruned if any of the following happens:

- the node bounds are inconsistent, that is,  $\ell_i > u_i$  for some  $i$ ;
- the LP or NLP relaxation is infeasible or numerically rejected;
- the node bound is not better than the incumbent, that is,  $z_{LP}(N) \geq z_{inc}$ ;
- the relaxation solution is already integral, in which case it becomes a candidate incumbent immediately.

## 22.4 MILP Presolve and Bound Strengthening

`milp_presolve()` runs before the root node is solved. It is not a general-purpose black-box presolve; it is a branch-and-cut-oriented propagation engine focused on bounds, fixings, and matrix simplification.

**1. Inequality and equality bound tightening** For an inequality row  $a^\top x \leq b$ , the code computes the minimum activity

$$L(a, \ell, u) = \sum_{a_j > 0} a_j \ell_j + \sum_{a_j < 0} a_j u_j, \quad (347)$$

and then derives tighter upper or lower bounds variable by variable. Equality rows use both minimum and maximum activity, so they can be stronger. Integer and binary bounds are rounded immediately with  $\lfloor \cdot \rfloor$  or  $\lceil \cdot \rceil$ .

**2. Singleton-row processing** If a row contains only one nonzero coefficient, it directly implies a bound on that variable. Equality singletons can fix a variable outright. Fixed singleton rows are then removed from the active matrix.

**3. Redundant-row elimination** If an inequality already satisfies  $\max a^\top x \leq b$  under current bounds, the row is redundant and removed from **A**.

**4. Binary probing inside presolve** For unfixed binary variables, the presolver probes  $x_j = 0$  and  $x_j = 1$ , applies three rounds of propagation in each branch, and then:

- fixes the variable if one side is infeasible, or
- merges implied bounds back into the root model if both sides are feasible.

**5. Fixed-variable substitution** If  $\ell_j = u_j$ , the contribution of that variable is absorbed into  $b$  and  $b_{eq}$ , and the corresponding matrix column is zeroed out during sparse-matrix rebuild. This reduces effective LP density before simplex begins.

A lighter version of the same idea is repeated at tree nodes through `node_bound_propagation()`. The node-level propagator does not rebuild matrices; it only tightens current node bounds for two or three rounds before the LP solve, which allows cheap infeasibility detection before simplex reoptimization.

## 22.5 Standard Form and LP Relaxation Kernel

LP node solves are not performed directly on  $(A, b, A_{eq}, b_{eq}, \ell, u)$ . The solver first constructs a bounded standard form via `build_standard_form_lp()`. For each original variable, it defines

$$x = \ell + y, \quad y \geq 0, \quad (348)$$

and stores finite upper bounds in `var_ub`, that is,  $0 \leq y_j \leq u_j - \ell_j$ . The resulting internal LP is represented as a maximization problem,

$$\max \quad c_{max}^\top z \quad (349)$$

$$\text{s.t.} \quad \tilde{A}z = \tilde{b}, \quad (350)$$

$$0 \leq z \leq \tilde{u}. \quad (351)$$

Important implementation details are:

- The internal objective is always maximization, with  $c_{max} = -c_{min}$ .
- $\leq$  rows receive slack variables;  $\geq$  rows receive surplus plus artificial variables; equality rows receive artificial variables.
- If a shifted right-hand side is negative, the whole row is multiplied by  $-1$ , and the row sense is flipped so that the stored RHS becomes nonnegative.
- Finite upper bounds are kept implicitly through `var_ub` and `at_upper`; they are not converted into explicit rows.

So the LP kernel is closer to a bounded-form revised simplex than to the textbook lower-bound-only standard form. This is the key reason why both bounded-form GMI generation and dual-simplex warm starts are efficient in the current implementation.

**Fast bound updates** Most children modify only one or two bounds. For that reason, `update_standard_form_bounds()` updates only `lb_shift`, `b`, `var_ub`, and `objective_const`, without rebuilding `A`. When no more than 50 lower-bound components change, the code applies sparse column-wise incremental updates; otherwise it falls back to a full sparse matrix-vector product.

**LP relaxation solve modes** `solve_lp_relaxation()` has two front-end modes:

- Default path: `use_simplex_lp_nodes=true`. The solver uses the native simplex kernel and returns a basis hint, reduced costs, and optionally a persisted sparse basis object for child-node warm starts and GMI separation.
- Fallback path: `use_simplex_lp_nodes=false`. The LP is wrapped as an NLP with zero Hessian and solved by the native IPM. This still gives a relaxation solution, but tableau-based cuts are unavailable unless a `SimplexResult` exists.

## 22.6 Warm Start by Dual Simplex

The main theoretical fact behind the tree search is simple: when branching only changes variable bounds, the constraint matrix  $A$  remains unchanged. Therefore the parent basis is often still structurally useful for the child. In bounded-form simplex, reduced costs are determined by

$$\bar{c} = c_{max} - A^\top y, \quad y = B^{-T} c_B, \quad (352)$$

so a basis reused under changed bounds often stays dual feasible while losing primal feasibility. That is exactly the setting in which dual simplex is effective.

The implementation currently contains three practical warm-start regimes.

**Path A: sparse warm start for large LPs** For  $m > \text{kSparseSimplexThreshold} = 200$ , the code prefers a sparse basis path built around `SparseBasis`. The main steps are:

- reuse the parent’s persisted sparse basis and reduced costs when available;
- validate basis indices and block artificial columns from entering;
- pre-clean the basis by swapping out basic artificial variables with row slacks whenever possible, which avoids false infeasibility after bound changes;
- recompute  $x_B$ , reduced costs, objective value, and `at_upper` status with `sparse_full_recompute()` or the equivalent sparse operations;
- if primal feasibility is violated but dual feasibility is retained, run `sparse_dual_simplex_reoptimize()`;
- if dual reoptimization fails numerically but the basis can be refactorized, fall back to sparse primal Phase II as a last warm-start rescue.

When sparse warm start succeeds, the solver persists the sparse basis again for descendants instead of computing a dense inverse.

**Path B: dense warm start for small LPs** For  $m \leq 200$ , the solver uses the dense inverse path. If a cached inverse is available, it reconstructs the basis state quickly; otherwise it recomputes the state from the basis itself. If the reconstructed state is primal infeasible but dual feasible, it calls `dual_simplex_reoptimize()`.

**Path C: cold start when warm start is unavailable or rejected** The simplex kernel still supports cold-start Phase I / Phase II solves with slack and artificial variables. For large LPs, the sparse cold-start path uses sparse primal simplex and may use a Big-M objective if crash replacement does not remove all artificials. For small LPs, the dense path uses a classic Phase I / Phase II sequence.

**A deliberate engineering tradeoff at tree nodes** In the main MILP tree search, `SimplexOptions.allow_cold_start` is set to `false`. Therefore, when a large child-node warm start fails, the code does *not* attempt an expensive cold-start Phase I / Phase II solve. Instead, that node LP is marked as failed and the branch-and-cut layer prunes it. This is a speed-first engineering choice: it reduces node cost substantially on large UC-style MILPs, but strict completeness depends on warm-start robustness. Root solves and root cut re-solves still retain cold-start capability so that the algorithm does not lose the model at the very beginning.

**Incumbent-aware LP early cutoff** Both sequential and parallel MILP paths provide an incumbent bound to the simplex solver. The LP kernel can then terminate early when the node’s LP bound is already provably worse than the incumbent, which avoids many unnecessary pivot iterations in nodes that would be pruned anyway.

## 22.7 Cut Families and Admission Rules

The current cut module implements three distinct families.

**1. Tableau-based Gomory / GMI cuts** `add_basis_gomory_cuts()` builds bounded-form GMI cuts from the current simplex basis. Conceptually, if row  $r$  corresponds to a basic integer variable with value  $\beta$  and fractional part  $f_0 = \beta - \lfloor \beta \rfloor$ , the code first extracts the tableau row

$$\rho = e_r^\top B^{-1} A, \quad (353)$$

using either a dense  $B^{-1}$  row or sparse BTRAN. It then computes bounded-form GMI coefficients for each nonbasic column, taking into account:

- whether the original variable is integer or continuous,
- whether that variable is currently represented at its upper bound, and
- the shifted lower-bound representation used by the standard form.

After that, slack and surplus columns are eliminated algebraically, and the cut is mapped back to the original variable space as

$$a_{cut}^\top x \leq b_{cut}. \quad (354)$$

Candidate GMI cuts are not inserted blindly. They are filtered by:

- violation at the current LP point,
- efficacy  $\text{viol} / \|a_{cut}\|_2$ ,
- density cap `gmi_max_density`,
- a binary-activity score that favors cuts acting on highly fractional binary support,
- a binary-support ratio filter, and
- near-parallel rejection against already selected cuts through `gmi_max_parallelism`.

The ranking score is

$$\text{score} = \text{efficacy} \cdot (1 + w_{act} \cdot \text{activity}), \quad (355)$$

where  $w_{act}$  is `gmi_activity_weight`.

**2. Cover cuts** `add_cover_cuts()` scans inequality rows that contain only positive coefficients on binary variables. For a cover set  $C$  with

$$\sum_{j \in C} a_j > b, \quad (356)$$

it generates the classical cover inequality

$$\sum_{j \in C} x_j \leq |C| - 1. \quad (357)$$

The implementation builds the cover greedily by sorting coefficients in descending order and accumulating until the right-hand side is exceeded.

**3. Legacy row-based integer-rounding cuts** `add_mir_like_cuts()` reads fractional right-hand sides directly from original inequality rows and builds an older integer-rounding cut from row coefficients. In the current public interface, this family is exposed primarily by `CutType::IntRounding`.

**Actual mapping of CutType** Table 127 records the *actual current code behavior*, not the ideal meaning suggested by enum names.

Table 127: Cut-type enum and current implementation mapping

| Enum value               | Actual dispatch target             | Notes                                                                                                    |
|--------------------------|------------------------------------|----------------------------------------------------------------------------------------------------------|
| <code>None</code>        | No cuts                            | Baseline mode.                                                                                           |
| <code>IntRounding</code> | <code>add_mir_like_cuts</code>     | Legacy row-based integer-rounding cuts on original inequalities.                                         |
| <code>MIR</code>         | <code>add_basis_mir_cuts</code>    | <code>add_basis_mir_cuts()</code> currently delegates directly to <code>add_basis_gomory_cuts()</code> . |
| <code>Gomory</code>      | <code>add_basis_gomory_cuts</code> | Main tableau-based GMI/Gomory generator.                                                                 |
| <code>Cover</code>       | <code>add_cover_cuts</code>        | Binary knapsack-style cover cuts.                                                                        |
| <code>All</code>         | Gomory $\rightarrow$ Cover         | The dispatcher now skips the MIR alias in this mode to avoid duplicate tableau GMI passes.               |

So if an upper-layer module says it is using “MIR cuts” through `CutType::MIR`, the current implementation should be understood as using the tableau GMI/Gomory generator under a different enum label. The only truly row-based family currently exposed is `IntRounding`.

## 22.8 Cut Pool and Solution Pool

**CutPool** The global `CutPool` stores cuts that were already generated and can be cheaply re-separated later in the tree. Each pool entry keeps a dense coefficient vector, right-hand side, age, best efficacy placeholder, and norm. Its current policy is:

- duplicate rejection by cosine similarity above 0.95,
- age-based replacement when the pool is full, and
- age reset for violated cuts plus purge after `cut_pool_max_age` consecutive non-violations.

**SolutionPool** The solution pool stores the best  $K$  feasible integer solutions and supports guided rounding. For a fractional integer variable, guided rounding uses a vote across stored solutions to choose the rounding direction. This is more structure-aware than plain `round( $x_j$ )` because it reuses historically successful integer patterns.

## 22.9 Branching, Node Selection, and Primal Heuristics

**Branch-variable selection** The code currently supports three branching strategies:

- **FirstFractional**: branch on the first fractional variable. Useful as a baseline or debugging strategy.
- **MostInfeasible**: pick the variable with fractional part closest to 0.5. The score is  $0.5 - |\text{frac}(x_j) - 0.5|$ .

- **Pseudocost:** use a product score

$$s_j = \max\left(f_j \bar{q}_j^\downarrow, \varepsilon\right) \max\left((1 - f_j) \bar{q}_j^\uparrow, \varepsilon\right), \quad f_j = x_j - \lfloor x_j \rfloor, \quad (358)$$

where  $\bar{q}_j^\downarrow$  and  $\bar{q}_j^\uparrow$  are average pseudo-cost estimates.

When pseudo-cost branching is enabled, shallow nodes may use `choose_branch_var_reliability()` first. That routine probes unreliable candidates by solving temporary down/up child LPs, updates pseudo-costs with true gains, and only then selects the branch variable. The current implementation behaves as follows:

- reliability branching is used only up to depth 10;
- candidates that already received root probing data require fewer additional observations;
- the probing budget is more aggressive at the root and lighter at deeper nodes.

**Node selection** The public API exposes `BestFirst`, `DepthFirst`, and `Hybrid`. The active implementation of `Hybrid` is slightly more specific than the enum name suggests:

- before an incumbent exists, new nodes are pushed to DFS;
- after an incumbent exists, new nodes are pushed to the best-bound priority queue;
- however, if the DFS stack is nonempty, it is still popped before the priority queue.

So the current `Hybrid` strategy is best read as DFS-biased with best-bound backfill, not as a strict one-time DFS-to-best-first mode switch.

**Primal heuristics used in the main solve path** The main branch-and-cut driver currently employs the following primal heuristics:

- quick rounding after the root LP,
- user warm-start acceptance and warm-start repair,
- a fix-and-solve style feasibility pump at the root,
- LP diving at the root,
- direct rounding and guided rounding at child nodes,
- crossover from the two best solution-pool incumbents, and
- a simple RINS-like heuristic in the sequential path.

The helper `try_rounding_heuristic()` still exists in `bc_relaxation.cpp`, but the production MILP path in `branch_and_cut.cpp` now performs the main rounding logic inline.

## 22.10 Root-Node Strengthening Pipeline

In the current implementation, the root node is much more than “solve one LP and start branching”. The actual pipeline is shown in Algorithm BC-R1.

**Algorithm BC-R1: Root strengthening**

```

1: procedure ROOTSTRENGTHEN(prob, opt)
2: run milp_presolve on the LPModel
3: solve the root LP relaxation to obtain x_0^{LP} , a basis hint, and the root bound
4: if cuts are enabled then
5: for $k = 1$ to root_cut_rounds do
6: generate up to cuts_per_round root cuts
7: if no cut was added then
8: break
9: end if
10: extend the basis hint to accommodate new slack/artificial columns
11: re-solve the cut-augmented root LP
12: if the re-solve fails then
13: roll back the added root cuts and stop root cutting
14: end if
15: if bound improvement stalls then
16: stop root cutting early
17: end if
18: end for
19: end if
20: seed the cut pool from accepted root cuts
21: try user warm start and warm-start repair
22: run the feasibility-pump loop
23: run LP diving
24: run two-phase root probing: domain propagation, then LP probing
25: if an incumbent exists, run reduced-cost fixing at the root
26: push the strengthened root node into the search queue
27: end procedure

```

Several implementation details are especially important.

**1. Root cut re-solve basis extension** After adding root cuts, the standard-form dimensions change. The old basis hint is therefore not directly compatible. The code explicitly rebuilds the basis index array so that:

- old inequality rows keep their relative order,
- new cut rows are inserted between the old inequality block and the equality block,
- old equality rows are shifted downward, and
- slack, surplus, and artificial column indices are renumbered consistently.

This is one of the key robustness mechanisms behind successful root cut re-solves.

**2. Root cut rollback and sparse-basis cleanup** If a root cut round adds cuts but the re-solve fails, the code truncates the newly appended inequality rows and restores the pre-cut basis, root relaxation point, and root bound. Even after successful root cutting, sparse-basis pointers created during cut re-solves are cleared before the fresh base standard form is built, because they may otherwise reference already-destroyed local matrices.

**3. Stall detection** Root cutting stops early if the relative bound improvement falls below  $10^{-4}$  for two consecutive rounds.

**4. Warm-start repair** If `prob.initial_solution` exists, the solver first clamps it to bounds and rounds integer variables. If it is not feasible, the solver can still attempt repair by fixing the integer variables to the warm-start pattern and re-solving an LP over the continuous variables.

**5. Feasibility pump** The current root feasibility-pump logic has two stages:

- a cheap direct rounding test, and
- up to five iterations of “round integer variables, fix them, re-solve the restricted LP for the continuous part”.

So the current implementation is closer to a fix-and-solve repair loop than to the more elaborate distance-minimization feasibility-pump variants found in the literature.

**6. LP diving** The root diving heuristic starts from the root LP solution and greedily fixes near-integral variables while repeatedly re-solving warm-started LPs. It:

- is skipped when a warm start already provided an incumbent,
- scales its LP budget with model size,
- uses a time budget of at most roughly 3% of the global time limit and no more than about 1 second,
- batch-fixes variables that are already very close to integrality, and
- tries the opposite rounding direction if the nearest-integer fix fails.

**7. Two-phase root probing** Root probing has two distinct phases.

- Domain-propagation probing: for each eligible unfixed binary variable, test  $x_j = 0$  and  $x_j = 1$  by bound propagation only. If one side becomes inconsistent, fix the variable to the other side.
- LP probing: for the most fractional unfixed binaries, solve temporary down/up LPs. This both initializes pseudo-costs and may produce additional fixings when one side is LP-infeasible or already dominated by the incumbent.

The LP probing budget is adaptive. It is smaller for large models and smaller again when a good incumbent already exists.

**8. Reduced-cost fixing** If an incumbent exists, the root can use reduced costs to fix nonbasic integer variables. The current implementation computes

$$\Delta = z_{inc} - z_{LP}(N), \quad (359)$$

and for a nonbasic integer variable at one of its bounds, fixes it at that bound if

$$|rc_j| \geq \Delta. \quad (360)$$

In the present code, reduced costs are handled in the internal minimization metric and applied only to nonbasic integer variables that are already sitting at their lower or upper bound.

## 22.11 Child Processing and Sequential Tree Search

For each child node, the helper `process_single_child()` applies the following steps in order:

1. check bound consistency;
2. run `node_bound_propagation()` on the node bounds;
3. update the standard-form bounds by `update_standard_form_bounds()` without rebuilding the matrix;
4. solve the child LP with the parent basis hint;
5. if an incumbent exists, run reduced-cost fixing and re-check consistency;
6. if the LP has at most 500 inequality rows, separate violated cuts from the global cut pool and re-solve the child LP with those pool cuts;
7. if `cuts!=None` and the child depth is within `max_cut_depth`, generate new node cuts, add them to the global cut pool, and re-solve the node on a transient cut-augmented LP;
8. attempt direct rounding and guided rounding to search for new incumbents.

The 500-row threshold is a pure engineering switch. Above that size, copying `LPModel`, rebuilding `StandardFormLP`, and re-solving the LP for pool separation becomes too expensive relative to the expected gain, so the implementation skips pool separation.

A second important detail is that local node cuts are handled on a transient cut-augmented LP copy. If that re-solve succeeds, the node keeps the improved bound and LP point, but the persistent basis hint is restored to the pre-cut basis. This design keeps descendants aligned with the base LP plus node bounds rather than with a private cut-augmented submodel.

The sequential main loop can be summarized as Algorithm BC-S1.

### Algorithm BC-S1: Sequential MILP B&C

```
1: procedure SEQUENTIALBC(root)
2: while time limit not reached, node limit not reached, and queue not empty do
3: pop the next live node N
4: if $z_{LP}(N) \geq z_{inc}$ then
5: continue
6: end if
7: if $x^{LP}(N)$ is integral then
8: update incumbent and solution pool
9: continue
10: end if
11: choose branch variable by reliability branching or pseudo-cost branching
12: create down and up children
13: process both children with process_single_child
14: push the more promising child to DFS first and the other to the global queue structure
15: periodically run crossover and the RINS-like heuristic
16: update live-node lower bound and global gap
17: if relative gap is below tolerance then
18: terminate
19: end if
20: end while
21: end procedure
```

In the sequential path:

- crossover is attempted every 100 explored nodes;
- the RINS-like heuristic is attempted every 200 explored nodes;
- the simplex kernel receives an atomic incumbent bound so that poor nodes can be cut off early during LP reoptimization itself.

## 22.12 Heterogeneous Parallel Tree Search

The parallel MILP path is not a homogeneous “all workers do the same thing” implementation. It is intentionally heterogeneous:

- if `opt.cuts!=None` and `max_cut_depth>0`, the solver runs one dedicated cut worker plus  $N - 1$  explorer threads;
- if cuts are only allowed at the root, all threads become explorers because a cut worker would be idle.

The explorer threads are split into two roles:

- **Diver:** incumbent-oriented, DFS-biased, pseudo-cost branching, and local plunging.
- **Prover:** bound-oriented, best-bound queue preference, reliability branching at shallow depths, and stronger focus on proving optimality.

**Shared parallel structures** The parallel code uses the following shared objects:

- **ThreadSafeNodeQueue:** a shared queue containing both a best-bound priority queue and a DFS stack.
- **SharedIncumbent:** objective bound shared by atomic compare-and-swap and solution vector shared by mutex.
- **SharedCutPool:** a `shared_mutex`-protected cut pool.
- **SharedSolutionPool:** a mutex-protected solution pool.
- **CutRequestQueue:** a bounded explorer-to-cut-worker request queue.
- **AtomicBCStats:** lock-free or low-lock parallel counters.

**Explorer behavior** Each explorer owns a private copy of the base standard form. This avoids repeated synchronization around LP matrix data. In addition:

- a Diver maintains a thread-local DFS stack;
- the maximum size of that local DFS plunge is now controlled by `kLocalDfsMax = max(1, opt.max_plunge_depth)`;
- when the local plunge stack is full, additional children are spilled to the shared queue;
- explorers still perform node LP solves, reduced-cost fixing, pool separation, rounding, and guided rounding locally.

So `max_plunge_depth` is no longer a dormant option: it now actively controls how aggressively Diver threads keep nodes in thread-local DFS before returning them to the shared queue.

**Cut worker behavior** Explorer threads do not perform expensive tableau separation themselves in the parallel path. Instead they package the current `SimplexResult`, node bounds, node depth, and LP relaxation point into a `CutRequest` and submit it non-blockingly to the cut worker. If the request queue is full, the request is dropped. This affects cut opportunities, not solution correctness.

The cut worker then:

- pops cut requests,
- generates cuts,
- inserts newly generated rows into the shared cut pool, and
- when idle, continues to search for incumbents via guided rounding and crossover from the shared solution pool.

**A code-level nuance: parallel tree cutting is currently Gomory-only** There is an important implementation asymmetry here. Root cutting and sequential child cutting both use the generic `add_cuts()` dispatcher, so they honor the configured `CutType`. The dedicated parallel cut worker, however, currently calls `add_basis_gomory_cuts()` directly. Therefore tree-level cut generation in the parallel path is effectively Gomory/GMI-only, even if the user selected `MIR`, `Cover`, or `All`. This is a real code fact and should be kept in mind when interpreting parallel runs.

**Monitor loop** The main thread acts as a monitor. Roughly every millisecond it checks:

- wall-clock time limit,
- node limit,
- optimality gap, and
- whether the shared queue is empty and no explorer is still active.

The last condition is verified twice with a short sleep in between, which reduces false termination due to queue/worker races.

## 22.13 MINLP Path

`solve_minlp_bc()` is currently a spatial branch-and-bound rather than a full branch-and-cut engine:

- the root and child relaxations are solved by `solve_nlp_relaxation()` through the native IPM,
- branching still reuses the same branch-variable selection framework and pseudo-cost table,
- there is no cut generation, no cut pool, no solution pool, and no heterogeneous parallel explorer/cut-worker design.

So the name “branch-and-cut” is fully accurate for the MILP path, but the MINLP path should currently be understood as branch-and-bound with NLP relaxations.

## 22.14 Options and Statistics

Table 128 summarizes the most important user-visible solver options.

Table 128: Key `BCOptions` fields

| Option                             | Default            | Purpose                                                                                        |
|------------------------------------|--------------------|------------------------------------------------------------------------------------------------|
| <code>max_nodes</code>             | 50000              | Maximum number of explored tree nodes.                                                         |
| <code>max_lp_iter</code>           | 500                | LP/NLP iteration budget parameter used in relaxation solves.                                   |
| <code>time_limit_sec</code>        | 120.0              | Global wall-clock time limit.                                                                  |
| <code>int_tol</code>               | $10^{-5}$          | Integrality tolerance.                                                                         |
| <code>gap_tol</code>               | $10^{-4}$          | Relative optimality-gap tolerance.                                                             |
| <code>lp_tol</code>                | $10^{-6}$          | LP tolerance used to derive simplex feasibility and optimality tolerances.                     |
| <code>root_cut_rounds</code>       | 5                  | Maximum number of root cut rounds.                                                             |
| <code>cuts_per_round</code>        | 10                 | Maximum new cuts per round or per node.                                                        |
| <code>max_cut_depth</code>         | 0                  | Tree nodes deeper than this do not generate fresh cuts.                                        |
| <code>cuts</code>                  | None               | Cut-family selection switch.                                                                   |
| <code>branching</code>             | Pseudocost         | Branch-variable selection strategy.                                                            |
| <code>node_sel</code>              | Hybrid             | Node-selection strategy.                                                                       |
| <code>use_simplex_lp_nodes</code>  | true               | Whether node LPs use the simplex path with basis reuse.                                        |
| <code>use_feasibility_pump</code>  | true               | Whether the root feasibility-pump loop is enabled.                                             |
| <code>cut_pool_max_size</code>     | 2000               | Maximum cut-pool capacity.                                                                     |
| <code>cut_pool_max_age</code>      | 50                 | Age threshold for purging stale pool cuts.                                                     |
| <code>solution_pool_size</code>    | 10                 | Number of best incumbents kept for guided rounding and crossover.                              |
| <code>num_threads</code>           | 1                  | Thread count; values above 1 enable the heterogeneous parallel path.                           |
| <code>cut_worker_queue_size</code> | 64                 | Maximum number of pending explorer-to-cut-worker requests.                                     |
| <code>max_plunge_depth</code>      | 10                 | Active limit on Diver thread-local DFS plunging before spillback to the shared queue.          |
| <code>gmi_*</code>                 | several thresholds | Controls GMI efficacy, density, near-parallel rejection, binary activity, and support filters. |

`BCStats` records diagnostics about the actual search behavior, as summarized in Table 129.

Table 129: Meaning of BCStats fields

| Field                               | Meaning                                                                            |
|-------------------------------------|------------------------------------------------------------------------------------|
| <code>nodes_explored</code>         | Number of explored nodes.                                                          |
| <code>lp_solves</code>              | Total number of LP or NLP relaxation solves.                                       |
| <code>cuts_added</code>             | Number of newly generated cuts inserted into models.                               |
| <code>best_bound</code>             | Current global lower bound over live nodes.                                        |
| <code>best_obj</code>               | Current incumbent objective value.                                                 |
| <code>gap</code>                    | Relative MIP gap in the solver's internal minimization metric.                     |
| <code>runtime_sec</code>            | Total runtime.                                                                     |
| <code>pool_cuts_separated</code>    | Number of cuts reintroduced from the global cut pool.                              |
| <code>crossover_incumbents</code>   | Number of incumbents found by the crossover heuristic.                             |
| <code>cut_requests_submitted</code> | Number of cut requests sent from explorers to the cut worker.                      |
| <code>cut_requests_dropped</code>   | Number of cut requests dropped because the queue was full.                         |
| <code>cut_worker_cuts</code>        | Number of cuts generated by the dedicated cut worker.                              |
| <code>cut_worker_incumbents</code>  | Number of incumbents found by the cut worker through guided rounding or crossover. |
| <code>status</code>                 | Solver status string, aligned with <code>SolveStats.status</code> .                |

## 22.15 Applications, Tests, and Code-Aligned Caveats

**1. Current in-toolbox usage** The native MILP branch-and-cut solver is currently used most directly in:

- ONR and topology-reconfiguration workflows,
- UC and scheduling-style MILP integration, and
- performance and regression tests that compare the native implementation against external solvers.

The current UC-oriented tuning used by `analysis/time_series_pf.cpp` is representative:

- `cuts=CutType::Gomory`,
- `root_cut_rounds=15`,
- `cuts_per_round=20`,
- `use_feasibility_pump=false`,
- `gap_tol=1e-3`,
- large `max_lp_iter`, and
- parallelism up to 8 hardware threads when available.

**2. Existing test coverage** The current implementation is directly exercised by at least the following test groups:

- `tests/test_branch_and_cut.cpp` for correctness on small MILP/MINLP instances,
- `tests/test_unit_commitment.cpp` for UC-style integration and solver behavior,
- `tests/test_uc_performance.cpp` for larger UC scaling and parallel performance, and
- the root-cut re-solve robustness tests associated with basis extension behavior.

**3. Important caveats for users and developers** The current codebase contains several behaviorally important caveats.

- **CutType::MIR is not a distinct MIR implementation.** It currently aliases the tableau GMI/Gomory generator.
- **CutType::All no longer duplicates MIR/GMI dispatch.** It now executes one tableau GMI pass plus cover cuts.
- **Large child-node LPs usually do not cold start.** This is a deliberate speed optimization, not a purely textbook completeness-first design.
- **Pool separation is intentionally skipped for larger LPs.** Once the LP is large, a second full re-solve per child becomes too expensive.
- **Parallel cut requests may be dropped.** This weakens cut generation opportunities but does not invalidate incumbents or bounds already proven.
- **The dedicated parallel cut worker is currently Gomory-only.** Parallel tree cuts therefore do not perfectly mirror the configured CutType semantics.
- **Diff-based tree management is prepared but not active.** `TreeNode` and `TreeDomainManager` exist in the type layer, but the running solver still uses the legacy full-bound `Node` representation.

**4. Practical interpretation** Taken together, the native branch-and-cut solver is not a minimal teaching implementation. It is an engineering-oriented MILP core that combines:

- bounded-form simplex with dual-simplex warm reoptimization,
- an aggressively strengthened root node,
- several incumbent-finding heuristics,
- reusable cut and solution pools,
- sequential and heterogeneous parallel tree search, and
- application-driven tuning for power-system optimization models such as ONR and UC.

The most natural next algorithmic extensions, if stricter generality is desired, are a truly distinct MIR implementation, a more explicit recovery path when large-node warm starts fail, activation of diff-based tree-state management, and richer cut or parallel logic for the MINLP path.

## 23 Validation Assets and Current Evidence

### 23.1 Validation Philosophy

The current repository already contains a meaningful cross-validation suite, but its evidence is distributed across individual test binaries rather than a single benchmark report. This notebook therefore documents the current validation assets by test intent and by covered module interaction.

## 23.2 Latest Release-Mode Core Snapshot

After the recent power-flow and OPF refactors, the current release-mode evidence set for the core notebook modules was rerun with the nine targeted binaries listed below. The run completed with 865 assertions and no failures.

| Test binary                              | Assertions | Main evidence added to this notebook                                                                       |
|------------------------------------------|------------|------------------------------------------------------------------------------------------------------------|
| <code>test_all_components_pf</code>      | 159        | Full-component PF coverage after the <code>power_flow</code> directory split and Jacobian-kernel refactor. |
| <code>test_annual_production_sim</code>  | 405        | Annual production simulation, L0/L2/L3 orchestration, and metric aggregation.                              |
| <code>test_carbon_analysis</code>        | 82         | Static hybrid carbon tracing, matrix carbon intensity, and result summaries.                               |
| <code>test_core_modules</code>           | 11         | Core module smoke and utility checks.                                                                      |
| <code>test_modeling_toolkit</code>       | 146        | AML/modeling-toolkit expression, bridge, and solver-facing contracts.                                      |
| <code>test_opf_pf_crossval</code>        | 24         | OPF dispatch replay through PF feasibility checks.                                                         |
| <code>test_powerflow_performance</code>  | 3          | Optimized PF kernel smoke/performance checks.                                                              |
| <code>test_problem_types</code>          | 14         | Engine problem-type contract coverage.                                                                     |
| <code>test_projection_equivalence</code> | 21         | Canonical projection equivalence and stable-result behavior.                                               |
| <b>Total</b>                             | <b>865</b> | <b>All assertions passed.</b>                                                                              |

## 23.3 Cross-Validation Map

| Test or evidence source                                                                             | What it validates                                                                                                                                                                    |
|-----------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>tests/test_opf_pf_crossval.cpp</code>                                                         | Consistency between AC OPF outputs and PF feasibility on representative AC/DC cases such as <code>ieee14_acdc</code> , <code>case33bw_acdc</code> , and <code>case33mg_acdc</code> . |
| <code>tests/test_ac_opf_jacobian_fd.cpp</code>                                                      | Finite-difference verification of the native AC OPF equality Jacobian assembly.                                                                                                      |
| <code>tests/test_parity_formulation_fd.cpp</code> and <code>tests/test_parity_hessian_fd.cpp</code> | Finite-difference verification of the parity formulation and its Hessian terms.                                                                                                      |
| <code>tests/test_parity_ipm.cpp</code> and <code>tests/test_parity_julia.cpp</code>                 | Parity-IPM convergence and parity-path cross-checks against reference behavior.                                                                                                      |
| <code>tests/test_distribution_pf_crossval.cpp</code>                                                | BFS-vs-Newton agreement on radial networks, plus validation of DER injections, hybrid feeder overlays, PV power curves, and stationary vs transit mobile storage behavior.           |

| Test or evidence source                                                                                                                                      | What it validates                                                                                                                                                                  |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>tests/test_short_circuit_crossval.cpp</code>                                                                                                           | Analytical and self-consistency checks for short-circuit levels, Thevenin impedances, and API consistency between per-bus and batch runs.                                          |
| <code>tests/test_topology_crossval.cpp</code>                                                                                                                | Connectivity/radiality checks, 6-bus ONR comparison against exhaustive spanning-tree enumeration, and IEEE-33 style post-reconfiguration verification with Newton PF.              |
| <code>tests/test_solver_handle.cpp</code>                                                                                                                    | Consistency between cached solver-handle runs and direct API runs.                                                                                                                 |
| <code>tests/test_case_builders.cpp</code> , <code>tests/test_json_roundtrip_new_components.cpp</code> , <code>tests/test_excel_comprehensive_case.cpp</code> | Input-model integrity and serialization round-trip coverage for the expanded information model.                                                                                    |
| <code>tests/test_acopf_dcopf_crossval.cpp</code>                                                                                                             | DC OPF convergence on MATPOWER cases (IEEE 14/30/57/118/300), objective and dispatch cross-validation against AC OPF, LMP extraction, and PF feasibility check of DC OPF dispatch. |
| <code>tests/test_annual_production_sim.cpp</code>                                                                                                            | Annual production-simulation smoke tests: L0 annual planning, L2 weekly UC, L3 daily replay pipeline, and annual metric aggregation on small benchmark systems.                    |
| <code>tests/test_lifecycle_smoke.cpp</code>                                                                                                                  | Multi-year lifecycle simulation smoke tests: degradation application, stratified sampling, NPV accumulation, and year-by-year result consistency.                                  |

## 23.4 What Is Well Covered Today

The current repository has comparatively strong evidence in the following areas.

- Main PF correctness on representative AC/DC test cases.
- Derivative correctness for both native and parity OPF paths, including finite-difference verification of DCDC converter  $I^2R$  Jacobian and Hessian terms.
- VSC converter loss model consistency between PF and OPF ( $P_{\text{loss}} = a + b I_{ac} + c I_{ac}^2$ ).
- DCDC converter model consistency: efficiency,  $I^2R$  conduction loss, and correct sign-convention handling across PF and OPF solvers.
- OPF→PF cross-validation on hybrid AC/DC cases, confirming that OPF dispatch is PF-feasible within documented tolerances.
- DC OPF vs AC OPF cross-validation on standard MATPOWER cases, confirming dispatch and objective agreement within expected DC-approximation error bounds, and PF feasibility of DC OPF dispatch.
- DPF agreement with Newton PF on radial benchmark feeders.
- ONR feasibility and post-solve verification through the full PF solver.
- Short-circuit self-consistency and small-network analytical checks.

- Input/output round trips for the enlarged public information model.
- Annual production simulation end-to-end pipeline (L0→L2→L3) on small benchmark systems.
- Lifecycle simulation multi-year degradation and NPV consistency.

### 23.5 What Still Needs Better Validation

The code structure itself highlights the most important evidence gaps.

- DPF and ONR now share the canonical projection path, but the repository still lacks explicit projected-vs-canonical equivalence tests for feeders containing transformers, switches, flexible loads, asymmetric loads, and chargers.
- The detailed IEC short-circuit path is now exercised by dedicated tests, but stronger reference-case validation is still needed for explicit external-grid contribution accounting and richer component-by-component breakdown semantics.
- The parity path now instantiates richer component-variable blocks including DCDC converter dispatch with analytical Jacobian and Hessian, but the public OPF export contract still lags the in-memory result struct, so result-I/O regression tests should be extended when those fields are serialized.
- A direct notebook-generation regression target should eventually verify that all documented formulas still match the implementation.
- The reactive-power optimisation (RPO) path has no dedicated test file; convergence and OA gap properties are exercised only indirectly through internal development runs.
- The annual production simulation has smoke-level coverage but no reference-case comparison against an external production-cost tool.
- The lifecycle simulation has smoke-level coverage; stronger validation would include comparison of NPV results against manual spreadsheet calculations on a known degradation curve.

### 23.6 Suggested Interpretation for Users

At the current maturity level, the strongest production path is:

projected main PF / native OPF / parity OPF / DC OPF / batch+detailed SC / annual sim,  
(361)

while DPF and ONR should still be used with awareness of their narrower hybrid overlay coverage and their currently lighter result contracts. This is not a weakness of the numerical cores themselves; it is primarily an interface- and consumer-contract issue between the rich public model layer and specialized distribution-planning workflows.

## 24 Current Project Implementation Audit

### 24.1 Audit Scope and Reading Order

This section is a code-aligned snapshot of the current project state as of May 5, 2026, after the recent refactors to the power-flow and OPF modules. It does not replace the detailed derivations in the individual chapters. Instead, it provides one consolidated ledger that connects the current

mathematical models, algorithms, implementation files, numerical evidence, and code-quality plan for each major module.

The current filesystem convention is:

- `include/hacdcpf/power_flow` and `src/power_flow` for steady-state PF kernels,
- `include/hacdcpf/optimal_power_flow` and `src/optimal_power_flow` for OPF kernels,
- `include/hacdcpf/engine` and `src/engine` for reusable numerical optimization machinery,
- `include/hacdcpf/analysis` and `src/analysis` for secondary analysis modules,
- `include/hacdcpf/planning` and `src/planning` for planning studies and reporting.

The C++ namespace `hacdcpf::powerflow` is intentionally unchanged; only the source and include paths were renamed.

## 24.2 Canonical Mathematical Form Shared by the Project

Most modules are built from one canonical hybrid-system representation. The runtime flow is

$$\text{HybridPowerSystem} \xrightarrow{\text{canonical projection}} \text{SolverData}, \quad (362)$$

$$\text{SolverData} \xrightarrow{\text{matrix / residual build}} \text{PF, OPF, analysis, and planning kernels.} \quad (363)$$

The shared steady-state equations have the compact form

$$F_{\text{pf}}(x) = \begin{bmatrix} P_{\mathcal{N}_{ac} \setminus \mathcal{N}_{sl}}^{\text{spec}}(x) - P_{\mathcal{N}_{ac} \setminus \mathcal{N}_{sl}}^{\text{calc}}(v, \theta) \\ Q_{\mathcal{N}_{PQ}}^{\text{spec}}(x) - Q_{\mathcal{N}_{PQ}}^{\text{calc}}(v, \theta) \\ P_{\mathcal{N}_{dc} \setminus \mathcal{N}_{dc,sl}}^{\text{spec}}(x) - v^{dc} \circ (G_{dc} v^{dc}) \end{bmatrix} = 0, \quad (364)$$

with ZIP demand, VSC converter losses, DCDC efficiency and  $I^2R$  loss, energy router ports, storage, renewable generation, VPPs, microgrids, and charging loads folded into the specified-injection terms. The nonlinear OPF family uses the standard constrained form

$$\min_x f(x) \quad (365)$$

$$\text{s.t. } g(x) = 0, \quad h(x) \leq 0, \quad x^{\min} \leq x \leq x^{\max}, \quad (366)$$

where the parity route exposes a full-space AC/DC formulation, while the native route and fast route use specialized reduced or projected forms. The engine layer exposes LP, QP, NLP, MILP, and MINLP kernels that all reduce to sparse matrix models with explicit bound, equality, and inequality blocks.

## 24.3 Mathematical Model Ledger

| Module             | Current mathematical model                                                                                                                                                                                                                                                                                                                                                                     | Current implementation boundary                                                                                                                                                                                                                                                                                     |
|--------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Model and I/O      | Rich hybrid component graph with AC buses, DC buses, branches, transformers, switches, VSCs, DCDCs, loads, ZIP demand, storage, renewables, VPPs, microgrids, mobile storage, energy routers, chargers, external grids, and metadata. Canonical projection normalizes equivalent structures while retaining merge metadata.                                                                    | <code>include/hacdcpf/model</code> , <code>src/model</code> , <code>include/hacdcpf/io</code> , <code>src/io</code> . JSON, Excel, MATPOWER, OpenDSS bridge, and case builders feed the same public model.                                                                                                          |
| Power flow         | Hybrid AC/DC nonlinear equations $F_{\text{pf}}(x) = 0$ . AC rows use polar $Y_{bus}$ power injections; DC rows use $V(GV)$ ; VSC rows include fixed, linear, or quadratic loss; DCDC rows include bidirectional efficiency, droop, and optional $I^2R$ loss; ZIP loads make demand voltage-dependent.                                                                                         | <code>src/power_flow: solver_data</code> , <code>admittance_builder</code> , <code>residual_evaluator</code> , <code>jacobian_builder</code> , <code>jacobian_pattern</code> , <code>ac_kernel</code> , <code>newton_solver</code> , <code>fdpf_solver</code> , <code>dc_solver</code> , <code>helm_solver</code> . |
| Optimal power flow | Nonlinear AC/DC OPF with generation cost, AC/DC balance, converter balance, voltage limits, generator bounds, branch MVA limits, converter limits, DC branch limits, load shedding, controllable renewables, storage, flexible load, and DCDC decision variables. DC OPF uses B- $\theta$ LP/QP approximations. RPO is a mixed-integer tap/shunt optimization with OPF subproblem evaluations. | <code>src/optimal_power_flow: ac_opf</code> , <code>dc_opf</code> , <code>parity_formulation</code> , <code>parity_ipm</code> , <code>reactive_power_opt</code> .                                                                                                                                                   |
| Engine solvers     | Generic mathematical-programming models: LP/QP/NLP/MILP/MINLP in bound/equality/inequality form. KKT systems, barrier systems, branch-and-bound trees, cut pools, pre-solve/postsolve transforms, simplex bases, and sparse factorization state are explicit data structures.                                                                                                                  | <code>include/hacdcpf/engine</code> , <code>src/engine</code> , and compatibility <code>include/hacdcpf/solver</code> . Native IPM, LP IPM, LCQP, dual simplex, branch-and-cut, PDLP/PMM-style solvers, and adapters.                                                                                               |
| Distribution PF    | Radial distribution feeder equations with backward/forward sweep and three-phase variants. The model tracks branch current, voltage drops, unbalance, DER injections, hybrid overlays, and optional time-series topology changes.                                                                                                                                                              | <code>include/hacdcpf/analysis/distribution_power_flow.hpp</code> , <code>src/analysis/distribution_power_flow.cpp</code> , plus <code>time_series_distribution</code> .                                                                                                                                            |

| Module                                | Current mathematical model                                                                                                                                                                                                                                  | Current implementation boundary                                                                                                                                                          |
|---------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Short circuit                         | Z-bus/Thevenin impedance model with IEC-style correction factors and multiple fault types. Three-phase, single-line-to-ground, line-to-line, and double-line-to-ground faults are represented through sequence-domain or equivalent impedance calculations. | <code>include/hacdcpf/analysis/short_circuit.hpp</code> , <code>src/analysis/short_circuit.cpp</code> .                                                                                  |
| Network reconfiguration               | LinDistFlow-style MILP with binary branch status, radiality/connectivity constraints, voltage-drop constraints, flow limits, load-service variables, switching costs, and post-solve PF feasibility checks.                                                 | <code>include/hacdcpf/analysis/topology_analysis.hpp</code> , <code>src/analysis/topology_analysis.cpp</code> , native branch-and-cut engine.                                            |
| Carbon analysis                       | Directed hybrid carrier graph, proportional tracing, matrix-based bus carbon-intensity solve, storage source/sink treatment, converter and branch loss allocation, and emission aggregation by source/load/carrier.                                         | <code>include/hacdcpf/analysis/carbon_analysis.hpp</code> , <code>src/analysis/carbon_analysis.cpp</code> , <code>visualization/reporting</code> helpers.                                |
| Time series and annual simulation     | Multi-horizon operation model: representative annual planning layer, weekly UC layer, daily replay/validation layer, PF/OPF feasibility checks, emissions and cost aggregation, degradation-aware lifecycle wrapper.                                        | <code>include/hacdcpf/analysis/time_series_pf.hpp</code> , <code>annual_production_sim.hpp</code> , <code>lifecycle_simulation.hpp</code> , and related <code>src/analysis</code> files. |
| Reliability, resilience, MESS         | Reliability/FMEA component catalog, outage/resilience state evolution, mobile energy storage dispatch heuristics, restoration scoring, and service-restoration metrics.                                                                                     | <code>include/hacdcpf/analysis/reliability_assessment.hpp</code> , <code>resilience_assessment.hpp</code> , docs sections 14–15, tests and validation demos.                             |
| Planning, market, stochastic dispatch | Microgrid and hybrid AC/DC distribution planning models; market-clearing and settlement models; two-stage/stochastic dispatch and decomposition formulations with extensive notebook derivations.                                                           | <code>include/hacdcpf/planning</code> , <code>src/planning</code> , <code>include/hacdcpf/market</code> , <code>include/hacdcpf/decomposition</code> , sections 10a, 10b, 16, and 17.    |

## 24.4 Algorithm and Implementation Ledger

| Module                  | Implemented algorithms                                                                                                                                                                                                                                                                   | Important implementation notes                                                                                                                                                                 |
|-------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Model/I/O               | Canonical projection, bus merge mapping, MATPOWER parsing, JSON/Excel round trips, canned case construction, OpenDSS snapshot adapters.                                                                                                                                                  | Projection is the common input boundary for PF/OPF/analysis. Compatibility headers under <code>core</code> and <code>solver</code> remain for older includes.                                  |
| Power flow              | Sparse Newton PF, FDPF, DC-only Newton, adaptive island solve, distributed slack, HELM with sparse $Y_{trans}$ , homotopy continuation, non-linear scaling, nonmonotone line search, LM/PTC trust-region control, Newton-Krylov/GMRES with Schur preconditioning, voltage-stability/CPF. | The AC kernel and Jacobian sparsity construction are split into <code>ac_kernel.cpp</code> and <code>jacobian_pattern.cpp</code> . Shared susceptance builders now serve FDPF $B'$ and $B''$ . |
| Optimal power flow      | Fast economic dispatch plus PF projection, native primal-dual AC/DC OPF, parity full-space formulation, dedicated parity IPM with Mehrotra predictor-corrector and extra correctors, DC OPF backend dispatch, RPO OA search over taps/shunts.                                            | The parity route is the strongest current nonlinear AC/DC OPF path. RPO is self-contained and calls AC OPF repeatedly with a solution cache and OA lower bounds.                               |
| Engine solvers          | Dual simplex, sparse dual simplex, LP IPM, LCQP IPM, native NLP IPM, PDLP/PMM-family solvers, branch-and-cut with presolve, cuts, conflict/domain propagation, warm starts, parallel search and sharing.                                                                                 | The engine is now a real in-tree optimization layer rather than a thin wrapper. Generic native NLP IPM is improving but still lags the parity-specific OPF solver on large OPF-class NLPs.     |
| Distribution PF         | Backward/forward sweep, radial topology traversal, DER/load aggregation, three-phase validation, time-series feeder replay.                                                                                                                                                              | Cross-validation against Newton PF exists for radial cases; hybrid component projection is still the main area needing broader cases.                                                          |
| Short circuit           | Batch all-bus and per-bus fault analysis, Thevenin/Zbus solve, sequence fault formulas, contribution book-keeping.                                                                                                                                                                       | The module is mathematically usable; stronger external-grid and reference standard cases are still needed before calling the reporting contract complete.                                      |
| Network reconfiguration | Native MILP build, branch-and-cut solve, stable branch-id result emission, radially/connectivity checks, post-reconfiguration Newton PF validation.                                                                                                                                      | Stable branch ids are mostly solved; projected/canonical equivalence cases with non-trivial ids still need stronger assertions.                                                                |

| Module                             | Implemented algorithms                                                                                                                               | Important implementation notes                                                                                                                                   |
|------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Carbon analysis                    | Hybrid graph construction, directed flow extraction from PF/OPF, proportional source/load tracing, matrix bus-intensity solve, Sankey/report output. | Current tests cover core carbon accounting; richer time-coupled storage carbon-state validation remains a useful next step.                                      |
| Time series, annual sim, lifecycle | L0/L2/L3 hierarchy, UC/OPF/PF replay, annual metric aggregation, multi-year degradation, stratified sampling, NPV accumulation.                      | Smoke and contract tests exist; external reference-cost cases and spreadsheet lifecycle baselines would strengthen numerical credibility.                        |
| Reliability, resilience, MESS      | Reliability catalog, fault/service metrics, restoration heuristics, mobile storage movement/energy constraints, validation demo reporting.           | The current solver is pragmatic and demonstrative; full MILP/rolling-horizon restoration remains the intended high-fidelity upgrade path.                        |
| Planning, market, stochastic       | Benders/progressive decomposition studies, low-carbon planning cases, market settlement and reserve products, stochastic dispatch summaries.         | These chapters contain strong mathematical material; implementation coverage varies by study and should be documented per executable path as experiments mature. |

## 24.5 Numerical Evidence and Performance Snapshot

The latest release-mode regression snapshot recorded for the core notebook evidence set is summarized below. These are not exhaustive benchmarks; they are the current fast-to-medium confidence gates that demonstrate the refactored modules still execute coherently after the recent source split and directory rename.

| Evidence source                         | Assertions | Interpretation                                                                                                         |
|-----------------------------------------|------------|------------------------------------------------------------------------------------------------------------------------|
| <code>test_all_components_pf</code>     | 159        | Main AC/DC PF component integration: loads, DERs, converters, storage, hybrid buses, and result assembly.              |
| <code>test_annual_production_sim</code> | 405        | Annual production simulation stack: L0/L2/L3 orchestration, metric aggregation, and representative operation pipeline. |
| <code>test_carbon_analysis</code>       | 82         | Hybrid carbon tracing, matrix intensity solve, source/load aggregation, and report-contract evidence.                  |
| <code>test_core_modules</code>          | 11         | Core utility and module smoke checks.                                                                                  |
| <code>test_modeling_toolkit</code>      | 146        | AML/modeling-toolkit variable, expression, bridge, and solver interaction coverage.                                    |
| <code>test_opf_pf_crossval</code>       | 24         | OPF dispatch replay through PF feasibility checks on representative cases.                                             |

| Evidence source                            | Assertions | Interpretation                                                             |
|--------------------------------------------|------------|----------------------------------------------------------------------------|
| test_powerflow_performance                 | 3          | Performance-oriented PF smoke checks for the optimized/refactored kernels. |
| test_problem_types                         | 14         | Engine problem-type contracts and validation paths.                        |
| test_projection_equivalence                | 21         | Canonical projection equivalence checks.                                   |
| <b>Total in this release-mode snapshot</b> | <b>865</b> | <b>All assertions passed in the last recorded run.</b>                     |

Additional numerical evidence is distributed in the detailed chapters:

- Section 12 records the power-flow refactor performance status and the D1–D6 deduplication completion state.
- Section 13 records AC/DC OPF, parity IPM, DC OPF, and RPO formulations.
- Section 5 records structured sparse NLP/IPM and parity-OPF comparison results, including large OPF cases where the parity path remains stronger than the generic native NLP path.
- The 10a microgrid-planning chapter records the corrected numerical study and replay-feasibility evidence.
- Section 29 and the standalone stochastic manuscript record two-stage/stochastic dispatch verification and decomposition behavior.

## 24.6 Code-Quality Investigation by Module

| Module               | Current quality assessment                                                                                                                                                                                                                                | Focused improvement plan                                                                                                                                                 |
|----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Model and projection | Strong public information model and useful canonical projection, but some consumer paths still rely on implicit contiguous bus ids or vector order assumptions.                                                                                           | Add explicit id-to-position mapping utilities, use them in replay/unprojection/export code, and add non-contiguous id regression cases.                                  |
| I/O contracts        | JSON/Excel/MATPOWER coverage is broad, but the runtime model has grown faster than some result/input dictionaries.                                                                                                                                        | Close remaining DC PV/static-generator serialization gaps; extend OPF and PF result exports for new dispatch vectors; add round-trip equality tests for every new table. |
| Power flow           | Numerically central and recently refactored: shared susceptance builder, split Jacobian pattern/kernel/evaluator, and robust nonlinear aids. Main remaining risk is duplicated injection accounting across PF residual, OPF parity, and validation paths. | Keep <code>pf_injection_assembly</code> as the shared DC source of truth; add FD tests for every AC/DC cross derivative; add one stress case per VSC/DCDC control mode.  |

| Module                                    | Current quality assessment                                                                                                                                               | Focused improvement plan                                                                                                                                                                                                       |
|-------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Optimal power flow                        | Parity path is robust and feature-rich; generic native path still has performance and globalization gaps for large NLP-class OPF. RPO lacks dedicated tests.             | Continue parity/native convergence comparison; add RPO regression tests; expose all parity result vectors in output contracts; fix DCDC directional derivative consistency if current-iterate sign and reference sign diverge. |
| Engine solvers                            | LP/MILP center is mature, with strong presolve, warm-start, and branch-and-cut work. Generic NLP IPM is much improved but not yet parity-competitive on large OPF cases. | Stabilize generic IPM globalization and scaling; stratify benchmark-scale tests from fast correctness gates; add per-kernel performance baselines.                                                                             |
| Distribution PF                           | BFS and radial work-flows are usable and cross-validated on representative feeders. Hybrid overlays and projected component views are the weaker edge.                   | Expand cross-validation to transformers, switches, chargers, asymmetric loads, flexible loads, and projected feeder ids.                                                                                                       |
| Short circuit                             | Functional mathematical core with batch/per-bus APIs. Reporting semantics still need richer component-source breakdown.                                                  | Add external-grid contribution bucket; compare against named IEC reference cases; lock per-fault-type expected values.                                                                                                         |
| Network reconfiguration                   | Stable ids and PF post-validation are mostly in place. Branch-id semantics under projection remain the main subtle risk.                                                 | Add projected-vs-canonical branch-id set equality tests and deliberately non-trivial branch indices.                                                                                                                           |
| Carbon analysis                           | Core tracing and matrix methods are covered; current model is strong for static snapshots.                                                                               | Add time-coupled storage carbon-intensity state tests and OPF/PF replay consistency checks for losses.                                                                                                                         |
| Time-series, annual simulation, lifecycle | End-to-end paths exist and have meaningful smoke coverage. They need stronger external reference cases to become benchmark-grade.                                        | Add small reference production-cost case; add lifecycle spreadsheet baseline; validate UC initial state/startup accounting.                                                                                                    |
| Reliability, resilience, MESS             | The documents and demos give a coherent practical workflow; optimization depth is still heuristic in places.                                                             | Add MILP restoration prototype or rolling-horizon benchmark; quantify service-restoration optimality gaps on fixed toy systems.                                                                                                |
| Planning, market, stochastic              | Mathematical documentation is extensive and several studies have numerical artifacts, but implementation maturity varies by sub-study.                                   | For each study, maintain a code map, input contract, reproducible run command, result table, and smoke/reference test.                                                                                                         |

| Module               | Current quality assessment                                                   | Focused improvement plan                                                                                                                  |
|----------------------|------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| Documentation and CI | Notebook coverage is broad but historically lagged fast-changing code paths. | Build the technical notebook in release gates; include the current implementation audit; keep validation tables tied to executable tests. |

## 24.7 Immediate Documentation Contract

For future development, every new module or major refactor should update five documentation surfaces together:

1. the mathematical model section that defines variables, objective, and constraints;
2. the algorithm section that gives code-aligned pseudocode;
3. the implementation map that names source/header files and public APIs;
4. the validation section that names exact tests, benchmarks, and expected outputs;
5. the improvement-plan table that records residual code-quality risks.

This audit section should remain the high-level index, while individual chapters remain the detailed derivation and experiment records.

## 25 Hybrid AC/DC Distribution DER Planning Under Carbon and Peak $N-1$ Reliability Constraints

This section defines a planning model for hybrid AC/DC distribution systems with distributed photovoltaic generation, distributed battery energy storage, and converter-coupled AC/DC power exchange. The model is intended for feeder-level planning rather than transmission expansion. Its distinguishing feature is the joint treatment of annual carbon control, peak-time reliability security, and extreme-event resilience: the normal-operation layer uses weighted representative operating states, the reliability layer uses a peak-snapshot  $N-1$  contingency set evaluated through an FMEA-style frequency-duration model, and the resilience layer uses extreme-state  $N-k$  scenarios with post-fault partitioning and island support.

The linear network model follows the sign conventions and AC/DC variable semantics of Section 12. In particular, bus injections are taken as positive when they inject into the network, the AC feeder is represented by a P/Q LinDistFlow model with voltage-magnitude variables, and the DC subnetwork is written using conductance-based voltage differences.

### 25.1 Study Object and Assumptions

Consider a hybrid distribution system with AC bus set  $\mathcal{N}^{ac}$ , DC bus set  $\mathcal{N}^{dc}$ , AC branch set  $\mathcal{E}^{ac}$ , DC branch set  $\mathcal{E}^{dc}$ , and voltage-source converter set  $\mathcal{E}^{vsc}$ . Let  $\mathcal{N} = \mathcal{N}^{ac} \cup \mathcal{N}^{dc}$ . The planning horizon is one representative year.

Normal operation is represented by a weighted set of operating snapshots  $\Omega$ , with weight  $\pi_\omega \geq 0$  and equivalent duration  $\Delta_\omega$  hours, satisfying

$$\sum_{\omega \in \Omega} \pi_\omega = 1, \quad \sum_{\omega \in \Omega} \pi_\omega \Delta_\omega = 8760. \quad (367)$$

Among these snapshots, let  $\omega^{pk} \in \Omega$  denote the system peak snapshot used for reliability planning.

Let  $\Xi$  denote a finite library of low-probability extreme states, such as storm, wildfire, flood, or heat-driven common-mode outage conditions. Each  $\xi \in \Xi$  has scenario weight  $\varpi_\xi \geq 0$ , and for each extreme state let  $\mathcal{K}_\xi$  denote the associated  $N-k$  contingency family induced by that event.

Reliability is enforced through a single-contingency set  $\mathcal{C}$  covering the AC/DC components to be protected under an  $N-1$  criterion. Only the peak snapshot is analyzed under contingencies. For each failed component  $k \in \mathcal{C}$ , let  $\lambda_k$  denote annual failure frequency and let  $\tau_k$  denote the equivalent outage duration used in the peak-time FMEA approximation.

The following assumptions define the intended model scope.

**Assumption 1** (Linear hybrid network model). *The planning model uses a linearized AC/DC power-flow approximation consistent with the variable conventions in Section 12. The AC feeder is assumed radial or radially operated and is represented by a P/Q LinDistFlow approximation with branch active-power, branch reactive-power, and squared-voltage variables. DC active-power flow is represented by conductance-weighted DC-voltage differences, and each AC/DC converter is treated as a lossless controllable interlinking device.*

**Assumption 2** (DER-assisted voltage regulation). *Reactive support may be supplied by PV inverters, BESS inverters, AC/DC converters, and upstream substations through linear box-type capability envelopes. Hence the AC voltage profile is regulated jointly by network reinforcement and controllable DER-side reactive power rather than by substations alone.*

**Assumption 3** (Hierarchical reliability screening). *The reliability architecture has two security layers. The first evaluates only the peak snapshot  $\omega^{\text{pk}}$  under  $N-1$  contingencies. The second evaluates extreme states  $\xi \in \Xi$  under event-induced  $N-k$  contingencies, where post-fault partitioning and islanded operation are allowed.*

**Assumption 4** (Tree-structured partition restoration). *Under extreme states, surviving substations and selected grid-forming BESS units may energize multiple supply partitions. Each energized partition must remain radial, so the post-fault topology is a forest rooted at activated substations or activated grid-forming BESS sources.*

**Assumption 5** (FMEA weighting). *Reliability impact is measured through a frequency-duration-weighted peak load curtailment functional, consistent with the deterministic FMEA logic in Section 18.*

**Assumption 6** (Snapshot storage approximation). *The planning model is not a chronological unit-commitment problem. Storage energy limits are therefore enforced by linear snapshot energy envelopes based on effective discharge/charge durations and a prescribed pre-contingency state-of-charge factor at the peak snapshot.*

## 25.2 Sets, Parameters, and Planning Variables

### 25.2.1 Network and device sets

$$\mathcal{R}^{ac} \subseteq \mathcal{N}^{ac} \quad \text{AC root or substation buses,} \quad (368)$$

$$\mathcal{R}^{dc} \subseteq \mathcal{N}^{dc} \quad \text{DC root or substation buses,} \quad (369)$$

$$\mathcal{N}^{pv} \subseteq \mathcal{N} \quad \text{candidate PV buses,} \quad (370)$$

$$\mathcal{N}^{bess} \subseteq \mathcal{N} \quad \text{candidate BESS buses,} \quad (371)$$

$$\mathcal{N}^{gfm} \subseteq \mathcal{N}^{bess} \cap \mathcal{N}^{ac} \quad \text{candidate AC-side grid-forming BESS buses,} \quad (372)$$

$$\mathcal{N}^{crit} \subseteq \mathcal{N} \quad \text{critical-load buses,} \quad (373)$$

$$\delta_i^-, \delta_i^+ \quad \text{incoming and outgoing incident branches of bus } i. \quad (374)$$

For each converter  $c \in \mathcal{E}^{vsc}$ , let  $i(c) \in \mathcal{N}^{ac}$  be the associated AC bus and  $k(c) \in \mathcal{N}^{dc}$  be the associated DC bus.

For each AC branch  $e = (i, j) \in \mathcal{E}^{ac}$ , the branch orientation is taken from the upstream bus  $i$  to the downstream bus  $j$ , consistent with a radial feeder embedding used by LinDistFlow.

### 25.2.2 Planning variables

The first-stage decision vector is

$$x := \left( \overline{P}_i^{pv} \right)_{i \in \mathcal{N}^{pv}}, \left( \overline{P}_i^{bess}, \overline{E}_i^{bess} \right)_{i \in \mathcal{N}^{bess}}, \left( \overline{S}_c^{vsc} \right)_{c \in \mathcal{E}^{vsc}}, \quad (375)$$

$$\left( \overline{P}_r^{sub} \right)_{r \in \mathcal{R}^{ac} \cup \mathcal{R}^{dc}}, \zeta^{CO_2}, \zeta^{rel}, \zeta^{ext}, \quad (376)$$

where the components represent PV capacity, BESS power capacity, BESS energy capacity, converter capacity, substation import capability, carbon slack, peak-reliability slack, and extreme-resilience slack.

The annualized investment cost is written as

$$C^{inv}(x) = \sum_{i \in \mathcal{N}^{pv}} \hat{K}_i^{pv} \overline{P}_i^{pv} + \sum_{i \in \mathcal{N}^{bess}} \left( \hat{K}_i^{bp} \overline{P}_i^{bess} + \hat{K}_i^{be} \overline{E}_i^{bess} \right) \quad (377)$$

$$+ \sum_{c \in \mathcal{E}^{vsc}} \hat{K}_c^{vsc} \overline{S}_c^{vsc} + \sum_{r \in \mathcal{R}^{ac} \cup \mathcal{R}^{dc}} \hat{K}_r^{sub} \overline{P}_r^{sub}. \quad (378)$$

## 25.3 Three-Layer Planning Model

The planning model contains three recourse layers:

1. a normal-operation layer over  $\omega \in \Omega$ , used for expected operating cost and annual carbon accounting;
2. a peak-contingency layer over  $k \in \mathcal{C}$ , used for  $N-1$  reliability enforcement by FMEA weighting;
3. an extreme-state resilience layer over  $(\xi, \kappa) \in \Xi \times \mathcal{K}_\xi$ , used for  $N-k$  resilience enforcement under partitioned post-fault operation. In the current code base, this layer has both a legacy heuristic restoration simulator and a new strict multi-period MIP/LinDistFlow restoration skeleton for unexpected-fault replacement.

The complete planning problem is

$$\min_{x, y, z} \quad C^{inv}(x) + \sum_{\omega \in \Omega} \pi_\omega Q^{nor}(x, \omega) + \rho_{CO_2} \zeta^{CO_2} + \rho_{rel} \zeta^{rel} + \rho_{ext} \zeta^{ext} \quad (379)$$

$$\text{s.t.} \quad x \in \mathcal{X}, \quad (380)$$

$$\sum_{\omega \in \Omega} \pi_\omega G^{CO_2}(x, \omega) \leq \overline{E}_{CO_2}^{ann} + \zeta^{CO_2}, \quad (381)$$

$$\sum_{k \in \mathcal{C}} \lambda_k \tau_k H_k^{pk}(x) \leq \overline{E}_{rel}^{pk} + \zeta^{rel}, \quad (382)$$

$$\sum_{\xi \in \Xi} \varpi_\xi \sum_{\kappa \in \mathcal{K}_\xi} \lambda_{\xi, \kappa} \tau_{\xi, \kappa} H_{\xi, \kappa}^{ext}(x) \leq \overline{E}_{rel}^{ext} + \zeta^{ext}, \quad (383)$$

$$\zeta^{CO_2} \geq 0, \quad \zeta^{rel} \geq 0, \quad \zeta^{ext} \geq 0. \quad (384)$$

Here  $Q^{nor}(x, \omega)$  is normal-state operating cost,  $G^{CO_2}(x, \omega)$  is snapshot carbon contribution,  $H_k^{pk}(x)$  is peak-snapshot unsupplied energy under contingency  $k$ , and  $H_{\xi, \kappa}^{ext}(x)$  is unsupplied

energy under extreme state  $\xi$  and higher-order contingency  $\kappa$ . The scalar  $\overline{E}_{CO_2}^{ann}$  is the annual carbon budget,  $\overline{E}_{rel}^{pk}$  is an admissible FMEA-weighted peak EENS threshold, and  $\overline{E}_{rel}^{ext}$  is the resilience budget for extreme-state  $N-k$  events.

## 25.4 Normal-Operation Recourse Model

### 25.4.1 Operating variables

For each operating snapshot  $\omega \in \Omega$ , the normal-state recourse variables are

$$y_\omega := \left( f_{e,\omega}^{ac}, q_{e,\omega}^{ac}, u_{i,\omega}^{ac}, f_{\ell,\omega}^{dc}, v_{k,\omega}^{dc}, p_{c,\omega}^{vsc}, q_{c,\omega}^{vsc}, \right. \quad (385)$$

$$\left. p_{i,\omega}^{pv}, q_{i,\omega}^{pv}, p_{i,\omega}^{bess,ch}, p_{i,\omega}^{bess,dis}, q_{i,\omega}^{bess}, p_{r,\omega}^{sub,+}, p_{r,\omega}^{sub,-}, q_{r,\omega}^{sub} \right), \quad (386)$$

where  $p_{c,\omega}^{vsc} \in \mathbb{R}$  is the signed converter transfer. A positive value denotes active-power transfer from the DC side to the AC side, while a negative value denotes transfer from the AC side to the DC side.

### 25.4.2 Operating objective

For fixed  $(x, \omega)$ , define

$$Q^{nor}(x, \omega) := \min_{y_\omega} \sum_{r \in \mathcal{R}^{ac} \cup \mathcal{R}^{dc}} \left( c_{r,\omega}^{buy} p_{r,\omega}^{sub,+} - c_{r,\omega}^{sell} p_{r,\omega}^{sub,-} \right) \quad (387)$$

$$+ \sum_{i \in \mathcal{N}^{bess}} c_i^{bess} \left( p_{i,\omega}^{bess,ch} + p_{i,\omega}^{bess,dis} \right) \quad (388)$$

subject to the following constraints.

**PV and storage capability.** For all  $i \in \mathcal{N}^{pv} \cap \mathcal{N}^{bess}$  and  $\omega \in \Omega$ ,

$$0 \leq p_{i,\omega}^{pv} \leq \alpha_{i,\omega}^{pv} \overline{P}_i^{pv}, \quad (389)$$

$$-\overline{\gamma}_i^{pv} \alpha_{i,\omega}^{pv} \overline{P}_i^{pv} \leq q_{i,\omega}^{pv} \leq \overline{\gamma}_i^{pv} \alpha_{i,\omega}^{pv} \overline{P}_i^{pv}, \quad (390)$$

$$0 \leq p_{i,\omega}^{bess,ch} \leq \overline{P}_i^{bess}, \quad (391)$$

$$0 \leq p_{i,\omega}^{bess,dis} \leq \overline{P}_i^{bess}, \quad (392)$$

$$-\overline{\gamma}_i^{bess} \overline{P}_i^{bess} \leq q_{i,\omega}^{bess} \leq \overline{\gamma}_i^{bess} \overline{P}_i^{bess}, \quad (393)$$

$$p_{i,\omega}^{bess,ch} \Delta_\omega \leq (1 - \underline{s}_i) \overline{E}_i^{bess}, \quad (394)$$

$$p_{i,\omega}^{bess,dis} \Delta_\omega \leq \overline{s}_i \overline{E}_i^{bess}. \quad (395)$$

If a candidate bus hosts only PV or only BESS, the irrelevant variables are omitted.

**Substation capacity.** For each root bus  $r$ ,

$$0 \leq p_{r,\omega}^{sub,+} \leq \overline{P}_r^{sub}, \quad 0 \leq p_{r,\omega}^{sub,-} \leq \overline{P}_r^{sub}. \quad (396)$$

For each AC root bus  $r \in \mathcal{R}^{ac}$ ,

$$-\overline{Q}_r^{sub} \leq q_{r,\omega}^{sub} \leq \overline{Q}_r^{sub}. \quad (397)$$

**Lossless converter capability.** For each converter  $c \in \mathcal{E}^{vsc}$ ,

$$-\overline{S}_c^{vsc} \leq p_{c,\omega}^{vsc} \leq \overline{S}_c^{vsc}. \quad (398)$$

On the AC side, the converter may also provide reactive support within a linear capability box:

$$-\overline{\gamma}_c^{vsc} \overline{S}_c^{vsc} \leq q_{c,\omega}^{vsc} \leq \overline{\gamma}_c^{vsc} \overline{S}_c^{vsc}. \quad (399)$$

Under the lossless approximation, the converter contributes equal and opposite injections on the two subnetworks. Define

$$p_{c,\omega}^{vsc,ac} = p_{c,\omega}^{vsc}, \quad (400)$$

$$p_{c,\omega}^{vsc,dc} = -p_{c,\omega}^{vsc}. \quad (401)$$

The converter reactive injection appears only on the AC side and is denoted by

$$q_{c,\omega}^{vsc,ac} = q_{c,\omega}^{vsc}. \quad (402)$$

Hence the converter only transfers active power between the AC and DC subnetworks, while AC-side reactive support is supplied locally by its converter station.

**Linear AC P/Q LinDistFlow constraints.** For each AC branch  $e = (i, j) \in \mathcal{E}^{ac}$ ,

$$u_{j,\omega}^{ac} = u_{i,\omega}^{ac} - 2(r_e^{ac} f_{e,\omega}^{ac} + x_e^{ac} q_{e,\omega}^{ac}), \quad (403)$$

$$-\overline{F}_e^{ac} \leq f_{e,\omega}^{ac} \leq \overline{F}_e^{ac}, \quad (404)$$

$$-\overline{Q}_e^{ac} \leq q_{e,\omega}^{ac} \leq \overline{Q}_e^{ac}, \quad (405)$$

$$\underline{u}_i^{ac} \leq u_{i,\omega}^{ac} \leq \overline{u}_i^{ac}. \quad (406)$$

For each AC root bus  $r \in \mathcal{R}^{ac}$ , the feeder head is fixed at

$$u_{r,\omega}^{ac} = u_r^{ref}. \quad (407)$$

Here  $u_{i,\omega}^{ac}$  denotes the squared voltage magnitude at AC bus  $i$ . The P/Q LinDistFlow approximation retains both active- and reactive-power contributions to voltage drop, while neglecting the higher-order current and loss terms of the full nonlinear DistFlow equations.

**Linear DC branch flow.** For each DC branch  $\ell = (i, j) \in \mathcal{E}^{dc}$ ,

$$f_{\ell,\omega}^{dc} = g_\ell^{dc} (v_{i,\omega}^{dc} - v_{j,\omega}^{dc}), \quad (408)$$

$$-\overline{F}_\ell^{dc} \leq f_{\ell,\omega}^{dc} \leq \overline{F}_\ell^{dc}, \quad (409)$$

$$\underline{v}_i^{dc} \leq v_{i,\omega}^{dc} \leq \overline{v}_i^{dc}. \quad (410)$$

**AC nodal balance.** For each AC bus  $i \in \mathcal{N}^{ac}$ ,

$$\sum_{e \in \delta_i^- \cap \mathcal{E}^{ac}} f_{e,\omega}^{ac} - \sum_{e \in \delta_i^+ \cap \mathcal{E}^{ac}} f_{e,\omega}^{ac} + p_{i,\omega}^{inj,ac} = 0, \quad (411)$$

where the net AC injection is

$$p_{i,\omega}^{inj,ac} := \mathbf{1}_{\{i \in \mathcal{N}^{pv}\}} p_{i,\omega}^{pv} + \mathbf{1}_{\{i \in \mathcal{N}^{bess}\}} \left( p_{i,\omega}^{bess,dis} - p_{i,\omega}^{bess,ch} \right) \quad (412)$$

$$+ \mathbf{1}_{\{i \in \mathcal{R}^{ac}\}} \left( p_{i,\omega}^{sub,+} - p_{i,\omega}^{sub,-} \right) + \sum_{c:i(c)=i} p_{c,\omega}^{vsc,ac} - P_{i,\omega}^{load,ac}. \quad (413)$$

For each AC bus  $i \in \mathcal{N}^{ac}$ , reactive balance is enforced by

$$\sum_{e \in \delta_i^- \cap \mathcal{E}^{ac}} q_{e,\omega}^{ac} - \sum_{e \in \delta_i^+ \cap \mathcal{E}^{ac}} q_{e,\omega}^{ac} + q_{i,\omega}^{inj,ac} = 0, \quad (414)$$

where

$$q_{i,\omega}^{inj,ac} := \mathbf{1}_{\{i \in \mathcal{N}^{pv}\}} q_{i,\omega}^{pv} + \mathbf{1}_{\{i \in \mathcal{N}^{bess}\}} q_{i,\omega}^{bess} + \mathbf{1}_{\{i \in \mathcal{R}^{ac}\}} q_{i,\omega}^{sub} + \sum_{c: i(c)=i} q_{c,\omega}^{vsc,ac} - Q_{i,\omega}^{load,ac}. \quad (415)$$

This equation explicitly captures DER-assisted voltage regulation through PV, BESS, VSC, and substation reactive-power decisions.

**DC nodal balance.** For each DC bus  $k \in \mathcal{N}^{dc}$ ,

$$\sum_{\ell \in \delta_k^- \cap \mathcal{E}^{dc}} f_{\ell,\omega}^{dc} - \sum_{\ell \in \delta_k^+ \cap \mathcal{E}^{dc}} f_{\ell,\omega}^{dc} + p_{k,\omega}^{inj,dc} = 0, \quad (416)$$

where

$$p_{k,\omega}^{inj,dc} := \mathbf{1}_{\{k \in \mathcal{N}^{pv}\}} p_{k,\omega}^{pv} + \mathbf{1}_{\{k \in \mathcal{N}^{bess}\}} \left( p_{k,\omega}^{bess,dis} - p_{k,\omega}^{bess,ch} \right) \quad (417)$$

$$+ \mathbf{1}_{\{k \in \mathcal{R}^{dc}\}} \left( p_{k,\omega}^{sub,+} - p_{k,\omega}^{sub,-} \right) + \sum_{c: k(c)=k} p_{c,\omega}^{vsc,dc} - P_{k,\omega}^{load,dc}. \quad (418)$$

### 25.4.3 Carbon accounting

The annual carbon functional is defined from normal-state substation imports:

$$G^{CO_2}(x, \omega) := \Delta_\omega \sum_{r \in \mathcal{R}^{ac} \cup \mathcal{R}^{dc}} \kappa_{r,\omega} p_{r,\omega}^{sub,+}, \quad (419)$$

where  $\kappa_{r,\omega}$  is the emission factor associated with upstream supply at root bus  $r$  during snapshot  $\omega$ . Export is not credited unless a policy-adjusted negative emission factor is explicitly adopted.

## 25.5 Peak-Time $N-1$ Reliability Model

### 25.5.1 Contingency variables

For each contingency  $k \in \mathcal{C}$ , the peak-time recourse vector is

$$z_k := \left( f_{e,k}^{ac}, q_{e,k}^{ac}, u_{i,k}^{ac}, f_{\ell,k}^{dc}, v_{m,k}^{dc}, p_{c,k}^{vsc}, q_{c,k}^{vsc}, \right. \quad (420)$$

$$\left. p_{i,k}^{pv}, q_{i,k}^{pv}, p_{i,k}^{bess,ch}, p_{i,k}^{bess,dis}, q_{i,k}^{bess}, p_{r,k}^{sub,+}, p_{r,k}^{sub,-}, q_{r,k}^{sub}, p_{n,k}^{ls} \right), \quad (421)$$

where  $p_{n,k}^{ls} \geq 0$  is curtailed load at peak bus  $n$  under contingency  $k$ .

Let  $\zeta_{e,k}^{ac}$ ,  $\zeta_{\ell,k}^{dc}$ , and  $\zeta_{c,k}^{vsc}$  be binary availability parameters equal to zero if the corresponding asset fails in contingency  $k$ , and one otherwise.

### 25.5.2 Peak-contingency constraints

The peak-time post-contingency model reuses the normal-state equations with the peak data  $(P_{\omega^{pk}}^{load}, Q_{\omega^{pk}}^{load}, \alpha_{\omega^{pk}}^{pv})$ , but modifies device bounds by contingency availability.

**Contingency network limits.** For each AC branch  $e \in \mathcal{E}^{ac}$ , DC branch  $\ell \in \mathcal{E}^{dc}$ , and converter  $c \in \mathcal{E}^{vsc}$ ,

$$-M_e^{ac}(1 - \zeta_{e,k}^{ac}) \leq u_{j,k}^{ac} - u_{i,k}^{ac} + 2(r_e^{ac} f_{e,k}^{ac} + x_e^{ac} q_{e,k}^{ac}) \leq M_e^{ac}(1 - \zeta_{e,k}^{ac}), \quad (422)$$

$$-\zeta_{e,k}^{ac} \overline{F}_e^{ac} \leq f_{e,k}^{ac} \leq \zeta_{e,k}^{ac} \overline{F}_e^{ac}, \quad (423)$$

$$-\zeta_{e,k}^{ac} \overline{Q}_e^{ac} \leq q_{e,k}^{ac} \leq \zeta_{e,k}^{ac} \overline{Q}_e^{ac}, \quad (424)$$

$$\underline{u}_i^{ac} \leq u_{i,k}^{ac} \leq \overline{u}_i^{ac}, \quad (425)$$

$$-M_\ell^{dc}(1 - \zeta_{\ell,k}^{dc}) \leq f_{\ell,k}^{dc} - g_\ell^{dc}(v_{i,k}^{dc} - v_{j,k}^{dc}) \leq M_\ell^{dc}(1 - \zeta_{\ell,k}^{dc}), \quad (426)$$

$$-\zeta_{\ell,k}^{dc} \overline{F}_\ell^{dc} \leq f_{\ell,k}^{dc} \leq \zeta_{\ell,k}^{dc} \overline{F}_\ell^{dc}, \quad (427)$$

$$-\zeta_{c,k}^{vsc} \overline{S}_c^{vsc} \leq p_{c,k}^{vsc} \leq \zeta_{c,k}^{vsc} \overline{S}_c^{vsc}, \quad (428)$$

$$-\zeta_{c,k}^{vsc} \overline{\gamma}_c^{vsc} \overline{S}_c^{vsc} \leq q_{c,k}^{vsc} \leq \zeta_{c,k}^{vsc} \overline{\gamma}_c^{vsc} \overline{S}_c^{vsc}. \quad (429)$$

For each AC root bus  $r \in \mathcal{R}^{ac}$ ,

$$u_{r,k}^{ac} = u_r^{ref}, \quad (430)$$

$$-\overline{Q}_r^{sub} \leq q_{r,k}^{sub} \leq \overline{Q}_r^{sub}. \quad (431)$$

The big- $M$  terms decouple post-contingency voltage-drop equations from outaged AC or DC branches. This prevents a failed line from imposing an artificial algebraic tie between the voltages of buses that should no longer be connected by that asset.

**Peak PV and BESS envelopes.** For all candidate DER buses,

$$0 \leq p_{i,k}^{pv} \leq \alpha_{i,\omega^{pk}}^{pv} \overline{P}_i^{pv}, \quad (432)$$

$$-\overline{\gamma}_i^{pv} \alpha_{i,\omega^{pk}}^{pv} \overline{P}_i^{pv} \leq q_{i,k}^{pv} \leq \overline{\gamma}_i^{pv} \alpha_{i,\omega^{pk}}^{pv} \overline{P}_i^{pv}, \quad (433)$$

$$0 \leq p_{i,k}^{bess,ch} \leq \overline{P}_i^{bess}, \quad 0 \leq p_{i,k}^{bess,dis} \leq \overline{P}_i^{bess}, \quad (434)$$

$$-\overline{\gamma}_i^{bess} \overline{P}_i^{bess} \leq q_{i,k}^{bess} \leq \overline{\gamma}_i^{bess} \overline{P}_i^{bess}, \quad (435)$$

$$p_{i,k}^{bess,dis} \tau_k \leq s_i^{pk} \overline{E}_i^{bess}, \quad p_{i,k}^{bess,ch} \tau_k \leq (1 - s_i^{pk}) \overline{E}_i^{bess}, \quad (436)$$

where  $s_i^{pk} \in [\underline{s}_i, \overline{s}_i]$  is a prescribed pre-contingency state-of-charge factor at the peak snapshot.

**Peak nodal balance with curtailment.** For each AC bus  $i \in \mathcal{N}^{ac}$ , active and reactive balances are enforced by

$$\sum_{e \in \delta_i^- \cap \mathcal{E}^{ac}} f_{e,k}^{ac} - \sum_{e \in \delta_i^+ \cap \mathcal{E}^{ac}} f_{e,k}^{ac} + p_{i,k}^{inj,pk} + p_{i,k}^{ls} = 0, \quad (437)$$

$$\sum_{e \in \delta_i^- \cap \mathcal{E}^{ac}} q_{e,k}^{ac} - \sum_{e \in \delta_i^+ \cap \mathcal{E}^{ac}} q_{e,k}^{ac} + q_{i,k}^{inj,pk} + \eta_i^{Q,load} p_{i,k}^{ls} = 0, \quad (438)$$

where the peak active and reactive injections are

$$p_{i,k}^{inj,pk} := \mathbf{1}_{\{i \in \mathcal{N}^{pv}\}} p_{i,k}^{pv} + \mathbf{1}_{\{i \in \mathcal{N}^{bess}\}} \left( p_{i,k}^{bess,dis} - p_{i,k}^{bess,ch} \right) \quad (439)$$

$$+ \mathbf{1}_{\{i \in \mathcal{R}^{ac}\}} \left( p_{i,k}^{sub,+} - p_{i,k}^{sub,-} \right) + \sum_{c:i(c)=i} p_{c,k}^{vsc,ac} - p_{i,\omega^{pk}}^{load,ac}, \quad (440)$$

$$q_{i,k}^{inj,pk} := \mathbf{1}_{\{i \in \mathcal{N}^{pv}\}} q_{i,k}^{pv} + \mathbf{1}_{\{i \in \mathcal{N}^{bess}\}} q_{i,k}^{bess} + \mathbf{1}_{\{i \in \mathcal{R}^{ac}\}} q_{i,k}^{sub} + \sum_{c:i(c)=i} q_{c,k}^{vsc} - q_{i,\omega^{pk}}^{load,ac}. \quad (441)$$

For each DC bus  $m \in \mathcal{N}^{dc}$ , active balance remains

$$\sum_{\ell \in \delta_m^- \cap \mathcal{E}^{dc}} f_{\ell,k}^{dc} - \sum_{\ell \in \delta_m^+ \cap \mathcal{E}^{dc}} f_{\ell,k}^{dc} + p_{m,k}^{inj,pk} + p_{m,k}^{ls} = 0, \quad (442)$$

with

$$p_{m,k}^{inj,pk} := \mathbf{1}_{\{m \in \mathcal{N}^{pv}\}} p_{m,k}^{pv} + \mathbf{1}_{\{m \in \mathcal{N}^{bess}\}} \left( p_{m,k}^{bess,dis} - p_{m,k}^{bess,ch} \right) \quad (443)$$

$$+ \mathbf{1}_{\{m \in \mathcal{R}^{dc}\}} \left( p_{m,k}^{sub,+} - p_{m,k}^{sub,-} \right) + \sum_{c:k(c)=m} p_{c,k}^{vsc,dc} - P_{m,\omega^{pk}}^{load,dc}. \quad (444)$$

The coefficient  $\eta_i^{Q,load}$  is the fixed reactive-to-active load ratio used to translate active load shedding into proportional reactive-demand reduction at AC bus  $i$ .

**Critical-load  $N-1$  security.** For each critical-load bus  $n \in \mathcal{N}^{crit}$  and each contingency  $k \in \mathcal{C}$ ,

$$p_{n,k}^{ls} = 0. \quad (445)$$

This is the strict  $N-1$  condition for the critical load set. If full-load  $N-1$  service is required, (445) may be imposed for all buses.

### 25.5.3 FMEA-weighted reliability metric

The peak-contingency unsupplied-energy functional is

$$H_k^{pk}(x) := \sum_{n \in \mathcal{N}} p_{n,k}^{ls}. \quad (446)$$

The reliability constraint (382) therefore becomes

$$\sum_{k \in \mathcal{C}} \lambda_k \tau_k \sum_{n \in \mathcal{N}} p_{n,k}^{ls} \leq \bar{E}_{rel}^{pk} + \zeta^{rel}. \quad (447)$$

Equation (447) is a peak-only FMEA proxy for expected annual unserved energy. It is more informative than a pure deterministic no-shed rule because it retains failure-frequency and repair-duration information, yet it is far cheaper than full chronological contingency simulation.

## 25.6 Extreme-State $N-k$ Resilience Model

The third recourse layer addresses low-probability, high-impact states  $\xi \in \Xi$  in which weather, wildfire, flood, or cascading damage can induce simultaneous outages. For each extreme state  $\xi$ , let  $\mathcal{K}_\xi$  denote the associated  $N-k$  contingency family. Unlike the peak  $N-1$  layer, the extreme-state layer permits network reconfiguration, multiple supply partitions, and grid-forming operation of selected BESS units.

### 25.6.1 Extreme-state variables

For each  $(\xi, \kappa) \in \Xi \times \mathcal{K}_\xi$ , define the resilience recourse vector

$$w_{\xi,\kappa} := \left( \beta_{a,\xi,\kappa}, F_{a,\xi,\kappa}, y_{n,\xi,\kappa}^{en}, z_{s,\xi,\kappa}^{src}, z_{i,\xi,\kappa}^{gfm}, \right. \quad (448)$$

$$f_{e,\xi,\kappa}^{ac}, q_{e,\xi,\kappa}^{ac}, u_{i,\xi,\kappa}^{ac}, f_{\ell,\xi,\kappa}^{dc}, v_{m,\xi,\kappa}^{dc}, \quad (449)$$

$$p_{c,\xi,\kappa}^{vsc}, q_{c,\xi,\kappa}^{vsc}, p_{i,\xi,\kappa}^{pv}, q_{i,\xi,\kappa}^{pv}, p_{i,\xi,\kappa}^{bess,ch}, p_{i,\xi,\kappa}^{bess,dis}, q_{i,\xi,\kappa}^{bess}, \quad (450)$$

$$p_{r,\xi,\kappa}^{sub,+}, p_{r,\xi,\kappa}^{sub,-}, q_{r,\xi,\kappa}^{sub}, p_{n,\xi,\kappa}^{ls} \left. \right), \quad (451)$$

where  $\beta_{a,\xi,\kappa} \in \{0,1\}$  is the energized status of hybrid network asset  $a \in \mathcal{E}^{ac} \cup \mathcal{E}^{dc} \cup \mathcal{E}^{vsc}$ ,  $y_{n,\xi,\kappa}^{en} \in \{0,1\}$  indicates whether bus  $n$  is energized,  $z_{s,\xi,\kappa}^{src} \in \{0,1\}$  activates a source partition rooted at substation or grid-forming BESS source  $s$ , and  $z_{i,\xi,\kappa}^{gfm} \in \{0,1\}$  activates BESS  $i \in \mathcal{N}^{gfm}$  in grid-forming mode.

### 25.6.2 Partition topology and source activation

Let  $\mathcal{S}^{src} := \mathcal{R}^{ac} \cup \mathcal{R}^{dc} \cup \mathcal{N}^{gfm}$  be the candidate source set. Introduce a super-root node 0 and virtual source arcs  $\mathcal{E}^{sup} = \{(0, s) : s \in \mathcal{S}^{src}\}$ . Then tree-structured partitioning is imposed on the augmented graph by

$$\beta_{a,\xi,\kappa} \leq \zeta_{a,\xi,\kappa}, \quad \forall a \in \mathcal{E}^{ac} \cup \mathcal{E}^{dc} \cup \mathcal{E}^{vsc}, \quad (452)$$

$$\sum_a \beta_{a,\xi,\kappa} + \sum_{s \in \mathcal{S}^{src}} z_{s,\xi,\kappa}^{src} = \sum_{n \in \mathcal{N}} y_{n,\xi,\kappa}^{en}, \quad (453)$$

$$0 \leq F_{a,\xi,\kappa} \leq M_a^F \beta_{a,\xi,\kappa}, \quad \forall a \in \mathcal{E}^{ac} \cup \mathcal{E}^{dc} \cup \mathcal{E}^{vsc}, \quad (454)$$

$$0 \leq F_{0s,\xi,\kappa} \leq M_s^F z_{s,\xi,\kappa}^{src}, \quad \forall s \in \mathcal{S}^{src}, \quad (455)$$

$$\sum_{a \in \delta_n^- \cap (\mathcal{E} \cup \mathcal{E}^{sup})} F_{a,\xi,\kappa} - \sum_{a \in \delta_n^+ \cap (\mathcal{E} \cup \mathcal{E}^{sup})} F_{a,\xi,\kappa} = y_{n,\xi,\kappa}^{en}, \quad \forall n \in \mathcal{N}, \quad (456)$$

$$\sum_{s \in \mathcal{S}^{src}} F_{0s,\xi,\kappa} = \sum_{n \in \mathcal{N}} y_{n,\xi,\kappa}^{en}. \quad (457)$$

Equations (453)–(457) guarantee that the energized post-fault topology is a forest of radial partitions, each connected to exactly one activated source.

For grid-forming BESS activation,

$$z_{i,\xi,\kappa}^{src} \leq z_{i,\xi,\kappa}^{gfm} \leq y_{i,\xi,\kappa}^{en}, \quad \forall i \in \mathcal{N}^{gfm}, \quad (458)$$

$$0 \leq p_{i,\xi,\kappa}^{bess,dis} \leq \bar{P}_i^{gfm} z_{i,\xi,\kappa}^{gfm}, \quad \forall i \in \mathcal{N}^{gfm}, \quad (459)$$

$$-\bar{Q}_i^{gfm} z_{i,\xi,\kappa}^{gfm} \leq q_{i,\xi,\kappa}^{bess} \leq \bar{Q}_i^{gfm} z_{i,\xi,\kappa}^{gfm}, \quad \forall i \in \mathcal{N}^{gfm}, \quad (460)$$

$$p_{i,\xi,\kappa}^{bess,dis} \tau_{\xi,\kappa} \leq s_i^{ext} \bar{E}_i^{bess}, \quad \forall i \in \mathcal{N}^{gfm}. \quad (461)$$

Thus a BESS can form a partition only when it is explicitly activated as a grid-forming source and has sufficient discharge energy to support that partition during the extreme event.

### 25.6.3 Extreme-state electrical and load-service constraints

Conditional on  $\beta_{a,\xi,\kappa}$ , the AC/DC power-flow relations are enforced explicitly. For AC branches  $e = (i, j) \in \mathcal{E}^{ac}$ , DC branches  $\ell = (i, j) \in \mathcal{E}^{dc}$ , and converters  $c \in \mathcal{E}^{vsc}$ ,

$$-M_e^{ac}(1 - \beta_{e,\xi,\kappa}) \leq u_{j,\xi,\kappa}^{ac} - u_{i,\xi,\kappa}^{ac} + 2(r_e^{ac} f_{e,\xi,\kappa}^{ac} + x_e^{ac} q_{e,\xi,\kappa}^{ac}) \leq M_e^{ac}(1 - \beta_{e,\xi,\kappa}), \quad (462)$$

$$-\bar{F}_e^{ac} \beta_{e,\xi,\kappa} \leq f_{e,\xi,\kappa}^{ac} \leq \bar{F}_e^{ac} \beta_{e,\xi,\kappa}, \quad (463)$$

$$-\bar{Q}_e^{ac} \beta_{e,\xi,\kappa} \leq q_{e,\xi,\kappa}^{ac} \leq \bar{Q}_e^{ac} \beta_{e,\xi,\kappa}, \quad (464)$$

$$\underline{u}_i^{ac} y_{i,\xi,\kappa}^{en} \leq u_{i,\xi,\kappa}^{ac} \leq \bar{u}_i^{ac} y_{i,\xi,\kappa}^{en}, \quad (465)$$

$$-M_\ell^{dc}(1 - \beta_{\ell,\xi,\kappa}) \leq f_{\ell,\xi,\kappa}^{dc} - g_\ell^{dc}(v_{i,\xi,\kappa}^{dc} - v_{j,\xi,\kappa}^{dc}) \leq M_\ell^{dc}(1 - \beta_{\ell,\xi,\kappa}), \quad (466)$$

$$-\bar{F}_\ell^{dc} \beta_{\ell,\xi,\kappa} \leq f_{\ell,\xi,\kappa}^{dc} \leq \bar{F}_\ell^{dc} \beta_{\ell,\xi,\kappa}, \quad (467)$$

$$\underline{v}_i^{dc} y_{i,\xi,\kappa}^{en} \leq v_{i,\xi,\kappa}^{dc} \leq \bar{v}_i^{dc} y_{i,\xi,\kappa}^{en}, \quad (468)$$

$$-\bar{S}_c^{vsc} \beta_{c,\xi,\kappa} \leq p_{c,\xi,\kappa}^{vsc} \leq \bar{S}_c^{vsc} \beta_{c,\xi,\kappa}, \quad (469)$$

$$-\bar{\gamma}_c^{vsc} \bar{S}_c^{vsc} \beta_{c,\xi,\kappa} \leq q_{c,\xi,\kappa}^{vsc} \leq \bar{\gamma}_c^{vsc} \bar{S}_c^{vsc} \beta_{c,\xi,\kappa}. \quad (470)$$

For activated AC source buses  $r \in \mathcal{R}^{ac}$  and activated DC source buses  $m \in \mathcal{R}^{dc}$ , reference voltages are imposed by

$$-M_r^{src}(1 - z_{r,\xi,\kappa}^{src}) \leq u_{r,\xi,\kappa}^{ac} - u_r^{ref} \leq M_r^{src}(1 - z_{r,\xi,\kappa}^{src}), \quad \forall r \in \mathcal{R}^{ac}, \quad (471)$$

$$-M_m^{src}(1 - z_{m,\xi,\kappa}^{src}) \leq v_{m,\xi,\kappa}^{dc} - v_m^{ref} \leq M_m^{src}(1 - z_{m,\xi,\kappa}^{src}), \quad \forall m \in \mathcal{R}^{dc}. \quad (472)$$

These constraints ensure that de-energized buses collapse to zero voltage, energized buses remain inside admissible envelopes, and only activated substation roots impose voltage references on their restored partitions. For AC buses,

$$\sum_{e \in \delta_i^- \cap \mathcal{E}^{ac}} f_{e,\xi,\kappa}^{ac} - \sum_{e \in \delta_i^+ \cap \mathcal{E}^{ac}} f_{e,\xi,\kappa}^{ac} + p_{i,\xi,\kappa}^{inj,ext} + p_{i,\xi,\kappa}^{ls} = 0, \quad (473)$$

$$\sum_{e \in \delta_i^- \cap \mathcal{E}^{ac}} q_{e,\xi,\kappa}^{ac} - \sum_{e \in \delta_i^+ \cap \mathcal{E}^{ac}} q_{e,\xi,\kappa}^{ac} + q_{i,\xi,\kappa}^{inj,ext} + \eta_i^{Q,load} p_{i,\xi,\kappa}^{ls} = 0, \quad (474)$$

and for DC buses,

$$\sum_{\ell \in \delta_m^- \cap \mathcal{E}^{dc}} f_{\ell,\xi,\kappa}^{dc} - \sum_{\ell \in \delta_m^+ \cap \mathcal{E}^{dc}} f_{\ell,\xi,\kappa}^{dc} + p_{m,\xi,\kappa}^{inj,ext} + p_{m,\xi,\kappa}^{ls} = 0. \quad (475)$$

The extreme-state injection terms are written explicitly as

$$p_{i,\xi,\kappa}^{inj,ext} := \mathbf{1}_{\{i \in \mathcal{N}^{pv}\}} p_{i,\xi,\kappa}^{pv} + \mathbf{1}_{\{i \in \mathcal{N}^{bess} \setminus \mathcal{N}^{gfm}\}} \left( p_{i,\xi,\kappa}^{bess,dis} - p_{i,\xi,\kappa}^{bess,ch} \right) \quad (476)$$

$$+ \mathbf{1}_{\{i \in \mathcal{N}^{gfm}\}} \left( p_{i,\xi,\kappa}^{bess,dis} - p_{i,\xi,\kappa}^{bess,ch} \right) + \mathbf{1}_{\{i \in \mathcal{R}^{ac}\}} z_{i,\xi,\kappa}^{src} \left( p_{i,\xi,\kappa}^{sub,+} - p_{i,\xi,\kappa}^{sub,-} \right) \quad (477)$$

$$+ \sum_{c:i(c)=i} p_{c,\xi,\kappa}^{vsc} - P_{i,\xi}^{load,ac}, \quad (478)$$

$$q_{i,\xi,\kappa}^{inj,ext} := \mathbf{1}_{\{i \in \mathcal{N}^{pv}\}} q_{i,\xi,\kappa}^{pv} + \mathbf{1}_{\{i \in \mathcal{N}^{bess}\}} q_{i,\xi,\kappa}^{bess} + \mathbf{1}_{\{i \in \mathcal{R}^{ac}\}} z_{i,\xi,\kappa}^{src} q_{i,\xi,\kappa}^{sub} \quad (479)$$

$$+ \sum_{c:i(c)=i} q_{c,\xi,\kappa}^{vsc} - Q_{i,\xi}^{load,ac}, \quad (480)$$

$$p_{m,\xi,\kappa}^{inj,ext} := \mathbf{1}_{\{m \in \mathcal{N}^{pv}\}} p_{m,\xi,\kappa}^{pv} + \mathbf{1}_{\{m \in \mathcal{N}^{bess}\}} \left( p_{m,\xi,\kappa}^{bess,dis} - p_{m,\xi,\kappa}^{bess,ch} \right) \quad (481)$$

$$+ \mathbf{1}_{\{m \in \mathcal{R}^{dc}\}} z_{m,\xi,\kappa}^{src} \left( p_{m,\xi,\kappa}^{sub,+} - p_{m,\xi,\kappa}^{sub,-} \right) + \sum_{c:k(c)=m} p_{c,\xi,\kappa}^{vsc,dc} - P_{m,\xi}^{load,dc}. \quad (482)$$

Thus only activated substations and activated grid-forming BESS units may act as partition roots in the resilience layer, while non-source DER units remain dispatchable support devices inside an energized partition.

Load service is tied to bus energization by

$$0 \leq p_{n,\xi,\kappa}^{ls} \leq P_{n,\xi}^{load}, \quad \forall n \in \mathcal{N}, \quad (483)$$

$$p_{n,\xi,\kappa}^{ls} \geq P_{n,\xi}^{load}(1 - y_{n,\xi,\kappa}^{en}), \quad \forall n \in \mathcal{N}. \quad (484)$$

Hence a bus that is not energized must be fully shed, while an energized bus may still experience partial curtailment if local generation and transfer capacity are insufficient.

The corresponding extreme-state unsupplied-energy functional is

$$H_{\xi,\kappa}^{ext}(x) := \sum_{n \in \mathcal{N}} v_n^{ext} p_{n,\xi,\kappa}^{ls}, \quad (485)$$

where  $v_n^{ext}$  is a load-priority weight that may emphasize critical-service buses during severe events. Constraint (383) then limits the weighted resilience loss across the extreme-state library.

This resilience layer is a MILP because it combines binary topology variables, source-activation decisions, and grid-forming mode selection. It extends the peak  $N-1$  reliability screen to a restoration-aware planning surrogate for higher-order faults.

## 25.7 Discussion of Modeling Choices

The 10b model is deliberately asymmetric in time resolution:

1. carbon accounting and operating cost are driven by annual representative states;
2. peak-time  $N-1$  reliability is screened at the annual stress point, because feeder adequacy violations are typically most severe there;
3. extreme-state  $N-k$  resilience is screened on a compact event library with restoration-aware partitioning;
4. FMEA weighting and extreme-state severity weighting preserve contingency information without requiring full chronological multi-contingency simulation.

This structure makes the model suitable for distribution-level planning studies where the main design variables are DER siting and sizing, the carbon target is annual, the reliability target is peak adequacy under single failures, and the resilience target is survivability under a limited number of severe higher-order events.

**Remark 1** (Interpretation of the reliability layer). *The peak-only  $N-1$  screening layer should still be interpreted as a planning-security approximation rather than a full reliability-assessment replacement. The new extreme-state layer partly closes that gap by introducing restoration-aware  $N-k$  events, but it remains a reduced event library rather than a full sequential Monte Carlo simulation. If seasonal restoration behavior is also important, additional event states can be added to  $\Xi$  without changing the model structure.*

## 25.8 Illustrative Test System for Case 10b

To support reproducible numerical experiments, the planning model above is paired with a synthetic but engineering-oriented hybrid AC/DC distribution benchmark. The test system is intended to be large enough to expose the coordination value of AC/DC coupling and DER siting, while remaining small enough to support repeated sensitivity runs.

### 25.8.1 Network structure

The proposed 10b benchmark is a radial hybrid feeder with the following topology.

$$\text{AC buses: } |\mathcal{N}^{ac}| = 33, \tag{486}$$

$$\text{DC buses: } |\mathcal{N}^{dc}| = 9, \tag{487}$$

$$\text{AC branches: } |\mathcal{E}^{ac}| = 32, \tag{488}$$

$$\text{DC branches: } |\mathcal{E}^{dc}| = 8, \tag{489}$$

$$\text{interlinking converters: } |\mathcal{E}^{vsc}| = 3. \tag{490}$$

The AC side is organized as a modified radial feeder derived from a standard medium-voltage distribution template, so voltage regulation can be represented explicitly through LinDistFlow-type drop equations. The DC side represents converter-fed industrial or campus subzones with strong electronic load penetration. One AC root bus and one DC root bus are connected to the upstream utility through import-limited substations. The three AC/DC converters link the AC backbone to three DC pockets positioned at upstream, middle-feeder, and downstream locations, respectively.

The network is partitioned into three load regions:

1. an upstream mixed commercial region with high daytime PV coincidence;

2. a mid-feeder residential region with evening peak stress;
3. a downstream critical-service region including hospital, communication, and emergency-support loads.

The critical-service region defines the set  $\mathcal{N}^{crit}$  used in the strict  $N-1$  reliability constraints.

### 25.8.2 Candidate DER placement

Candidate PV sites are placed at buses with suitable rooftop or community-solar potential, while candidate BESS sites are placed at electrically strategic buses that can support both congestion relief and contingency restoration. A practical initial candidate set is:

$$|\mathcal{N}^{pv}| = 10, \quad |\mathcal{N}^{bess}| = 6. \quad (491)$$

The candidate buses should be selected so that at least:

1. two PV candidates are near the AC root,
2. four PV candidates are distributed over middle and downstream AC buses,
3. two PV candidates are located on DC buses behind converters,
4. two BESS candidates are placed in the critical-service region,
5. one BESS candidate is placed near each converter-coupled DC pocket.

This placement pattern allows the planning model to test whether converter capacity, local storage, feeder siting, and grid-forming source placement act as substitutes or complements under carbon, reliability, and resilience pressure.

### 25.8.3 Representative operating states

The annual operating layer uses a compact representative-state library rather than full chronological simulation. A suitable baseline set is:

$$\Omega = \{\text{summer peak, summer shoulder, winter peak, winter shoulder,} \quad (492)$$

$$\text{spring high-PV, autumn high-PV, night valley, } \omega^{pk}\}. \quad (493)$$

The peak snapshot  $\omega^{pk}$  is the annual system peak and is also the anchor state for the  $N-1$  contingency layer. The remaining snapshots are used to approximate annual operating cost and annual carbon exposure. Their weights should be calibrated from feeder smart-meter and PV-availability data or from a synthetic seasonal load/PV library.

### 25.8.4 Extreme-state library and $N-k$ families

The resilience layer should use a compact but explicit extreme-state library such as

$$\Xi = \{\text{heat-wave import stress, storm-driven multi-line outage, flooded downstream corridor}\}. \quad (494)$$

For each  $\xi \in \Xi$ , the family  $\mathcal{K}_\xi$  should include geographically or functionally coupled higher-order failures, for example:

1. simultaneous outage of two to four adjacent AC feeder sections,
2. joint outage of a converter and its downstream DC branch pocket,

3. common-mode loss of one substation and one to two nearby feeders,
4. storm-induced outage of one corridor plus one large DER block.

For a first reproducible benchmark, it is sufficient to instantiate three event classes with explicit stress assumptions:

1. **heat-wave import stress:** upstream import capability reduced by 20–40%, annual-peak-like demand retained, and one additional feeder outage added to mimic protection-triggered overload isolation;
2. **storm-driven corridor damage:** two to four adjacent AC lines and one converter or DC tie simultaneously unavailable in the same geographical corridor;
3. **flooded downstream corridor:** one downstream substation path and all vulnerable downstream laterals inside the flooded zone unavailable, requiring local partition support for critical loads.

The purpose of  $\Xi$  is not exhaustive event enumeration, but a planning-grade set of severe conditions that activate islanding, reactive support scarcity, and multi-source partition restoration.

### 25.8.5 Contingency set for peak-time FMEA

The  $N-1$  contingency set  $\mathcal{C}$  should include all single failures whose loss can materially affect peak feasibility:

$$\mathcal{C} = \mathcal{C}^{ac\_line} \cup \mathcal{C}^{dc\_line} \cup \mathcal{C}^{vsc} \cup \mathcal{C}^{sub} \cup \mathcal{C}^{der}, \quad (495)$$

where:

1.  $\mathcal{C}^{ac\_line}$  contains all primary AC feeder branches;
2.  $\mathcal{C}^{dc\_line}$  contains all DC feeder links;
3.  $\mathcal{C}^{vsc}$  contains the three AC/DC converters;
4.  $\mathcal{C}^{sub}$  contains the AC and DC root-supply outages;
5.  $\mathcal{C}^{der}$  optionally contains large BESS outages and the largest individual PV block outage.

For the base 10b study, it is sufficient to start with line, converter, and substation contingencies only, then add major DER outages in a secondary sensitivity study.

## 25.9 Baseline Parameter Set

Tables 136–137 collect a compact baseline parameter set for the first full numerical study. The numbers below are not meant to be universal constants; they provide a coherent starting point for calibration and sensitivity analysis.

The main purpose of Tables 136–137 is to freeze a reporting basis. Once numerical runs begin, exact values should be mapped to the component fields already documented in Section 18 and the I/O contract chapters.

## 25.10 Experimental Design

The numerical study for 10b should answer four questions simultaneously:

1. how the hybrid feeder uses DER siting and converter capacity to reduce annual carbon exposure;
2. how much additional capacity is required to satisfy peak-time  $N-1$  reliability targets;
3. how much DER-assisted voltage regulation reduces reinforcement and load curtailment under stressed conditions;
4. whether AC/DC coupling and grid-forming BESS change survivability under extreme  $N-k$  events relative to a pure AC planning baseline.

To answer those questions cleanly, the experiments should be organized into a small number of structured cases.

### 25.10.1 Evaluation metrics

Every case should report the same output set:

|                                                                     |       |
|---------------------------------------------------------------------|-------|
| installed PV capacity by bus and total,                             | (496) |
| installed BESS power and energy by bus and total,                   | (497) |
| installed converter and substation capacities,                      | (498) |
| planning objective and its investment / operating decomposition,    | (499) |
| annual carbon emission and carbon-budget slack,                     | (500) |
| peak FMEA-weighted EENS and contingency-wise shed-load diagnostics, | (501) |
| extreme-state weighted ENS and partition-count diagnostics,         | (502) |
| critical-load service status under every $N-1$ contingency,         | (503) |
| critical-load service status under every extreme $N-k$ event,       | (504) |
| root import dependence of the AC and DC subnetworks,                | (505) |
| solver runtime and model size.                                      | (506) |

In addition, the following structural diagnostics are recommended:

|                                                                               |       |
|-------------------------------------------------------------------------------|-------|
| utilization of each converter at $\omega^{pk}$ ,                              | (507) |
| fraction of peak contingency support supplied by local BESS,                  | (508) |
| change in downstream branch loading after DER installation,                   | (509) |
| number and composition of energized tree partitions under each extreme event, | (510) |
| reactive-support contribution shares of PV, BESS, VSC, and substations.       | (511) |

### 25.10.2 Case 1: Base AC/DC planning without carbon or reliability pressure

The first case is the economic reference point:

|                                                          |       |
|----------------------------------------------------------|-------|
| annual operating states enabled,                         | (512) |
| $\zeta^{CO_2}$ inactive or budget very loose,            | (513) |
| $\zeta^{rel}$ inactive or reliability budget very loose, | (514) |
| $\zeta^{ext}$ inactive or resilience budget very loose,  | (515) |
| lossless converter model enabled.                        | (516) |

This case identifies the purely economic DER pattern selected by the hybrid feeder without policy or security pressure. It serves as the denominator for all later comparisons.

### 25.10.3 Case 2: Carbon-constrained planning

The second case activates the annual carbon constraint (381) while keeping the reliability layer soft or inactive. Its purpose is to quantify how much PV, BESS, and converter reinforcement are induced by decarbonization alone.

Recommended sensitivities are:

$$\overline{E}_{CO_2}^{ann} \in \{1.0, 0.9, 0.8, 0.7\} \times E_{CO_2, \text{base}}^{ann}. \quad (517)$$

The main outputs of interest are marginal carbon abatement cost, relocation of DER from AC to DC buses, and the extent to which converter expansion substitutes for local storage.

### 25.10.4 Case 3: Peak-time $N-1$ reliability-constrained planning

The third case activates the peak FMEA constraint (447) and the strict critical-load condition (445), while keeping the carbon budget loose. This isolates the planning cost of reliability.

Two variants are especially informative:

1. **critical-load security only:**  $p_{n,k}^{ls} = 0$  only on  $\mathcal{N}^{crit}$ ;
2. **systemwide adequacy:**  $\overline{E}_{rel}^{pk}$  tightened for all buses.

This case reveals whether the model prefers local BESS in the critical-service region, stronger upstream import capacity, additional converter reinforcement, or more flexible inverter VAR support to survive single contingencies at peak.

### 25.10.5 Case 4: Joint carbon- and reliability-constrained planning

The fourth case activates (381) and (382) together with DER-assisted voltage regulation. This isolates the value of coordinated PV/BESS/VSC VAR support before the resilience layer is added.

The most useful outputs are not only total cost and total capacity, but also the interaction effects:

1. whether carbon-constrained PV siting improves or worsens peak  $N-1$  feasibility;
2. whether reliability-driven BESS placement also reduces carbon exposure by smoothing root imports;
3. whether DER-assisted voltage regulation reduces the reinforcement needed to sustain the carbon-reliability combination.

### 25.10.6 Case 5: Extreme-state $N-k$ resilience planning

The fifth case activates the full model, including the resilience constraint (383), source-partition binaries, and grid-forming BESS constraints. This is the main resilience case.

The most informative outputs are:

1. how often the model chooses multiple energized partitions rather than a single surviving backbone,
2. whether grid-forming BESS is concentrated in the critical-service region or distributed across the feeder,
3. how much extreme-event ENS is reduced by reactive support and converter flexibility,
4. whether extreme-state preparedness changes the carbon and peak  $N-1$  design selected by the first stage.

### 25.10.7 Case 6: Architecture and device-value sensitivities

To isolate the value of hybridization itself, the integrated resilience-aware design should be compared against the following structural ablations:

$$\text{pure AC benchmark: } \mathcal{N}^{dc} = \emptyset, \quad (518)$$

$$\text{fixed-converter benchmark: } \overline{S}_c^{usc} \text{ not expandable}, \quad (519)$$

$$\text{no-BESS benchmark: } \overline{P}_i^{bess} = \overline{E}_i^{bess} = 0, \quad (520)$$

$$\text{no-DER benchmark: only substation reinforcement allowed.} \quad (521)$$

These ablations show whether the main benefit arises from topology, from storage, from inverter VAR support, from grid-forming capability, or from the AC/DC coupling device itself.

## 25.11 Recommended Reporting Tables and Figures

For a first complete 10b numerical study, the following outputs should be prepared.

1. A system-data table listing buses, candidate DER sites, critical-load buses, converters, and contingency counts.
2. Baseline-parameter tables, starting from Tables 136–137.
3. A case-summary table comparing Cases 1–6 in objective value, carbon emission, peak FMEA-EENS, extreme-state ENS, and installed capacities.
4. A bus-level DER-siting figure showing where PV and BESS are built under the joint case.
5. A contingency heatmap showing peak shed load by contingency and by load region.
6. An extreme-event partition figure showing energized islands and their source buses under one representative  $N-k$  event.
7. A carbon–reliability–resilience tradeoff plot obtained by varying  $\overline{E}_{CO_2}^{ann}$ ,  $\overline{E}_{rel}^{pk}$ , and  $\overline{E}_{rel}^{ext}$ .

## 25.12 Expected Experimental Findings

Although the exact numerical values must be determined computationally, the 10b model is designed to test five hypotheses.

1. Hybrid AC/DC coupling should reduce the cost of meeting peak  $N-1$  reliability relative to a pure AC feeder because mobile power support across subnetworks is enabled through the interlinking converters.
2. Carbon tightening should increase PV adoption first, then BESS adoption, and finally converter/substation reinforcement once local renewable hosting limits become active.
3. Reliability tightening should shift BESS placement toward downstream critical-service buses and increase the option value of converter capacity even when converters are modeled as lossless.
4. DER-assisted voltage regulation should reduce both peak-time curtailment and the need for substation VAR expansion compared with a substation-only reactive support policy.
5. The extreme-state resilience layer should induce additional BESS energy reservation and more spatially distributed inverter placement, because multiple tree-structured partitions become valuable under severe  $N-k$  events.

These hypotheses provide a clear roadmap for the first computational campaign and convert the 10b section from a pure formulation note into an executable numerical-study plan.

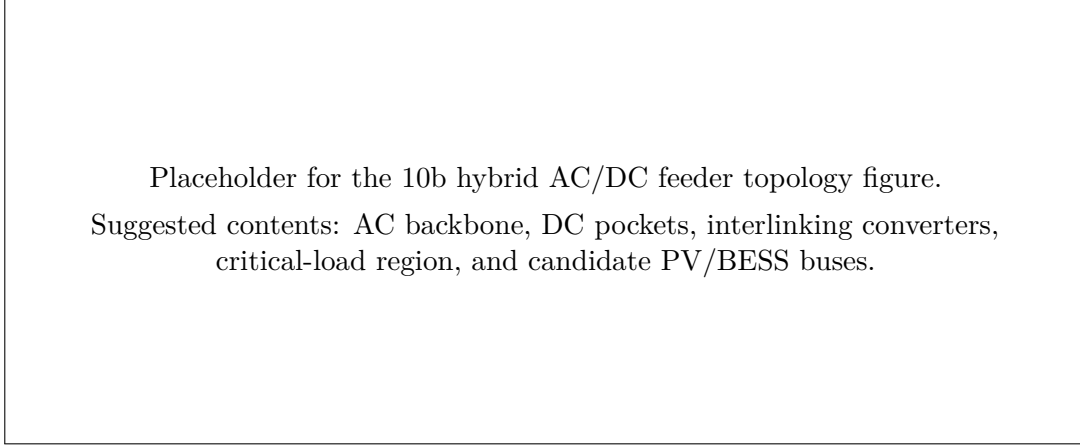


Figure 12: Topology placeholder for the 10b benchmark system.

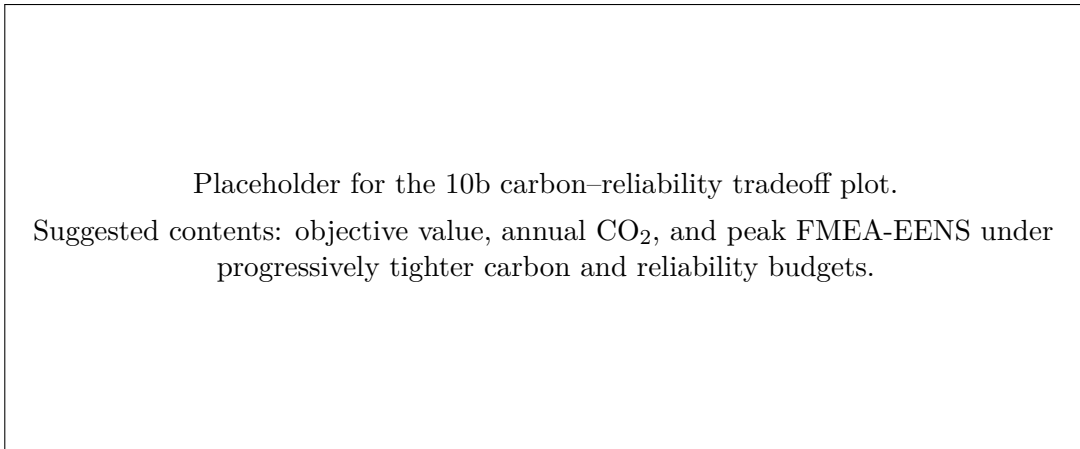


Figure 13: Tradeoff-curve placeholder for the 10b study.

### 25.13 Result-Display Skeleton

To reduce setup overhead in the first numerical campaign, the following reporting templates are prepared directly in the notebook workspace.

#### 25.13.1 Case-summary and parameter-table templates

The case-summary table template is stored in `visualization/hybrid_distribution_case10b/tables/table_10b_case_summary.tex`, and the compact parameter-summary template is stored in `visualization/hybrid_distribution_case10b/tables/table_10b_parameter_summary.tex`. Their CSV companions are stored in the matching `data/` directory so that result-generation scripts can populate the same reporting layout automatically.

#### 25.13.2 Figure placeholders

Three figure placeholders are also prepared for the first paper draft: feeder topology, carbon-reliability tradeoff, and DER siting under the joint case.

## 26 Improvement Plan

### 26.1 Completion Audit of the Previous Plan

Placeholder for the 10b DER siting and capacity-allocation figure.  
Suggested contents: bus-level PV siting, BESS siting, and converter reinforcement under the joint planning case.

Figure 14: DER-siting placeholder for the 10b joint planning case.

| Previous priority                      | Status          | Evidence in the current code                                                                                                                                            | Remaining gap                                                                                                                                                                                                  |
|----------------------------------------|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Zero-impedance bus-merge contract      | partial         | Canonical preprocessing merges zero-impedance buses and propagates <code>bus_merge_map</code> ; PF APIs unproject merged $v_m/\theta$ back to original bus cardinality. | <code>unproject_bus_vector()</code> still maps by <code>orig_pos = ext_bus - 1</code> , which is only safe under contiguous 1..N external bus numbering. The contract is still implicit, not machine-enforced. |
| Result unprojection policy across APIs | partial         | <code>solve_power_flow()</code> , <code>solve_handle()</code> , and <code>solve_power_flow_fdcpf()</code> now unproject PF voltages when merges exist.                  | Non-voltage artifacts (branch-oriented tables, OPF/DPF/ONR side outputs) still do not expose one unified projected-vs-original indexing policy with explicit mapping metadata for every artifact.              |
| DCDC directional Jacobian consistency  | partial         | Parity equality equations use iterate-dependent bidirectional efficiency ( $p_{in} \geq 0 \Rightarrow \eta$ , $p_{in} < 0 \Rightarrow 1/\eta$ ).                        | Parity Jacobian still picks derivative sign from reference <code>p_ref_mw</code> (warm-start sign), not from current iterate $x$ . This can mis-linearize exactly when $p_{in}$ changes direction.             |
| ONR stable-id regression semantics     | mostly complete | ONR outputs now emit stable <code>ACBranch::index</code> ; projection-equivalence tests verify disjoint/open-closed partition and radially.                             | The suite still lacks a strict projected-vs-canonical branch-id set equality assertion on a case with deliberately nontrivial branch indices.                                                                  |

| Previous priority                     | Status   | Evidence in the current code                                                                                                                                                                                                                            | Remaining gap                                                                                                     |
|---------------------------------------|----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| Visualization verification depth      | complete | Net-work/geo/dashboard/profile outputs carry stable ids, DC losses, and loading fields; <code>test_visualization</code> now compares branch loss/loading and carbon intensity across JSON, workbook, HTML dashboard payloads, and Plotly figure inputs. | Contract locked by the payload-level numeric consistency test.                                                    |
| Notebook and benchmark release gating | complete | <code>technical_notebook_pdf</code> is part of <code>.github/workflows/release-gate.yml</code> ; the gate builds contract tests, runs a solver benchmark smoke, and emits <code>release_gate_summary.json</code> .                                      | Contract locked by <code>test_aper_study_contracts</code> and <code>scripts/compact_benchmark_summary.py</code> . |

## 26.2 Native Branch-and-Cut Proof-Tail Direction

The highest-value Native B&C work is no longer root LP objective alignment or threshold tuning. The root-certified path is now close to the intended regime: on the UC root-certificate family, the vendored HiGHS certificate can close the gap with one LP and no native tree. The remaining performance gap is the non-root proof tail: Native can produce local proof information, but the critical question is whether that information becomes a reusable certificate that raises the live frontier lower bound.

The proof invariant to optimize is

$$z_{\text{LB}} = \min_{N \in \mathcal{F}_{\text{proof}}} z_N.$$

Incumbent quality alone cannot certify optimality. Every major B&C change should therefore answer one question: did it lift, prune, or refresh the proof frontier so that this lower bound moves toward the incumbent?

**Primary chain.** The next proof-tail implementation work should be organized around the following closed loop:

local proof artifact  $\rightarrow$  resolved reason-side frontier  $\rightarrow$  bound-lifting certificate  $\rightarrow$  queue lower-bound movement

The Native code already has the main hooks for this chain: `resolve_dual_proof_target_bound_from_local_trail()`, `resolve_dual_proof_conflict_from_local_trail()`, `BoundLiftingCertificate`, `NodeQueue::propagate_bound_lifting_certificates()`, and `NodeQueue::propagate_by_dual_proof_domain()`. The remaining work is to make the chain auditable and to verify that it moves the frontier rather than merely increasing artifact counts.

| Area                              | Current implementation anchor                                                                                                                                                                                                                         | Next improvement target                                                                                                                                                                                                                     |
|-----------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Local trail resolution            | Native already resolves target bounds and conflicts through the local domain trail, including previous-bound relaxation and activity revalidation fields.                                                                                             | Add explicit counters for target/conflict attempts, successes, missing reasons, scope blocks, and activity failures. Reject every artifact whose resolved frontier does not revalidate the proof activity after substitution.               |
| Bound-lifting certificates        | <code>BoundLiftingCertificate</code> is a first-class object with frontier literals, target bound, source literal, hash, depth, and proof-activity audit values.                                                                                      | Strengthen the type contract so cross-frontier consumption requires resolved-frontier, activity-revalidated, target-validated, and space-validated flags. Keep local artifacts and published certificates separate in names and call paths. |
| Queue certificate replay          | Queue replay can tighten domains, append <code>DomainReasonBound</code> , prune infeasible nodes, mark LP refresh, and publish movement statistics.                                                                                                   | Treat replay success as lower-bound movement, not as scan count. For every pass, record queue lower bound before and after, frontier-domain movement, LP-refresh count, prunes, and refreshed-node bound lift.                              |
| Queue index consistency           | Sequential and threaded queues maintain separate priority and bound views through <code>priority_</code> , <code>bounds_</code> , and <code>dfs_bounds_</code> ; the direct dual-proof replay path rekeys nodes when <code>node.bound</code> changes. | Add an audit path that recomputes the minimum bound from live nodes and compares it with the indexed queue lower bound after certificate replay, post-update callbacks, LP refresh, and prune passes.                                       |
| Dual proof rows                   | <code>build_full_dual_proof_row()</code> and <code>build_reduced_cost_mass_dual_proof_row()</code> already check identity, stationarity, activity range, and mapped cost errors.                                                                      | Attach an LP-state signature to each row: basis hash, row-dual hash, reduced-cost hash, nonbasic-side hash, objective, and standard-form signature. Reject proof-row reuse on basis, side, or space mismatch.                               |
| Root and non-root proof semantics | <code>root_gap_certificate_ready</code> and <code>require_tree_exhaustion_certificate</code> already separate root shortcut logic from strict tree exhaustion.                                                                                        | Make the result proof mode explicit (vendored root certificate, native root gap certificate, strict tree exhaustion) so future maintenance cannot replace strict exhaustion with an ordinary gap shortcut.                                  |
| Parallel proof frontier           | <code>WorkStealingPool</code> and <code>ActiveNodeBounds</code> expose lower-bound views for worker dequeues and active nodes.                                                                                                                        | Centralize the parallel proof lower bound as the minimum of shared queue, worker dequeues, and active nodes, and log those components when proof-tail mode is active.                                                                       |

**Acceptance metrics.** A non-root proof-tail benchmark should be judged by movement

metrics, not by artifact volume. The minimum useful diagnostic set is:

- `resolved_target_attempts`, `resolved_target_success`, `resolved_conflict_attempts`, and `resolved_conflict_success`;
- `resolution_activity_failed`, `resolution_missing_reason`, and `resolution_scope_blocked`;
- `bound_lifting_certificates_published` and `bound_lifting_certificates_rejected_audit`;
- `certificate_queue_tightenings`, `certificate_queue_prunes`, `certificate_queue_lower_bound_lift_nodes`, `certificate_queue_lp_refresh_marked`, and `certificate_queue_frontier_bound_move_sum`;
- queue audit records of the form `before`, `after`, `changed_nodes`, `tightened`, `pruned`, `computed_min`, and `indexed_min`.

The proof-tail target is to observe all of the following on a case where the vendored root frontier is disabled or unavailable:

`resolved_target_success > 0`, `bound_lifting_certificates_published > 0`, `certificate_queue_lo`

#### **Execution order for Native B&C.**

1. Harden local trail resolution and activity revalidation before adding new proof sources.
2. Publish only resolved, audited `BoundLiftingCertificate` objects; keep proof artifacts local until resolution succeeds.
3. Make queue replay prove real movement: tighten/prune/mark LP refresh, then refresh marked nodes and audit the lower-bound index.
4. Add proof-row LP-state signatures after the certificate path is moving the frontier.
5. Optimize parallel proof drain only after the single-frontier certificate path shows lower-bound movement.

## **26.3 Current Code-Quality Priorities**

**Priority 1: Remove hidden index assumptions in time-series and merged-network paths.**

**Why this is now first.** The new UC→OPF→PF pipeline has good functional coverage, but still contains bus-position assumptions (for example `bus-1` indexing when replaying OPF voltages) while bus-merge unprojection also assumes contiguous external indices. These are correctness risks on real planning cases with nontrivial bus numbering.

#### **Minimal action list.**

- Introduce one shared utility for `bus_index->vector_position` mapping and use it in time-series replay + unprojection code paths.
- Extend `BusMergeMap` with explicit original-position mapping, and use that instead of `ext_bus-1`.
- Add regression cases with non-contiguous indices for PF, OPF replay, and merged-network unprojection.

**Priority 2: Make parity DCDC Jacobian consistent with the active operating direction.**

**Why this matters.** Equality constraints already use iterate-dependent efficiency; Jacobian entries still use reference-sign logic. This creates local model inconsistency near direction reversals and can degrade IPM/Newton steps.

**Minimal action list.**

- Compute  $\partial g / \partial p_{dcdc}$  from current iterate  $x$  (or a smooth transition near zero).
- Add finite-difference tests that force sign reversals during solving.
- Add one economic dispatch case where bidirectional DCDC operation is active.

**Priority 3: Clarify UC initial-state semantics and startup-cost accounting.**

**Why this matters.** UC currently filters generators by `in_service` and derives  $u_{init}$  from the same flag. This conflates “available in model” with “initially online”, and can hide startup-cost errors at  $t = 0$ .

**Minimal action list.**

- Separate availability (`in_service`) from initial commitment (for example `u_init` field or UC input override).
- Allow initially-off but available units to participate in UC.
- Add unit tests for  $t = 0$  startup behavior and cold-start transitions.

**Priority 4: Make cross-validation metrics physically complete for AC/DC systems.**

**Why this matters.** Current OPF-vs-PF loss comparison uses simplified OPF supply-demand arithmetic (AC-side only) and does not fully replay load-shedding on component-level load models. This can distort loss and mismatch diagnostics.

**Minimal action list.**

- Compute OPF-side loss with the same accounting boundary as PF-side metrics (AC+DC loads, converters, controllable demand).
- Replay load shedding consistently onto the component tables used by PF residual assembly.
- Add tolerance-based AC/DC cross-validation tests on loss and power-balance metrics.

**Priority 5: Close remaining DC model I/O contract gaps.**

**Why this matters.** Runtime now uses `dc_pv_arrays` and `dc_static_generators` paths, but top-level JSON I/O still serializes only `dc.static_generators`. This creates silent data loss when exchanging cases.

**Minimal action list.**

- Add JSON (and Excel, if applicable) read/write support for `dc.pv_arrays` and `dc.dc_static_generators`.
- Add round-trip tests that assert value equality for all DC generation subtypes.
- Update notebook field dictionary and consumer contract tables accordingly.

**Priority 6: Split benchmark-scale tests from fast correctness gates.**

**Why this matters.** `test_unit_commitment` is comprehensive but benchmark-heavy; it is unsuitable as a fast regression gate and obscures real correctness failures when CI timeouts occur.

**Minimal action list.**

- Keep a fast UC smoke/contract test in default CI; move full benchmark comparison to nightly/perf jobs.
- Add explicit runtime budgets and labels for test tiers (smoke/contract/perf).
- Gate releases on `technical_notebook_pdf` + core contract tests.

**Priority 7: Add dedicated RPO regression tests.**

**Why this matters.** The reactive-power optimisation (RPO) module is fully implemented with a 4-phase outer-approximation MINLP, but has no dedicated test file. Convergence properties and OA gap behaviour are currently exercised only through internal development runs.

**Minimal action list.**

- Add at least one test case per RPO objective (loss minimisation, voltage deviation, combined).
- Verify that discrete tap/shunt positions satisfy integrality after convergence.
- Check that OA gap decreases monotonically across the 4 phases.

**Priority 8: Strengthen annual production simulation and lifecycle validation.**

**Why this matters.** Both the annual production simulation (L0/L2/L3 hierarchy) and the multi-year lifecycle simulation are implemented but only have smoke-level test coverage. There is no reference-case comparison against an external production-cost tool, and lifecycle NPV results are not cross-checked against manual calculations.

**Minimal action list.**

- Add a small reference case (e.g. 5-bus, 24-hour scaled to 1 week) with known optimal cost from an external solver.
- Add lifecycle regression test comparing NPV against a spreadsheet on a known degradation curve.
- Verify that L0 energy budgets correctly constrain L2 weekly UC dispatch.

## 26.4 Module-Level Investigation and Improvement Matrix

The current project is now large enough that a single priority list is not sufficient. The table below records the module-by-module code-quality investigation plan that should be updated whenever a major implementation path changes.

| Module                       | Current investigation finding                                                                                                                                       | Concrete improvement plan                                                                                                                                                                        |
|------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Model / canonical projection | Rich public component model and useful canonical projection; remaining risk is hidden id/vector-position assumptions in replay, unprojection, and result consumers. | Introduce a shared id-position map utility; extend <code>BusMergeMap</code> with explicit original-position data; test non-contiguous bus and branch ids through PF, OPF, DPF, ONR, and exports. |

| Module                   | Current investigation finding                                                                                                                                                                                                  | Concrete improvement plan                                                                                                                                                                                                         |
|--------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| I/O and result contracts | Input and output support is broad, but the runtime model now includes fields that are not uniformly serialized or reported.                                                                                                    | Complete JSON/Excel support for DC PV arrays and DC static-generator subtypes; serialize parity OPF DER/storage/DCDC/flex vectors; add round-trip and golden-file tests.                                                          |
| Power flow               | Main kernels are recently cleaned up: shared susceptance builder, split AC kernel, split Jacobian pattern, shared DC injection assembly. Remaining risk is formula drift across PF, OPF, and validation code.                  | Make every converter and DC injection formula available through shared helpers; add finite-difference derivative tests for VSC and DCDC controls; keep FDPF and Newton susceptance/admittance construction under common builders. |
| Optimal power flow       | Parity path is the strongest AC/DC NLP route; native generic path still has globalization/performance gaps; RPO has no dedicated regression file.                                                                              | Fix DCDC directional derivative consistency around sign changes; add RPO loss/voltage/combined objective tests; keep parity-vs-native benchmark table current; expand output dictionaries.                                        |
| Engine solvers           | LP/MILP stack is mature; generic NLP IPM is improving but not yet parity-competitive for large OPF.                                                                                                                            | Separate smoke, contract, and performance test tiers; add persistent benchmark baselines for dual simplex, LP IPM, LCQP, branch-and-cut, and NLP IPM; keep presolve/postsolve round-trip checks mandatory.                        |
| Distribution PF          | BFS and radial tests are credible, but coverage is still too feeder-simple for the full hybrid component catalog.                                                                                                              | Add cross-validation feeders with transformers, switches, chargers, flexible loads, storage, asymmetric load, PV curves, and projected bus/branch ids.                                                                            |
| Short circuit            | IEC-style Zbus/fault reference coverage now includes a single-source named case for three-phase, single-line-ground, line-line, and line-line-ground faults, with explicit external-grid and zero component-bucket assertions. | Extend the named-reference set to transformer/vector-group and converter-rich feeders; keep contribution-bucket conservation checks mandatory for every new short-circuit component model.                                        |
| Network reconfiguration  | Stable branch ids and PF post-validation are mostly complete.                                                                                                                                                                  | Add projected-vs-canonical branch-id set equality tests; test nontrivial physical branch ids; report infeasibility reasons consistently.                                                                                          |

| Module                               | Current investigation finding                                                                                                                                                                                                  | Concrete improvement plan                                                                                                                                                                    |
|--------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Carbon analysis                      | Static tracing, matrix solve, branch/VSC/DCDC loss attribution, and time-coupled ESS SOC/carbon-inventory dynamics are now covered by hand-calculated reference tests, including per-step system dispatch in dynamic series.   | Add larger multi-storage chronological cases with nonzero self-discharge and mixed AC/DC storage, and keep explicit loss-bucket reference values whenever converter formulas change.         |
| Time series / annual sim / lifecycle | End-to-end smoke tests are strong for regressions but not yet enough for benchmark-grade numerical claims.                                                                                                                     | Add a small external reference production-cost case; add spreadsheet-verified lifecycle NPV/degradation case; test UC initial commitment and startup accounting explicitly.                  |
| Reliability / resilience / MESS      | Heuristic restoration is now pinned against a strict HiGHS-backed restoration MIP on a toy MESS feeder, and MESS movement/energy edge cases cover in-transit dispatch blocking, travel-energy infeasibility, and SOC clamping. | Broaden the strict reference tier to multi-island repair sequencing and rolling-horizon feeders; track heuristic-vs-MIP optimality gaps as benchmark metadata instead of ad hoc demo output. |
| Planning / market / stochastic       | Paper-style studies are indexed by <code>reports/study_contracts/paper_studies.json</code> , with one reproducible CLI command, exact input manifest, reference result artifact, and smoke/reference regression per study.     | Keep the manifest updated whenever a study command, input fixture, artifact schema, or regression target changes.                                                                            |
| Visualization / reporting            | JSON result contracts, workbooks, HTML dashboard payloads, and Plotly figure inputs now carry stable ids plus shared loss/loading/carbon metrics.                                                                              | Keep <code>test_visualization</code> as the consumer contract for new reporting fields and chart payloads.                                                                                   |

| Module             | Current investigation finding                                                                                                                                          | Concrete improvement plan                                                                 |
|--------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| Documentation / CI | Release branches are gated on notebook compilation, core contract tests, and compact benchmark summarization through <code>.github/workflows/release-gate.yml</code> . | Require docs updates when public formulas, result fields, or paper-study commands change. |

## 26.5 Recommended Execution Order

The highest-value technical sequence is now:

1. remove index assumptions first (time-series replay + merge unprojection),
2. fix parity DCDC Jacobian directional consistency and lock it with FD tests,
3. solidify UC initial-state/startup semantics,
4. align OPF-vs-PF cross-validation boundaries for AC/DC metrics,
5. complete DC JSON/Excel contracts and corresponding notebook tables,
6. split benchmark-scale tests from fast correctness gates,
7. add dedicated RPO regression tests (convergence + integrality + OA gap),
8. then strengthen annual production simulation and lifecycle validation with reference-case comparisons.

## 27 Time-Series Analysis

### 27.1 Problem Form and Runtime Scope

This chapter covers chronological operations analysis built from two linked problem classes:

- a multi-period unit-commitment MILP for commitment and dispatch decisions,
- a sequential per-time-step hybrid power-flow replay for electrical feasibility and reporting.

The same chapter also documents the hierarchical annual production-simulation extension layered on top of those solve paths.

### 27.2 Code Map

The current implementation is concentrated in:

- `src/analysis/time_series_pf.cpp` for UC model build, backend selection, and chronological PF replay,
- `src/analysis/annual_production_sim.cpp` for the implemented L0/L2/L3 annual-production workflow,
- `src/analysis/lifecycle_simulation.cpp` for multi-year orchestration built on the annual simulation,
- the engine adapter layer and native branch-and-cut infrastructure for MILP solving.

## 27.3 Pseudocode Map

The relevant code-aligned workflow summaries are:

- Algorithm A20 in Section 21 for annual production simulation,
- Algorithm A21 for lifecycle simulation,
- the UC backend-selection and MILP mechanics summarized in Section 4 and Section 22.

## 27.4 Current Status

This is a live analysis stack with implemented UC, chronological PF replay, and annual production simulation. The main architectural boundary is that different time-scale layers reuse common solver infrastructure rather than a single unified monolithic optimization model.

This section documents the *current* C++ implementation in `src/analysis/time_series_pf.cpp` and defines the next framework-aligned extension for **annual production simulation**. The runtime pipeline has two stages over a discrete horizon  $\mathcal{T} = \{0, 1, \dots, T-1\}$  with step length  $\Delta t$  (hours):

1. **Unit Commitment (UC) MILP**: build and solve a linear mixed-integer dispatch/commitment model for in-service AC generators, AC storage, and AC renewable generators.
2. **Sequential Time-Domain PF**: for each time step, write UC setpoints into a system snapshot, apply load/PV profiles, and run the hybrid AC/DC Newton power-flow solver.

The public APIs are:

- `solve_unit_commitment(sys, ts_data, opts)` returning `UCSchedule`,
- `solve_time_series_pf(sys, ts_data, opts)` returning `TimeSeriesPFResult`.

## 27.5 Code-Execution Path

**Top-level sequence (`solve_time_series_pf`).**

1. If  $T \leq 0$ , return an empty result immediately.
2. If `skip_uc=false`, call `solve_unit_commitment` first.
3. Build index lists for in-service generators, storage units, and renewable generators.
4. Create a reusable PF `SolverHandle`.
5. For each  $t \in \mathcal{T}$ :
  - (a) Copy the base system to `syst`.
  - (b) If UC is active and feasible, apply schedule setpoints to `Generator::pg_mw`, `Generator::in_service`, `Storage::p_mw`, and `RenewableGen::p_mw`.
  - (c) Apply load profiles to `Load::p_mw/q_mvar`.
  - (d) Apply PV profiles to `PVSystem::irradiance`.
  - (e) Reset solver handle with `syst`, solve hybrid PF, and store `PowerFlowResult` for step  $t$ .
6. Destroy handle and return `TimeSeriesPFResult`.

**UC solver backend routing.** `TimeSeriesPFOptions::uc_solver` supports `Auto/Native/HiGHS/Gurobi`. In `Auto` mode the adapter registry tries MILP backends in order:

$$\text{Gurobi} \rightarrow \text{HiGHS} \rightarrow \text{NativeBranchAndCut}. \quad (522)$$

## 27.6 Input Profiles and Runtime Scaling

A profile is a vector  $\pi(t)$  looked up by `profile_id`. Missing profile IDs or out-of-range indices fall back to default value 1.0.

| Component    | Runtime update         | Implemented rule                                                           |
|--------------|------------------------|----------------------------------------------------------------------------|
| Load         | $P_\ell(t), Q_\ell(t)$ | $P_\ell(t) = P_\ell^0 \sigma_\ell(t), Q_\ell(t) = Q_\ell^0 \sigma_\ell(t)$ |
| RenewableGen | availability in UC     | $\bar{P}_{r,t}^{\text{ren}} = P_r^{\text{rated}} \rho_r(t)$                |
| PVSystem     | irradiance in PF stage | $G_p(t) = 1000 \rho_p(t) \text{ W/m}^2$                                    |

## 27.7 Annual Production-Simulation Model (Implemented — L0/L2/L3)

### 27.7.1 Implementation Status

The annual production simulation is **implemented** in `src/analysis/annual_production_sim.cpp` with the following level coverage:

- **L0 (Annual Planning):** Implemented. Builds monthly or weekly block boundaries, computes per-block energy budgets, maintenance masks (default: no maintenance), storage SOC boundaries, and block-averaged cost estimates.
- **L1 (Monthly Sub-problems):** **Not implemented.** The orchestrator skips directly from L0 blocks to weekly UC windows. Monthly sub-problem constraints (refined energy budgets, hydro allocation) are not solved; L0 block boundaries serve as the coupling interface.
- **L2 (Weekly Rolling UC):** Implemented. Each week calls `solve_unit_commitment()` on a sliced sub-horizon with look-ahead buffer, inheriting maintenance masks and SOC boundaries from L0.
- **L3 (Daily OPF/PF Replay):** Implemented. Each day within a weekly binding window is replayed through `solve_time_series_pf()` with the pre-solved UC schedule injected via `skip_uc=true`.

The public entry point is `solve_annual_production_simulation(sys, ts_data, opts)` returning `AnnualProductionSimResult`.

A companion module, `src/analysis/lifecycle_simulation.cpp`, wraps the annual simulation in a multi-year loop with component degradation (battery capacity fade, solar degradation), stratified sampling over operating strata (7 bins), Neyman allocation, and NPV discounting. Its entry point is `run_lifecycle_simulation(sys, ts_data, opts)`.

### 27.7.2 Position in the Existing Modeling Framework

The annual production-simulation module is designed as a **chronological year-ahead scheduling layer** that reuses the existing time-series abstractions rather than introducing a parallel framework. The intended runtime order is

$$\text{TimeSeriesData} \xrightarrow{\text{annual schedule}} \text{UCSchedule-like yearlyplan} \quad (523)$$

$$\xrightarrow{\text{step-wise replay}} \text{ACOPF/hybridPFvalidation}. \quad (524)$$

Concretely, the module fits the current code architecture as follows:

- `TimeSeriesData::num_steps` carries the full chronological horizon, typically  $T = 8760$  for a non-leap year with  $\Delta t = 1$  hour.
- Existing component `profile_id` hooks continue to provide load, renewable, PV, DC-load, and DC-generation trajectories.
- The scheduling stage computes yearly commitment/dispatch/SOC trajectories.
- The physical network is validated afterwards by the already-documented OPF and PF models in Sections 13 and 12, preserving one shared sign convention and one shared converter model family.

This separation is deliberate: the annual scheduling model remains tractable, while the electrical feasibility check still uses the detailed hybrid AC/DC models already implemented elsewhere in the notebook.

### 27.7.3 Chronological Horizon and Sets

Let the yearly horizon be

$$\mathcal{T}^{yr} = \{0, 1, \dots, T_{yr} - 1\}, \quad T_{yr} = \frac{8760}{\Delta t}. \quad (525)$$

For the base annual production-simulation layer, define:

$$\mathcal{G} : \text{dispatchable AC generators}, \quad \mathcal{S}^{ac} : \text{AC storage units}, \quad (526)$$

$$\mathcal{S}^{dc} : \text{DC storage units}, \quad \mathcal{R}^{ac} : \text{AC renewable generators / PV}, \quad (527)$$

$$\mathcal{R}^{dc} : \text{DC renewable or static generators} \quad (528)$$

$$\text{with profiles}, \quad \mathcal{L}^{ac}, \mathcal{L}^{dc} : \text{AC/DC loads}, \quad (529)$$

$$\mathcal{C}^{vsc} : \text{VSC converters}, \quad \mathcal{D}^{dcdc} : \text{DCDC converters}. \quad (530)$$

The per-step exogenous profiles are inherited directly from Section 27.6:

$$\sigma_\ell(t) : \text{load scaling}, \quad \rho_r(t) : \text{renewable availability}, \quad (531)$$

$$\gamma_k(t) : \text{optional forced-availability or} \quad (532)$$

$$\text{maintenance factor}. \quad (533)$$

### 27.7.4 Decision Variables

For each  $t \in \mathcal{T}^{yr}$ , the annual production-simulation model uses the following primary variables:

$$P_{g,t}, u_{g,t}, y_{g,t}^{su}, y_{g,t}^{sd}, \quad g \in \mathcal{G}, \quad (534)$$

$$P_{s,t}^{ch}, P_{s,t}^{dis}, E_{s,t}, \quad s \in \mathcal{S}^{ac} \cup \mathcal{S}^{dc}, \quad (535)$$

$$P_{r,t}^{ac}, P_{r,t}^{dc}, C_{r,t}^{curt}, \quad r \in \mathcal{R}^{ac} \cup \mathcal{R}^{dc}, \quad (536)$$

$$P_{c,t}^{ac}, Q_{c,t}^{ac}, P_{c,t}^{dc}, \quad c \in \mathcal{C}^{vsc}, \quad (537)$$

$$P_{d,t}^{in}, P_{d,t}^{out}, \quad d \in \mathcal{D}^{dcdc}, \quad (538)$$

$$L_{i,t}^{shed}, \quad i \in \mathcal{L}^{ac} \cup \mathcal{L}^{dc}. \quad (539)$$

where  $u_{g,t} \in \{0, 1\}$  is commitment status,  $y^{su}$  and  $y^{sd}$  are startup/shutdown indicators,  $E_{s,t}$  is stored energy, and  $L_{i,t}^{shed}$  is optional unserved energy for adequacy studies.

### 27.7.5 Objective Function

The recommended yearly objective is a weighted production-cost functional:

$$\min J^{\text{yr}} = \sum_{t \in \mathcal{T}^{\text{yr}}} \left[ \sum_{g \in \mathcal{G}} (c_{2g} P_{g,t}^2 + c_{1g} P_{g,t} + c_{0g} u_{g,t} + C_g^{\text{su}} y_{g,t}^{\text{su}} + C_g^{\text{sd}} y_{g,t}^{\text{sd}}) \right. \quad (540)$$

$$\left. + \sum_{s \in \mathcal{S}^{\text{ac}} \cup \mathcal{S}^{\text{dc}}} (c_s^{\text{cyc}} P_{s,t}^{\text{dis}} + c_s^{\text{deg}} (P_{s,t}^{\text{ch}} + P_{s,t}^{\text{dis}})) \right. \quad (541)$$

$$\left. + \sum_{r \in \mathcal{R}^{\text{ac}} \cup \mathcal{R}^{\text{dc}}} c_r^{\text{curt}} C_{r,t}^{\text{curt}} + \sum_i c_i^{\text{ens}} L_{i,t}^{\text{shed}} + \sum_g c_g^{\text{co2}} P_{g,t} \right] \Delta t. \quad (542)$$

This objective keeps the framework consistent with the existing linear UC cost model, but extends it in the directions needed for a production-simulation module: startup/shutdown accounting, storage cycling cost, renewable curtailment, optional loss-of-load penalty, and optional emissions accounting.

### 27.7.6 Core Constraints

**(1) AC and DC carrier balance at the schedule stage.** The year-ahead schedule can remain carrier-aware without requiring a full nonlinear network solve in the master problem. For each time step:

$$\sum_{g \in \mathcal{G}_i^{\text{ac}}} P_{g,t} + \sum_{r \in \mathcal{R}_i^{\text{ac}}} P_{r,t}^{\text{ac}} + \sum_{s \in \mathcal{S}_i^{\text{ac}}} (P_{s,t}^{\text{dis}} - P_{s,t}^{\text{ch}}) + \sum_{c \in \mathcal{C}_i^{\text{vsc}}} P_{c,t}^{\text{ac}} = P_{i,t}^{\text{load,ac}} - L_{i,t}^{\text{shed}} + \hat{P}_{i,t}^{\text{loss,ac}}, \quad (543)$$

$$\sum_{r \in \mathcal{R}_k^{\text{dc}}} P_{r,t}^{\text{dc}} + \sum_{s \in \mathcal{S}_k^{\text{dc}}} (P_{s,t}^{\text{dis}} - P_{s,t}^{\text{ch}}) + \sum_{c \in \mathcal{C}_k^{\text{vsc}}} P_{c,t}^{\text{dc}} + \sum_{d \in \mathcal{D}_k^{\text{out}}} P_{d,t}^{\text{out}} = P_{k,t}^{\text{load,dc}} - L_{k,t}^{\text{shed}} + \sum_{d \in \mathcal{D}_k^{\text{in}}} P_{d,t}^{\text{in}} + \hat{P}_{k,t}^{\text{loss,dc}}. \quad (544)$$

Here  $\hat{P}^{\text{loss}}$  is a schedule-stage loss proxy. The exact AC/DC network losses and voltage feasibility are checked later by the step-wise OPF/PF validation stage.

**(2) Load and renewable profile embedding.**

$$P_{i,t}^{\text{load,ac}} = P_i^{0,\text{ac}} \sigma_i(t), \quad Q_{i,t}^{\text{load,ac}} = Q_i^{0,\text{ac}} \sigma_i(t), \quad (545)$$

$$P_{k,t}^{\text{load,dc}} = P_k^{0,\text{dc}} \sigma_k(t), \quad 0 \leq P_{r,t}^{\text{ac}} \leq \bar{P}_r^{\text{ac}} \rho_r(t) \gamma_r(t), \quad (546)$$

$$0 \leq P_{r,t}^{\text{dc}} \leq \bar{P}_r^{\text{dc}} \rho_r(t) \gamma_r(t), \quad C_{r,t}^{\text{curt}} = \bar{P}_{r,t} - P_{r,t}. \quad (547)$$

**(3) Generator commitment logic.**

$$u_{g,t} - u_{g,t-1} = y_{g,t}^{\text{su}} - y_{g,t}^{\text{sd}}, \quad (548)$$

$$P_g^{\text{min}} u_{g,t} \leq P_{g,t} \leq P_g^{\text{max}} u_{g,t}, \quad (549)$$

$$0 \leq y_{g,t}^{\text{su}}, y_{g,t}^{\text{sd}} \leq 1, \quad y_{g,t}^{\text{su}}, y_{g,t}^{\text{sd}} \in \{0, 1\}. \quad (550)$$

(4) **Minimum up/down time and ramping.** For each generator:

$$\sum_{\tau=t-T_g^{\text{up}}+1}^t y_{g,\tau}^{\text{su}} \leq u_{g,t}, \quad (551)$$

$$\sum_{\tau=t-T_g^{\text{dn}}+1}^t y_{g,\tau}^{\text{sd}} \leq 1 - u_{g,t}, \quad (552)$$

$$P_{g,t} - P_{g,t-1} \leq R_g^{\text{up}} \Delta t, \quad (553)$$

$$P_{g,t-1} - P_{g,t} \leq R_g^{\text{dn}} \Delta t. \quad (554)$$

This is the natural yearly extension of the existing short-horizon UC model and uses the unit-commitment metadata already present in **Generator**.

(5) **Storage dynamics and annual cyclic boundary.** For every storage unit:

$$E_{s,t} = \alpha_s E_{s,t-1} + \eta_s^{\text{ch}} P_{s,t}^{\text{ch}} \Delta t - \frac{1}{\eta_s^{\text{dis}}} P_{s,t}^{\text{dis}} \Delta t, \quad (555)$$

$$E_s^{\text{min}} \leq E_{s,t} \leq E_s^{\text{max}}, \quad (556)$$

$$0 \leq P_{s,t}^{\text{ch}} \leq P_s^{\text{ch,max}}, \quad (557)$$

$$0 \leq P_{s,t}^{\text{dis}} \leq P_s^{\text{dis,max}}, \quad (558)$$

$$E_{s,T_{yr}-1} = E_{s,-1} + \Delta E_s^{\text{target}}. \quad (559)$$

The terminal condition is essential for annual studies; the default choice is  $\Delta E_s^{\text{target}} = 0$ , i.e. end-of-year SOC equals start-of-year SOC.

(6) **Converter coupling with schedule-stage proxies.** To stay consistent with the AC/DC component models already defined in Sections 12 and 13, the yearly schedule uses the same sign convention but allows a simplified master formulation:

$$P_{c,t}^{\text{ac}} + P_{c,t}^{\text{dc}} + \hat{P}_{c,t}^{\text{loss}} = 0, \quad c \in \mathcal{C}^{\text{vsc}}, \quad (560)$$

$$P_{d,t}^{\text{out}} = \eta_d^{\pm} P_{d,t}^{\text{in}} - \hat{P}_{d,t}^{\text{loss}}, \quad d \in \mathcal{D}^{\text{dcdc}}. \quad (561)$$

At the annual scheduling stage,  $\hat{P}^{\text{loss}}$  may be linear, piecewise-linear, or even zero if loss accounting is deferred. During per-step validation, these proxies are replaced by the exact nonlinear VSC and DCDC models already documented in the PF/OPF sections, including  $P_{\text{loss}} = a + bI + cI^2$  for VSC and  $I^2 R$  loss for DCDC.

(7) **Optional adequacy and reserve constraints.** For yearly production simulation, reserve or resource-adequacy constraints can be added without changing the replay pipeline:

$$\sum_{g \in \mathcal{G}} (P_g^{\text{max}} u_{g,t} - P_{g,t}) + \sum_s (P_s^{\text{dis,max}} - P_{s,t}^{\text{dis}}) \geq R_t^{\text{req}}, \quad \forall t. \quad (562)$$

If desired,  $L_{i,t}^{\text{shed}}$  can be kept strictly at zero for economic production simulation, or activated with a high penalty for adequacy studies.

### 27.7.7 Hierarchical Temporal Decomposition

**Motivation.** The monolithic formulation (542)–(562) contains  $\mathcal{O}(|\mathcal{G}| T_{yr})$  binary commitment variables. For a system with 50 generators and hourly resolution, this yields  $\sim 4.4 \times 10^5$  binary variables over 8760 time steps — well beyond the practical reach of general-purpose MILP solvers. The standard remedy is to split the annual horizon into a hierarchy of nested sub-horizons, where each level trades temporal resolution for problem size.

**Decomposition levels.** We partition the year into four nested levels with decreasing granularity in the master problems and increasing detail in the inner sub-problems:

| Level     | Scope   | Typical horizon | Resolution           | Key decisions                           |
|-----------|---------|-----------------|----------------------|-----------------------------------------|
| <b>L0</b> | Annual  | 1 year          | monthly              | maintenance, fuel and energy budgets    |
| <b>L1</b> | Monthly | 1 month         | weekly               | generation targets and hydro allocation |
| <b>L2</b> | Weekly  | 1 week          | hourly               | unit commitment, storage cycling        |
| <b>L3</b> | Daily   | 1–2 days        | hourly or sub-hourly | detailed UC and replay                  |

Each level solves a smaller optimisation problem embedded within the constraints inherited from the level above. Level L3 maps directly to the existing `solve_unit_commitment()` and `solve_time_series_pf()` pipeline.

**Level L0: annual planning.** The annual level operates on a coarsened representation of the year, using  $K$  blocks (e.g. 12 months or 52 weeks). Let  $\mathcal{B}^{(0)} = \{B_1, \dots, B_K\}$  partition  $\mathcal{T}^{\text{yr}}$ . For each block  $k$ :

$$\bar{E}_{g,k}^{\text{gen}} : \text{energy budget for generator } g, \quad (563)$$

$$\bar{E}_{s,k}^{\text{sto}} : \text{net energy throughput for storage } s, \quad (564)$$

$$m_{g,k} \in \{0, 1\} : \text{maintenance flag (forced outage),} \quad (565)$$

$$F_k^{\text{fuel}} : \text{fuel/emission budget for block } k. \quad (566)$$

The L0 objective approximates yearly cost using block-averaged energy prices and capacity availability:

$$\min J^{(0)} = \sum_{k=1}^K \left[ \sum_{g \in \mathcal{G}} \hat{c}_g \bar{E}_{g,k}^{\text{gen}} + \sum_g C_g^{\text{maint}} m_{g,k} + \sum_r c_r^{\text{curt}} \bar{C}_{r,k}^{\text{curt}} + \sum_i c_i^{\text{ens}} \bar{L}_{i,k}^{\text{shed}} \right], \quad (567)$$

subject to block energy balance, maintenance window constraints (minimum consecutive outage length, seasonal windows), and annual fuel/emission caps:

$$\sum_{k=1}^K F_k^{\text{fuel}} \leq F^{\text{annual}}, \quad (568)$$

$$\sum_{k=k_0}^{k_0+M_g-1} m_{g,k} \geq M_g \cdot w_{g,k_0}, \quad (\text{maintenance window of length } M_g), \quad (569)$$

$$\sum_{k=1}^K \bar{E}_{g,k}^{\text{gen}} \leq E_g^{\text{annual,max}} (1 - m_{g,k}). \quad (570)$$

**Level L1: monthly sub-problems.** Each month  $k$  inherits from L0:

- Generator availability mask:  $m_{g,k} = 1 \Rightarrow$  generator  $g$  is on forced outage for all weeks in month  $k$ .
- Energy budget ceiling:  $\sum_{t \in B_k} P_{g,t} \Delta t \leq \bar{E}_{g,k}^{\text{gen}}$ .
- Fuel allocation: total fuel consumption in month  $k$  must not exceed  $F_k^{\text{fuel}}$ .
- Boundary SOC:  $E_{s,t_{\text{start}(k)}} = E_{s,k}^{\text{init}}$ , where  $E_{s,k}^{\text{init}}$  is either fixed by L0 or carried from the terminal state of month  $k-1$ .

The monthly sub-problem uses weekly representative blocks or full hourly resolution within the month, solving a UC-like problem with the inherited constraints tightening the feasible region.

**Level L2: weekly rolling commitment.** Within each month, the weekly level solves a rolling-horizon UC over  $T^{(2)} = 168/\Delta t$  steps (one week) plus a look-ahead buffer  $T^{\text{la}}$  (typically 24–48 hours):

$$\mathcal{T}_w^{(2)} = \{t_w, t_w+1, \dots, t_w+T^{(2)}+T^{\text{la}}-1\}, \quad (571)$$

where commitment decisions in the look-ahead window  $[t_w+T^{(2)}, t_w+T^{(2)}+T^{\text{la}}-1]$  are advisory (not binding). Only the binding window’s schedule is retained before the horizon rolls forward.

Boundary conditions from L1:

$$u_{g,t_w-1}, P_{g,t_w-1}, E_{s,t_w-1} : \quad \text{initial commitment, dispatch, and SOC from end of previous week.} \quad (572)$$

The formulation within each weekly window is identical to the monolithic model (542)–(562), but restricted to  $\mathcal{T}_w^{(2)}$  and augmented with the L1 energy-budget constraint:

$$\sum_{t \in \mathcal{T}_w^{(2)} \cap B_k} P_{g,t} \Delta t \leq \bar{E}_{g,k}^{\text{gen}} - \sum_{t' < t_w, t' \in B_k} P_{g,t'}^* \Delta t, \quad (573)$$

where  $P_{g,t'}^*$  denotes already-committed dispatch from earlier weeks in the same month.

**Level L3: daily detailed UC + OPF/PF.** The innermost level processes each day (or 1–2 day window) at full temporal resolution. This level is *identical* to the existing short-horizon `solve_unit_commitment()` followed by `solve_time_series_pf()`, inheriting:

- Commitment status and dispatch from L2 as initial conditions or must-run/must-off constraints.
- SOC from the end of the previous day.
- Generator availability from L0 maintenance flags.

The replayed OPF/PF gives per-step voltage, loss, and flow results that feed into the annual metrics in §27.7.9.

**Coordination protocol.** The levels are linked by the following cascade of complicating (coupling) variables:

| From → To | Coupling variables                                                              | Direction |
|-----------|---------------------------------------------------------------------------------|-----------|
| L0 → L1   | $m_{g,k}, \bar{E}_{g,k}^{\text{gen}}, F_k^{\text{fuel}}, E_{s,k}^{\text{init}}$ | top-down  |
| L1 → L2   | $\bar{E}_{g,k}^{\text{gen,remaining}}, E_{s,t_w}^{\text{init}}, u_{g,t_w-1}$    | top-down  |
| L2 → L3   | $u_{g,t_d-1}, P_{g,t_d-1}, E_{s,t_d-1}$                                         | top-down  |
| L3 → L2   | Actual dispatch, SOC, and cost realisation                                      | bottom-up |
| L2 → L1   | Accumulated energy, fuel consumption                                            | bottom-up |

The default execution mode is a **single forward pass** (top-down only), which is the most common approach in industry production-simulation tools. An optional **iterative feedback** mode re-solves upper levels using realised cost/energy information from lower levels, which can improve inter-temporal coordination (e.g. hydro scheduling or long-duration storage) at the cost of additional solve time.

**Algorithmic outline.** The hierarchical production-simulation pipeline is:

```

1: procedure SOLVEANNUALPRODUCTIONSIMULATION(sys, ts_data, opts)
2: L0: Solve annual planning LP/MILP $\rightarrow$ maintenance schedule $\{m_{g,k}\}$, energy budgets
 $\{\bar{E}_{g,k}\}$, fuel allocations $\{F_k\}$, boundary SOC $\{E_{s,k}^{\text{init}}\}$
3: for each month $k = 1, \dots, 12$ do
4: L1: Solve monthly sub-problem with L0 constraints $\rightarrow$ weekly targets, refined SOC
 trajectories
5: for each week w in month k do
6: L2: Solve rolling weekly UC over $\mathcal{T}_w^{(2)}$ with look-ahead
7: Inherit u_{g,t_w-1} , E_{s,t_w-1} from previous week
8: Enforce remaining energy budget from L1 via (573)
9: for each day d in binding window of week w do
10: L3: Solve daily UC + sequential OPF/PF via existing solve_time_series_
 pf() pipeline
11: Store per-step results: voltage, losses, flows, costs
12: end for
13: end for
14: if opts.iterative_feedback then
15: Feed realised monthly cost/energy back to L0; re-solve if budget violation >
 tolerance
16: end if
17: end for
18: Aggregate all per-step results into annual metrics (§27.7.9)
19: end procedure

```

**Computational complexity reduction.** Let  $N_g = |\mathcal{G}|$  and assume hourly resolution. The monolithic problem has  $\sim N_g \cdot 8760$  binaries. With the hierarchy:

- L0:  $\sim N_g \cdot 12$  maintenance binaries (easily solvable).
- L1:  $\sim N_g \cdot 4$  weekly blocks per month (or full hourly:  $\sim N_g \cdot 720$  — still tractable).
- L2:  $\sim N_g \cdot (168 + T^{\text{la}})$  binaries per window (standard UC size).
- L3:  $\sim N_g \cdot 24$  binaries per day (existing pipeline).

The total solve effort becomes  $\sum_{w=1}^{52} \mathcal{O}(N_g \cdot 192)$  plus overhead, which is orders of magnitude cheaper than a single  $\mathcal{O}(N_g \cdot 8760)$  MILP. The trade-off is that inter-temporal optimality across weeks is approximate rather than exact, but this is the standard and accepted practice in production-simulation studies.

**Mapping to the C++ framework.** The hierarchical structure maps to the existing API as follows:

- **L0:** `solve_annual_plan()` returns `AnnualPlanResult` carrying maintenance masks, energy budgets, and boundary SOC per block.
- **L1:** Not implemented; the orchestrator proceeds directly from L0 blocks to L2 weekly windows.
- **L2:** `solve_weekly_uc()` calls `solve_unit_commitment()` with a 168-step `TimeSeriesData` sliced from the yearly horizon, plus a configurable look-ahead buffer.
- **L3:** `solve_daily_replay()` calls `solve_time_series_pf()` with `skip_uc=true` and the pre-solved weekly schedule sliced to the daily window.

- The outer loop is `solve_annual_production_simulation()`, which assembles sub-horizons and manages boundary handoff.

### 27.7.8 Coupling to OPF/PF Replay

Whether the annual problem is solved monolithically or via the hierarchical decomposition in §27.7.7, the scheduled dispatch for each time step is not the final electrical truth. Instead, each step is converted into a physical snapshot and replayed through the existing OPF/PF pipeline:

$$\Pi^{\text{yr}} \xrightarrow{\text{write } \text{sys}_t} \text{solve\_ac\_opf}(\text{sys}_t, \text{opt}_t) \xrightarrow{\text{validation}} \text{solve\_power\_flow}(\text{sys}_t, \text{opt}_t). \quad (574)$$

This gives the production-simulation module three nested fidelity levels:

1. **Schedule level:** chronological annual MILP/QP on dispatch, commitment, storage, curtailment, and converter transfer proxies.
2. **Operational refinement:** optional per-step OPF with the exact AC/DC formulation and converter equations.
3. **Physical validation:** per-step hybrid PF to verify that the replayed operating point remains feasible in the detailed network model.

### 27.7.9 Annual Output Metrics

The yearly result object should aggregate both economics and physical quality:

$$C^{\text{tot}} = J^{\text{yr}}, \quad (575)$$

$$E_g^{\text{gen}} = \sum_{t \in \mathcal{T}^{\text{yr}}} P_{g,t} \Delta t, \quad (576)$$

$$E_r^{\text{curt}} = \sum_{t \in \mathcal{T}^{\text{yr}}} C_{r,t}^{\text{curt}} \Delta t, \quad (577)$$

$$E^{\text{ens}} = \sum_{t \in \mathcal{T}^{\text{yr}}} \sum_i L_{i,t}^{\text{shed}} \Delta t, \quad (578)$$

$$E^{\text{loss}} = \sum_{t \in \mathcal{T}^{\text{yr}}} P_t^{\text{loss}} \Delta t, \quad (579)$$

$$\text{CF}_g = \frac{E_g^{\text{gen}}}{P_g^{\text{max}} T_{\text{yr}} \Delta t}. \quad (580)$$

These annual metrics align naturally with the existing `TimeSeriesPFResult` style and can be exported together with per-step cross-validation statistics already supported by the UC→OPF→PF path.

## 27.8 Unit Commitment MILP (Implemented)

### 27.8.1 Sets and Variables

Let:

$$\mathcal{G} = \text{in-service AC generators}, \quad \mathcal{S} = \text{in-service AC storage units}, \quad (581)$$

$$\mathcal{R} = \text{in-service AC renewable generators}, \quad \mathcal{T} = \{0, 1, \dots, T-1\}. \quad (582)$$

Decision variables:

$$P_{g,t}, u_{g,t} \in \{0, 1\}, s_{g,t} \geq 0, P_{s,t}^{\text{ess}}, E_{s,t}^{\text{soc}}, P_{r,t}^{\text{ren}}. \quad (583)$$

where:

- $P_{g,t}$ : generator active output,
- $u_{g,t}$ : generator commitment,
- $s_{g,t}$ : startup-cost auxiliary variable,
- $P_{s,t}^{\text{ess}}$ : signed storage dispatch (+ discharge, – charge),
- $E_{s,t}^{\text{soc}}$ : storage SOC state,
- $P_{r,t}^{\text{ren}}$ : renewable dispatch.

### 27.8.2 Objective Function

The implemented objective is linear:

$$\min J = \sum_{t \in \mathcal{T}} \left[ \sum_{g \in \mathcal{G}} c_{1g} P_{g,t} \Delta t + \sum_{g \in \mathcal{G}} s_{g,t} - \sum_{r \in \mathcal{R}} c_r^{\text{curt}} P_{r,t}^{\text{ren}} \Delta t \right]. \quad (584)$$

The negative renewable term is equivalent (up to a constant) to a curtailment penalty  $c_r^{\text{curt}}(\bar{P}_{r,t}^{\text{ren}} - P_{r,t}^{\text{ren}})$ , therefore it drives renewable utilization toward availability bounds.

### 27.8.3 Constraints

#### (1) System-wide active-power balance (single-bus approximation).

$$\sum_{g \in \mathcal{G}} P_{g,t} + \sum_{s \in \mathcal{S}} P_{s,t}^{\text{ess}} + \sum_{r \in \mathcal{R}} P_{r,t}^{\text{ren}} = P_t^{\text{load}}, \quad \forall t \in \mathcal{T}. \quad (585)$$

Load aggregation from AC loads:

$$P_t^{\text{load}} = \sum_{\ell \in \mathcal{L}} P_\ell^0 \sigma_\ell(t) \kappa_\ell, \quad (586)$$

with  $\kappa_\ell = \text{Load}::\text{scaling}$ .

#### (2) Generator output-commitment coupling.

$$0 \leq P_{g,t} \leq P_g^{\text{max}} u_{g,t}, \quad P_{g,t} \geq P_g^{\text{min}} u_{g,t}, \quad \forall g, t. \quad (587)$$

#### (3) Generator ramp limits.

$$P_{g,t} - P_{g,t-1} \leq R_g^{\text{up}} (60\Delta t), \quad (588)$$

$$P_{g,t-1} - P_{g,t} \leq R_g^{\text{dn}} (60\Delta t), \quad \forall g, t \geq 1. \quad (589)$$

#### (4) Startup-cost linearization.

$$s_{g,0} \geq C_g^{\text{su}} u_{g,0}, \quad (590)$$

$$s_{g,t} \geq C_g^{\text{su}} (u_{g,t} - u_{g,t-1}), \quad t \geq 1. \quad (591)$$

In matrix form this is implemented as  $-s + C^{\text{su}} u_t - C^{\text{su}} u_{t-1} \leq 0$ .

(5) **Storage SOC dynamics (linearized signed-power form).** For storage unit  $s$ , define

$$\eta_s = \sqrt{\eta_s^{\text{ch}} \eta_s^{\text{dis}}}, \quad \alpha_s = 1 - \frac{\text{self\_discharge\_pct}_s}{100}, \quad (592)$$

$$\beta_s = \frac{\Delta t}{\eta_s E_s^{\text{rated}}}. \quad (593)$$

Then:

$$E_{s,0}^{\text{soc}} + \beta_s P_{s,0}^{\text{ess}} = \alpha_s E_s^{\text{soc,init}}, \quad (594)$$

$$E_{s,t}^{\text{soc}} - \alpha_s E_{s,t-1}^{\text{soc}} + \beta_s P_{s,t}^{\text{ess}} = 0, \quad t \geq 1, \quad (595)$$

with bounds

$$E_s^{\text{soc,min}} \leq E_{s,t}^{\text{soc}} \leq E_s^{\text{soc,max}}, \quad P_s^{\text{ess,min}} \leq P_{s,t}^{\text{ess}} \leq P_s^{\text{ess,max}}. \quad (596)$$

(6) **Renewable availability bounds.**

$$0 \leq P_{r,t}^{\text{ren}} \leq \bar{P}_{r,t}^{\text{ren}}, \quad \bar{P}_{r,t}^{\text{ren}} = P_r^{\text{rated}} \rho_r(t), \quad \forall r, t. \quad (597)$$

#### 27.8.4 Model-Class and Solution Output

The current formulation is a **MILP** and is solved by the selected MILP adapter. Solution vector  $x$  is unpacked into `UCSchedule::gen_dispatch/gen_commit/ess_dispatch/ess_soc/renewable_dispatch`, plus `total_cost`, `feasible`, and `solver_name`.

#### 27.8.5 Implementation Scope Notes (Important)

The present UC implementation is intentionally compact and differs from a full network-constrained SCUC:

- DC network matrices are constructed, but UC constraints currently use the aggregated single-bus balance (585).
- No line-flow, voltage, reserve, N-1, or zonal constraints are included.
- `min_up_time_hr`, `min_dn_time_hr`, and shutdown cost are not yet enforced in this runtime UC model.
- Objective currently uses linear `cost_c1`; quadratic `cost_c2`/constant `cost_c0` are not part of the active UC MILP.

### 27.9 Distribution-System Counterpart: Time-Series Topology Reconfiguration

**Scope separation (important).** The UC model in §27.8 is the scheduling layer for **transmission-oriented** studies (commit/decommit central generators). For **distribution systems**, the analogous scheduling layer is *time-series topology reconfiguration* (TS-TR), where binary decisions operate feeder switches and tie lines across time.

In short:

$$\text{Transmission: } \{u_{g,t}\}_{g,t} \iff \text{Distribution: } \{z_{e,t}\}_{e,t}. \quad (598)$$

### 27.9.1 Sets, Parameters, and Decision Variables

Let the hybrid distribution network be split into AC and DC subgraphs:

$$\mathcal{G}^{\text{ac}} = (\mathcal{N}^{\text{ac}}, \mathcal{E}^{\text{ac}}), \quad \mathcal{G}^{\text{dc}} = (\mathcal{N}^{\text{dc}}, \mathcal{E}^{\text{dc}}), \quad (599)$$

with switchable sets  $\mathcal{E}^{\text{ac,sw}} \subseteq \mathcal{E}^{\text{ac}}$ ,  $\mathcal{E}^{\text{dc,sw}} \subseteq \mathcal{E}^{\text{dc}}$ , root sets  $\mathcal{N}_0^{\text{ac}}, \mathcal{N}_0^{\text{dc}}$ , converters  $\mathcal{C}^{\text{vsc}}$  (AC/DC), DC/DC converters  $\mathcal{D}^{\text{dcdc}}$ , and  $\mathcal{T} = \{0, 1, \dots, T-1\}$ .

Distributed-resource sets are modeled on both sides:

$$\mathcal{G}^{\text{ac}}, \mathcal{G}^{\text{dc}} : \text{dispatchable DG}, \quad (600)$$

$$\mathcal{R}^{\text{ac}}, \mathcal{R}^{\text{dc}} : \text{renewables}, \quad (601)$$

$$\mathcal{S}^{\text{ac}}, \mathcal{S}^{\text{dc}} : \text{storage}, \quad (602)$$

$$\mathcal{F}^{\text{ac}}, \mathcal{F}^{\text{dc}} : \text{flexible loads / charger-type controllable demand}. \quad (603)$$

Main decision variables (per  $t$ ):

$$z_{e,t}^{\text{ac}}, z_{\ell,t}^{\text{dc}} \in \{0, 1\} \quad (\text{AC/DC branch closed/open}), \quad (604)$$

$$y_{e,t}^{\text{ac,on}}, y_{e,t}^{\text{ac,off}} \in \{0, 1\}, y_{\ell,t}^{\text{dc,on}}, y_{\ell,t}^{\text{dc,off}} \in \{0, 1\} \quad (\text{switching actions}), \quad (605)$$

$$P_{e,t}^{\text{ac}}, Q_{e,t}^{\text{ac}}, \ell_{e,t}^{\text{ac}}, v_{i,t}^{\text{ac}} \quad (\text{AC DistFlow}), \quad (606)$$

$$P_{\ell,t}^{\text{dc}}, \ell_{\ell,t}^{\text{dc}}, v_{k,t}^{\text{dc}} \quad (\text{DC branch variables}), \quad (607)$$

$$P_{c,t}^{\text{ac}}, Q_{c,t}^{\text{ac}}, P_{c,t}^{\text{dc}}, p_{c,t}^{\text{loss,vsc}} \quad \forall c \in \mathcal{C}^{\text{vsc}}, \quad (608)$$

$$p_{d,t}^{\text{in}}, p_{d,t}^{\text{out}} \quad \forall d \in \mathcal{D}^{\text{dcdc}}, \quad (609)$$

$$p_{g,t}^{\text{dg}}, p_{r,t}^{\text{curt}}, p_{s,t}^{\text{ch}}, p_{s,t}^{\text{dis}}, E_{s,t}, \Delta p_{f,t}^{\text{flex}} \quad (\text{DER schedule}), \quad (610)$$

$$p_{i,t}^{\text{shed,ac}}, p_{k,t}^{\text{shed,dc}} \geq 0 \quad (\text{load shedding}). \quad (611)$$

All nodal injection equations below use the same sign convention as PF: **bus injection is positive**.

### 27.9.2 Objective Function (Hybrid TS-TR)

A practical hybrid objective is:

$$\begin{aligned} \min J^{\text{tr}} = \sum_{t \in \mathcal{T}} & \left[ c^{\text{ac,loss}} \sum_{e \in \mathcal{E}^{\text{ac}}} r_e^{\text{ac}} \ell_{e,t}^{\text{ac}} \Delta t + c^{\text{dc,loss}} \sum_{\ell \in \mathcal{E}^{\text{dc}}} r_{\ell}^{\text{dc}} \ell_{\ell,t}^{\text{dc}} \Delta t \right. \\ & + c^{\text{vsc,loss}} \sum_{c \in \mathcal{C}^{\text{vsc}}} p_{c,t}^{\text{loss,vsc}} \Delta t + c^{\text{dcdc,loss}} \sum_{d \in \mathcal{D}^{\text{dcdc}}} (p_{d,t}^{\text{in}} - p_{d,t}^{\text{out}}) \Delta t \\ & + c^{\text{shed}} \left( \sum_{i \in \mathcal{N}^{\text{ac}}} p_{i,t}^{\text{shed,ac}} + \sum_{k \in \mathcal{N}^{\text{dc}}} p_{k,t}^{\text{shed,dc}} \right) \Delta t + c^{\text{curt}} \sum_r p_{r,t}^{\text{curt}} \Delta t \\ & \left. + c^{\text{sw,ac}} \sum_{e \in \mathcal{E}^{\text{ac,sw}}} (y_{e,t}^{\text{ac,on}} + y_{e,t}^{\text{ac,off}}) + c^{\text{sw,dc}} \sum_{\ell \in \mathcal{E}^{\text{dc,sw}}} (y_{\ell,t}^{\text{dc,on}} + y_{\ell,t}^{\text{dc,off}}) \right]. \quad (612) \end{aligned}$$

### 27.9.3 Core Constraints

(1) **AC and DC nodal balance with DER and converter injections.** For  $i \in \mathcal{N}^{\text{ac}}$ ,  $k \in \mathcal{N}^{\text{dc}}$ :

$$\sum_{e:(m \rightarrow i)} (P_{e,t}^{\text{ac}} - r_e^{\text{ac}} \ell_{e,t}^{\text{ac}}) - \sum_{e:(i \rightarrow m)} P_{e,t}^{\text{ac}} + p_{i,t}^{\text{inj,ac}} + p_{i,t}^{\text{shed,ac}} = p_{i,t}^{\text{load,ac}}, \quad (613)$$

$$\sum_{e:(m \rightarrow i)} (Q_{e,t}^{\text{ac}} - x_e^{\text{ac}} \ell_{e,t}^{\text{ac}}) - \sum_{e:(i \rightarrow m)} Q_{e,t}^{\text{ac}} + q_{i,t}^{\text{inj,ac}} = q_{i,t}^{\text{load,ac}}, \quad (614)$$

$$\sum_{\ell:(u \rightarrow k)} (P_{\ell,t}^{\text{dc}} - r_{\ell}^{\text{dc}} \ell_{\ell,t}^{\text{dc}}) - \sum_{\ell:(k \rightarrow u)} P_{\ell,t}^{\text{dc}} + p_{k,t}^{\text{inj,dc}} + p_{k,t}^{\text{shed,dc}} = p_{k,t}^{\text{load,dc}}. \quad (615)$$

Injection decomposition (same sign convention):

$$\begin{aligned} p_{i,t}^{\text{inj,ac}} = & \sum_{g \in \mathcal{G}_i^{\text{ac}}} p_{g,t}^{\text{dg}} + \sum_{r \in \mathcal{R}_i^{\text{ac}}} (p_{r,t}^{\text{ava}} - p_{r,t}^{\text{curt}}) + \sum_{s \in \mathcal{S}_i^{\text{ac}}} (p_{s,t}^{\text{dis}} - p_{s,t}^{\text{ch}}) \\ & - \sum_{f \in \mathcal{F}_i^{\text{ac}}} \Delta p_{f,t}^{\text{flex}} + \sum_{c: \text{ac}(c)=i} P_{c,t}^{\text{ac}}, \end{aligned} \quad (616)$$

$$\begin{aligned} p_{k,t}^{\text{inj,dc}} = & \sum_{g \in \mathcal{G}_k^{\text{dc}}} p_{g,t}^{\text{dg}} + \sum_{r \in \mathcal{R}_k^{\text{dc}}} (p_{r,t}^{\text{ava}} - p_{r,t}^{\text{curt}}) + \sum_{s \in \mathcal{S}_k^{\text{dc}}} (p_{s,t}^{\text{dis}} - p_{s,t}^{\text{ch}}) \\ & - \sum_{f \in \mathcal{F}_k^{\text{dc}}} \Delta p_{f,t}^{\text{flex}} + \sum_{c: \text{dc}(c)=k} P_{c,t}^{\text{dc}} + \sum_{d: \text{dst}(d)=k} p_{d,t}^{\text{out}} - \sum_{d: \text{src}(d)=k} p_{d,t}^{\text{in}}. \end{aligned} \quad (617)$$

(2) **AC/DC branch constraints under switching.**

$$|v_{j,t}^{\text{ac}} - v_{i,t}^{\text{ac}} + 2(r_e^{\text{ac}} P_{e,t}^{\text{ac}} + x_e^{\text{ac}} Q_{e,t}^{\text{ac}}) - ((r_e^{\text{ac}})^2 + (x_e^{\text{ac}})^2) \ell_{e,t}^{\text{ac}}| \leq M_e^{\text{ac,v}} (1 - z_{e,t}^{\text{ac}}), \quad (618)$$

$$|v_{v,t}^{\text{dc}} - v_{u,t}^{\text{dc}} + 2r_{\ell}^{\text{dc}} P_{\ell,t}^{\text{dc}} - (r_{\ell}^{\text{dc}})^2 \ell_{\ell,t}^{\text{dc}}| \leq M_{\ell}^{\text{dc,v}} (1 - z_{\ell,t}^{\text{dc}}), \quad (619)$$

$$(P_{e,t}^{\text{ac}})^2 + (Q_{e,t}^{\text{ac}})^2 \leq v_{i,t}^{\text{ac}} \ell_{e,t}^{\text{ac}}, \quad (620)$$

$$(P_{\ell,t}^{\text{dc}})^2 \leq v_{u,t}^{\text{dc}} \ell_{\ell,t}^{\text{dc}}, \quad (621)$$

$$|P_{e,t}^{\text{ac}}| \leq \bar{P}_e^{\text{ac}} z_{e,t}^{\text{ac}}, \quad |Q_{e,t}^{\text{ac}}| \leq \bar{Q}_e^{\text{ac}} z_{e,t}^{\text{ac}}, \quad 0 \leq \ell_{e,t}^{\text{ac}} \leq \bar{\ell}_e^{\text{ac}} z_{e,t}^{\text{ac}}, \quad (622)$$

$$|P_{\ell,t}^{\text{dc}}| \leq \bar{P}_{\ell}^{\text{dc}} z_{\ell,t}^{\text{dc}}, \quad 0 \leq \ell_{\ell,t}^{\text{dc}} \leq \bar{\ell}_{\ell}^{\text{dc}} z_{\ell,t}^{\text{dc}}. \quad (623)$$

(3) **Converter coupling constraints.**

$$P_{c,t}^{\text{ac}} + P_{c,t}^{\text{dc}} + p_{c,t}^{\text{loss,vsc}} = 0, \quad \forall c \in \mathcal{C}^{\text{vsc}}, \quad (624)$$

$$p_{c,t}^{\text{loss,vsc}} \geq a_c^{\text{vsc}} + b_c^{\text{vsc}} p_{c,t}^{\text{abs,vsc}}, \quad p_{c,t}^{\text{abs,vsc}} \geq \pm P_{c,t}^{\text{dc}}, \quad (625)$$

$$(P_{c,t}^{\text{ac}})^2 + (Q_{c,t}^{\text{ac}})^2 \leq (S_c^{\text{rated}})^2 u_{c,t}^{\text{vsc}}, \quad |P_{c,t}^{\text{dc}}| \leq \bar{P}_c^{\text{dc,vsc}} u_{c,t}^{\text{vsc}}, \quad (626)$$

$$0 \leq p_{d,t}^{\text{in}} \leq \bar{P}_d^{\text{in}} u_{d,t}^{\text{dc,dc}}, \quad p_{d,t}^{\text{out}} = \eta_d^{\text{dc,dc}} p_{d,t}^{\text{in}}, \quad \forall d \in \mathcal{D}^{\text{dc,dc}}. \quad (627)$$

#### (4) DER operating constraints (AC+DC).

$$\underline{P}_g u_{g,t} \leq p_{g,t}^{\text{dg}} \leq \bar{P}_g u_{g,t}, \quad \forall g, t, \quad (628)$$

$$0 \leq p_{r,t}^{\text{curt}} \leq p_{r,t}^{\text{ava}}, \quad \forall r, t, \quad (629)$$

$$E_{s,t} = E_{s,t-1} + \eta_s^{\text{ch}} p_{s,t}^{\text{ch}} \Delta t - \frac{1}{\eta_s^{\text{dis}}} p_{s,t}^{\text{dis}} \Delta t, \quad \forall s, t \geq 1, \quad (630)$$

$$\underline{E}_s \leq E_{s,t} \leq \bar{E}_s, \quad 0 \leq p_{s,t}^{\text{ch}} \leq \bar{P}_s^{\text{ch}}, \quad 0 \leq p_{s,t}^{\text{dis}} \leq \bar{P}_s^{\text{dis}}, \quad \forall s, t, \quad (631)$$

$$-\Delta P_f^{\text{dn}} \leq \Delta p_{f,t}^{\text{flex}} \leq \Delta P_f^{\text{up}}, \quad \sum_{t \in \mathcal{T}} \Delta p_{f,t}^{\text{flex}} = 0, \quad \forall f. \quad (632)$$

#### 27.9.4 Component-Wise Sub-Equations (One-to-One with C++ Models)

This subsection refines (616)–(617) into component-level sub-equations aligned with runtime structs in `ACSystem`, `DCSystem`, and `HybridPowerSystem`. The sign rule remains unchanged: **bus injection is positive**.

**AC generation and inverter-based DER (`Generator`, `StaticGenerator`, `RenewableGen`, `PVSystem`).**

$$\underline{P}_g u_{g,t} \leq p_{g,t}^{\text{gen}} \leq \bar{P}_g u_{g,t}, \quad \underline{Q}_g u_{g,t} \leq q_{g,t}^{\text{gen}} \leq \bar{Q}_g u_{g,t}, \quad (633)$$

$$-R_g^{\text{dn}} \Delta t \leq p_{g,t}^{\text{gen}} - p_{g,t-1}^{\text{gen}} \leq R_g^{\text{up}} \Delta t, \quad \forall g \in \text{Generator},$$

$$p_{s,t}^{\text{s-gen,ac}} = \alpha_{s,t}^{\text{s-gen}} P_s^{\text{rated}}, \quad \underline{P}_s \leq p_{s,t}^{\text{s-gen,ac}} \leq \bar{P}_s, \quad \underline{Q}_s \leq q_{s,t}^{\text{s-gen,ac}} \leq \bar{Q}_s, \quad \forall s \in \text{StaticGenerator}, \quad (634)$$

$$p_{r,t}^{\text{ren,ac}} = p_{r,t}^{\text{ava}} - p_{r,t}^{\text{curt}}, \quad 0 \leq p_{r,t}^{\text{curt}} \leq p_{r,t}^{\text{ava}}, \quad \underline{Q}_r \leq q_{r,t}^{\text{ren,ac}} \leq \bar{Q}_r, \quad \forall r \in \text{RenewableGen}, \quad (635)$$

$$0 \leq p_{p,t}^{\text{pv}} \leq p_{p,t}^{\text{pv,ava}}, \quad (p_{p,t}^{\text{pv}})^2 + (q_{p,t}^{\text{pv}})^2 \leq (S_p^{\text{pv}})^2, \quad \forall p \in \text{PVSystem}. \quad (636)$$

**AC demand-side models (`Load`, `FlexibleLoad`, `AsymmetricLoad`, `ChargingStation/Charger`).** Let  $i = b(\ell)$  be the host bus of load-like component  $\ell$ , and  $\hat{v}_{i,t}^{\text{ac}} = \sqrt{v_{i,t}^{\text{ac}}}$ .

$$p_{\ell,t}^{\text{load}} = P_{\ell,t}^0 (k_{\ell,p}^Z v_{i,t}^{\text{ac}} + k_{\ell,p}^I \hat{v}_{i,t}^{\text{ac}} + k_{\ell,p}^P) - p_{\ell,t}^{\text{shed}}, \quad (637)$$

$$q_{\ell,t}^{\text{load}} = Q_{\ell,t}^0 (k_{\ell,q}^Z v_{i,t}^{\text{ac}} + k_{\ell,q}^I \hat{v}_{i,t}^{\text{ac}} + k_{\ell,q}^P), \quad \forall \ell \in \text{Load},$$

$$p_{f,t}^{\text{flex}} = P_{f,t}^0 + \Delta p_{f,t}^{\text{flex}}, \quad -\Delta P_f^{\text{dn}} \leq \Delta p_{f,t}^{\text{flex}} \leq \Delta P_f^{\text{up}}, \quad \sum_t \Delta p_{f,t}^{\text{flex}} \Delta t = 0, \quad \forall f \in \text{FlexibleLoad}, \quad (638)$$

$$p_{a,t}^{\text{asym}} = \kappa_{a,t} (p_{a\phi a,t} + p_{a\phi b,t} + p_{a\phi c,t}),$$

$$q_{a,t}^{\text{asym}} = \kappa_{a,t} (q_{a\phi a,t} + q_{a\phi b,t} + q_{a\phi c,t}), \quad \forall a \in \text{AsymmetricLoad}, \quad (639)$$

$$p_{c,t}^{\text{cs}} = \sum_{h \in \mathcal{H}(c)} p_{h,t}^{\text{charger}}, \quad \forall c \in \mathcal{C}^{\text{cs}},$$

$$q_{c,t}^{\text{cs}} = p_{c,t}^{\text{cs}} \tan(\varphi_c), \quad \varphi_c = \arccos(\text{pf}_c), \quad \forall c \in \mathcal{C}^{\text{cs}},$$

$$-\bar{p}_h^{\text{v2g}} \leq p_{h,t}^{\text{charger}}, \quad (640)$$

$$p_{h,t}^{\text{charger}} \leq \bar{p}_h^{\text{ch}}.$$

where  $\mathcal{H}^{\text{ev}}$  is the set of active chargers aggregated by all charging stations, and (640) applies to all  $h \in \mathcal{H}^{\text{ev}}$ .

**Storage and aggregation component equations.** This block corresponds to `Storage`, `MobileStorage`, `VirtualPowerPlant`, `Microgrid`, `Shunt`, and `ExternalGrid`. Define

$\mathcal{S}^{\text{all}} = \mathcal{S}^{\text{ac}} \cup \mathcal{S}^{\text{dc}}$  for stationary storage tables in AC and DC subsystems.

$$E_{s,t} = (1 - \lambda_s \Delta t) E_{s,t-1} + \eta_s^{\text{ch}} p_{s,t}^{\text{ch}} \Delta t - \frac{1}{\eta_s^{\text{dis}}} p_{s,t}^{\text{dis}} \Delta t, \quad \forall s \in \mathcal{S}^{\text{all}}, \quad (641)$$

$$\begin{aligned} 0 &\leq p_{s,t}^{\text{ch}} \leq \bar{P}_s^{\text{ch}} u_{s,t}^{\text{ch}}, \\ 0 &\leq p_{s,t}^{\text{dis}} \leq \bar{P}_s^{\text{dis}} (1 - u_{s,t}^{\text{ch}}), \end{aligned}$$

$$\underline{E}_s \leq E_{s,t} \leq \bar{E}_s,$$

$$\sum_{i \in \mathcal{N}^{\text{ac}}} x_{m,i,t}^{\text{mob}} + x_{m,t}^{\text{travel}} = 1,$$

$$p_{m,t}^{\text{mob}} = p_{m,t}^{\text{mob,dis}} - p_{m,t}^{\text{mob,ch}},$$

$$E_{m,t}^{\text{mob}} = E_{m,t-1}^{\text{mob}} + \Delta E_{m,t}^{\text{mob}}, \quad \forall m \in \text{MobileStorage}, \quad (642)$$

$$\underline{P}_v^{\text{vpp}} \leq p_{v,t}^{\text{vpp}} \leq \bar{P}_v^{\text{vpp}},$$

$$\underline{R}_v^{\text{dn}} \leq p_{v,t}^{\text{vpp}} - p_{v,t-1}^{\text{vpp}},$$

$$p_{v,t}^{\text{vpp}} - p_{v,t-1}^{\text{vpp}} \leq \bar{R}_v^{\text{up}}, \quad \forall v \in \text{VirtualPowerPlant}, \quad (643)$$

$$\underline{P}_m^{\text{mg}} \leq p_{m,t}^{\text{mg}} \leq \bar{P}_m^{\text{mg}}, \quad \forall m \in \text{Microgrid},$$

$$p_{m,t}^{\text{mg}} = 0, \quad \forall m \in \mathcal{M}_t^{\text{isl}}, \quad (644)$$

$$p_{h,t}^{\text{sh}} = g_h v_{i,t}^{\text{ac}},$$

$$q_{h,t}^{\text{sh}} = -b_{h,t}^{\text{eff}} v_{i,t}^{\text{ac}},$$

$$b_{h,t}^{\text{eff}} = b_h^0 + n_{h,t}^{\text{step}} \Delta b_h, \quad \forall h \in \text{Shunt}, \quad (645)$$

$$v_{b(e),t}^{\text{ac}} = v_e^{\text{set}}, \quad \forall e \in \mathcal{E}^{\text{ext}},$$

$$\theta_{b(e),t} = \theta_e^{\text{set}}, \quad \forall e \in \mathcal{E}^{\text{ext}}. \quad (646)$$

**AC topology projection models.**

$$z_{e,t}^{\text{ac}} = u_{e,t}^{\text{line}}, \quad \forall e \in \text{ACBranch},$$

$$(647)$$

$$r_{\tau}^{\text{eq}} = \frac{\text{vkr\_percent}_{\tau}}{100} \frac{S_{\tau}^{\text{base}}}{S_{\tau}^{\text{rated}}}, \quad x_{\tau}^{\text{eq}} = \sqrt{\left( \frac{\text{vkr\_percent}_{\tau}}{100} \frac{S_{\tau}^{\text{base}}}{S_{\tau}^{\text{rated}}} \right)^2 - (r_{\tau}^{\text{eq}})^2}, \quad \forall \tau \in \text{Transformer2W}, \quad (648)$$

$$\{e_1, e_2, e_3\} = \Pi^{3\text{w} \rightarrow \text{branch}}(\tau^{3\text{w}}), \quad z_{e_j,t}^{\text{ac}} = u_{\tau,t}^{\text{tr3w}}, \quad \forall \tau^{3\text{w}} \in \text{Transformer3W}, \quad j \in \{1, 2, 3\}, \quad (649)$$

$$z_{e(\sigma),t}^{\text{ac}} = s_{\sigma,t}, \quad s_{\sigma,t} - s_{\sigma,t-1} = y_{\sigma,t}^{\text{on}} - y_{\sigma,t}^{\text{off}}, \quad \forall \sigma \in \text{Switch}, \quad (650)$$

$$(P^{\text{eq}}, Q^{\text{eq}}, Y^{\text{eq}}) = \Pi^{3\phi \rightarrow 1\phi}(\mathcal{X}^{3\phi}). \quad (651)$$

where  $\mathcal{X}^{3\phi}$  denotes the projected three-phase tables: **ThreePhaseACBus**, **ThreePhaseACLine**, **ThreePhaseTransformer**, **ThreePhaseLoad**, **ThreePhaseGenerator**, and **ThreePhaseExternalGrid**.

**DC-side component equations.**

$$p_{\ell,t}^{\text{load,dc}} = P_{\ell,t}^{0,\text{dc}} \left( k_{\ell}^Z v_{k,t}^{\text{dc}} + k_{\ell}^I \sqrt{v_{k,t}^{\text{dc}}} + k_{\ell}^P \right) - p_{\ell,t}^{\text{shed,dc}}, \quad \forall \ell \in \text{DCLoad}, \quad (652)$$

$$\underline{P}_g^{\text{dc}} \leq p_{g,t}^{\text{sgen,dc}} \leq \bar{P}_g^{\text{dc}}, \quad \forall g \in \text{StaticGeneratorDC} \cup \text{DC}::\text{StaticGenerator}, \quad (653)$$

$$0 \leq p_{a,t}^{\text{pv,dc}} \leq p_{a,t}^{\text{pv,dc,ava}}, \quad p_{a,t}^{\text{pv,dc,ava}} = f_a(\text{irradiance}_{a,t}, \text{temperature}_{a,t}), \quad \forall a \in \text{PVArrayDC}. \quad (654)$$

**Interlinking converters (VSCConverter, DCDCCConverter, EnergyRouterPort, EnergyRouter).**

$$\text{PQ mode: } P_{c,t}^{\text{ac}} = P_{c,t}^{\text{set}}, \quad Q_{c,t}^{\text{ac}} = Q_{c,t}^{\text{set}}, \quad c \in \text{VSCConverter},$$

$$\text{DC-voltage mode: } v_{k,t}^{\text{dc}} = v_c^{\text{dc,set}} + k_c^{\text{vdc}} \left( P_{c,t}^{\text{dc}} - P_{c,t}^{\text{set}} \right), \quad k = b^{\text{dc}}(c), \quad (655)$$

$$\text{Power mode: } p_{d,t}^{\text{in}} = p_{d,t}^{\text{ref}}, \quad d \in \text{DCDCCConverter},$$

$$\text{Voltage-droop mode: } p_{d,t}^{\text{in}} = p_{d,t}^{\text{ref}} + k_d^{\text{droop}} \left( v_{\text{out}(d),t}^{\text{dc}} - v_d^{\text{ref}} \right), \quad p_{d,t}^{\text{out}} = \eta_d p_{d,t}^{\text{in}}, \quad (656)$$

$$\sum_{p \in \mathcal{P}(r)} p_{p,t}^{\text{er}} + p_{r,t}^{\text{loss,er}} = 0, \quad \forall r \in \mathcal{R}^{\text{er}},$$

$$p_{r,t}^{\text{loss,er}} \geq \lambda_r \sum_{p \in \mathcal{P}(r)} |p_{p,t}^{\text{er}}|, \quad \forall r \in \mathcal{R}^{\text{er}}, \quad (657)$$

$$\underline{P}_p \leq p_{p,t}^{\text{er}} \leq \bar{P}_p, \quad \forall p \in \mathcal{P}^{\text{er}},$$

$$\underline{Q}_p \leq q_{p,t}^{\text{er}} \leq \bar{Q}_p, \quad \forall p \in \mathcal{P}^{\text{er,ac}}.$$

**TS-TR inactive in this horizon.** AsynchronousMotor is short-circuit-only in the current code path and is therefore excluded from TS-TR steady-state scheduling constraints.

**Integration map (struct  $\leftrightarrow$  sub-equation).**

| C++ struct               | TS-TR sub-equation block                                             |
|--------------------------|----------------------------------------------------------------------|
| ACBus, DCBus             | Nodal balances in (613) and (615); voltage bounds in (622) and (623) |
| ACBranch, DCBranch       | Branch laws in (618) and (619); status in (647)                      |
| Transformer2W            | Projection to equivalent AC branch by (648)                          |
| Transformer3W            | Projection to three equivalent AC branches by (649)                  |
| Switch                   | Binary switch status and transitions by (650), (660)                 |
| ExternalGrid             | Slack/root treatment by (646)                                        |
| Generator                | Dispatch/ramp limits by (633)                                        |
| StaticGenerator          | AC static-DER injections by (634)                                    |
| RenewableGen             | Availability-curtailment model by (635)                              |
| PVSystem                 | AC PV inverter envelope by (636)                                     |
| Load                     | ZIP load model by (637)                                              |
| FlexibleLoad             | Shiftable-load model by (638)                                        |
| AsymmetricLoad           | Three-phase-to-single-phase aggregation by (639)                     |
| ChargingStation, Charger | EV aggregation and V2G envelope by (640)                             |
| Storage                  | SOC and charge/discharge exclusivity by (641)                        |
| MobileStorage            | Time-space occupancy and SOC with travel by (642)                    |
| VirtualPowerPlant        | Aggregate dispatch/ramp by (643)                                     |
| Microgrid                | PCC exchange by (644)                                                |
| Shunt                    | Voltage-dependent shunt injections by (645)                          |

| C++ struct                     | TS-TR sub-equation block                          |
|--------------------------------|---------------------------------------------------|
| DCLoad                         | DC ZIP demand by (652)                            |
| StaticGeneratorDC              | DC static generation by (653)                     |
| PVArrayDC                      | DC PV availability by (654)                       |
| VSCConverter                   | AC/DC coupling and mode equations by (624), (655) |
| DCDCConverter                  | Transfer and droop equations by (627), (656)      |
| EnergyRouterPort, EnergyRouter | Multi-port balance/limits by (657)                |
| ThreePhase*                    | Projection operator in (651)                      |
| AsynchronousMotor              | Out of TS-TR horizon (short-circuit only)         |

### (5) Topology constraints (AC mandatory radiality, DC configurable).

$$\sum_{e \in \mathcal{E}^{\text{ac}}} z_{e,t}^{\text{ac}} = |\mathcal{N}^{\text{ac}}| - |\mathcal{N}_0^{\text{ac}}|, \quad \forall t, \quad (658)$$

plus standard single-commodity AC connectivity-flow constraints that ensure every energized AC bus is connected to one substation root. For DC, choose one of:

$$\text{radial DC: } \sum_{\ell \in \mathcal{E}^{\text{dc}}} z_{\ell,t}^{\text{dc}} = |\mathcal{N}^{\text{dc}}| - |\mathcal{N}_0^{\text{dc}}|, \quad (659)$$

or mesh-permitted DC (no edge-count equation, only electrical and thermal limits).

### (6) Inter-temporal switching dynamics (AC and DC).

$$z_{e,t}^{\text{ac}} - z_{e,t-1}^{\text{ac}} = y_{e,t}^{\text{ac,on}} - y_{e,t}^{\text{ac,off}}, \quad y_{e,t}^{\text{ac,on}} + y_{e,t}^{\text{ac,off}} \leq 1, \quad (660)$$

$$z_{\ell,t}^{\text{dc}} - z_{\ell,t-1}^{\text{dc}} = y_{\ell,t}^{\text{dc,on}} - y_{\ell,t}^{\text{dc,off}}, \quad y_{\ell,t}^{\text{dc,on}} + y_{\ell,t}^{\text{dc,off}} \leq 1. \quad (661)$$

## 27.9.5 Position in Time-Series Runtime Pipeline

The hybrid TS-TR layer is the distribution counterpart of UC:

$$\text{Stage A: } \Pi^{\text{tr}} = \{z^{\text{ac}}, z^{\text{dc}}, \Pi^{\text{vsc}}, \Pi^{\text{dcdc}}, \Pi^{\text{der}}\}_{t \in \mathcal{T}} \xrightarrow{\text{write schedule}} \text{Stage B: step-wise validation.} \quad (662)$$

At each step  $t$ , the runtime writes AC/DC branch statuses, VSC and DCDC setpoints, and DER dispatch to  $sys_t$ , then runs either hybrid PF/OPF (recommended for AC/DC-coupled feeders) or DPF-only validation on AC-only subsets.

## 27.9.6 Code-Aligned Pseudocode (Proposed)

**Algorithm TS-TR-1: hybrid topology schedule stage.**

- 1: **procedure** SOLVE\_TOPOLOGY\_RECONFIGURATION\_SCHEDULE( $sys, ts\_data, opts$ )
- 2:   Validate  $T = ts\_data.num\_steps > 0$ ; else return empty infeasible schedule
- 3:   Build per-step AC/DC load and DER availability from `profile_id`
- 4:   Build hybrid TS-TR model using (612)–(661)
- 5:   Include AC/DC switch states, VSC/DCDC coupling, and DER temporal constraints
- 6:   Select MILP/MISOCP backend from `opts.tr_solver`
- 7:   Solve and extract:  
 $\{z_{e,t}^{\text{ac}}, z_{\ell,t}^{\text{dc}}\}$ , converter schedules, DER schedules
- 8:   Pack all arrays into `TRSchedule` and return
- 9: **end procedure**

**Algorithm TS-TR-2: full time-series distribution pipeline.**

```

1: procedure SOLVE_TIME_SERIES_DISTRIBUTION_PF(sys, ts_data, opts)
2: Initialize TimeSeriesDistroResult with T
3: if opts.skip_tr==false then
4: tr_schedule $\leftarrow$ solve_topology_reconfiguration_schedule(sys, ts_data, opts)
5: else
6: Build default schedule from base in-service AC/DC topology and fixed DER
7: end if
8: if opts.run_validation==false then
9: Return schedule-only result
10: end if
11: for $t = 0$ to $T - 1$ do
12: Copy base system to syst
13: Apply $z_{e,t}^{\text{ac}}$ and $z_{\ell,t}^{\text{dc}}$ to branch in_service
14: Apply VSC/DCDC schedule and time- t DER dispatch to syst
15: if opts.validation_mode includes HybridPF then
16: Solve solve_hybrid(syst, opts.pf_options)
17: end if
18: if opts.validation_mode includes DPF then
19: Solve solve_distribution_pf(syst, opts.dpf_options)
20: end if
21: Store step convergence, losses, and interface transfer metrics
22: end for
23: Aggregate objective decomposition and convergence counts
24: Return full result
25: end procedure

```

**27.9.7 C++ Interface Draft (time\_series\_pf-style, Proposed)**

To keep runtime style aligned with `time_series_pf.cpp`, use a dual-entry API:

- `solve_topology_reconfiguration_schedule(...)` for schedule stage only;
- `solve_time_series_distribution_pf(...)` for schedule + sequential validation.

```
namespace hacdcpf::analysis {
```

```
enum class TRSolverChoice { Auto, Native, HiGHS, Gurobi };
```

```
enum class TRValidationMode { HybridPF, DistributionPF, Both };
```

```
struct TRSchedule {
```

```
 // topology status
```

```
 std::vector<std::vector<int>> ac_branch_closed; // [ac_branch] [t]
```

```
 std::vector<std::vector<int>> dc_branch_closed; // [dc_branch] [t]
```

```
 std::vector<std::vector<int>> ac_switch_on, ac_switch_off;
```

```
 std::vector<std::vector<int>> dc_switch_on, dc_switch_off;
```

```
 // converter schedules
```

```
 std::vector<std::vector<double>> vsc_p_ac_mw; // [vsc] [t]
```

```
 std::vector<std::vector<double>> vsc_q_ac_mvar; // [vsc] [t]
```

```
 std::vector<std::vector<double>> vsc_p_dc_mw; // [vsc] [t]
```

```
 std::vector<std::vector<double>> dcdc_p_in_mw; // [dcdc] [t]
```

```
 std::vector<std::vector<double>> dcdc_p_out_mw; // [dcdc] [t]
```

```

// DER schedules (AC + DC)
std::vector<std::vector<double>> dg_ac_dispatch_mw, dg_dc_dispatch_mw;
std::vector<std::vector<double>> ren_ac_dispatch_mw, ren_dc_dispatch_mw;
std::vector<std::vector<double>> ren_ac_curtail_mw, ren_dc_curtail_mw;
std::vector<std::vector<double>> storage_ac_ch_mw;
std::vector<std::vector<double>> storage_ac_dis_mw;
std::vector<std::vector<double>> storage_ac_soc_mwh;
std::vector<std::vector<double>> storage_dc_ch_mw;
std::vector<std::vector<double>> storage_dc_dis_mw;
std::vector<std::vector<double>> storage_dc_soc_mwh;
std::vector<std::vector<double>> flex_ac_shift_mw, flex_dc_shift_mw;

double total_objective{0.0};
bool feasible{false};
std::string solver_name;
};

struct TimeSeriesDistroOptions {
 PowerFlowOptions pf_options;
 DPFOptions dpf_options;
 ONROptions onr_options;
 TRSolverChoice tr_solver{TRSolverChoice::Auto};
 TRValidationMode validation_mode{TRValidationMode::HybridPF};
 bool skip_tr{false}; // true: no schedule optimization
 bool run_validation{true}; // true: run per-step physical validation
 bool enforce_ac_radiality{true};
 bool enforce_dc_radiality{false};
 bool verbose{false};
};

struct TimeSeriesDistroResult {
 TRSchedule tr_schedule;
 std::vector<PowerFlowResult> hybrid_pf_results;
 std::vector<DPFResult> dpf_results;
 int num_steps{0};
 int num_hybrid_converged{0};
 int num_dpf_converged{0};
 int total_switch_actions{0};
 double total_ac_loss_mwh{0.0};
 double total_dc_loss_mwh{0.0};
};

TRSchedule solve_topology_reconfiguration_schedule(
 const HybridPowerSystem& sys,
 const TimeSeriesData& ts_data,
 const TimeSeriesDistroOptions& opts = {});

TimeSeriesDistroResult solve_time_series_distribution_pf(
 const HybridPowerSystem& sys,
 const TimeSeriesData& ts_data,

```

```

 const TimeSeriesDistroOptions& opts = {});
} // namespace hacdcpf::analysis

```

Recommended source layout (draft):

```

include/hacdcpf/analysis/time_series_distribution.hpp,
src/analysis/time_series_distribution.cpp.

```

(663)

## 27.10 Time-Domain Power Flow (Sequential Hybrid PF)

Given UC schedule  $\Pi^{\text{uc}}$ , each time step  $t$  solves:

$$F_t(x_t; \zeta_t) = 0, \quad (664)$$

where  $x_t$  is the hybrid PF state and  $\zeta_t$  is the step-updated component injection set.

### 27.10.1 Step State and Residual Equations

Using the same notation as §12,

$$x_t = \left[ \theta_{\mathcal{N}_{ac} \setminus \mathcal{N}_{sl}}(t) \quad v_{\mathcal{N}_{PQ}}(t) \quad v_{\mathcal{N}_{dc} \setminus \mathcal{N}_{sl}}^{\text{dc}}(t) \right]^\top, \quad (665)$$

and residual rows are

$$\Delta P_i(t) = P_i^{\text{spec}}(t) - P_i^{\text{calc}}(x_t), \quad (666)$$

$$\Delta Q_i(t) = Q_i^{\text{spec}}(t) - Q_i^{\text{calc}}(x_t), \quad (667)$$

$$\Delta P_k^{\text{dc}}(t) = P_k^{\text{dc,spec}}(t) - P_k^{\text{dc,calc}}(x_t). \quad (668)$$

### 27.10.2 Per-Step Injection Update Rules

For each  $t \in \mathcal{T}$ :

$$P_g^{\text{set}}(t) = P_{g,t}^{\text{uc}}, \quad \text{in\_service}_g(t) = u_{g,t}^{\text{uc}}, \quad (669)$$

$$P_s^{\text{ess,set}}(t) = P_{s,t}^{\text{ess,uc}}, \quad P_r^{\text{ren,set}}(t) = P_{r,t}^{\text{ren,uc}}, \quad (670)$$

$$P_\ell(t) = P_\ell^0 \sigma_\ell(t), \quad Q_\ell(t) = Q_\ell^0 \sigma_\ell(t), \quad (671)$$

$$G_p(t) = 1000 \rho_p(t). \quad (672)$$

Then `reset_solver_handle(handle, sys_t, ...)` and `solve_handle(handle, pf_option s)` are executed to obtain `PowerFlowResult` at step  $t$ .

### 27.10.3 Sequential Outputs and Aggregation

The time-series result stores:

$$\text{pf\_results}[t], \quad \text{num\_converged} = \sum_{t \in \mathcal{T}} \mathbb{1}[\text{pf\_results}[t].\text{converged}]. \quad (673)$$

The reported linear generation-cost accumulator is

$$C^{\text{gen,total}} = \sum_{t \in \mathcal{T}} \sum_{g \in \mathcal{G}} c_{1g} P_{g,t}^{\text{uc}} \Delta t, \quad (674)$$

which is stored as `TimeSeriesPFResult::total_generation_cost`.

## 27.11 Post-Processing Metrics

For a solved horizon, typical compliance metrics are:

$$\text{VCR} = \frac{1}{NT} \sum_{i=1}^N \sum_{t=0}^{T-1} \mathbb{I}[V_i^{\min} \leq V_i(t) \leq V_i^{\max}], \quad (675)$$

$$\text{FVR} = \frac{1}{BT} \sum_{b=1}^B \sum_{t=0}^{T-1} \mathbb{I}[|S_b(t)| > S_b^{\text{rated}}]. \quad (676)$$

## 27.12 Current C++ API (Implemented)

| Structure / Function                                   | Current behavior                                                                                                     |
|--------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| <code>TimeSeriesProfile</code>                         | <code>id</code> , <code>name</code> , and a step vector <code>values</code> .                                        |
| <code>TimeSeriesData</code>                            | <code>num_steps</code> , <code>step_duration_hr</code> , and profile list.                                           |
| <code>TimeSeriesPFOptions</code>                       | per-step <code>PowerFlowOptions</code> , UC backend selector, and <code>skip_uc</code> flag.                         |
| <code>solve_unit_commitment(sys, ts_data, opts)</code> | builds the MILP in §27.8 and returns <code>UCSchedule</code> .                                                       |
| <code>solve_time_series_pf(sys, ts_data, opts)</code>  | executes UC (optional) + sequential hybrid PF per §27.10.                                                            |
| <code>TimeSeriesPFResult</code>                        | UC schedule, per-step <code>PowerFlowResult</code> array, convergence count, and linear generation-cost accumulator. |

## 28 Market Clearing Mathematical Models

### 28.1 Problem Form and Runtime Scope

This chapter documents the day-ahead market-clearing stack built in `src/analysis/market_simulation.cpp`. The runtime sequence is a layered optimization pipeline:

- SCUC as a multi-period MILP,
- SCED as an LP re-dispatch with fixed commitment,
- LMP extraction from LP dual variables on the cleared network model.

### 28.2 Code Map

The current implementation is concentrated in:

- `src/analysis/market_simulation.cpp` for SCUC, SCED, LMP model assembly, and reporting,
- `src/engine/adapter_registry.cpp`, `src/engine/native_adapters.cpp`, and `src/engine/external_adapters.cpp` for backend routing,
- `src/engine/branch_and_cut.cpp` and associated LP backends for native MILP and LP execution when external solvers are unavailable.

### 28.3 Pseudocode Map

The detailed algorithm chapter does not yet include a dedicated market-clearing pseudocode block. The solver infrastructure underneath this chapter is covered in Section 4 and Section 22. This chapter therefore remains the primary problem-form and runtime-sequencing reference for SCUC, SCED, and LMP.

### 28.4 Current Status

The market-clearing route is implemented, but it remains one of the more active integration surfaces in the repository because it depends on MILP backends, pricing duals, and network-constraint assembly. This chapter should therefore be read as an implementation-facing description of the current solver stack, not as a claim that every possible market-model extension is already active.

This section presents the complete mathematical formulations for day-ahead electricity market clearing implemented in `market_simulation.cpp`. The models follow the *Southern China Regional Electricity Market Spot Energy Trading Implementation Rules* (Southern China regional spot-market rules, 2025 V1.0), Section 2.6.

The day-ahead clearing consists of three sequential stages:

1. **SCUC** (Security-Constrained Unit Commitment, §2.6.3): Determines the  $T$ -period commitment schedule via mixed-integer linear programming (MILP).
2. **SCED** (Security-Constrained Economic Dispatch, §2.6.4): Fixes the commitment from SCUC and re-optimises dispatch as a linear program (LP).
3. **LMP** (Locational Marginal Price, §2.6.5–2.6.6): Computes nodal prices from the LP dual variables of a  $\delta$ -neighbourhood re-dispatch model.

## 28.5 Nomenclature

### 28.5.1 Sets and Indices

| Symbol                     | Description                                                                                                      |
|----------------------------|------------------------------------------------------------------------------------------------------------------|
| $\mathcal{G}$              | Set of all generators (thermal, nuclear, hydro), indexed by $i$ , $ \mathcal{G}  = N$ ; does not include storage |
| $\mathcal{W}$              | Set of wind farms, indexed by $w$ , $ \mathcal{W}  = N_W$                                                        |
| $\mathcal{S}$              | Set of solar PV plants, indexed by $s$ , $ \mathcal{S}  = N_{PV}$                                                |
| $\mathcal{R}$              | $\mathcal{W} \cup \mathcal{S}$ , set of all renewable units, $ \mathcal{R}  = N_R$                               |
| $\mathcal{E}$              | Set of energy storage units, indexed by $e$ , $ \mathcal{E}  = N_{ES}$                                           |
| $\mathcal{D}$              | Set of loads, indexed by $k$ , $ \mathcal{D}  = K$                                                               |
| $\mathcal{L}$              | Set of AC monitored transmission lines, $ \mathcal{L}  = N_L$                                                    |
| $\mathcal{S}_{\text{sec}}$ | Set of monitored sections/corridors, $ \mathcal{S}_{\text{sec}}  = N_S$                                          |
| $\mathcal{DC}$             | Set of intra-regional DC interconnections, $ \mathcal{DC}  = N_{TD}$                                             |
| $\mathcal{T}_O$            | Set of inter-regional (external) tie-lines, $ \mathcal{T}_O  = N_T$                                              |
| $\mathcal{A}$              | Set of balancing areas (provinces), $ \mathcal{A}  = N_A$                                                        |
| $\mathcal{T}$              | Set of time periods, $ \mathcal{T}  = T$ (D-day: $96 \times 15$ min; D+1: 2 peak/valley; total 98)               |
| $\mathcal{M}_i$            | Set of bid segments for generator $i$ , $ \mathcal{M}_i  = N_M$                                                  |
| $\mathcal{I}_s$            | Set of must-run / CHP / commissioning units                                                                      |
| $\mathcal{H}$              | Set of hydro stations with reservoir, $ \mathcal{H}  = N_H$                                                      |
| $\mathcal{J}$              | Set of generator groups, indexed by $j$                                                                          |

### 28.5.2 Decision Variables

| Variable                     | Description                                                                                |
|------------------------------|--------------------------------------------------------------------------------------------|
| $P_{i,t}$                    | Generation output of unit $i$ in period $t$ [MW]                                           |
| $\Delta P_{i,t,m}$           | Dispatch in bid segment $m$ of unit $i$ in period $t$ [MW]                                 |
| $u_{i,t}$ ( $\alpha_{i,t}$ ) | Commitment status of unit $i$ in period $t$ (binary: 0/1)                                  |
| $v_{i,t}$                    | Startup indicator of unit $i$ in period $t$ (binary: 1 iff $u_{i,t} = 1, u_{i,t-1} = 0$ )  |
| $w_{i,t}$                    | Shutdown indicator of unit $i$ in period $t$ (binary: 1 iff $u_{i,t} = 0, u_{i,t-1} = 1$ ) |
| $R_{i,t}^{\text{spin}}$      | Spinning reserve of unit $i$ in period $t$ [MW]                                            |
| $R_{i,t}^{\text{up}}$        | Regulation-up reserve of unit $i$ in period $t$ [MW]                                       |
| $R_{i,t}^{\text{dn}}$        | Regulation-down reserve of unit $i$ in period $t$ [MW]                                     |
| $P_{w,t}^W$                  | Wind generation of farm $w$ in period $t$ [MW]                                             |
| $P_{s,t}^{PV}$               | Solar PV generation of plant $s$ in period $t$ [MW]                                        |
| $P_{e,t}^{\text{ch}}$        | Storage charging power of unit $e$ in period $t$ [MW]                                      |
| $P_{e,t}^{\text{dis}}$       | Storage discharging power of unit $e$ in period $t$ [MW]                                   |
| $E_{e,t}$                    | State of charge (SOC) of storage $e$ at end of period $t$ [MWh]                            |
| $\xi_{e,t}^+, \xi_{e,t}^-$   | Binary indicators for storage charge/discharge status                                      |
| $T_{d,t}^{DC}$               | Intra-regional DC line $d$ power transfer in period $t$ [MW]                               |

| Variable                   | Description                                                      |
|----------------------------|------------------------------------------------------------------|
| $T_{j,t}^O$                | Inter-regional tie-line $j$ scheduled power in period $t$ [MW]   |
| $SL_{l,t}^+, SL_{l,t}^-$   | Forward/reverse line flow slack for line $l$ in period $t$       |
| $SL_{s,t}^+, SL_{s,t}^-$   | Forward/reverse section flow slack for section $s$ in period $t$ |
| $SL_k$                     | Tie-line priority schedule slack for component $k$               |
| $P_{r,t}^d$                | Renewable curtailment of unit $r$ in period $t$ [MW]             |
| $P_{i,t}^{d,\text{hydro}}$ | Hydro spillage power of station $i$ in period $t$ [MW]           |
| $LS_t$                     | Load shedding in period $t$ [MW]                                 |
| $GC_t$                     | Generation curtailment in period $t$ [MW]                        |

### 28.5.3 Parameters

| Parameter                                      | Description                                                       |
|------------------------------------------------|-------------------------------------------------------------------|
| $C_{i,m}$                                      | Bid price of generator $i$ in segment $m$ [¥/MWh]                 |
| $C_i^U$                                        | Startup cost of unit $i$ (hot/warm/cold state dependent) [¥]      |
| $C_i^{\min,p}$                                 | Minimum-output cost of unit $i$ [¥/h]; applies when $u_{i,t} = 1$ |
| $P_i^{\min}$                                   | Minimum stable output of unit $i$ [MW]                            |
| $P_i^{\max}$                                   | Maximum capacity of unit $i$ [MW]                                 |
| $\Delta P_{i,m}^{\min}, \Delta P_{i,m}^{\max}$ | Bid segment $m$ lower/upper bound [MW]                            |
| $\Delta P_i^U$                                 | Ramp-up rate of unit $i$ [MW/period]                              |
| $\Delta P_i^D$                                 | Ramp-down rate of unit $i$ [MW/period]                            |
| $T_i^U$                                        | Minimum up-time of unit $i$ [periods]                             |
| $T_i^D$                                        | Minimum down-time of unit $i$ [periods]                           |
| $N_i^{\max,\text{start}}$                      | Maximum startup count of unit $i$ over the horizon                |
| $N_i^{\max,\text{stop}}$                       | Maximum shutdown count of unit $i$ over the horizon               |
| $UD_i$                                         | Startup process duration (periods from sync to min output)        |
| $DD_i$                                         | Shutdown process duration (periods from min output to desync)     |
| $D_t^a$                                        | System load in balancing area $a$ , period $t$ [MW]               |
| $R_t^{U,a}$                                    | Positive reserve requirement for area $a$ in period $t$ [MW]      |
| $R_t^{D,a}$                                    | Negative reserve requirement for area $a$ in period $t$ [MW]      |
| $R_t^{\text{red},a}$                           | Network-constrained reserve reduction for area $a$ [MW]           |
| $R_t^{\text{pf}}$                              | Primary frequency regulation reserve requirement [MW]             |
| $P_l^{\max}, P_l^{\min}$                       | Forward/reverse thermal limits of line $l$ [MW]                   |
| $P_s^{\max}, P_s^{\min}$                       | Forward/reverse limits of section $s$ [MW]                        |
| $G_{l,k}$                                      | PTDF of bus $k$ on line $l$                                       |
| $G_{s,k}$                                      | PTDF of bus $k$ on section $s$                                    |
| $\eta_e^{\text{ch}}, \eta_e^{\text{dis}}$      | Charging/discharging efficiency of storage $e$                    |
| $\bar{E}_e, \underline{E}_e$                   | SOC upper/lower limits of storage $e$ [MWh]                       |
| $E_e^{\text{ini}}$                             | Initial SOC of storage $e$ [MWh]                                  |
| $E_e^{\text{fin}}$                             | Required final SOC of storage $e$ [MWh]                           |
| $N_e^{\text{cycle}}$                           | Maximum daily charge/discharge cycles of storage $e$              |
| $P_{r,t}^{\text{st}}$                          | Short-term forecast power of renewable $r$ in period $t$ [MW]     |

| Parameter                                       | Description                                                                |
|-------------------------------------------------|----------------------------------------------------------------------------|
| $\alpha_n$                                      | Renewable minimum output coefficient (province-specific)                   |
| $\lambda_e^{\text{dis}}, \lambda_e^{\text{ch}}$ | Storage discharge/charge bid prices [¥/MWh]                                |
| $P_{\text{gwf}}$                                | Inter-provincial transmission fee (incl. network losses, wheeling) [¥/MWh] |
| $M_1$                                           | Network flow constraint relaxation penalty [¥/MW]                          |
| $M_2$                                           | Renewable curtailment penalty [¥/MW]                                       |
| $M_3$                                           | Hydro spillage penalty [¥/MW]                                              |
| $M_4$                                           | Tie-line priority schedule relaxation penalty [¥/MW]                       |
| $\Delta t$                                      | Period length [hours] (0.25 for 15-min periods)                            |
| $\delta$                                        | LMP $\delta$ -neighbourhood parameter (fraction)                           |

## 28.6 SCUC – Security-Constrained Unit Commitment (§2.6.3)

### 28.6.1 Objective Function

The SCUC minimises total system cost over the planning horizon  $T$  (§2.6.3 Objective):

$$\begin{aligned}
\min \quad & \sum_{i=1}^N \sum_{t=1}^T [C_i(P_{i,t}) + C_{i,t}^U + C_{i,t}^{\text{min},p}] + \sum_{i=1}^n \sum_{t=1}^T P_{\text{gwf}} \cdot P_{L,i,t} \\
& + M_1 \sum_{l=1}^{N_L} \sum_{t=1}^T (SL_{l,t}^+ + SL_{l,t}^-) + M_1 \sum_{s=1}^{N_S} \sum_{t=1}^T (SL_{s,t}^+ + SL_{s,t}^-) \\
& + M_2 \sum_{r=1}^{N_R} \sum_{t=1}^T P_{r,t}^d + M_3 \sum_{i=1}^{N_H} \sum_{t=1}^T P_{i,t}^{d,\text{hydro}} + M_4 \sum_{k=1}^a \sum_{t=1}^T SL_k \\
& + \sum_{e=1}^{N_{ES}} \sum_{t=1}^T (\lambda_e^{\text{dis}} \cdot P_{e,t}^{\text{dis}} + \lambda_e^{\text{ch}} \cdot P_{e,t}^{\text{ch}})
\end{aligned} \tag{677}$$

**Generator operating cost (§2.6.3.7).** The operating cost is a piecewise-linear function of the bid segments:

$$C_i(P_{i,t}) = \sum_{m=1}^{N_M} C_{i,m} \cdot \Delta P_{i,t,m} \tag{678}$$

**Startup cost (§2.6.3.13).** Startup cost depends on off-time duration (cold/warm/hot state):

$$C_{i,t}^U = C_i^U \cdot v_{i,t} \tag{679}$$

**Minimum-output cost.** Minimum-output cost applies when the unit is committed:

$$C_{i,t}^{\text{min},p} = C_i^{\text{min},p} \cdot u_{i,t} \cdot \Delta t \tag{680}$$

**C++ mapping.** In `build_scuc_formulation()`:

- Segment cost: `lp.c(seg_idx)=price*period_length_hr`
- Startup cost: `lp.c(su_idx)+=startup_cost_hot`
- Min-output cost: `lp.c(ig_idx)+=no_load_cost/intervals_per_hour`
- $M_1$  slack: `lp.c(SL_LINE/SEC)=M1_line_slack_penalty`
- Load shedding: `lp.c(LOAD_SHED)=VOLL*period_length_hr`

- Gen curtailment:  $\text{lp.c}(\text{GEN\_CURT}) = \text{VOCC} \cdot \text{period\_length\_hr}$
- Storage bids:  $\text{lp.c}(\text{PSTO\_OUT/IN})$  stores the discharge or charge bid coefficient  $\lambda_e^{\text{dis}}$  or  $\lambda_e^{\text{ch}}$

### 28.6.2 Constraint 1: Power Balance (§2.6.3.1)

For each balancing area  $a$  and period  $t$ :

$$\sum_{i \in \mathcal{G}^a} P_{i,t} + \sum_{j=1}^{NT^a} T_{j,t}^{O,a} + \sum_{k=1}^{NTI^a} T_{k,t}^{DC,a} = D_t^a \quad (681)$$

where  $P_{i,t}$  includes generators, renewables, and storage net output (discharge minus charge). The implementation adds load-shedding ( $LS_t$ ) and generation-curtailment ( $GC_t$ ) slack variables to the balance.

### 28.6.3 Constraint 2: Positive Reserve (§2.6.3.2)

For each province  $a$  and period  $t$ :

$$\sum_{i \in \mathcal{G}^a} u_{i,t} \cdot P_{i,t}^{\max,a} + \sum_{j=1}^{NT^a} T_{j,t}^{O,a} + \sum_{k=1}^{NTI^a} T_{k,t}^{DC,a} \geq D_t^a + R_t^{U,a} - R_t^{\text{red},a} \quad (682)$$

In C++, reserves are modelled via explicit reserve variables with coupling:

$$P_{i,t} + R_{i,t}^{\text{spin}} + R_{i,t}^{\text{up}} \leq P_i^{\max} \cdot u_{i,t} \quad \forall i \in \mathcal{G}, t \in \mathcal{T} \quad (683)$$

### 28.6.4 Constraint 3: Negative Reserve (§2.6.3.3)

For each province  $a$  and period  $t$ :

$$\sum_{i \in \mathcal{G}^a} (P_{i,t} - \beta_{i,t} \cdot P_{i,t}^{\min,a}) + \sum_q \beta_{q,t} \cdot P_{q,t}^{\max,a} + \sum_j T_{j,t}^{O,a} + \sum_k T_{k,t}^{DC,a} \leq D_t^a - R_t^{D,a} \quad (684)$$

where  $\beta_{i,t}$  indicates whether unit  $i$  can provide negative reserve (thermal: same as  $u_{i,t}$ ; hydro/VPP: separately determined).

### 28.6.5 Constraint 4: Primary Frequency Regulation Reserve (§2.6.3.4)

$$\sum_{f=1}^{NF} P_{f,t}^{\text{pf}} + \sum_{p=1}^{NHP} P_{p,t}^{\text{pf}} + \sum_{h=1}^{NHC} P_{h,t}^{\text{pf}} \geq R_t^{\text{pf}} \quad \forall t \quad (685)$$

where  $NF$ ,  $NHP$ ,  $NHC$  are the counts of thermal, pumped-storage, and conventional hydro units, and each unit's PFR capacity is:

$$P_{f,t}^{\text{pf}} = \min(\alpha_f^{\text{pf}} \cdot P_{f,t}^{\max}, P_{f,t}^{\max} - P_{f,t}) \quad (686)$$

### 28.6.6 Constraint 5: Must-Run Units (§2.6.3.5)

$$u_{i,t} = 1 \quad \forall i \in \mathcal{I}_s, t \in \mathcal{T} \quad (687)$$

$\mathcal{I}_s$  includes must-run generators, CHP units (during heating periods), and commissioning units.

### 28.6.7 Constraint 6: Generator Output Expression (§2.6.3.6)

Output is decomposed into a minimum block and piecewise segments:

$$P_{i,t} = P_i^{\min} \cdot u_{i,t} + \sum_{m=1}^{N_M} \Delta P_{i,t,m} \quad (688)$$

$$\Delta P_{i,m}^{\min} \leq \Delta P_{i,t,m} \leq \Delta P_{i,m}^{\max} \quad \forall m \in \mathcal{M}_i \quad (689)$$

### 28.6.8 Constraint 7: Generator Output Limits (§2.6.3.8)

$$P_{i,t}^{\min} \cdot u_{i,t} \leq P_{i,t} \leq P_{i,t}^{\max} \cdot u_{i,t} \quad \forall i \in \mathcal{G}, t \in \mathcal{T} \quad (690)$$

For units in startup/shutdown transition (§2.6.3.8 detailed modelling), the limits account for  $UD_i$  and  $DD_i$  ramp trajectories:

$$P_{i,t}^{\min} = \begin{cases} P_i^{\min} & \text{if } u_{i,t} = 1 \text{ (steady state)} \\ \text{startup trajectory value} & \text{if startup in progress} \end{cases} \quad (691)$$

### 28.6.9 Constraint 8: Generator Group Output Limits (§2.6.3.9)

$$P_{j,t}^{\min} \leq \sum_{i \in j} P_{i,t} \leq P_{j,t}^{\max} \quad \forall j \in \mathcal{J}, t \in \mathcal{T} \quad (692)$$

### 28.6.10 Constraint 9: Generator Group Energy Limits (§2.6.3.10)

$$Q_j^{\min} \leq \sum_{i \in j} \sum_{t=1}^T Q_{i,t} \leq Q_j^{\max} \quad (693)$$

where  $Q_{i,t} = P_{i,t} \cdot \Delta t$  is the cleared energy of unit  $i$  in period  $t$ .

### 28.6.11 Constraint 10: Ramp Rate (§2.6.3.11)

$$P_{i,t} - P_{i,t-1} \leq \Delta P_i^U + P_i^{\max} \cdot v_{i,t} \quad \forall i, t \geq 2 \quad (694)$$

$$P_{i,t-1} - P_{i,t} \leq \Delta P_i^D + P_i^{\max} \cdot w_{i,t} \quad \forall i, t \geq 2 \quad (695)$$

The startup/shutdown terms allow larger ramps during transitions.

### 28.6.12 Constraint 11: Minimum Up/Down Time (§2.6.3.12)

Commitment transition logic:

$$u_{i,t} - u_{i,t-1} = v_{i,t} - w_{i,t} \quad \forall i, t \geq 2 \quad (696)$$

Minimum up-time: if a unit started up, it must stay on for  $T_i^U$  periods:

$$\sum_{\tau=\max(1, t-T_i^U+1)}^t v_{i,\tau} \leq u_{i,t} \quad \forall i, t \quad (697)$$

Minimum down-time: if a unit shut down, it must stay off for  $T_i^D$  periods:

$$\sum_{\tau=\max(1, t-T_i^D+1)}^t w_{i,\tau} \leq 1 - u_{i,t} \quad \forall i, t \quad (698)$$

### 28.6.13 Constraint 12: Maximum Startup/Shutdown Count (§2.6.3.13)

$$\sum_{t=1}^T v_{i,t} \leq N_i^{\max, \text{start}} \quad \forall i \in \mathcal{G} \quad (699)$$

$$\sum_{t=1}^T w_{i,t} \leq N_i^{\max, \text{stop}} \quad \forall i \in \mathcal{G} \quad (700)$$

where a value of 0 means unlimited. In C++, these constraints are only added when `gen_max_startups[g]>0` or `gen_max_shutdowns[g]>0`.

### 28.6.14 Constraint 13: Line Flow Limits (§2.6.3.14)

Using PTDF (Power Transfer Distribution Factor):

$$P_l^{\min} \leq \sum_{i=1}^N G_{l,i} P_{i,t} + \sum_{j=1}^{N_T} G_{l,j} T_{j,t}^O + \sum_{d=1}^{N_{TD}} G_{l,d} T_{d,t}^{DC} - \sum_{k=1}^K G_{l,k} D_{k,t} + SL_{l,t}^- \leq P_l^{\max} + SL_{l,t}^+ \quad (701)$$

In the C++ implementation, this is decomposed into two  $\leq$  constraints per line per period, with explicit slack variables:

$$(\text{Fwd}): \quad \sum_b G_{l,b} \cdot \text{NetInj}_{b,t} - SL_{l,t}^+ \leq P_l^{\max} \quad (702)$$

$$(\text{Rev}): \quad - \sum_b G_{l,b} \cdot \text{NetInj}_{b,t} - SL_{l,t}^- \leq -P_l^{\min} \quad (703)$$

where  $\text{NetInj}_{b,t} = \sum_{g \in b} P_{g,t} + \sum_{w \in b} P_{w,t}^W + \sum_{s \in b} P_{s,t}^{PV} + \sum_{e \in b} (P_{e,t}^{\text{dis}} - P_{e,t}^{\text{ch}}) + T_{b,t}^{DC} - D_{b,t}$ .

### 28.6.15 Constraint 14: Section Flow Limits (§2.6.3.15)

$$P_s^{\min} \leq \sum_{i=1}^N G_{s,i} P_{i,t} + \sum_{j=1}^{N_T} G_{s,j} T_{j,t}^O + \sum_{d=1}^{N_{TD}} G_{s,d} T_{d,t}^{DC} - \sum_{k=1}^K G_{s,k} D_{k,t} + SL_{s,t}^- \leq P_s^{\max} + SL_{s,t}^+ \quad (704)$$

where  $G_{s,k}$  is the aggregate PTDF for section  $s$  on bus  $k$ , computed as a weighted combination of the PTDFs of lines belonging to the section. In C++, section PTDFs are precomputed by `section_constraints` and slack variables `SL_SEC_POS/NEG` are added analogously to line flow.

### 28.6.16 Constraint 15: Storage (§2.6.3.16)

(a) Charge/discharge power limits (§2.6.3.16(1)):

$$P_e^{\text{ch}, \min} \cdot \xi_{e,t}^+ \leq P_{e,t}^{\text{ch}} \leq P_e^{\text{ch}, \max} \cdot \xi_{e,t}^+ \quad (705)$$

$$P_e^{\text{dis}, \min} \cdot \xi_{e,t}^- \leq P_{e,t}^{\text{dis}} \leq P_e^{\text{dis}, \max} \cdot \xi_{e,t}^- \quad (706)$$

$$\xi_{e,t}^+ + \xi_{e,t}^- \leq 1 \quad (707)$$

(b) SOC dynamics (§2.6.3.16(2)):

$$E_{e,t} = E_{e,t-1} + \eta_e^{\text{ch}} \cdot P_{e,t}^{\text{ch}} \cdot \Delta t - \frac{P_{e,t}^{\text{dis}}}{\eta_e^{\text{dis}}} \cdot \Delta t \quad (708)$$

$$\underline{E}_{e,t} \leq E_{e,t} \leq \bar{E}_{e,t} \quad \forall t \quad (709)$$

(c) **Initial and final SOC (§2.6.3.16(3)):**

$$E_{e,0} = E_e^{\text{ini}}, \quad E_{e,T} \geq E_e^{\text{fin}} \quad (710)$$

(d) **No simultaneous charge/discharge within hour (§2.6.3.16(4)):** For each hour  $n \in [1, 24]$ , covering the 4 periods  $t \in [4n-3, 4n]$ :

$$\sum_{t'=4n-3}^{4n} \xi_{e,t'}^+ \leq 4 \xi_{e,n}^+, \quad \sum_{t'=4n-3}^{4n} \xi_{e,t'}^- \leq 4(1 - \xi_{e,n}^+) \quad (711)$$

In the LP relaxation (C++ implementation), this is approximated by:

$$P_{e,t}^{\text{ch}} + P_{e,t}^{\text{dis}} \leq \max(P_e^{\text{ch,max}}, P_e^{\text{dis,max}}) \quad (712)$$

(e) **Cycle limit (§2.6.3.16(5)):**

$$\sum_{t=1}^T \left( \frac{P_{e,t}^{\text{dis}}}{\eta_e^{\text{dis}}} + \eta_e^{\text{ch}} \cdot P_{e,t}^{\text{ch}} \right) \cdot \Delta t \leq 2 \cdot N_e^{\text{cycle}} \cdot \bar{E}_e \quad (713)$$

#### 28.6.17 Constraint 16: DC Line (§2.6.3.17)

(a) **Multi-terminal DC power balance:** For a multi-terminal DC group  $MT$ :

$$\sum_{s \in J_{MT}^{\text{send}}} T_{s,t}^{DC} = \sum_{d \in J_{MT}^{\text{rec}}} T_{d,t}^{DC} \cdot (1 - \rho_d) \quad (714)$$

where  $\rho_d$  is the loss factor for DC line  $d$ .

(b) **Power limits:**

$$T_{j,t}^{DC,\min} \leq T_{j,t}^{DC} \leq T_{j,t}^{DC,\max} \quad (715)$$

(c) **Ramp constraints with direction change prohibition:**

$$T_{j,t}^{DC} - T_{j,t-1}^{DC} \leq x_{j,t,r} \cdot \Delta P_j^+ \quad (716)$$

$$T_{j,t-1}^{DC} - T_{j,t}^{DC} \leq x_{j,t,l} \cdot \Delta P_j^- \quad (717)$$

$$x_{j,t,r} + x_{j,t,l} \leq 1 \quad (718)$$

(d) **No reversal between adjacent periods (§2.6.3.17(4)):**

$$x_{j,t-1,r} + x_{j,t,l} \leq 1, \quad x_{j,t,r} + x_{j,t-1,l} \leq 1 \quad (719)$$

#### 28.6.18 Constraint 17: Hydro Water Level (§2.6.3.18)

For hydro station  $i$  with reservoir:

$$Z_{i,t}^{\min} \leq Z_{i,0} + \frac{1}{S_i} \sum_{\tau=1}^t \left( I_{i,\tau} - P_{i,\tau} \cdot h_i - Q_{i,\tau}^d + P_{\text{up}(i),\tau-s(i)} \cdot h_{\text{up}(i)} + Q_{\text{up}(i),\tau-s(i)}^d \right) \leq Z_{i,t}^{\max} \quad (720)$$

where  $Z_{i,0}$  is the initial water level,  $S_i$  is the reservoir surface area,  $h_i$  is the water consumption rate,  $I_{i,\tau}$  is inflow,  $Q_i^d$  is spillage flow,  $\text{up}(i)$  is the upstream station, and  $s(i)$  is the upstream delay.

### 28.6.19 Constraint 18: Hydro Energy Limits (§2.6.3.19)

$$Q_i^{\min} \leq \sum_{t=1}^T Q_{i,t} \leq Q_i^{\max} \quad \forall i \in \mathcal{H} \quad (721)$$

### 28.6.20 Constraint 19: Renewable Output (§2.6.3.20)

For bid-quantity-and-price renewable units:

$$\alpha_n \cdot P_{r,t}^{\text{st}} \leq P_{r,t} \leq P_{r,t}^{\text{st}} \quad \forall r \in \mathcal{R}, t \in \mathcal{T} \quad (722)$$

$P_{r,t}^d = P_{r,t}^{\text{st}} - P_{r,t}$  is the curtailed power.

For bid-quantity-only (price-taker) units:

$$P_{r,t} + P_{r,t}^d = P_{r,t}^F \quad (723)$$

### 28.6.21 Constraint 20: Inter-Provincial Priority Schedule (§2.6.3.21)

$$\sum_{t=1}^T Q_{k,t} \geq Q_k^{\text{sch}} \quad \forall k \quad (724)$$

where  $Q_{k,t}$  is the cleared energy of inter-provincial trading component  $k$  and  $Q_k^{\text{sch}}$  is the D-2 adjusted priority schedule energy.

## 28.7 SCED – Security-Constrained Economic Dispatch (§2.6.4)

### 28.7.1 SCED Objective (§2.6.4)

The SCED objective is identical in structure to SCUC but without startup and minimum-output costs (these are sunk costs given fixed commitment):

$$\begin{aligned} \min \quad & \sum_{i=1}^N \sum_{t=1}^T C_i(P_{i,t}) + \sum_{i=1}^n \sum_{t=1}^T P_{\text{gwf}} \cdot P_{L,i,t} \\ & + M_1 \sum_l \sum_t (SL_{l,t}^+ + SL_{l,t}^-) + M_1 \sum_s \sum_t (SL_{s,t}^+ + SL_{s,t}^-) \\ & + M_2 \sum_r \sum_t P_{r,t}^d + M_3 \sum_i \sum_t P_{i,t}^{d,\text{hydro}} + M_4 \sum_k \sum_t SL_k \\ & + \sum_e \sum_t (\lambda_e^{\text{dis}} P_{e,t}^{\text{dis}} + \lambda_e^{\text{ch}} P_{e,t}^{\text{ch}}) \end{aligned} \quad (725)$$

### 28.7.2 SCED Key Differences from SCUC

1. **Fixes commitment:**  $u_{i,t}$ ,  $v_{i,t}$ ,  $w_{i,t}$  are fixed to the SCUC optimal values; the problem becomes a pure LP.
2. **Output limits (§2.6.4.5):** For units that are off in SCUC,  $P_{i,t}^{\min} = P_{i,t}^{\max} = 0$ . For units in startup/shutdown transition, limits are fixed to trajectory values.
3. **Ramp constraints (§2.6.4.8):** Simplified—no startup/shutdown ramp relaxation terms:

$$|P_{i,t} - P_{i,t-1}| \leq \Delta P_i^{U/D} \quad (726)$$

4. All other constraints (balance, reserves, line/section flow, storage, DC line, renewables, priority schedule) carry over from SCUC.

**C++ implementation.** In `solve_sced()`, the SCUC formulation is rebuilt, then binary variables (IG, SU, SD) are fixed via their bounds. The lists `binary_idx` and `integer_idx` are cleared so the solver treats the problem as a pure LP.

## 28.8 LMP – Locational Marginal Price (§2.6.5–2.6.6)

### 28.8.1 LMP Objective (§2.6.5)

The LMP model has the same objective structure as SCED, with potentially different penalty factors  $M'_1, M'_2, M'_3, M'_4$ :

$$\begin{aligned}
\min \quad & \sum_{i=1}^N \sum_{t=1}^T C_i(P_{i,t}) + \sum_{i=1}^n \sum_{t=1}^T P_{\text{gwf}} \cdot P_{L,i,t} \\
& + M'_1 \sum_l \sum_t (SL_{l,t}^+ + SL_{l,t}^-) + M'_1 \sum_s \sum_t (SL_{s,t}^+ + SL_{s,t}^-) \\
& + M'_2 \sum_r \sum_t P_{r,t}^d + M'_3 \sum_i \sum_t P_{i,t}^{d,\text{hydro}} + M'_4 \sum_k \sum_t SL_k \\
& + \sum_e \sum_t (\lambda_e^{\text{dis}} P_{e,t}^{\text{dis}} + \lambda_e^{\text{ch}} P_{e,t}^{\text{ch}})
\end{aligned} \tag{727}$$

### 28.8.2 $\delta$ -Neighbourhood Bounds (§2.6.5.5)

For committed, price-setting generators:

$$P_{i,t}^{\min, \text{LMP}} = \max((1 - \delta) P_{i,t}^{\text{SCED}}, P_{i,t}^{\min}) \tag{728}$$

$$P_{i,t}^{\max, \text{LMP}} = \min((1 + \delta) P_{i,t}^{\text{SCED}}, P_{i,t}^{\max}) \tag{729}$$

For non-price-setting generators (must-run, in transition, non-market):

$$P_{i,t}^{\min, \text{LMP}} = P_{i,t}^{\max, \text{LMP}} = P_{i,t}^{\text{SCED}} \tag{730}$$

For storage units (§2.6.5.11), analogous  $\delta$ -bounds apply:

$$P_{e,t}^{\text{ch}, \max} = \min((1 + \delta) P_{e,t}^{\text{ch}, \text{SCED}}, P_e^{\text{ch}, \max}) \tag{731}$$

$$P_{e,t}^{\text{dis}, \max} = \min((1 + \delta) P_{e,t}^{\text{dis}, \text{SCED}}, P_e^{\text{dis}, \max}) \tag{732}$$

### 28.8.3 LMP Decomposition (§2.6.6)

The nodal LMP at bus  $k$  in period  $t$  is:

$$\boxed{
\begin{aligned}
\text{LMP}_{k,t} = & \underbrace{\lambda_t}_{\text{Energy (SMP)}} + \underbrace{\sum_{l=1}^{N_L} (\mu_{l,t}^{\max} - \mu_{l,t}^{\min}) \cdot G_{l,k}}_{\text{Congestion component}} + \sum_{s=1}^{N_S} (\mu_{s,t}^{\max} - \mu_{s,t}^{\min}) \cdot G_{s,k}
\end{aligned}
} \tag{733}$$

where:

- $\lambda_t$ : Lagrange multiplier of the power balance constraint (system marginal price, SMP).
- $\mu_{l,t}^{\max}$ : multiplier of line  $l$  forward flow constraint; equals  $M'_1$  when the slack variable absorbs flow violation.
- $\mu_{l,t}^{\min}$ : multiplier of line  $l$  reverse flow constraint.
- $\mu_{s,t}^{\max}, \mu_{s,t}^{\min}$ : analogous multipliers for section  $s$ .
- $G_{l,k}$ : PTDF of bus  $k$  on line  $l$ ;  $G_{s,k}$ : PTDF of bus  $k$  on section  $s$ .

**C++ implementation.** In `solve_1mp()`, the LP is solved with a simplex method. SMP is extracted from the power-balance equality dual. Congestion components come from line/section flow inequality duals.

## 28.9 Market Clearing Pipeline

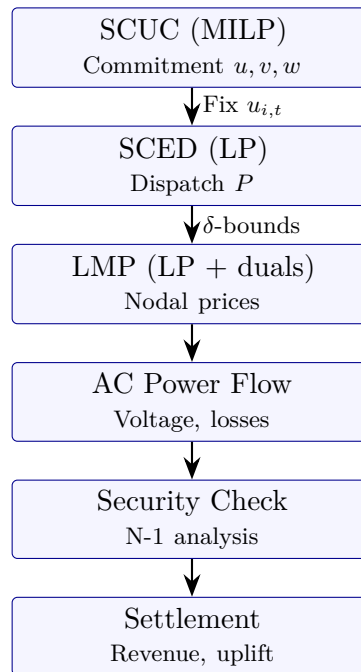


Figure 15: Day-ahead market clearing pipeline (§2.6.2).

## 28.10 Summary of C++ Implementation Mapping

| PDF §     | Constraint                 | C++ Location                                                          |
|-----------|----------------------------|-----------------------------------------------------------------------|
| 2.6.3 Obj | Objective function         | <code>build_scuc_formulation</code> , objective block                 |
| 2.6.3.1   | Power balance              | <code>Systempowerbalanceeachperiod</code>                             |
| 2.6.3.2   | Positive reserve           | <code>Reserverequirements(system+BA)</code>                           |
| 2.6.3.3   | Negative reserve           | <code>Reserverequirements(system+BA)</code>                           |
| 2.6.3.4   | Primary freq. regulation   | <code>Spinningreserveconstraints</code>                               |
| 2.6.3.5   | Must-run units             | <code>gen_must_run <math>\Rightarrow</math> ig.lb=ig.ub=1</code>      |
| 2.6.3.6   | Output expression          | <code>PG=Pmin*IG+sum(segments)</code>                                 |
| 2.6.3.7   | Operating cost             | <code>lp.c(seg)=price*dt</code>                                       |
| 2.6.3.8   | Output limits              | <code>PG&lt;=pmax*IG, PG&gt;=pmin*IG</code>                           |
| 2.6.3.9   | Group output limits        | <code>gen_group_constraints</code>                                    |
| 2.6.3.10  | Group energy limits        | <code>gen_group_energy_constraints</code>                             |
| 2.6.3.11  | Ramp rate                  | <code>PG-PG_prev&lt;=ramp_up+Pmax*SU</code>                           |
| 2.6.3.12  | Min up/down time           | <code>SumSU\$<math>\leq</math>IG, SumSD\$<math>\leq</math>1-IG</code> |
| 2.6.3.13  | Max startup/shutdown count | <code>SumSU\$<math>\leq</math>gen_max_startups</code>                 |
| 2.6.3.14  | Line flow (PTDF)           | <code>Forward/reverse PTDF + SL_LINE_POS/NEG</code>                   |
| 2.6.3.15  | Section flow               | <code>section_constraints + SL_SEC_POS/NEG</code>                     |

| PDF §    | Constraint                  | C++ Location                                                          |
|----------|-----------------------------|-----------------------------------------------------------------------|
| 2.6.3.16 | Storage                     | SOC dynamics, limits, final SOC, anti-simul, cycle                    |
| 2.6.3.17 | DC line                     | Bounds & ramp on PDC variables                                        |
| 2.6.3.18 | Hydro water level           | <code>hydro_water_level_constraints</code>                            |
| 2.6.3.19 | Hydro energy limits         | <code>hydro_energy_constraints</code>                                 |
| 2.6.3.20 | Renewable output            | Variable bounds from forecast profiles                                |
| 2.6.3.21 | Priority schedule           | <code>priority_schedule_constraints</code>                            |
| 2.6.4    | SCED fixes commitment       | <code>solve_sced</code> : fix IG/SU/SD bounds                         |
| 2.6.5    | LMP $\delta$ -neighbourhood | <code>solve_lmp</code> : delta bounds on PG/storage                   |
| 2.6.6    | Nodal price formula         | Dual extraction: $\text{SMP} + \sum(\mu^{\max} - \mu^{\min}) \cdot G$ |

## 28.11 MILP Formulation Tightening: Problem-Specific Cutting Planes

The SCUC MILP (677)–(713) admits an LP relaxation whose root gap arises from two distinct structural sources: *commitment-logic relaxation* (temporal coupling of  $u_{i,t}$ ,  $v_{i,t}$ ,  $w_{i,t}$ ) and *storage-mode relaxation* (fractional charging/discharging that violates energy conservation). This section documents eleven families of valid inequalities that are separated and appended to the branch-and-cut LP to close these gaps. All cuts are sparse (number of non-zeros independent of problem size), certificate-preserving, and compatible with the native solver’s Append Gate and density cap.

### 28.11.1 LP Relaxation Gap Sources

#### Commitment-logic weaknesses.

- (i) **Temporal coupling:**  $u_{i,t}$  can take fractional values; minimum up/down time constraints are correspondingly weakened.
- (ii) **Startup/shutdown logic:**  $v_{i,t}$  and  $w_{i,t}$  may simultaneously be fractional, violating their mutual-exclusion intent.
- (iii) **Output–state coupling:**  $P_{i,t}$  may be non-zero even when  $u_{i,t}$  is small, because the product  $P_i^{\max} \cdot u_{i,t}$  in (690) is only linearised at its LP extreme.
- (iv) **Startup/shutdown ramp coupling:** the relaxed ramp constraints (694)–(695) fail to capture the multi-period ramp-up trajectory from a cold start.

#### Storage-mode weaknesses.

- (i) **SOC path infeasibility:** the LP allows  $E_{e,t}$  to “jump” across time periods beyond what is physically achievable.
- (ii) **Simultaneous charge/discharge:**  $\xi_{e,t}^+ = \xi_{e,t}^- = 0.5$  is LP-feasible via (707), creating fictitious energy from roundtrip losses ( $\eta^{\text{ch}}\eta^{\text{dis}} < 1$ ).
- (iii) **Energy–duration decoupling:** the LP may schedule sustained full-power discharge across many periods in excess of usable capacity.

### 28.11.2 Generator Cut Families (Families 1–6)

Let  $SU_i$  denote the maximum output achievable in the first period after a cold start (startup ramp capacity, [MW]),  $SD_i$  the maximum output in the last period before shutdown, and define:

$$K_i = \left\lceil \frac{P_i^{\max} - SU_i}{\Delta P_i^U} \right\rceil, \quad \bar{P}_i^{(k)} = \min(P_i^{\max}, SU_i + k \Delta P_i^U), \quad k = 0, 1, \dots, K_i \quad (734)$$

$K_i$  is the number of periods required to ramp from cold-start output  $SU_i$  to full capacity  $P_i^{\max}$ .

**Family 1: Multi-Period Startup Ramp Inequality.** For each unit  $i \in \mathcal{G}$  and period  $t \in \mathcal{T}$ :

$$P_{i,t} \leq P_i^{\max} \cdot u_{i,t} - \sum_{k=1}^{\min(K_i, t)} (P_i^{\max} - \bar{P}_i^{(k)}) v_{i,t-k+1} \quad (735)$$

*Validity:* if  $u_{i,t} = 1$  and unit started  $k-1$  periods ago ( $v_{i,t-k+1} = 1$ ), the right-hand side equals  $\bar{P}_i^{(k)}$ , which is precisely the physical ramp-up ceiling after  $k$  periods.

**Family 2: Startup Clique and Strengthened Interval Up Inequality.** *Startup clique* (no two startups within one up-time window):

$$\sum_{t'=t}^{t+T_i^U-1} v_{i,t'} \leq 1 \quad \forall i, t \quad (736)$$

*Strengthened interval up inequality:* for any sub-window of length  $L \leq T_i^U$  starting at  $t_1$ :

$$\sum_{t'=t_1}^{t_1+L-1} u_{i,t'} \geq \sum_{k=0}^{L-1} (L-k) v_{i,t_1+k} \quad (737)$$

*Validity:* if a startup at  $t_1+k$  occurs (integer solution), then  $u_{i,t'} = 1$  for all  $t' \in [t_1+k, t_1+L-1]$ , contributing exactly  $L-k$  to the left-hand sum. The symmetric shutdown clique and interval-down inequality hold by replacing  $v_{i,t'}$  with  $w_{i,t'}$  and  $u_{i,t'}$  with  $1 - u_{i,t'}$ .

**Family 3: Multi-Period Shutdown Ramp-Down Inequality.** Let  $\bar{P}_i^{\text{dn},(k)} = \min(P_i^{\max}, SD_i + k \Delta P_i^D)$  and  $K_i^{\text{dn}} = \lceil (P_i^{\max} - SD_i) / \Delta P_i^D \rceil$ .

$$P_{i,t} \leq P_i^{\max} \cdot u_{i,t} - \sum_{k=1}^{\min(K_i^{\text{dn}}, T-t)} (P_i^{\max} - \bar{P}_i^{\text{dn},(k)}) w_{i,t+k} \quad (738)$$

This is the time-reverse of Family 1: if a shutdown will occur  $k$  periods ahead, the current output is bounded by the pre-shutdown ramp-down ceiling.

**Family 4: Multi-Level Startup Cost Tightening.** For units with state-dependent startup costs (hot/warm/cold), let  $\tau_i^{\text{hot}}$  and  $\tau_i^{\text{warm}}$  be the hot- and warm-start cooling thresholds (peri-

ods),  $DT_i \leq \tau_i^{\text{hot}} \leq \tau_i^{\text{warm}}$ . Introduce binary startup-type indicators  $\delta_{i,t}^{\text{warm}}$  and  $\delta_{i,t}^{\text{cold}}$ :

$$v_{i,t} \leq \sum_{t'=t-\tau_i^{\text{hot}}}^{t-T_i^D} u_{i,t'} + \delta_{i,t}^{\text{warm}} + \delta_{i,t}^{\text{cold}} \quad (739)$$

$$\delta_{i,t}^{\text{warm}} + \delta_{i,t}^{\text{cold}} \leq 1 - \sum_{t'=t-\tau_i^{\text{hot}}}^{t-T_i^D} u_{i,t'} \quad (740)$$

$$\delta_{i,t}^{\text{cold}} \leq 1 - \sum_{t'=t-\tau_i^{\text{warm}}}^{t-\tau_i^{\text{hot}}-1} u_{i,t'} \quad (741)$$

These inequalities force the LP to identify the startup type correctly, preventing fractional mixing of hot and cold costs.

**Family 5: Three-Period Window Inequality.** Simultaneously tightening the ramp-up, ramp-down, and commitment logic:

$$\boxed{P_{i,t} \leq P_i^{\max} \cdot u_{i,t} - (P_i^{\max} - SU_i) v_{i,t} - (P_i^{\max} - SD_i) w_{i,t+1}} \quad (742)$$

*Interpretation:* if the unit starts in period  $t$  it cannot exceed  $SU_i$ ; if it shuts down in period  $t+1$  it must be no greater than  $SD_i$ .

**Family 6: Symmetry-Breaking Inequalities.** For a group  $\mathcal{G}^{\text{sym}} = \{i_1, i_2, \dots, i_K\}$  of identical units (same  $P^{\max}$ ,  $P^{\min}$ ,  $T^U$ ,  $T^D$ , cost curves):

*Lexicographic ordering:* commit units in label order,

$$u_{i_j,t} \geq u_{i_{j+1},t} \quad \forall j = 1, \dots, K-1, \quad \forall t \quad (743)$$

*Cumulative startup ordering* (stronger):

$$\sum_{t'=1}^t v_{i_j,t'} \geq \sum_{t'=1}^t v_{i_{j+1},t'} \quad \forall j, t \quad (744)$$

Symmetry-breaking cuts are generated once at the root node and remain globally valid throughout the branch-and-cut tree.

### 28.11.3 Storage Cut Families (Families 7–11)

The original six generator cuts remain valid in the presence of storage (they only involve  $\mathcal{G}$  variables), but they cannot close the additional LP gap introduced by  $\mathcal{E}$  variables. The following five families address the storage-specific sources identified in Section 28.11.1.

**Family 7: SOC Reachability Inequality.** For each storage unit  $e \in \mathcal{E}$  and any interval  $[t_1, t_2]$ :

$$E_{e,t_2} - E_{e,t_1} \leq \eta_e^{\text{ch}} \cdot P_e^{\text{ch,max}} \cdot \Delta t \sum_{t=t_1+1}^{t_2} \xi_{e,t}^+ \quad (745)$$

$$E_{e,t_1} - E_{e,t_2} \leq \frac{P_e^{\text{dis,max}}}{\eta_e^{\text{dis}}} \cdot \Delta t \sum_{t=t_1+1}^{t_2} \xi_{e,t}^- \quad (746)$$

*Validity* (charging direction, (745)): from the SOC dynamics (708),  $E_{e,t_2} - E_{e,t_1} \leq \sum_t \eta_e^{\text{ch}} P_{e,t}^{\text{ch}} \cdot \Delta t \leq \sum_t \eta_e^{\text{ch}} P_e^{\text{ch,max}} \cdot \xi_{e,t}^+ \cdot \Delta t$ . The LP may violate (745) when  $\xi^+$  is fractional and the product  $P_e^{\text{ch,max}} \cdot \xi_{e,t}^+$  over-estimates achievable charging. The window length is capped at  $W \leq 12$  periods in the separator to control the number of cut candidates.

**Family 8: Simultaneous Charge/Discharge Disjunctive Cut.** Beyond the period-level exclusion (707), the following cut exploits the energy loss implied by simultaneous operation:

$$\eta_e^{\text{ch}} \cdot P_{e,t}^{\text{ch}} \cdot \Delta t + \frac{P_{e,t}^{\text{dis}}}{\eta_e^{\text{dis}}} \cdot \Delta t \leq \max\left(\eta_e^{\text{ch}} P_e^{\text{ch,max}}, \frac{P_e^{\text{dis,max}}}{\eta_e^{\text{dis}}}\right) \cdot (\xi_{e,t}^+ + \xi_{e,t}^-) \cdot \Delta t \quad (747)$$

In an integer solution  $\xi_{e,t}^+ + \xi_{e,t}^- \in \{0, 1\}$ , so the right-hand side equals either 0 (idle) or the larger of the two physical power rates. When the LP places both indicators at 0.5, the right-hand side becomes  $\max(\cdot)$ , which is tighter than the sum of the individual big- $M$  bounds given by (705)–(706).

**Family 9: Energy–Duration Coupling Inequality.** For any discharge window  $[t_1, t_2]$ :

$$\sum_{t=t_1}^{t_2} \frac{P_{e,t}^{\text{dis}}}{\eta_e^{\text{dis}}} \cdot \Delta t \leq (E_{e,t_1} - \underline{E}_e) + \eta_e^{\text{ch}} P_e^{\text{ch,max}} \cdot \Delta t \sum_{t=t_1+1}^{t_2} \xi_{e,t}^+ \quad (748)$$

*Interpretation:* the total energy extracted over the window cannot exceed the available energy at  $t_1$  above the minimum SOC, augmented by any charging that occurs during the same window.

**Family 10: Cross-Device Substitution Inequality (Cover Type).** For each period  $t$  and any minimal cover set  $\mathcal{C}_t \subseteq \mathcal{G}^a \cup \mathcal{E}^a$  satisfying

$$\sum_{i \in \mathcal{C}_t \cap \mathcal{G}} P_i^{\text{max}} + \sum_{e \in \mathcal{C}_t \cap \mathcal{E}} P_e^{\text{dis,max}} > D_t^a - \sum_{e \notin \mathcal{C}_t} P_e^{\text{dis,max}},$$

the resulting cover inequality is:

$$\sum_{i \in \mathcal{C}_t \cap \mathcal{G}} u_{i,t} + \sum_{e \in \mathcal{C}_t \cap \mathcal{E}} \xi_{e,t}^- \leq |\mathcal{C}_t| - 1 \quad (749)$$

This forbids every device in the cover from simultaneously operating at or near full output, which is unnecessary to meet demand. In practice, the separator identifies the most-violated cover of size  $|\mathcal{C}_t| \leq 5$  at the root node only, limiting the  $\mathcal{O}(|\mathcal{G}||\mathcal{E}|)$  enumeration to tractable runs.

**Family 11: Cyclic SOC Feasibility Inequality.** When the problem requires  $E_{e,T} \geq E_e^{\text{fin}} \approx E_e^{\text{ini}}$  (daily cycling), summing (708) over all periods gives:

$$\sum_{t=1}^T \eta_e^{\text{ch}} P_{e,t}^{\text{ch}} \cdot \Delta t \geq \sum_{t=1}^T \frac{P_{e,t}^{\text{dis}}}{\eta_e^{\text{dis}}} \cdot \Delta t + (E_e^{\text{fin}} - E_e^{\text{ini}}) \quad (750)$$

The tightened form replaces  $P_{e,t}^{\text{ch}}$  by its binary-state upper bound, yielding a cut on the charging indicator sum:

$$\eta_e^{\text{ch}} P_e^{\text{ch,max}} \cdot \Delta t \sum_{t=1}^T \xi_{e,t}^+ \geq \sum_{t=1}^T \frac{P_{e,t}^{\text{dis}}}{\eta_e^{\text{dis}}} \cdot \Delta t + (E_e^{\text{fin}} - E_e^{\text{ini}}) \quad (751)$$

This global cut is generated once per storage unit at the root node.

#### 28.11.4 Cut Properties and Separation Complexity

| Family | Description      | Max candidates                 | Non-zeros                | Separation cost              | Depth limit |
|--------|------------------|--------------------------------|--------------------------|------------------------------|-------------|
| 1      | Startup ramp     | $O(NT)$                        | $\leq K_i+2$             | $O(NTK_{\max})$              | 0–10        |
| 2      | Interval up      | $O(NT \cdot T_{\max}^U)$       | $\leq 2T_{\max}^U$       | $O(NT(T_{\max}^U)^2)$        | 0–10        |
| 3      | Shutdown ramp    | $O(NT)$                        | $\leq K_i^{\text{dn}}+2$ | $O(NTK_{\max}^{\text{dn}})$  | 0–10        |
| 4      | Startup cost     | $O(NT)$                        | $\leq 2\tau_{\max}$      | $O(NT\tau_{\max})$           | 0 only      |
| 5      | 3-period window  | $O(NT)$                        | 3                        | $O(NT)$                      | 0–10        |
| 6      | Symmetry break   | $O(N_{\text{sym}}T)$           | 2                        | $O(N_{\text{sym}}T)$         | 0 only      |
| 7      | SOC reachability | $O(N_{ES}TW)$                  | $\leq W+2$               | $O(N_{ES}TW^2)$              | 0–10        |
| 8      | Charge/discharge | $O(N_{ES}T)$                   | 4                        | $O(N_{ES}T)$                 | 0–10        |
| 9      | Energy–duration  | $O(N_{ES}TW)$                  | $\leq W+2$               | $O(N_{ES}TW)$                | 0–10        |
| 10     | Cross-device     | $O(T \cdot 2^{ \mathcal{C} })$ | $ \mathcal{C} $          | $O(T \cdot NG \cdot N_{ES})$ | 0 only      |
| 11     | Cyclic SOC       | $O(N_{ES})$                    | $\leq T+1$               | $O(N_{ES}T)$                 | 0 only      |

$W \leq 12$  is the window-length cap for SOC cuts. All families satisfy the native solver’s density cap ( $\leq 0.05 \cdot n$  non-zeros) because non-zero counts grow with machine parameters ( $K_i$ ,  $T_i^U$ ,  $W$ ), not with total problem size  $n$ .

#### 28.11.5 Layered Separation Strategy

Cuts are generated by a `UCSeparationOracle` (see `src/engine/branch_and_cut.cpp`) that dispatches to per-family separators according to tree depth:

1. **Root node** ( $d = 0$ ): Families 4, 6, 10, 11 are generated as pre-formulation rows and never revisited.
2. **Shallow nodes** ( $1 \leq d \leq 10$ ): Families 1, 2, 3, 5, 7, 8, 9 are active as B&B separation cuts.
3. **Deep nodes** ( $d > 10$ ): no problem-specific cuts; the solver relies on domain propagation and reduced-cost fixing.

**Pre-formulation vs. B&B separation.** Families 6 and 11 are the only generic families appended to the LP *before* branch-and-cut begins. Families 1, 2, 3, and 5 mix continuous dispatch outputs with startup/shutdown binaries ( $v_{i,t'}$ ,  $w_{i,t'}$ ). In the LP relaxation these binaries are fractional, which can drive the right-hand side of (735)–(742) below zero while the lower bound  $P_i^{\min}u_{i,t} > 0$  is simultaneously active, producing an infeasible LP. They must therefore be added lazily at integer nodes where  $v_{i,t'} \in \{0, 1\}$  is already fixed.

After each round of cut generation, violated cuts are filtered by an efficacy threshold  $\epsilon_{\text{cut}} \geq 10^{-4}$  and a maximum density cap before being appended via `AppendCuts()`. Numerical safety for Families 1 and 3 (coefficients proportional to  $P_i^{\max}$ ) is enforced by normalising each cut row by  $P_i^{\max}$  before appending, so all coefficients lie in  $[0, 1]$ .

**Interaction with RENS heuristic.** Problem-specific cuts tighten the LP solution, increasing the fraction of  $u_{i,t}$ ,  $v_{i,t}$ ,  $w_{i,t}$ , and  $\xi_{e,t}^{\pm}$  variables that are already integer-valued in the LP optimum. The Residual RENS heuristic (§22) then fixes this larger integer-valued set, reduces the residual sub-problem size, and finds an incumbent substantially faster. The expected effect is:

$$\underbrace{\text{Tighter LP}}_{\text{Cuts}} \xrightarrow{\text{more fixed } u/\xi} \underbrace{\text{Smaller RENS sub-problem}}_{\text{faster incumbent}} \xrightarrow{\text{small gap}} \underbrace{\text{Fewer proof nodes}}_{\text{faster termination}} \quad (752)$$

## 28.12 Market-Model-Specific Cutting Planes (SCUC Extensions)

The eleven generic cut families in Section 28.11 apply to any UC formulation. The Southern-China SCUC model (Section 28.6) exposes additional combinatorial structure that enables nine further families (A–I). These families exploit features unique to this market model: piecewise-linear bid curves, multi-type reserve requirements, monitored sections, intra- and inter-regional DC interconnections, hydro reservoirs, inter-provincial priority schedules, and a 96-period (15 min) time resolution.

### 28.12.1 Family A: Bid-Segment Cumulative Coupling

The output decomposition (688) and segment bounds (689) only enforce each segment  $m$  individually. Because bid prices are non-decreasing ( $C_{i,1} \leq C_{i,2} \leq \dots \leq C_{i,N_M}$ ), dispatch fills segments sequentially: segment  $m$  is used only after segment  $m - 1$  is saturated. The LP relaxation violates this ordering when  $u_{i,t}$  is fractional.

**Cumulative segment cut.** For each unit  $i$ , period  $t$ , and prefix  $k = 1, \dots, N_M$ :

$$\sum_{m=1}^k \Delta P_{i,t,m} \leq \left( \sum_{m=1}^k \overline{\Delta P}_{i,m} \right) \cdot u_{i,t} \quad (753)$$

This is tighter than the individual bounds  $\Delta P_{i,t,m} \leq \overline{\Delta P}_{i,m} u_{i,t}$  because it requires the *total* committed capacity of the first  $k$  segments to respect the binary state, preventing the LP from spreading fractional commitment uniformly across all segments.

### 28.12.2 Family B: Reserve–Commitment Cover Inequality

The positive-reserve constraint (682) and the reserve-coupling bound (683) jointly require sufficient committed capacity in each balancing area  $a$ . The LP relaxation allows fractional  $u_{i,t}$  to provide the required head-room at a fraction of the integer cost.

**Minimum-commitment cover for reserves.** For each area  $a$  and period  $t$ , let  $\mathcal{F}_t^a$  be the set of units that cannot individually cover the residual reserve requirement after subtracting storage and import. A minimum cover  $\mathcal{C}_t^a \subseteq \mathcal{G}^a$  satisfies

$$\sum_{i \in \mathcal{G}^a \setminus \mathcal{C}_t^a} (P_i^{\max} - P_i^{\min}) < R_t^{U,a} + D_t^a - \sum_{e \in \mathcal{E}^a} P_e^{\text{dis,max}} - T^{\text{import,max},a} \quad (754)$$

yielding the reserve cover inequality:

$$\sum_{i \in \mathcal{G}^a \setminus \mathcal{C}_t^a} u_{i,t} \geq \left\lceil \frac{R_t^{U,a} - \sum_e P_e^{\text{dis,max}} - \sum_{i \in \mathcal{C}_t^a} (P_i^{\max} - P_i^{\min})}{\max_{i \notin \mathcal{C}_t^a} (P_i^{\max} - P_i^{\min})} \right\rceil \quad (755)$$

**Reserve–ramp startup tightening.** A unit that just started ( $v_{i,t} = 1$ ) has a reduced upward ramp capability. The achievable spinning reserve at startup is bounded by:

$$R_{i,t}^{\text{spin}} \leq \min(\Delta P_i^U, P_i^{\max} - P_i^{\min}) u_{i,t} - (P_i^{\max} - P_i^{\min} - \Delta P_i^U)^+ v_{i,t} \quad (756)$$

### 28.12.3 Family C: Monitored-Section Commitment Inequality

Section flow constraints (704) aggregate multiple transmission lines. Cutting at the section level produces fewer, denser cuts that are more effective than per-line cuts for large networks.

Define the generation shift factor for section  $s$  and unit  $i$  as:

$$\text{GSF}_{s,i} = \sum_{l \in \text{lines}(s)} G_{l, \text{bus}(i)} \quad (757)$$

**Forward section commitment cut.** For the forward limit of section  $s$  in period  $t$ :

$$\boxed{\sum_{\substack{i \in \mathcal{G}: \\ \text{GSF}_{s,i} > 0}} \text{GSF}_{s,i} \cdot P_i^{\max} \cdot u_{i,t} \geq P_s^{\max} + \text{NetDemand}_t^{s,+}} \quad (758)$$

where  $\text{NetDemand}_t^{s,+}$  is the section-facing net demand (load minus storage dispatch and imports on the forward side). An analogous reverse-direction cut covers the lower section limit.

### 28.12.4 Family D: DC-Line Direction–Commitment Coupling

The DC line model (Section 28.6.17) prohibits direction reversal between adjacent periods and limits the ramp. Introducing a binary direction indicator  $\delta_{d,t} \in \{0, 1\}$  (1 = forward) gives:

$$T_{d,t}^{DC} \leq T_d^{DC, \max} \cdot \delta_{d,t} \quad (759)$$

$$T_{d,t}^{DC} \geq T_d^{DC, \min} \cdot (1 - \delta_{d,t}) \quad (760)$$

**Receiving-end commitment requirement.** When the DC link does not deliver to area  $a_{\text{recv}}$  ( $\delta_{d,t} = 0$ ), local generation must cover the full load plus reserve:

$$\boxed{\sum_{i \in \mathcal{G}^{a_{\text{recv}}}} P_i^{\max} \cdot u_{i,t} \geq D_t^{a_{\text{recv}}} + R_t^{U, a_{\text{recv}}} - T_d^{DC, \max} \cdot \delta_{d,t}} \quad (761)$$

**DC direction persistence cut (analogue of min-up-time).** Once a DC direction is established, forcing it to persist for  $L$  periods reduces LP oscillation:

$$\sum_{t'=t}^{t+L-1} \delta_{d,t'} \geq L \cdot (\delta_{d,t} - \delta_{d,t-1})^+ \quad L = 2, 3, 4 \quad (762)$$

### 28.12.5 Family E: Storage Cycle-Limit Strengthening

The cycle-energy bound (713) limits total throughput but does not constrain the number of discharge periods, allowing the LP to exploit fractional  $\xi_{e,t}^-$ .

**Discharge-period count bound.** Each full charge–discharge cycle occupies at least 2 active periods. Hence the total number of discharge periods is bounded:

$$\boxed{\sum_{t=1}^T \xi_{e,t}^- \leq 2 N_e^{\text{cycle}}} \quad (763)$$

**Window cycle quota cut.** For any sub-window  $[t_1, t_2]$  of length  $L$ :

$$\sum_{t=t_1}^{t_2} \frac{P_{e,t}^{\text{dis}}}{\eta_e^{\text{dis}}} \cdot \Delta t \leq N_e^{\text{cycle}} \cdot \frac{L}{T} \cdot (\bar{E}_e - \underline{E}_e) + (E_{e,t_1} - \underline{E}_e) \quad (764)$$

The right-hand side is the pro-rated cycle quota for the window plus the energy already available at the start of the window.

### 28.12.6 Family F: Hydro–Thermal Substitution Inequality

Because hydro generation can substitute for thermal commitment, the LP over-exploits fractional hydro flexibility to defer binary startups. Let  $E_h^{\text{total}}$  be the total available hydro energy of station  $h$  over the horizon, and  $\mathcal{T}_{\text{peak}}$  the set of peak-load periods.

**Thermal capacity sufficiency cut.** For each peak period  $t \in \mathcal{T}_{\text{peak}}$ :

$$\sum_{i \in \mathcal{G}^a} P_i^{\text{max}} \cdot u_{i,t} \geq D_t^a + R_t^{U,a} - \sum_{h \in \mathcal{H}^a} P_h^{\text{max}} - \sum_{e \in \mathcal{E}^a} P_e^{\text{dis,max}} - T^{\text{import,max},a} \quad (765)$$

**Hydro-energy–thermal minimum-output coupling.** The sum of thermal minimum output over peak periods cannot exceed the demand surplus after hydro and imports are accounted for:

$$\sum_{t \in \mathcal{T}_{\text{peak}}} \sum_{i \in \mathcal{G}^a} P_i^{\text{min}} \cdot u_{i,t} \leq \sum_{t \in \mathcal{T}_{\text{peak}}} D_t^a - \sum_{h \in \mathcal{H}^a} E_h^{\text{total}} - \sum_t T_t^{\text{import},a} \quad (766)$$

### 28.12.7 Family G: Extended Time-Window Clique (96-Period)

With  $T = 96$  periods and minimum up- plus down-times  $T_i^U + T_i^D$  that may span 16–48 periods, the extended window clique is substantially tighter than the standard startup clique (736).

**$(T_i^U + T_i^D)$ -period startup clique.** No two startups can occur within a window that spans one full on–off cycle:

$$\sum_{t'=t}^{t+T_i^U+T_i^D-1} v_{i,t'} \leq 1 \quad \forall i, t = 1, \dots, T - (T_i^U + T_i^D) + 1 \quad (767)$$

**Total startup count bound.** Over the full horizon, the maximum number of startups is:

$$\sum_{t=1}^T v_{i,t} \leq \left\lfloor \frac{T}{T_i^U + T_i^D} \right\rfloor \quad \forall i \quad (768)$$

**Strengthened 96-period interval-up inequality.** For windows of length  $W = T_i^U + T_i^D$ , the interval-up inequality (737) strengthens to:

$$\sum_{t'=t}^{t+W-1} u_{i,t'} \geq T_i^U \cdot v_{i,t} + T_i^U \cdot v_{i,t+T_i^U+T_i^D} \cdot \mathbf{1}[t + T_i^U + T_i^D \leq T] \quad (769)$$

requiring at least  $T_i^U$  consecutive on-periods for every startup within the window.

### 28.12.8 Family H: Inter-Provincial Schedule–Commitment Coupling

The priority schedule constraint (724) is enforced with a high-penalty slack  $SL_k$ . In integer solutions, meeting a large export schedule may require additional committed capacity beyond what the LP fractional solution uses.

**Export-commitment capacity cut.** For each area  $a$  and period  $t$  with net export plan  $T_t^{\text{plan,exp},a}$ :

$$\sum_{i \in \mathcal{G}^a} P_i^{\text{max}} \cdot u_{i,t} + \sum_{e \in \mathcal{E}^a} P_e^{\text{dis,max}} \cdot \xi_{e,t}^- \geq D_t^a + T_t^{\text{plan,exp},a} + R_t^{U,a} \quad (770)$$

**Export-ramp startup cut.** When the planned export ramps up by  $\Delta T_t^{\text{plan},a}$  between periods  $t-1$  and  $t$ :

$$\sum_{i \in \mathcal{G}^a} \Delta P_i^U \cdot u_{i,t-1} + \sum_{i \in \mathcal{G}^a} SU_i \cdot v_{i,t} \geq \Delta T_t^{\text{plan},a} \quad (771)$$

### 28.12.9 Family I: Primary Frequency Regulation–Storage Joint Cut

The PFR requirement (685) must be met by thermal and hydro units. When storage provides PFR, its contribution is limited by the available SOC headroom above  $\underline{E}_e$ .

**PFR–SOC joint cut.** Let  $\Delta t_{\text{pf}}$  be the PFR response window [hours] and  $P_e^{\text{pf,max}} = \min(P_e^{\text{dis,max}}, (\bar{E}_e - \underline{E}_e)/(2 \Delta t_{\text{pf}}))$ :

$$\sum_{i \in \mathcal{G}} R_i^{\text{pf,max}} \cdot u_{i,t} + \sum_{e \in \mathcal{E}} P_e^{\text{pf,max}} \cdot \xi_{e,t}^- \geq R_t^{\text{pf}} \quad \forall t \quad (772)$$

**SOC-dependent PFR cut.** When the LP SOC variable is available:

$$\sum_{i \in \mathcal{G}} R_i^{\text{pf,max}} \cdot u_{i,t} + \sum_{e \in \mathcal{E}} \frac{E_{e,t} - \underline{E}_e}{\Delta t_{\text{pf}}} \geq R_t^{\text{pf}} \quad (773)$$

This is a convex valid inequality: it tightens the PFR feasibility set by using the actual SOC value rather than the design maximum.

### 28.12.10 Market-Cut Priority and Separation Strategy

| Family | Description            | Candidates          | Non-zeros                  | Expected gap closing | Depth limit | Impl. |
|--------|------------------------|---------------------|----------------------------|----------------------|-------------|-------|
| G      | Extended clique        | $O(NT)$             | $\leq T_i^U + T_i^D$       | 25–30%               | 0–10        | ✓     |
| B      | Reserve–commitment     | $O( \mathcal{A} T)$ | $\leq N^a$                 | 20–25%               | 0–5         | ✓     |
| C      | Section commitment     | $O(N_S T)$          | $\leq N$                   | 15–20%               | 0–5         | ✓     |
| A      | Bid-segment cumul.     | $O(N N_M T)$        | $\leq N_M + 1$             | 10–15%               | 0–5         | ✓     |
| E      | Cycle strengthening    | $O(N_{ES} T)$       | $\leq T + 2$               | 8–12%                | 0–5         | ✓     |
| F      | Peak-period cover      | $O( \mathcal{A} )$  | $\leq N^a$                 | 3–5%                 | 0 only      | ✓     |
| D      | DC direction           | $O(N_{DC} T)$       | $\leq N^{\text{recv}} + 1$ | 5–10%                | 0–5         | —     |
| H      | Interprovincial sched. | $O( \mathcal{A} T)$ | $\leq N^a + N_{ES}$        | 5–8%                 | 0 only      | —     |
| I      | PFR–storage            | $O(T)$              | $\leq N + N_{ES}$          | 3–5%                 | 0–5         | —     |

The ✓ column indicates families currently implemented as pre-formulation rows in `market_simulation.cpp`. Families D, H, and I require model elements absent from the current formulation (DC direction binaries, provincial export schedules, and PFR requirement data, respectively) and are reserved for future extensions.

Among the implemented families, B, C, and F include an additional feasibility guard: the cut is skipped for any area whose total  $\sum_i P_i^{\max}$  is less than the residual requirement  $r_{a,h}$ . In a multi-area system with unconstrained inter-area imports the area could satisfy demand through imports, so requiring  $\sum P_i^{\max} \geq r_{a,h}$  at the LP level would produce an infeasible LP row.

**Relation to generic families.** Family G is a strict extension of generic Family 2 (startup clique), enlarging the window from  $T_i^U$  to  $T_i^U + T_i^D$ . Family B incorporates the reserve dimension absent from generic Family 5 and additionally subtracts wind, solar, hydro, and storage non-thermal maximum output from the requirement. Family F is a peak-period variant of B (applied to the single highest-demand commitment period per area) and replaces the earlier conservative “hydro-thermal only” formulation that incorrectly excluded renewable credit. Family E complements generic Family 9 by additionally exploiting the explicit cycle-count parameter  $N_e^{\text{cycle}}$ . Families A and the corrected F have no direct counterparts in the generic set.

### 28.13 Numerical Performance: Pre-Formulation Cuts on Five-Province SCUC

The cutting-plane implementation was validated on the five-province day-ahead SCUC benchmark in `tests/test_market_simulation.cpp` (`[market][cuts][benchmark]`). The test solves the same instance twice—without and with all implemented pre-formulation cut families (§28.11: 6, 11 and §28.12: A, B, C, E, F, G)—using Gurobi with default settings.

The 1 869 cut rows are dominated by Family A (one row per segment per unit per dispatch period) and Family G (one clique row per generator per commitment window). Families B and F each contribute one row per balancing area per relevant commit period; Families E and 11 contribute  $2N_{ES}$  rows; Family C contributes rows only when monitored sections are present.

The 56% solve-time reduction is consistent with the expected root-LP tightening effect: a tighter LP reduces both the initial MIP gap and the number of branch-and-bound nodes required for proof. The zero objective deviation confirms that every added row is LP-valid and does not cut off any feasible integer solution.

**Implementation notes.** Families 1, 2, 3, and 5 (§28.11.2) are *not* added as pre-formulation rows. In the LP relaxation, startup and shutdown variables  $v_{i,t'}$  and  $w_{i,t'}$  are fractional; the ramp-cut right-hand sides can consequently become negative while the lower bound  $P_i^{\min} u_{i,t} > 0$  is simultaneously active, making the LP infeasible. These families must be applied as B&B separation cuts at nodes where  $v_{i,t'} \in \{0, 1\}$  is already determined.

Families B, C, and F include an inter-area import guard: a cut is skipped for any area whose committed thermal capacity  $\sum_{i \in \mathcal{G}^a} P_i^{\max}$  falls below the residual requirement  $r_{a,h}$ . Without the guard, areas that rely on unconstrained imports would receive infeasible LP rows.

## 29 Stochastic Dispatch and Market-Boundary Optimization

### 29.1 Problem Form and Runtime Scope

This notebook chapter summarizes the stochastic market-boundary optimization material maintained separately in the standalone document `sections/17_two_stage_stochastic.tex`. The underlying formulation is a multi-stage stochastic dispatch and maintenance-planning model

with scenario tree structure, expected-cost and risk terms, and decomposition-oriented solve strategies.

## 29.2 Code Map

The material summarized here is aligned primarily with:

- `src/analysis/market_simulation.cpp` for market-clearing and scheduling-side model infrastructure,
- the MILP solver layer documented in Section 4,
- the standalone research-style derivation in `sections/17_two_stage_stochastic.tex` and its compiled PDF companion.

## 29.3 Pseudocode Map

The detailed technical notebook pseudocode already covers the main deterministic building blocks reused here:

- Algorithm A16 for DC OPF-type operational subproblems,
- Algorithm A20 for hierarchical annual and chronological dispatch orchestration,
- Algorithm E9 and Section 22 for MILP solve mechanics.

The full stochastic extensive-form and decomposition derivations remain in the standalone stochastic-dispatch manuscript rather than being duplicated inside this notebook.

## 29.4 Current Status

The stochastic-dispatch material in this repository is best understood as a hybrid of implementation-facing documentation and a standalone methods paper. For the notebook build, the standalone paper is kept external and this section acts as the normalized notebook-facing summary chapter.

## 29.5 Canonical Problem Form

In condensed form, the stochastic planning model can be read as

$$\min_{x, y(\omega)} \quad c^\top x + \sum_{\omega \in \Omega} p_\omega q_\omega^\top y(\omega) + \text{risk terms} \quad (774)$$

$$\text{s.t.} \quad Ax \leq b, \quad (775)$$

$$T_\omega x + W_\omega y(\omega) \leq h_\omega, \quad \forall \omega \in \Omega, \quad (776)$$

with first-stage boundary or maintenance decisions in  $x$  and scenario-wise recourse decisions in  $y(\omega)$ .

## 29.6 How This Chapter Relates to the Standalone Manuscript

The standalone file `sections/17_two_stage_stochastic.tex` is still the full derivation and should continue to be compiled on its own when the research paper version is needed. The technical notebook now keeps only the normalized summary chapter so the main notebook build remains a single coherent document instead of recursively including a second document preamble.

Table 136: Baseline planning and economic parameters for the 10b hybrid AC/DC distribution-planning benchmark.

| Quantity                                          | Baseline value                                               | Interpretation                                                    |
|---------------------------------------------------|--------------------------------------------------------------|-------------------------------------------------------------------|
| Annual discount rate $r$                          | 8%                                                           | annualization basis for DER and converter investment              |
| Planning horizon                                  | 1 representative year                                        | annualized planning objective                                     |
| Annual carbon budget $\bar{E}_{CO_2}^{ann}$       | 0.80 of base-case emissions                                  | moderate decarbonization target                                   |
| Peak FMEA reliability budget $\bar{E}_{rel}^{pk}$ | 0 for critical buses; 0.5–2.0 MWh for all-load sensitivities | strict critical-load security, softer systemwide adequacy options |
| Extreme resilience budget $\bar{E}_{rel}^{ext}$   | scenario-weighted 1.0–5.0 MWh equivalent                     | admissible loss under severe $N-k$ events                         |
| PV CAPEX                                          | 650–900 \$/kW                                                | installed AC-side equivalent PV cost                              |
| BESS power CAPEX                                  | 120–180 \$/kW                                                | inverter and power-electronics component                          |
| BESS energy CAPEX                                 | 180–260 \$/kWh                                               | cell and container energy component                               |
| Converter CAPEX                                   | 90–150 \$/kW                                                 | AC/DC interlinking converter sizing cost                          |
| Substation expansion CAPEX                        | 60–120 \$/kW                                                 | upstream import-capacity reinforcement                            |
| PV availability $\alpha_{i,\omega}^{pv}$          | state-dependent, in $[0, 1]$                                 | representative irradiance profile multiplier                      |
| BESS SOC bounds $[\underline{s}_i, \bar{s}_i]$    | $[0.1, 0.9]$                                                 | normal-state storage operating range                              |
| Peak pre-contingency SOC $s_i^{pk}$               | 0.6–0.8                                                      | reliability-prepared storage state at annual peak                 |
| Extreme-event pre-contingency SOC $s_i^{ext}$     | 0.7–0.9                                                      | resilience-prepared energy state for grid-forming support         |
| Upstream purchase tariff                          | time-state dependent                                         | used in (388)                                                     |
| Export tariff                                     | 0.2–0.6 of import tariff                                     | limited compensation for reverse power flow                       |
| Emission factor $\kappa_{r,\omega}$               | 0.45–0.70 kg/kWh                                             | upstream electricity carbon intensity                             |
| Failure frequency $\lambda_k$                     | component-specific                                           | annual contingency occurrence rate from FMEA catalog              |
| Outage duration $\tau_k$                          | component-specific                                           | equivalent switching plus repair duration at peak                 |

Table 137: Baseline electrical and resilience-support parameters for the 10b hybrid AC/DC distribution-planning benchmark.

| Quantity                                                   | Baseline value                                      | Interpretation                                                    |
|------------------------------------------------------------|-----------------------------------------------------|-------------------------------------------------------------------|
| PV reactive capability factor $\bar{\gamma}_i^{pv}$        | 0.3–0.5 p.u.                                        | inverter VAR capability relative to available active power        |
| BESS reactive capability factor $\bar{\gamma}_i^{bess}$    | 0.4–0.6 p.u.                                        | inverter VAR capability relative to BESS power rating             |
| VSC reactive capability factor $\bar{\gamma}_c^{vsc}$      | 0.3–0.6 p.u.                                        | AC-side converter VAR capability factor                           |
| AC line resistance $r_e^{ac}$                              | feeder-specific                                     | active-power contribution to LinDistFlow voltage drop             |
| AC line reactance $x_e^{ac}$                               | feeder-specific                                     | reactive-power contribution to LinDistFlow voltage drop           |
| AC active flow limits                                      | feeder-specific                                     | branch active-power bounds under AC P/Q LinDistFlow               |
| AC reactive flow limits                                    | feeder-specific                                     | branch reactive-power bounds under AC P/Q LinDistFlow             |
| AC voltage bounds $[\underline{u}_i^{ac}, \bar{u}_i^{ac}]$ | feeder-specific, typically $0.95^2$ – $1.05^2$ p.u. | squared-voltage operating envelope for AC buses                   |
| AC root voltage $u_r^{ref}$                                | $1.0^2$ p.u.                                        | feeder-head LinDistFlow reference                                 |
| AC root reactive capability $\bar{Q}_r^{sub}$              | feeder-specific                                     | upstream reactive support range at each AC substation             |
| AC load reactive ratio $\eta_i^{Q,load}$                   | bus-specific or region-specific                     | converts active load service and curtailment into reactive demand |
| DC line thermal limits                                     | feeder-specific                                     | branch active-power limits under linear DC flow                   |
| Contingency decoupling constants $M_e^{ac}, M_\ell^{dc}$   | sufficiently large, topology-specific               | deactivate constitutive equations for outaged branches            |
| Grid-forming BESS active capability $\bar{P}_i^{gfm}$      | subset-specific                                     | active-power headroom available in islanded source mode           |
| Grid-forming BESS reactive capability $\bar{Q}_i^{gfm}$    | subset-specific                                     | voltage support capability when BESS forms a partition            |
| Extreme-state weights $\varpi_\xi$                         | state-specific                                      | planning importance or occurrence weight of each severe event     |
| Extreme-event durations $\tau_{\xi,\kappa}$                | contingency-specific                                | effective restoration duration for each $N-k$ event               |

Table 138: Case-summary template for the 10b hybrid AC/DC distribution-planning study. Numerical values are placeholders to be replaced after solver runs.

| Case                               | Objective | Annual<br>CO <sub>2</sub> | Peak<br>ENS | Extreme<br>ENS | BESS<br>total | Notes                            |
|------------------------------------|-----------|---------------------------|-------------|----------------|---------------|----------------------------------|
| Case 1: eco-<br>nomic base         | –         | –                         | –           | –              | –             | baseline eco-<br>nomics          |
| Case 2: carbon-<br>constrained     | –         | –                         | –           | –              | –             | carbon-only ef-<br>fect          |
| Case 3: peak<br>reliability        | –         | –                         | –           | –              | –             | peak $N-1$ effect                |
| Case 4: VAR-<br>assisted joint     | –         | –                         | –           | –              | –             | carbon + relia-<br>bility + VARs |
| Case 5: ex-<br>treme resilience    | –         | –                         | –           | –              | –             | $N-k$ islands<br>with GFM BESS   |
| Case 6: archi-<br>tecture ablation | –         | –                         | –           | –              | –             | AC/DC and de-<br>vice values     |

Table 139: Compact parameter-summary template for the 10b benchmark. This table is intended to be synchronized with the CSV parameter sheet used to run the cases.

| Parameter group           | Placeholder<br>value | Comment                                                |
|---------------------------|----------------------|--------------------------------------------------------|
| Network size              | –                    | AC buses, DC buses, branches, con-<br>verters          |
| Candidate PV sites        | –                    | number and location rule                               |
| Candidate BESS sites      | –                    | number and location rule                               |
| Carbon budget             | –                    | annual cap or cap ratio                                |
| Peak reliability budget   | –                    | critical-load or all-load criterion                    |
| Extreme resilience budget | –                    | weighted severe-event loss cap                         |
| Reactive support policy   | –                    | PV/BESS/VSC/substation VAR<br>capability settings      |
| Extreme-state library     | –                    | severe events and associated $N-k$<br>families         |
| PV CAPEX                  | –                    | \$/kW                                                  |
| BESS CAPEX                | –                    | \$/kW and \$/kWh                                       |
| Converter CAPEX           | –                    | \$/kW                                                  |
| Substation CAPEX          | –                    | \$/kW                                                  |
| Tariff / emission factors | –                    | representative-state dependent                         |
| Failure frequencies       | –                    | FMEA input basis                                       |
| Outage durations          | –                    | switching plus repair basis                            |
| GFM BESS settings         | –                    | source-mode power, VAR, and SOC<br>reserve assumptions |

Table 152: Five-province SCUC benchmark: baseline vs. with pre-formulation cuts (Gurobi; 8 commit periods; 96-period 15 min dispatch).

| <b>Metric</b>            | <b>Baseline</b> | <b>With cuts</b> |
|--------------------------|-----------------|------------------|
| Converged                | yes             | yes              |
| Objective (\$/interval)  | 321 921.71      | 321 921.71       |
| Objective deviation      | —               | 0.000%           |
| Solve time (s)           | 0.619           | 0.270            |
| Solve-time change        | —               | −56.3%           |
| Final MIP gap            | 0.000%          | 0.000%           |
| Pre-formulation cut rows | 0               | 1 869            |